goxpyriment

Complete Documentation

Christophe Pallier

Contents

1 goxpyriment **13**
- 1.1 Documentation . . . 13
- 1.2 Gallery of ready-to-run experiments . . . 14
- 1.3 Support . . . 14
- 1.4 Background . . . 14
- 1.5 License & citation . . . 15

2 Goxpyriment installation instructions **17**
- 2.1 No installation is needed to run a goxpyriment app . . . 17
- 2.2 Minimal installation to execute a goxpyriment source code . . . 18
- 2.3 Full installation . . . 18
  - 2.3.1 One extra step on a Linux machine used for data collection . . . 19
  - 2.3.2 Program your own experiments . . . 19
  - 2.3.3 Use an AI coding agent . . . 20

3 Installing a Go Development Environment **21**
- 3.1 macOS . . . 21
  - 3.1.1 1. Install Apple Command Line Tools . . . 21
  - 3.1.2 2. Install Git . . . 22
  - 3.1.3 3. Install Go . . . 22
  - 3.1.4 4. Editor: Visual Studio Code (VS Code) . . . 22
- 3.2 Windows . . . 22
  - 3.2.1 1. Install Git . . . 22
  - 3.2.2 2. Install Go . . . 23
  - 3.2.3 3. Editor . . . 23
- 3.3 Linux (Ubuntu) . . . 24
  - 3.3.1 1. Install Git . . . 24
  - 3.3.2 2. Install Go . . . 24
  - 3.3.3 3. Editor: Visual Studio Code (VS Code) . . . 24
- 3.4 Your First Program (Hello World) . . . 25
  - 3.4.1 Step 0 — Open a terminal . . . 25
  - 3.4.2 Step 1 — Create a project folder . . . 25
  - 3.4.3 Step 2 — Create the program file . . . 26
  - 3.4.4 Step 3 — Run the program . . . 26
- 3.5 (Optional) Install an AI Coding Agent . . . 26
  - 3.5.1 First run . . . 27

4 Creating Your Own Experiment (a beginner's guide) 28
4.1 Before you start . . . 28
4.2 Step 1 — Create a folder for your experiment . . . 29
4.3 Step 2 — Write `main.go` . . . 29
4.4 Step 3 — Tell Go about its dependencies . . . 30
4.5 Step 4 — Run it while you work . . . 30
4.6 Step 5 — Build a standalone program . . . 31
4.7 Step 6 — A more advanced example . . . 31
4.8 Step 7 — Embed your stimuli (images, sounds, CSV) . . . 34
4.8.1 A single CSV file of trials . . . 35
4.8.2 A single image or sound . . . 35
4.8.3 A whole folder of files . . . 35
4.9 Step 8 — Build for other operating systems (to share with colleagues) . . . 36
4.10 Quick reference . . . 37
4.11 Where to go next . . . 38

5 Getting Started with goxpyriment 39
5.1 Tutorial 1: Your First Trial . . . 39
5.2 Tutorial 2: Blocks, Trials, and Data Logging . . . 41
5.3 Tutorial 3: Rapid Serial Visual Presentation (RSVP Streams) . . . 42
5.3.1 Text stream . . . 42
5.4 Tutorial 4: Hardware-Precision RT with Event Timestamps . . . 44
5.5 Tutorial 5 — Test AI Coding . . . 46
5.6 Next Steps . . . 47

6 Goxpyriment Example Experiments 48
6.1 Psychological Experiments . . . 48
6.2 Demonstrations . . . 58
6.3 Technical Tests . . . 60
6.4 Binaries . . . 62
6.5 Running the examples from source . . . 62

7 Pre-built binaries of examples and tests for goxpyriment 64
7.0.1 Windows (x86-64) . . . 64
7.0.2 macOS (Apple Silicon) . . . 65
7.0.3 Linux . . . 65

8 goxpyriment User's Manual 66
8.1 Table of Contents . . . 66
8.2 1. The Experiment Object . . . 67
8.2.1 Automatic setup dialog (launch without `-s`) . . . 67
8.2.2 Key fields . . . 67
8.3 2. Collecting Participant Information . . . 68
8.3.1 Pre-built field sets . . . 68
8.3.2 Defining custom fields . . . 68
8.3.3 Dialog interaction . . . 69
8.3.4 Session persistence . . . 69
8.3.5 Using the results . . . 69

8.4 3. The Run Loop and Error Handling . . . 70
8.4.1 The `exp.Run` wrapper . . . 70
8.4.2 Returning from the callback . . . 70
8.4.3 You don't need to check every error . . . 71
8.5 4. The Coordinate System . . . 71
8.5.1 Logical size . . . 72
8.5.2 Display scaling (HiDPI / fractional scaling) . . . 72
8.6 5. The Rendering Model . . . 72
8.6.1 The blank screen . . . 73
8.6.2 Never draw outside exp.Run . . . 73
8.7 6. Timing Architecture . . . 74
8.7.1 Frame cadence . . . 74
8.7.2 What `Update()` actually does with a frame . . . 74
8.7.3 Startup calibration . . . 76
8.7.4 `exp.Wait` vs `clock.Wait` . . . 76
8.7.5 Sub-millisecond timing is not guaranteed for `exp.Wait` . . . 77
8.7.6 Nanosecond event timestamps . . . 77
8.7.7 Two clocks: the Go clock and the SDL clock . . . 78
8.7.8 Disabling garbage collection . . . 79
8.7.9 Choosing a timing approach . . . 79
8.7.10 Display compositor bypass . . . 80
8.7.11 Variable Refresh Rate (VRR) — arbitrary stimulus durations . . . 81
8.8 7. Input Handling . . . 83
8.8.1 Keyboard . . . 83
8.8.2 Mouse . . . 86
8.8.3 Multi-device input — `WaitAnyEventTS` . . . 87
8.8.4 Key codes . . . 88
8.8.5 Response Devices . . . 88
8.9 8. Data Collection . . . 90
8.9.1 Output files . . . 90
8.9.2 Choosing the output directory . . . 90
8.9.3 Declaring column names . . . 90
8.9.4 Adding rows . . . 91
8.9.5 Example CSV output . . . 91
8.9.6 Saving mid-experiment . . . 91
8.10 9. Stimuli: Lifecycle and Preloading . . . 91
8.10.1 GPU textures are lazily allocated . . . 91
8.10.2 Releasing textures . . . 92
8.10.3 Writing a custom stimulus . . . 92
8.10.4 Interactive widgets . . . 93
8.11 10. High-Precision Streams (RSVP) . . . 93
8.11.1 Text stream . . . 93
8.11.2 Image / mixed stimulus stream . . . 94
8.11.3 Reading events from the stream . . . 94
8.11.4 Computing RT from stream events . . . 95
8.11.5 Reading the timing log . . . 95
8.11.6 Platform timing notes . . . 96
8.11.7 Audio stream . . . 97

8.11.8 Mixed stream . . . 97
8.11.9 Per-stimulus onset triggers . . . 97
8.12 11. Audio . . . 98
8.12.1 Sounds from files or embedded bytes . . . 98
8.12.2 Procedural tones . . . 98
8.12.3 Segment playback with fade . . . 98
8.12.4 Built-in feedback sounds . . . 99
8.12.5 Synchronous vs asynchronous . . . 99
8.12.6 Microphone recording . . . 99
8.12.7 Voice key — detecting vocal onset . . . 100
8.12.8 Timing: microphone and screen on the same clock . . . 100
8.12.9 Saving WAV files for verification . . . 101
8.12.10 Acoustic feedback (shadowing and AV tasks) . . . 101
8.13 12. Experimental Design and Randomization . . . 102
8.13.1 When to use `design.Block` / `design.Trial` . . . 102
8.13.2 Building a design . . . 102
8.13.3 Randomization helpers . . . 102
8.13.4 Between-subjects counterbalancing . . . 103
8.13.5 Constrained shuffling . . . 103
8.14 13. Embedding Stimuli and Trial Lists in the Executable . . . 103
8.14.1 13.1 Project layout . . . 104
8.14.2 13.2 Embedding individual image and sound files . . . 104
8.14.3 13.3 Embedding a whole stimulus directory . . . 104
8.14.4 13.4 Embedding a CSV trial list . . . 105
8.14.5 13.5 Mixing embedded and runtime-generated stimuli . . . 107
8.14.6 13.6 Build and deployment . . . 107
8.15 14. Animated Stimuli . . . 108
8.15.1 Moving dot cloud . . . 108
8.15.2 Drifting grating . . . 109
8.15.3 Drifting Gabor patch . . . 109
8.16 15. Video . . . 109
8.16.1 Start here . . . 110
8.16.2 15.1 `.gv` files: the timing-critical format . . . 111
8.16.3 15.2 MPEG-1 files: the convenient format . . . 116
8.17 16. Gamma Correction and Luminance Linearity . . . 118
8.17.1 The gamma problem . . . 118
8.17.2 Inverse-gamma LUT . . . 118
8.17.3 Enabling gamma correction . . . 118
8.17.4 Measuring your monitor’s gamma . . . 119
8.17.5 Practical example . . . 119
8.18 17. Hardware Triggers and TTL Devices . . . 119
8.18.1 Concepts . . . 119
8.18.2 Onset markers: photodiode square and TTL codes . . . 120
8.18.3 Sending triggers (EEG event codes) . . . 121
8.18.4 Timing advice . . . 122
8.18.5 `Pulse` blocks — and a goroutine is not the fix . . . 122
8.18.6 Reading response inputs (FORP pads) . . . 123
8.18.7 FT232H (Adafruit FT232H breakout, Linux) . . . 123

8.18.8 GPIO (Raspberry Pi and other Linux SBCs) . . . 124
8.18.9 LabJack T4 (Modbus TCP) . . . 124
8.18.10 OLP-IO20 (USB-CDC serial) . . . 124
8.18.11 NEUROSPEC MMBT-S (USB-CDC serial) . . . 125
8.18.12 Supported devices . . . 125
8.18.13 Networked recording control (non-TTL) . . . 127
8.19 18. Display Compositor Bypass . . . 129
8.20 19. Variable Refresh Rate (VRR) Stimulus Presentation . . . 130
8.21 20. Putting It All Together . . . 131

9 goxpyriment API Reference 134
9.1 Package Overview . . . 134
9.2 Package `control` . . . 135
9.2.1 Boilerplate . . . 135
9.2.2 Pre-experiment Setup Dialog . . . 135
9.2.3 Constructor Functions . . . 137
9.2.4 Lifecycle Methods . . . 137
9.2.5 Presentation Methods . . . 138
9.2.6 Input . . . 139
9.2.7 Design and Data . . . 140
9.2.8 Font and Display . . . 140
9.2.9 Gamma Correction . . . 141
9.2.10 Colors, Types, and Constants . . . 141
9.2.11 Onset markers (photodiode square and TTL codes) . . . 142
9.2.12 Audio . . . 143
9.2.13 Microphone . . . 143
9.3 Package `stimuli` . . . 144
9.3.1 Interfaces . . . 144
9.3.2 Visual Stimuli . . . 144
9.3.3 ChoiceGrid . . . 146
9.3.4 Menu . . . 146
9.3.5 Animated / VSYNC-locked Loops . . . 147
9.3.6 Stimulus Streams (High-Precision RSVP) . . . 148
9.3.7 Audio Stimuli . . . 152
9.4 Package `apparatus` and `results` . . . 153
9.4.1 Screen . . . 153
9.4.2 Laying out a block of text . . . 154
9.4.3 Keyboard . . . 155
9.4.4 Mouse . . . 156
9.4.5 GamePad . . . 157
9.4.6 Unified Input — `WaitAnyEventTS` . . . 157
9.4.7 ResponseDevice . . . 158
9.4.8 Microphone . . . 159
9.4.9 VoiceKey . . . 159
9.4.10 WriteWAV . . . 160
9.4.11 DataFile . . . 160
9.5 Package `media` . . . 161
9.5.1 Quick API summary . . . 161

9.5.2 Onset precision summary . . . 162
9.5.3 Platform status . . . 162
9.6 Package `design` . . . 163
9.6.1 Data Structures . . . 163
9.6.2 Randomization . . . 163
9.6.3 Latin Square (Between-Subjects Counterbalancing) . . . 163
9.7 Package `clock` . . . 164
9.8 Package `geometry` . . . 164
9.9 Package `triggers` . . . 165
9.9.1 Interfaces . . . 165
9.9.2 DLPIO8 (DLP-IO8-G, USB-CDC serial) . . . 165
9.9.3 DLPIO20 (DLP-IO20, USB-CDC serial) . . . 166
9.9.4 MEGTTLBox (NeuroSpin Arduino Mega TTL box) . . . 167
9.9.5 MMBTS (NEUROSPEC MMBT-S interface box) . . . 167
9.9.6 FT232HTrigger (Adafruit FT232H, Linux) . . . 168
9.9.7 LinuxGPIOTrigger (Raspberry Pi and other Linux SBCs) . . . 169
9.9.8 LabJackT4 (Modbus TCP) . . . 170
9.9.9 ParallelPort (Linux LPT) . . . 170
9.9.10 Null devices . . . 171
9.9.11 Opening a device by name . . . 171
9.10 Package `staircase` . . . 171
9.10.1 `UpDown` — transformed up/down (Levitt 1971) . . . 172
9.10.2 `Quest` — Bayesian estimation (Watson & Pelli 1983) . . . 172
9.10.3 `Runner` — interleaved staircases . . . 172
9.11 Package `units` . . . 173
9.12 Package `assets_embed` . . . 174
9.13 Coordinate System . . . 174
9.14 Typical Experiment Structure . . . 174

10 Migrating to goxpyriment **176**
10.1 From Expyriment (Python) . . . 176
10.1.1 Concept map . . . 176
10.1.2 Side-by-side: simple RT trial . . . 178
10.1.3 Key differences . . . 179
10.2 From PsychoPy (Python) . . . 179
10.2.1 Concept map . . . 180
10.2.2 Side-by-side: simple RT trial . . . 181
10.2.3 Key differences . . . 182
10.3 From Psychtoolbox (MATLAB) . . . 183
10.3.1 Concept map . . . 183
10.3.2 Side-by-side: simple RT trial . . . 184
10.3.3 Key differences . . . 185
10.4 Common gotchas (all three tools) . . . 186

11 goxpyriment compared with PsychoPy **188**
11.1 The structural difference . . . 188
11.2 Where goxpyriment matches or has an edge . . . 189
11.3 Where PsychoPy is ahead . . . 191

11.3.11. Eye tracking . . . 191
11.3.22. Breadth of visual stimuli . . . 191
11.3.33. Online experiments . . . 191
11.3.44. Video . . . 192
11.3.55. Ecosystem and tooling . . . 192
11.3.66. Camera and speech . . . 192
11.3.77. The Builder itself . . . 192
11.4 Choosing between them . . . 192
11.5 See also . . . 193

12 A discussion between Gemini and Me about Go vs. Python (March 2026) 194
12.0.11. Minimalism: The “One Way” Philosophy . . . 194
12.0.22. Explicit vs. Implicit . . . 194
12.0.33. Tooling and Deployment . . . 195
12.0.4 Comparison at a Glance . . . 195
12.0.5 Comparison at a Glance . . . 195
12.1 1. Why Python is Easier for *Humans* . . . 196
12.2 2. Why Go is Easier for *AI Agents* . . . 196

13 Timing tests 197
13.1 Part 1 — Measuring your own machine . . . 197
13.1.1 Read this before you trust any number . . . 197
13.1.2 What the display actually does . . . 198
13.1.3 The six tests . . . 199
13.1.4 Recording the machine (`-sysinfo`) . . . 200
13.1.5 The `Frame pacing` block . . . 201
13.1.6 `av` — the stimulus timing test . . . 202
13.1.7 `vrr`: arbitrary stimulus durations . . . 204
13.1.8 `rt` — reaction-time timestamp precision . . . 204
13.1.9 `timing-drift`: is the flip timestamp tracking the panel? . . . 205
13.1.10 Rules of thumb for your own numbers . . . 205
13.1.11 Loading the data in Python . . . 206
13.1.12 Improving timing on your system . . . 207
13.2 Part 2 — What we found . . . 209
13.2.1 The display stack is the dominant term . . . 209
13.2.2 What a returning `SDL_RenderPresent` actually means . . . 210
13.2.3 Flip timestamps track the panel . . . 211
13.2.4 Audio: the buffer size is the whole story . . . 212
13.2.5 How much the trigger device costs . . . 213
13.2.6 What we have not measured . . . 213
13.3 Related tests . . . 214
13.4 Contributing a report . . . 214

14 goxpyriment against the timing mega-study 215
14.1 Conditions . . . 215
14.2 The stack decides it, and it is not a single axis . . . 216
14.3 Against Table 2 . . . 217
14.4 Threshold sensitivity, and where the second instrument stops being usable 218

14.5 What this licenses saying . . . 219

15 frame-syncing **220**

16 Minimising trigger-to-stimulus jitter (EEG / MEG) **221**
16.1 The short version . . . 221
16.2 Where the time actually goes . . . 222
16.3 Why sleeping is the mistake . . . 222
16.4 Fire the trigger synchronously . . . 223
16.5 What is left, and why it is quantised . . . 223
16.6 Measure over a realistic run length, not a pilot . . . 224
16.7 The jitter is in the timestamp, not in the display . . . 224
16.7.1 The display stack is worth sixteen times more than anything else . . . 225
16.7.2 Removing the compositor is worth twelve times more than anything else . . . 226
16.7.3 Real-time priority halves it — and that is measured, not assumed . . . 227
16.8 Discard the first trials — they are genuinely different . . . 227
16.9 Two instruments agree . . . 228
16.10 How this compares with other packages . . . 228
16.11 The floor you cannot code around . . . 229
16.12 The honest assessment for EEG / MEG . . . 229
16.13 Verify it on your own hardware . . . 230
16.14 Appendix: the DLP-IO8-G . . . 231
16.14.1 Lower the FTDI latency timer before *reading* anything . . . 231
16.14.2 A multi-bit code is not atomic . . . 231
16.14.3 Pulse width is only as good as your scheduling . . . 232
16.14.4 Parallel port and GPIO alternatives . . . 232

17 Step 1: Create the Group **234**
17.0.1 Step 2: Create the Limits Configuration . . . 234
17.0.2 Step 3: Add Yourself (and others) to the Group . . . 235
17.0.3 Step 4: Apply the Changes . . . 235
17.0.4 Step 5: Actually Use It . . . 235
17.0.5 Step 6: The other groups the same machine usually needs . . . 236
17.0.6 How to Verify it Worked . . . 238
17.1 Running from a virtual console (VT) . . . 239
17.1.1 Getting there . . . 239
17.1.2 The console's mode is the mode you get . . . 239
17.1.3 Pinning the mode . . . 240
17.1.4 One display, or name the one you mean . . . 241
17.1.5 **!** A Friendly "Warning" . . . 241
17.1.6 **!** The throttle also wrecks the timing you asked for . . . 242
17.1.7 **!** The speed the CPU runs at is not fixed either . . . 243

18 Setting priority under Windows **245**
18.0.1 **!** None of this has been measured on Windows . . . 245
18.1 The short version . . . 245
18.2 Why a launcher prefix at all . . . 245

18.3 Step 1: launch through `start` . . . 246
18.3.1 ! In PowerShell, `start` is a different command . . . 246
18.4 Step 2: decide about `/realtime` — the honest answer is “probably not” . . . 247
18.5 Step 3: the thing that probably matters more — timer resolution . . . 248
18.6 Step 4: verify . . . 248
18.7 How to find out whether it helped, on your machine . . . 249
18.8 Other Windows-specific things worth knowing . . . 249
18.9 See also . . . 250

19 Setting priority under macOS 251
19.0.1 ! None of this has been measured on macOS . . . 251
19.1 The short version . . . 251
19.2 Why a launcher prefix at all . . . 251
19.3 Caveat 1: `nice` is not the lever that governs real-time behaviour on Darwin 252
19.4 Caveat 2: `sudo` leaves your data files owned by root . . . 253
19.5 Step 1: run it . . . 253
19.6 Step 2: verify . . . 254
19.7 How to find out whether it helped, on your machine . . . 254
19.8 Other macOS-specific things worth knowing . . . 255
19.9 See also . . . 255

20 Fullscreen + HIGH_PIXEL_DENSITY issue 256
20.1 Symptom . . . 256
20.2 Root cause analysis . . . 256
20.2.1 The asymmetry in `apparatus/NewScreen` . . . 256
20.2.2 What `SDL_WINDOW_HIGH_PIXEL_DENSITY` does . . . 256
20.2.3 Consequence . . . 257
20.3 Why the Pi issue was different . . . 257
20.4 The Mac issue — implemented fix . . . 258
20.4.1 Approach — `SetLogicalPresentation` (implemented) . . . 258

21 SDL3 Fullscreen in a Linux Virtual Console (TTY) 259
21.1 Problem . . . 259
21.2 Root Cause . . . 259
21.3 Diagnostics . . . 259
21.4 Fix . . . 260
21.4.1 Option 1 — environment variable (quickest) . . . 260
21.4.2 Option 2 — programmatic hint in screen.go . . . 260
21.5 Notes . . . 260

22 Media — Multi-Movie Playback with Hardware-Verified Display Onsets 261
22.1 Table of Contents . . . 261
22.2 1. Scope and design . . . 261
22.2.1 What `media/` is . . . 261
22.2.2 Hard scope choices . . . 262
22.2.3 What problems it solves . . . 262
22.3 2. Quick start . . . 263
22.4 3. Architecture in one diagram . . . 264

22.54. API reference (package `media`) . . . 265
22.5.14.1 `MovieManager` . . . 265
22.5.24.2 `Movie` . . . 267
22.5.34.3 `MasterClock` . . . 269
22.5.44.4 Onset events . . . 269
22.5.54.5 Time-spec helpers . . . 270
22.65. API reference (package `media/present`) . . . 270
22.76. Multi-movie synchronisation: what is guaranteed . . . 271
22.7.16.1 Mechanism . . . 271
22.7.26.2 What that means in numbers . . . 271
22.7.36.3 The atomic-burst pattern . . . 272
22.7.46.4 What is *not* synchronised . . . 272
22.87. External-trigger synchronisation with frames on screen . . . 272
22.8.17.1 The two timestamps that matter . . . 272
22.8.27.2 The wire-arrival delay . . . 273
22.8.37.3 LookAhead vs HardwareVerified vs VsyncEstimated . . . 273
22.8.47.4 The deferral pattern (macOS specifically) . . . 274
22.8.57.5 Calibration . . . 275
22.8.67.6 What is *not* measured . . . 275
22.8.77.7 Recommended pattern . . . 276
22.98. Platform support (Stage 5) . . . 276
22.9.18.1 What Windows users get today . . . 277
22.9.28.2 What needs to be done for Windows . . . 277
22.109. Mapping to PsyScope movie commands . . . 278
22.1110. References . . . 279

23 Running goxpyriment experiments in a web browser (WASM) 281
23.1 Architecture . . . 281
23.1.1 Where the pieces live . . . 281
23.1.2 The go-sdl3 replace — what dependents need to know . . . 282
23.2 Building and serving an example . . . 283
23.2.1 Per-example launcher pages . . . 283
23.2.2 Session settings via URL parameters . . . 284
23.2.3 Headless verification (no display needed) . . . 284
23.3 Publishing to downloads.pallier.org . . . 285
23.4 Running on JATOS . . . 285
23.5 What works today (verified 2026-07-13) . . . 286
23.5.1 Why the results arrive as a .zip . . . 287
23.5.2 The key design points . . . 288
23.6 What remains for a complete port . . . 288
23.7 Audio in the browser . . . 289
23.8 Timing in the browser . . . 289
23.8.1 Clock / timestamp resolution (measured 2026-07-13, headless Chrome) . . . 289
23.8.2 Frame pacing (measured 2026-07-13, headless Chrome) . . . 290
23.9 Notes for developers . . . 290
23.10 Appendix: historical manual Emscripten recipe . . . 291

24 Deploying an experiment on a JATOS server **292**
24.1 What you need . . . 292
24.1.1 JATOS vocabulary . . . 292
24.2 1. Make sure the experiment runs in a browser . . . 293
24.2.1 If your experiment is in its own folder . . . 293
24.3 2. Build the study archive . . . 294
24.4 3. Import it into JATOS . . . 294
24.5 4. Try it yourself first . . . 295
24.6 5. Create study links . . . 295
24.6.1 Passing parameters . . . 295
24.7 6. What participants see . . . 296
24.8 7. Getting the results . . . 296
24.8.1 When something goes wrong . . . 297
24.9 Limits and caveats . . . 297
24.10 Troubleshooting . . . 298

25 Packaging Your GoXpyriment Experiment **299**
25.1 Prerequisites . . . 299
25.2 Project Structure Recommendation . . . 299
25.3 Step 1: Handling Assets . . . 300
25.3.1 A. Embedding (Small Assets) . . . 300
25.3.2 B. Bundling (Very Large Assets) . . . 300
25.4 Step 2: GoReleaser Configuration . . . 300
25.5 Step 3: Building . . . 301
25.5.1 Local Snapshot Build . . . 301
25.5.2 Official Release . . . 301
25.6 Platform-Specific Notes . . . 301
25.6.1 Windows . . . 301
25.6.2 Linux . . . 301
25.6.3 macOS . . . 302
25.7 Using GitHub Actions . . . 302

26 Publishing an example as a standalone repository **303**
26.1 When to use it . . . 304
26.2 Prerequisites . . . 304
26.3 Step 1 — Generate the standalone module . . . 304
26.4 Step 2 — Prune goxpyriment-internal files . . . 305
26.5 Step 3 — Add the repo scaffolding . . . 305
26.6 Step 4 — Add the CI workflows . . . 306
26.6.1 `.github/workflows/release.yml` (native binaries) . . . 306
26.6.2 `.github/workflows/pages.yml` (browser build — optional) . . . 306
26.7 Building the browser bundle locally . . . 306
26.8 Step 5 — Create the GitHub repo and enable Pages . . . 307
26.9 Step 6 — Cut a release . . . 307
26.10 Updating the pinned goxpyriment later . . . 307
26.11 Verifying a browser build . . . 307
26.12 See also . . . 308

27 How `control/experiment.go` works **309**
27.1 1. The `Experiment` struct (the facade) . . . 309
27.2 2. Construction & initialization . . . 309
27.3 3. The run loop . . . 310
27.4 4. Event handling . . . 310
27.5 5. Convenience methods (the ergonomic layer) . . . 310
27.6 6. Teardown . . . 310

28 Library usage: reducing direct SDL use **312**
28.1 Run-loop exit and error handling . . . 312
28.2 Key and button constants . . . 312
28.3 Points and colors . . . 313
28.4 Screen . . . 313
28.5 Timing . . . 313
28.6 When you still need SDL . . . 313

29 Viewing the documentation locally **314**
29.1 API reference (pkgsite) . . . 314
29.2 This documentation site (zensical) . . . 314

Chapter 1

goxpyriment

- Homepage
- GitHub repository

  Goxpyriment is a software framework for programming behavioral and cognitive experiments using the Go programming language. It was designed to address some limitations of existing Python-based experiment tools, particularly the runtime environment complexity that frequently complicates deployment across laboratories. Because Go is a compiled language that can natively embed assets (e.g., graphics, audio files, and stimulus lists), Goxpyriment compiles entire experiments into single, self-contained executable binaries with zero runtime dependencies. This drastically simplifies distribution to collaborators and testing computers. The library includes an array of visual stimuli (text, shapes, images, Gabor patches, motion clouds, …) and audio capabilities (WAV playback and tone generation). A lot of effort was devoted to ensure timing reliability. For instance, input events are timestamped by the operating system at hardware-interrupt time, so reaction times are computed by subtracting two OS-level timestamps rather than relying on continuous polling. The framework also supports sending and receiving TTL signals to synchronize with various external equipment. Go's garbage collector can be disabled, greatly reducing the probability of unpredictable pauses that could corrupt stimulus timing. Finally, a set of over forty psychology experiments implemented in Goxpyriment are provided that promote not only learning by humans but also improve the ability of modern AI-assisted coding tools to help program experiments. The framework is released under the Apache License 2.0.

Remark: If you are looking for a simple experiment generator - with fixed stimulus presentation schedule, check out Gostim2 which does not require any coding.

1.1 Documentation

The top navigation bar provides access to various documentation guides.

Everything in one file: goxpyriment-docs.pdf — every page below as a single searchable PDF, one chapter each, with a full table of contents and bookmarks. Use it when you want to search the whole documentation at once rather than a page at a time. The individual PDFs linked beside each guide are the same content, split.

Start here — install and build your first experiment

1. Read **Install** and build the bundled examples. [PDF]
2. **Creating Your Own Experiment** — a step-by-step beginner guide from an empty folder to a shareable executable.
3. **Getting Started** — worked tutorials: trials, data logging, RSVP streams, reaction times. [PDF]
4. **Running your experiment online (JATOS)** — put an experiment on a JATOS server, send participants a link, and collect their data on the server.

User Manual — core concepts explained in depth: rendering model, timing, input, data, streams, audio, and experimental design. [PDF]

API Reference — complete function and type reference, organized by package. [PDF]

Misc — focused how-to guides: migrating from PsychoPy / Expyriment / Psychtoolbox, checking your computer's timing, video playback, packaging & sharing binaries, giving your experiment real-time priority on Linux, and more, all found under the **Misc** tab in the navigation bar at the top of the page.

In addition, if you clone goxpyriment, you can run `pkgsite` from the root folder to view the source code documentation.

1.2 Gallery of ready-to-run experiments

Browse the gallery of over fifty example experiments and demos (pre-built binaries (ready-to-run apps) are available for Windows, macOS, and Linux).

1.3 Support

- Google group — Forum
- Report bugs at https://github.com/chrplr/goxpyriment/issues

---

1.4 Background

The framework is described in a short paper: *Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments*.

Goxpyriment is written in Go and relies on the SDL3 library through the go-sdl3 bindings.

The original inspiration was expyriment.org, a lightweight Python library for cognitive and neuroscientific experiments (Krause & Lindemann, 2014. *Behavior Research Methods*, 46(2), 416–428. https://doi.org/10.3758/s13428-013-0390-6).

1.5 License & citation

Apache License 2.0 — see LICENSE and NOTICE.

Please cite as: > Christophe Pallier (2026) chrplr/goxpyriment. Zenodo. https://doi.org/10.5281/zenodo.19200598

Christophe Pallier, 2026.



Chapter 2

Goxpyriment installation instructions

There are four ways to use goxpyriment, depending on what you already have and what you want to do. Find your situation below and jump to the matching section — each choice builds on the previous one, so you only need to read as far as your goal requires.

Your situation	What you need	Go to
You want to **try the experiments from the gallery**	Nothing	Pre-built binaries
You were given a **ready-to-run app** (a compiled file) and just want to launch it	Nothing	No installation
You have the **source code** of an experiment (`.go` files) and want to run or build it	Go	Minimal installation
You want to **write your own experiments** or explore the framework	Go + Git + a code editor	Full installation

2.1 No installation is needed to run a goxpyriment app

If you were given a ready-to-run goxpyriment app (a compiled file, such as the ones provided in the compiled examples), you do not need to install anything to use it. Just launch the app by double-clicking its icon in your file folder, or by typing its name in your command line (Terminal or Command Prompt).

! WARNING One potential issue can arise from the antivirus or protection system of your computer if these binaries are unsigned: macOS Gatekeeper and Windows Defender will show security warnings or worse, *misleading messages* such as 'this

program is damaged'. Your antivirus may quarantine the files. Don't let them intimidate you:

- macOS: Right-click the app → **Open**, or run `xattr -dr com.apple.quarantine <AppName>.app` in Terminal. See macOS installation and security for step-by-step instructions.
- Windows: Just click on "More info" then "Run anyway". (Note: These warnings will only pop out the first time you try to execute a given program).

2.2 Minimal installation to execute a goxpyriment source code

Let us consider the case where you have the *source code* of a goxperiment program.

If Go is not already installed on your computer (you can check if Go installed by typing `go version` on a command line. If an error message is displayed, then Go is not installed), download and install it following the instructions at https://go.dev/doc/install

Then there are two possibilities. You have either:

1. A folder containing at least the files `go.mod`, `go.sum` and the source code, say `experiment.go` (it can be any file with the `.go` extension).

   Then, execute the command line:

   ```
   go run experiment.go    # replace experiment by the actual name
   ```

   Or build the app, then run it, by typing:

   ```
   go build experiment.go
   ./experiment
   ```

2. A single `.go` file, say `experiment.go`.

   Then, you need to copy this file into a folder, e.g. `experiment` and create `go.mod` and `go.sum` by typing

   ```
   go mod init experiment
   go mod tidy
   ```

   This will create the missing files and you are in the situation of the previous paragraph: you can run the program with the command `go run .` or `go build .`.

Remark: the very first time you run or build a goxpyriment program, it will take a while as Go needs to download a number of modules from the Internet. Afterwards, running or building an app should be near instantaneous.

2.3 Full installation

If you want to do serious development, in addition to Go, you need to install Git, a code editor and possibly an AI-coding agent like Claude Code, Gemini cli or Aider. If you are new to this, consult the detailed instructions

Then:

1. Clone goxpyriment Github repository, by opening a shell (e.g. launch `Git Bash` under Windows), and executing the command-line

   ```
   git clone https://github.com/chrplr/goxpyriment.git
   ```

   This will create a folder `goxpyriment` contain the entire source code, examples and documentation of the framework. In the future, you will be able run the `git pull` command within this folder to update to the latest version.

2. Navigate to the `goxpyriment` folder and type the command-line:

   ```
   ./build-all.sh
   ```

   (Note: On Linux/macOS you can instead use `make all`)

   This will compile the programs in examples/* and tests/* (Note: this may take a while because Go needs to download several modules. Once this is done, future compilations will be fast.)

   After this operation, the programs should be found in the `goxpyriment/_build` folder. You can either launch them by clicking on their icon in the file explorer (the "Finder" " under macOS), or by typing the ir name in a terminal, e.g.:

   "' cd _build ./

3. If you would like to read the documentation locally — both the API reference (via pkgsite) and this site (via zensical) — see Viewing the documentation locally.

2.3.1 One extra step on a Linux machine used for data collection

An experiment that has to present stimuli on time competes with other processes running on the machine. You can ask the operating system to give the preference to your goxpyriment programs by granting your user real-time scheduling. The full procedure is described in Setting priority under Linux. In a nutshell, you have create a group `goxpyriment`, configure its permissions in `/etc/security/limits.d/`, add yourself to the group, then **log out and log back in**.

Once that is in place you can verify the machine's timing end to end with Timing Tests, which also lists the other Linux tuning worth doing (disabling the compositor is the single largest improvement).

2.3.2 Program your own experiments

Follow the step-by-step guide in Creating Your Own Experiment, which walks you through writing, running, building, and sharing your own experiment.

For background and reference, see Getting Started, the examples' source codes, and the available functions.

2.3.3 Use an AI coding agent

If you launch `claude` (Claude Code) from the `goxpyriment` folder, the many `CLAUDE.md` files will help the AI coding agent to create experiments for you. You can describe the experimental paradigm in plain language (see the `description.md` files in `examples` subfolders) and ask claude code to program it using the goxpyriment framework.

Chapter 3

Installing a Go Development Environment

This guide sets up **Git**, **Go**, and a code editor, then has you write and run your first Go program. By the end you'll have a working Go environment and a program printing to your terminal. Once done, you can go back to the instructions for installing goxpyriment

What you'll do

1. Install the tools for your operating system — macOS · Windows · Linux (Ubuntu)
2. Write and run your first program
3. (Optional) Install Claude Code, an AI coding assistant

3.1 macOS

If Homebrew is installed, you can install Git and Go with two commands:

```
brew install git
brew install go
```

Otherwise, follow the steps below.

3.1.1 1. Install Apple Command Line Tools

Before installing anything else, macOS needs its basic developer foundation.

1. Open your **Terminal** (press `Cmd + Space` and type "Terminal").
2. Run:

   ```
   xcode-select --install
   ```

3. A popup will appear — click **Install** and agree to the terms.

3.1.2 2. Install Git

macOS ships with an outdated Git; install the official version.

1. **Download:** Go to git-scm.com/download/mac.
2. Use the **binary installer** package.
3. **Verify:**

```
git --version
```

3.1.3 3. Install Go

1. **Download:** Visit go.dev/dl and select the **macOS** installer (`.pkg`).
   - Apple Silicon (M1/M2/M3/…): choose **ARM64**.
   - Intel Mac: choose **x86-64**.
2. Open the package and follow the wizard.
3. **Verify** (open a **new** Terminal window):

```
go version
```

3.1.4 4. Editor: Visual Studio Code (VS Code)

If you do not already have a favourite editor (Vim or Emacs work great too), install VS Code.

1. **Download:** code.visualstudio.com.
2. Unzip the download and drag **Visual Studio Code** into your Applications folder.
3. **Add the `code` command to PATH:**
   - Open VS Code, press `Cmd + Shift + P`.
   - Type `shell command` and select **"Shell Command: Install 'code' command in PATH"**.
4. **Install the Go extension:**
   - Click the **Extensions** icon (four squares) in the left sidebar.
   - Search for **"Go"** and install the one by the Go Team at Google.

 Tools installed. Continue to Your First Program.

3.2 Windows

3.2.1 1. Install Git

1. **Download:** git-scm.com/download/win.
2. Run the `.exe` installer. On most screens you can click **Next**, but note:
   - **Editor:** select "nano" if it not the default (you can change it later).

- **PATH:** select “Git from the command line and also from 3rd-party software”.
- **Line endings:** choose “Checkout Windows-style, commit Unix-style line endings”.

3. **Verify** — open **Command Prompt** (`Win + R`, type `cmd`) and run:

```
git --version
```

3.2.2 2. Install Go

1. **Download:** go.dev/dl — choose the **Windows** installer (`.msi`).
2. Run the installer; it defaults to `C:\Program Files\Go`.
3. Close any open Command Prompts, then open a new one.
4. **Verify:**

```
go version
```

You should see something like `go version go1.26.x windows/amd64`.

3.2.3 3. Editor

Any text editor works. If you are new to programming, start with one of the simpler options below and switch later if you outgrow it.

Notepad++ (simplest) — tiny and instant, with Go syntax highlighting. Ideal if you just want to edit a file (no autocomplete or error checking).

1. **Download:** notepad-plus-plus.org.
2. Run the installer and accept the defaults.

Zed (lightweight, with Go support) — a fast, uncluttered editor that still gives you autocomplete and inline error checking.

1. **Download:** zed.dev.
2. Run the installer. The first time you open a `.go` file, Zed offers to install the Go tools — accept.

Visual Studio Code (most popular, more features) — powerful but can feel overwhelming at first.

1. **Download:** code.visualstudio.com.
2. Run the installer and make sure **“Add to PATH”** is checked.
3. **Set up Go support:**
   - Open VS Code and click the **Extensions** icon (four squares).
   - Search for **“Go”** (by the Go Team at Google) and click **Install**.
   - Press `Ctrl + Shift + P`, type `Go: Install/Update Tools`, select all boxes, and click **OK**.

 Tools installed. Continue to Your First Program.

3.3 Linux (Ubuntu)

3.3.1 1. Install Git

Git is available in Ubuntu's package repository:

```
sudo apt update
sudo apt install git
```

Verify:

```
git --version
```

3.3.2 2. Install Go

Install the latest version directly from the official site.

1. **Download** the Linux tarball from go.dev/dl:

   ```
   wget https://go.dev/dl/go1.26.4.linux-amd64.tar.gz
   ```

   Note: Replace the filename with the latest version shown on the downloads page (and the version for your computer arch (amd vs. arm)).

2. **Extract** to `/usr/local` (this is the standard location):

   ```
   sudo rm -rf /usr/local/go
   sudo tar -C /usr/local -xzf go1.26.4.linux-amd64.tar.gz
   ```

3. **Add Go to your PATH.** Append the following lines to `~/.bashrc` (or `~/.zshrc` if you use Zsh):

   ```
   export PATH=$PATH:/usr/local/go/bin
   ```

   Then reload your shell:

   ```
   source ~/.bashrc
   ```

4. **Verify:**

   ```
   go version
   ```

5. **(Optional) Install a few additional tools**

- If you want a debugger for step by step execution, you can install delve.
- To read the documentation locally — both the API reference (via pkgsite) and this site (via zensical) — see Viewing the documentation locally.

3.3.3 3. Editor: Visual Studio Code (VS Code)

Do this only if you do not already have a favorite code editor.

1. **Install** via Snap (simplest method):

```
sudo snap install --classic code
```

Alternatively, download the `.deb` package from code.visualstudio.com and install with `sudo apt install ./code_*.deb`.

2. **Install the Go extension:**
   - Click the **Extensions** icon (four squares) in the left sidebar.
   - Search for **"Go"** and install the one by the Go Team at Google.
   - Press `Ctrl + Shift + P`, type `Go: Install/Update Tools`, select all, and click **OK**.

 Tools installed. Continue to Your First Program.

3.4 Your First Program (Hello World)

Once Go is installed, let's check that everything works by writing and running a tiny program. If you have never used a command line before, don't worry — every step is spelled out below.

3.4.1 Step 0 — Open a terminal

Almost everything in Go is done from a **terminal** (also called a *command line*, *command prompt*, or *shell*). This is a window where you type commands and press **Enter** to run them, instead of clicking buttons. Open it like this:

- **Windows:** the easiest option is the **Git Bash** terminal that was installed together with Git. Click the **Start** menu, type `Git Bash`, and press **Enter**. (You can also use the built-in **Command Prompt**: press `Win + R`, type `cmd`, and press **Enter** — but the commands below assume Git Bash.)
- **macOS:** press `Cmd + Space`, type `Terminal`, and press **Enter**.
- **Linux (Ubuntu):** press `Ctrl + Alt + T`, or search for **Terminal** in the applications menu.

A new window opens with a blinking cursor, ready for you to type. From here on, each time you see a command in a grey box, **type it into this terminal and press Enter**.

> **Tip:** If you want to understand what is going on when you interact with your computer from a terminal, I recommend that you read https://linuxcommand.org/lc3_learning_the_shell.php

3.4.2 Step 1 — Create a project folder

A Go program lives in its own folder. The commands below create a folder called `hello`, move into it, and tell Go that this folder is a *module* (a self- contained project):

```
mkdir hello
cd hello
```

```
go mod init hello
```

- `mkdir hello` — **m**a**k**e a new **dir**ectory (folder) named `hello`.
- `cd hello` — **c**hange **d**irectory: step inside the folder you just made. Every command you type from now on runs *inside* this folder.
- `go mod init hello` — set up the folder as a Go project.

  Where did the folder go? It is created inside whatever folder the terminal is currently "in" — usually your home folder (e.g. `C:\Users\YourName` on Windows, or `/home/yourname` on Linux). That is fine for now.

3.4.3 Step 2 — Create the program file

Now create a text file named `main.go` inside the `hello` folder, using your editor (VS Code, Zed, Notepad++, ...). Open the editor, create a **new file**, paste the lines below, and save it as `main.go` **inside the `hello` folder**:

```go
package main

import "fmt"

func main() {
    fmt.Println("Hello! Your environment is ready.")
}
```

> **Make sure the file is named exactly `main.go`** (not `main.go.txt`). In Notepad++ or VS Code, choose *Save As* and type the full name `main.go`.

3.4.4 Step 3 — Run the program

Back in the terminal (still inside the `hello` folder), type:

```
go run .
```

The `.` means "run the program in the current folder". You should see this line printed in the terminal:

```
Hello! Your environment is ready.
```

 Congratulations — your Go environment is fully working, and you have just written and run your first program.

3.5 (Optional) Install an AI Coding Agent

An AI Coding agent runs **inside your terminal**. You describe what you want in plain English, and it reads your files, writes code, runs commands, and explains what it is doing. It is entirely optional, but many people find it a helpful companion when learning Go or working with this project.

There exists several AI coding agents. Here I describe Claude Code. Instructions for installation are available at https://code.claude.com/docs/en/quickstart

> **You will need an account.** Claude Code requires either a Claude subscription (Pro or Max) or an Anthropic API account with billing set up. The first time you run it, it walks you through signing in.

3.5.1 First run

> You may need to **close and reopen your terminal** after installing so that the `claude` command becomes available.

1. In your terminal, move into a project folder (for example the `hello` folder you created above, or your copy of this repository):

   ```
   cd hello
   ```

2. Start Claude Code:

   ```
   claude
   ```

3. The first time, it asks you to log in — follow the on-screen instructions to sign in with your Claude or Anthropic account.

4. Once you see the prompt, just type a request in plain English and press **Enter**, for example:

   ```
   Explain what the main.go file does, line by line.
   ```

To leave Claude Code, type `/exit` (or press `Ctrl + C` twice).

> **Verify the install** at any time with:
>
> ```
> claude --version
> ```

Chapter 4

Creating Your Own Experiment (a beginner's guide)

This guide is for people who have **never written Go before**. It walks you through making a brand-new experiment from scratch: creating a folder, writing a `main.go` file, running it, building a standalone program, embedding your own images/sounds/CSV files, and finally producing programs you can hand to colleagues on Windows, macOS, or Linux.

You do **not** need to learn the whole Go language. For most experiments you only copy an example, change a few lines, and run it.

> **Tip:** The easiest way to start is to copy an existing example from the `examples/` folder that resembles your paradigm, then change a few lines. That is far easier than writing from a blank page.
>
> * **Vibe-coding:** You can also let an AI coding agent (Claude, Gemini, etc.) write the experiment for you. Open the agent inside the `goxpyriment` folder and describe your experiment (stimuli, design, etc.) in plain language — asking it to add a new experiment to the `examples` folder leads it to read the existing examples for context. Recommendation: save your prompt in a `description.md` file.

4.1 Before you start

Make sure goxpyriment is installed first — follow Installation, which walks you through installing Git and Go, cloning the repository, and building the examples. (If you are new to setting up Go, that page links to a more detailed development-environment guide.)

Once that is done, open a terminal and confirm the Go compiler is available:

```
go version
```

If you see something like `go version go1.25 ...`, you are ready.

> **What is a terminal?** A text window where you type commands. On Windows, use **Git Bash** (installed with Git). On macOS use **Terminal**, on Linux any terminal app.

4.2 Step 1 — Create a folder for your experiment

Each experiment lives in its own folder. Pick a short name with no spaces:

```
mkdir my_experiment
cd my_experiment
```

Everything you do from now on happens inside this folder.

4.3 Step 2 — Write `main.go`

Create a file called `main.go` in that folder with the following content. This is the smallest complete experiment: it shows a message and waits for a key press.

```
package main

import (
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    // Title, background color, foreground color, default font size.
    exp := control.NewExperimentFromFlags("My Experiment", control.Black,
 ↪  control.White, 32)
    defer exp.End()

    hello := stimuli.NewTextBox("Hello, World!\n\nPress any key to quit.",
 ↪  600, control.FPoint{}, control.White)

    exp.Run(func() error {
        exp.Show(hello)
        exp.Keyboard.Wait()
        return control.EndLoop // leaving the loop ends the experiment
    })
}
```

You don’t need to understand every line yet. The pattern is always the same:

- `control.NewExperimentFromFlags(...)` sets up the screen, audio, keyboard, and data file.
- `defer exp.End()` makes sure everything is closed cleanly when the program stops.
- `exp.Run(func() error { ... })` is your experiment body. Return `control.EndLoop` to finish.

Remember: you rarely start from a blank page like this. The copy-an-example and vibe-coding tips at the top of this page are the easiest way in — this section just shows what the generated code looks like.

4.4 Step 3 — Tell Go about its dependencies

Your code uses the `goxpyriment` library, so Go needs to know how to fetch it. Run these two commands **once**, inside your folder:

```
go mod init my_experiment   # creates go.mod, the project's identity card
go mod tidy                 # downloads goxpyriment and everything it needs
```

`go mod tidy` reads your `import` lines, downloads the missing packages, and records exact versions in `go.mod` / `go.sum`. Run it again any time you add a new `import` or when Go complains it can't find a package.

You should now have these files:

```
my_experiment/
├── go.mod
├── go.sum
└── main.go
```

4.5 Step 4 — Run it while you work

To compile *and* run in one step:

```
go run .
```

The `.` means "the program in this folder". A window opens, your experiment runs, and the window closes when it ends.

This is your everyday loop: **edit `main.go` → `go run .` → look → edit again.** You don't need to "build" anything during development.

Useful command-line flags that every experiment understands automatically:

```
go run . -w          # windowed mode (1024×768) instead of fullscreen —
 ↪  great for development
go run . -s 007      # set the subject/participant ID to 007
```

```
go run . -d 1        # use monitor #1 (for a second screen)
```

When the experiment ends, two files are written to `~/goxpy_data/`: a `.csv` with your data and a `-info.txt` with session metadata.

4.6 Step 5 — Build a standalone program

When you are happy with the experiment, turn it into a single executable file that runs **without Go installed**:

```
go build .
```

This produces a file named after your folder (`my_experiment` on macOS/Linux, `my_experiment.exe` on Windows). It is fully self-contained — SDL3 and all assets are baked in — so you can copy that one file to another machine and run it:

```
./my_experiment          # macOS / Linux
my_experiment.exe        # Windows
```

You can rename the output or put it somewhere specific:

```
go build -o builds/my_experiment .
```

4.7 Step 6 — A more advanced example

Here is a script implementing a parity decision task (inspired from https://docs.expyriment.org/0.6.3/Examples.html)

```
package main

import (
    "slices"
    "strconv"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("Parity Decision",
        control.Black, control.White, 32)
    defer exp.End()

    Instructions := "When you'll see a number, your task to decide, " +
```

```
        "as quickly as possible, whether it is even or odd.\n\n" +
        "if it is even, press 'F'\n\nif it is odd, press 'J'"

    Targets := []int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
    NTrialsPerTarget := 2
    trials := slices.Repeat(Targets, NTrialsPerTarget)
    design.ShuffleList(trials)

    EvenResponse := control.K_F
    OddResponse := control.K_J

    cue := stimuli.NewFixCross(25, 2, control.DefaultTextColor)

    // creates a map number -> image
    Image := make(map[int]*stimuli.TextLine)
    for _, num := range Targets {
        Image[num] = stimuli.NewTextLine(strconv.Itoa(num),
            0, 0, control.DefaultTextColor)
    }

    exp.AddDataVariableNames([]string{"number", "key", "rt",
        "correct"})

    exp.Run(func() error {
        exp.ShowInstructions(Instructions)

        for _, t := range trials {
            exp.Blank(1000)
            exp.ShowTimed(cue, 500)
            key, rt, _ := exp.ShowAndGetRT(Image[t],
                []control.Keycode{EvenResponse, OddResponse},
                -1)
            correct := (t%2 == 0) == (key == EvenResponse)
            exp.Data.Add(t, key, rt, correct)
            if !correct {
                exp.Audio.PlayBuzzer()
            }
            exp.Wait(500)
        }

        return control.EndLoop
    })

    exp.ShowEndMessage("Experiment complete. Thank you!\n\n" +
        "Press any key to exit.")
}
```

Save this script in a folder ‘parity-decision’, open a Terminal, cd to this folder and type:

```
go mod init parity-decision
go mod tidy
go run .
```

This script may look a bit intinmidating. This is because especially the lines:

```
// creates a map number -> image
Image := make(map[int]*stimuli.TextLine)
for _, num := range Targets {
    Image[num] = stimuli.NewTextLine(strconv.Itoa(num),
        0, 0, control.DefaultTextColor)
}
```

Their purpose is to prepare in advanced the images (`stimuli.TextLine`) that will displayed on the screen during the main loop: One image is created for each number and saved in a map associating a number to an image (Go maps are similar to Python's dict).

Note: it is a generally a good habit to prepare the stimuli in advance, but we could have written the script without creating the map, and calling NewTextLine in the main loop, like this:

```
package main

import (
    "slices"
    "strconv"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("Parity Decision",
        control.Black, control.White, 32)
    defer exp.End()

    Instructions := "When you'll see a number, your task to decide, " +
        "as quickly as possible, whether it is even or odd.\n\n" +
        "if it is even, press 'F'\n\nif it is odd, press 'J'"

    Targets := []int{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
    NTrialsPerTarget := 2
    trials := slices.Repeat(Targets, NTrialsPerTarget)
    design.ShuffleList(trials)

    EvenResponse := control.K_F
    OddResponse := control.K_J

    cue := stimuli.NewFixCross(25, 2, control.DefaultTextColor)
```

```
    exp.AddDataVariableNames([]string{"number", "key", "rt",
        "correct"})

    exp.Run(func() error {
        exp.ShowInstructions(Instructions)

        for _, t := range trials {
            exp.Blank(1000)
            exp.ShowTimed(cue, 500)
            key, rt, _ :=
             ↪  exp.ShowAndGetRT(stimuli.NewTextLine(strconv.Itoa(t), 0, 0,
             ↪  control.White),
                []control.Keycode{EvenResponse, OddResponse},
                -1)
            correct := (t%2 == 0) == (key == EvenResponse)
            exp.Data.Add(t, key, rt, correct)
            if !correct {
                exp.Audio.PlayBuzzer()
            }
            exp.Wait(500)
        }

        return control.EndLoop
    })

    exp.ShowEndMessage("Experiment complete. Thank you!\n\n" +
        "Press any key to exit.")
}
```

The function `stimuli.NewTextLine()` converts a string into displayable image. The `stimuli` package provides many other functions, such as `NewPicture()` which loadds an imapge, `NewSound()` which loads a sound file, etc. All stimuli can be presented with the method `Present()` (in the above example, the function `ShowAndGetRT()` called `Present()` behind the scene)

If you have installed pkgsite and launched it from the `goxperiment` folder, you can check the list of available functions in `stimuli` by pointing your browser to http://localhost:8080/github.com/chrplr/goxpyriment@v0.0.0/stimuli

---

4.8 Step 7 — Embed your stimuli (images, sounds, CSV)

A built program is just one file. If your experiment loads images, sounds, or a list of trials from a CSV, those files would normally be missing on someone else's computer (unless the user copied them at the expected location). The solution is to **embed** them inside the program with Go's `//go:embed` directive. Embedded files travel inside the executable — nothing to copy alongside it.

Put your asset files in the folder (or a subfolder), then declare them at the top of `main.go`.

4.8.1 A single CSV file of trials

```
import (
    "bytes"
    "encoding/csv"
    _ "embed" // needed for //go:embed
)

//go:embed trials.csv
var trialsCSV []byte

func loadTrials() [][]string {
    r := csv.NewReader(bytes.NewReader(trialsCSV))
    rows, _ := r.ReadAll()
    return rows
}
```

4.8.2 A single image or sound

```
//go:embed apple.png
var appleImg []byte

//go:embed beep.wav
var beepSnd []byte

// ... later, inside your experiment:
picture := stimuli.NewPictureFromMemory(appleImg, 0, 0) // x=0, y=0 →
 ↪  centered
sound := stimuli.NewSoundFromMemory(beepSnd)
```

4.8.3 A whole folder of files

When you have many files, embed the folder as a virtual filesystem:

```
import "embed"

//go:embed images
var imagesFS embed.FS

// ... read one file by name:
data, err := imagesFS.ReadFile("images/apple.png")
if err != nil {
    exp.Fatal("could not load image: %v", err)
}
```

```
pic := stimuli.NewPictureFromMemory(data, 0, 0)
```

Rules to remember:

- The `//go:embed` line must sit **directly above** the variable, with no blank line.
- Paths are **relative to the `.go` file** and use forward slashes `/` on every OS.
- Add `import "embed"` (for `embed.FS`) or the blank import `_ "embed"` (for a `[]byte` / `string`).
- After adding embeds, run `go run .` again to recompile.

  When *not* to embed: very large files such as long videos. Embedding a 500 MB video makes a 500 MB program and slows builds. For those, ship the file next to the program instead — see Packaging Your Experiment.

For the full reference on embedding — folders as virtual filesystems, trial-list CSVs, binary size, and cross-compilation of embedded assets — see User Manual §13, *Embedding Stimuli and Trial Lists*.

4.9 Step 8 — Build for other operating systems (to share with colleagues)

Here is where Go shines. Because `goxpyriment` is pure Go (no C compiler needed), you can build a Windows program **from a Mac**, a Mac program **from Linux**, and so on — just by setting two variables: `GOOS` (operating system) and `GOARCH` (processor type).

```
# Windows (Intel/AMD 64-bit) → produces my_experiment.exe
GOOS=windows GOARCH=amd64 go build -o my_experiment-windows.exe .

# macOS (Apple Silicon: M1/M2/M3/M4)
GOOS=darwin  GOARCH=arm64 go build -o my_experiment-macos-arm .

# macOS (older Intel Macs)
GOOS=darwin  GOARCH=amd64 go build -o my_experiment-macos-intel .

# Linux (Intel/AMD 64-bit)
GOOS=linux   GOARCH=amd64 go build -o my_experiment-linux .
```

Common values:

Target machine	`GOOS`	`GOARCH`
Windows PC	`windows`	`amd64`
Mac with Apple Silicon	`darwin`	`arm64`
Older Intel Mac	`darwin`	`amd64`
Linux PC	`linux`	`amd64`
Raspberry Pi (64-bit)	`linux`	`arm64`

A small script that builds all of them at once (save as `cross-build.sh`):

```
#!/bin/bash
set -e
mkdir -p dist
GOOS=windows GOARCH=amd64 go build -o dist/my_experiment-windows.exe .
GOOS=darwin  GOARCH=arm64 go build -o dist/my_experiment-macos-arm .
GOOS=darwin  GOARCH=amd64 go build -o dist/my_experiment-macos-intel .
GOOS=linux   GOARCH=amd64 go build -o dist/my_experiment-linux .
echo "Done — see the dist/ folder."
```

Then:

```
bash cross-build.sh
```

Each file in `dist/` is a complete, standalone program. Email it, drop it on a USB stick, or share it — your colleague just double-clicks (or runs it from a terminal). No Go, no Python, no SDL install required.

> **On Windows**, a program built with plain `go build` is a *console* executable: double-clicking it opens a black terminal window next to the fullscreen experiment. For a pure GUI program that opens no console, add the `windowsgui` linker flag:
>
> ```
> GOOS=windows GOARCH=amd64 go build -ldflags="-H windowsgui" -o my_
> ↪  experiment-windows.exe .
> ```
>
> The trade-off: such a program has no terminal to print to, so `log` messages and any startup error become invisible — a failure shows up as a program that exits at once with nothing on screen. Keep a console build around for debugging, and ship the `windowsgui` one.
>
> **On macOS**, a binary downloaded from the internet may be blocked by Gatekeeper. The recipient can allow it once via **System Settings → Privacy & Security**, or run `xattr -d com.apple.quarantine my_experiment-macos-arm` in a terminal.

For a more polished distribution — `.deb`/`.rpm` packages, Windows installers, macOS `.app` bundles, and automated GitHub releases — see Packaging Your Experiment.

4.10 Quick reference

Task	Command
Create the project	`go mod init my_experiment && go mod tidy`
Run while developing	`go run .`
Run in a window	`go run . -w`
Add a new imported package	`go mod tidy`

Task	Command
Build a standalone program	`go build .`
Build for Windows from another OS	`GOOS=windows GOARCH=amd64 go build -o app.exe .`
Embed a file	`//go:embed file.png` above a variable

4.11 Where to go next

- Getting Started — worked tutorials (trials, data logging, RSVP, reaction times).
- Gallery of Examples — dozens of complete experiments to copy from.
- User Manual — the concepts behind the framework.
- API Reference — every function and type.
- Packaging Your Experiment — installers and automated releases.
- Deploying on a JATOS server — run your experiment in participants' browsers and collect the data on a server.

Happy experimenting!

Chapter 5

Getting Started with goxpyriment

Welcome! If you are a psychologist or neuroscientist used to building experiments in Python (with **Expyriment** or **PsychoPy**), you are in the right place.

`goxpyriment` is a alternative framework to build experiments using the **Go** programming language, offering significant advantages in timing precision and ease of sharing your work.

This document provides a few examples for you to get acquainted with the framework (if you want to run the examples, you will need goxpyriment installed and a working Go toolchain. See Installation for instructions).

5.1 Tutorial 1: Your First Trial

A classic sequence: **Fixation Cross** (500 ms) → **Target Circle** → **Wait for Spacebar**.

```
package main

import (
    "log"

    . "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := NewExperimentFromFlags("SimpleTrial", Black, White, 32)
    defer exp.End()

    fixation := stimuli.NewFixCross(20, 3, control.White)
    target   := stimuli.NewCircle(50, control.White)

    exp.ShowInstructions("Press SPACE when you see the circle.")
```

```
    exp.ShowTimed(fixation, 500)  // hold fixation for 500 ms

    exp.Show(target)
    exp.Keyboard.WaitKey(K_SPACE)
}
```

The following example goesa bit furhter: it runs a loops where a white square and a short tone are presented every second.

```
package main

import (
    . "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := NewExperimentFromFlags("Test", Black, White, 32)
    defer exp.End()

    w, h, _ := exp.Screen.Size()
    rect := stimuli.NewRectangle(0, 0, float32(w)/2.0, float32(h)/2.0,
 ↪  White)

    sound := stimuli.NewTone(440, 120, 0.5)
    sound.PreloadDevice(exp.AudioDevice)    // generates the sound

    instr := stimuli.NewTextBox(
        "A white rectangle will be displayed periodicallay\n\nPress any key
         ↪  to start\n\n'Esc' to interrupt",
        600, FPoint{}, White)

    exp.AddDataVariableNames([]string{"onset", "duration"})

    exp.Run(func() error {
        exp.Show(instr)
        exp.Keyboard.Wait()

        for {
            onsetNS, _ := exp.ShowTS(rect)
            sound.Play()
            exp.Wait(200)
            exp.Screen.Clear()
            offsetNS, _ := exp.Screen.FlipTS()
            duration_ms := (offsetNS - onsetNS) / 1_000_000
            exp.Wait(800)
```

```
            exp.Data.Add(onsetNS/1_000_000, duration_ms)
        }
    })

}
```

5.2 Tutorial 2: Blocks, Trials, and Data Logging

For real research you need multiple trials and a CSV result file.

```
package main

import (
    "fmt"
    "log"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("ParityTask", control.Black,
↪   control.White, 32)
    defer exp.End()

    exp.AddDataVariableNames([]string{"number", "is_even", "rt_ms",
↪   "correct"})

    err := exp.Run(func() error {
        exp.ShowInstructions(
            "Is the number EVEN or ODD?\n\nPress F for Even, J for Odd.",
        )

        numbers := []int{1, 2, 3, 4, 5, 6, 7, 8}
        trials  := design.MakeMultipliedShuffledList(numbers, 4) // 32
↪   trials, randomized

        for _, n := range trials {
            exp.Blank(1000)

            stim := stimuli.NewTextLine(fmt.Sprint(n), 0, 0, control.White)
            exp.Show(stim)

            // WaitKeysRT returns the key pressed, the reaction time in ms,
             ↪  and any error.
```

```
            key, rt, _ := exp.Keyboard.WaitKeysRT(
                []control.Keycode{control.K_F, control.K_J}, -1,
            )

            isEven := (n % 2 == 0)
            correct := (isEven && key == control.K_F) || (!isEven && key ==
↪   control.K_J)

            // Subject ID and timestamp are prepended automatically.
            exp.Data.Add(n, isEven, rt, correct)
        }
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```

`exp.Run()` handles the detection of the ESC keypress to interrupt the experiment. Two files are written to `~/goxpy_data/` automatically when the experiment ends:

- `<expName>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>.csv` — pure CSV data rows, directly importable by Excel, R, or pandas.
- `<expName>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>-info.txt` — session metadata (start/end time, hostname, OS, framework version, display and audio configuration, participant info).

5.3 Tutorial 3: Rapid Serial Visual Presentation (RSVP Streams)

Many paradigms — RSVP, Attentional Blink, priming — need stimuli flashed at high speed with precise timing. `goxpyriment` has dedicated stream functions that:

- Disable GC during the stream (no garbage-collection pauses).
- Lock every onset and offset to a VSYNC boundary.
- Record a `TimingLog` (predicted vs. actual onset) for every stimulus.
- Capture any key or mouse event that occurs during the stream.

5.3.1 Text stream

The simplest entry point is `stimuli.PresentStreamOfText`:

```
package main

import (
    "fmt"
```

```
    "log"
    "time"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("RSVP Demo", control.Black,
↪   control.White, 36)
    defer exp.End()

    exp.AddDataVariableNames([]string{"target_pos", "detected", "rt_ms"})

    // A stream of words; "TIGER" is the target at position 3 (0-indexed).
    words     := []string{"CHAIR", "RIVER", "LAMP", "TIGER", "CLOCK",
↪   "STONE", "BREAD", "HOUSE"}
    targetPos := 3

    err := exp.Run(func() error {
        exp.ShowInstructions(
            "Words will flash rapidly on screen.\n\n" +
            "Press SPACE as quickly as possible when you see TIGER.\n\n" +
            "Press any key to start.",
        )

        // 200 ms on, 50 ms off per word → ~4 words per second
        on  := 200 * time.Millisecond
        off :=  50 * time.Millisecond

        events, logs, err := stimuli.PresentStreamOfText(
            exp.Screen, words, on, off,
            0, 0,          // centred on screen
            control.White,
        )
        if err != nil {
            return err
        }

        // Did the participant press SPACE during the stream?
        ev, detected := stimuli.FirstKeyPress(events, control.K_SPACE)
        rtMs := ev.Timestamp.Milliseconds()

       // Optional: inspect timing quality. Read the authoritative SDL-clock
        // fields (OnsetNS/OffsetNS), not the Go-clock
         ↪  ActualOnset/ActualOffset
        // diagnostics. Achieved on-screen duration vs the target is the
         ↪  jitter.
```

```
        for i, l := range logs {
            shownMs := int64(l.OffsetNS-l.OnsetNS) / 1_000_000
            jitterMs := shownMs - l.TargetOn.Milliseconds()
            fmt.Printf("word %d: target %d ms  shown %d ms  jitter %d ms\n",
                i, l.TargetOn.Milliseconds(), shownMs, jitterMs)
        }

        exp.Data.Add(targetPos, detected, rtMs)
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```

Similar functions exist to present streams of images or sounds:

- Images:

```
stims := []stimuli.VisualStimulus{pic1, pic2, fixation, pic3}
on  := 100 * time.Millisecond
off :=   0 * time.Millisecond

elements := stimuli.MakeRegularVisualStream(stims, on, off)
events, logs, err := stimuli.PresentStreamOfImages(exp.Screen,
 ↪  elements, 0, 0)
```

- Sounds

```
tones     := []stimuli.AudioPlayable{tone440, tone880, tone440}
onsets    := []int{0, 500, 1000}   // ms from stream start
durations := []int{200, 200, 200}

elements, _ := stimuli.MakeSoundStream(tones, onsets, durations)
events, logs, err := stimuli.PlayStreamOfSounds(elements)
```

All stream functions return `([]UserEvent, []TimingLog, error)` to analyse timing quality and log participant responses.

5.4 Tutorial 4: Hardware-Precision RT with Event Timestamps

`WaitKeysRT` measures reaction time from the moment the function is called. That works well for a single-stimulus trial, but breaks down when several stimuli appear in sequence and you need RT from a specific onset.

Consider a **masked priming** paradigm: a prime word appears briefly, then a tar-

get appears 500 ms later, and you want RT measured from the prime onset — not from when `GetKeyEventTS` was called. With the standard approach you would need to record a timestamp before the prime and do arithmetic afterward. The event-timestamp API handles this directly.

SDL3 stamps every keyboard event with a hardware-interrupt time (`KeyboardEvent.Timestamp`, nanoseconds). `exp.ShowTS(stim)` returns the SDL nanosecond time captured immediately after the VSYNC flip. Because both values are on the same clock, their difference is hardware-precision RT — no arithmetic needed:

```
package main

import (
    "fmt"
    "log"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("PrimingRT", control.Black,
↪   control.White, 36)
    defer exp.End()

    exp.AddDataVariableNames([]string{"prime", "target", "key", "rt_prime_
↪   ns", "rt_target_ns"})

    primes  := []string{"DOCTOR", "NURSE", "BREAD", "TABLE"}
    targets := []string{"NURSE", "DOCTOR", "TABLE", "BREAD"}  // related /
↪   unrelated pairs
    responseKeys := []control.Keycode{control.K_F, control.K_J}

    err := exp.Run(func() error {
        exp.ShowInstructions(
            "A word will flash, then a second word will appear.\n\n" +
            "F = Living thing   J = Non-living thing\n\n" +
            "Respond to the SECOND word as quickly as possible.\n\n" +
            "Press SPACE to start.",
        )

        for i := range primes {
            exp.Blank(500)   // inter-trial interval

            // 1. Show prime — record its VSYNC flip timestamp
            prime := stimuli.NewTextLine(primes[i], 0, 0, control.Gray)
            primeOnset, _ := exp.ShowTS(prime)
            prime.Unload()
```

```
            exp.Wait(500)  // prime–target SOA

            // 2. Show target — record its VSYNC flip timestamp
            target := stimuli.NewTextLine(targets[i], 0, 0, control.White)
            targetOnset, _ := exp.ShowTS(target)
            target.Unload()

            // 3. Wait for response — get hardware event timestamp
           key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, 3000)

            // 4. Compute RTs from each stimulus onset
            rtFromPrime  := int64(eventTS - primeOnset)   // nanoseconds
            rtFromTarget := int64(eventTS - targetOnset)  // nanoseconds

            exp.Data.Add(primes[i], targets[i], key, rtFromPrime,
↪   rtFromTarget)
            fmt.Printf("prime RT: %.1f ms   target RT: %.1f ms\n",
                float64(rtFromPrime)/1e6, float64(rtFromTarget)/1e6)
        }
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```

Key observations:

- `exp.ShowTS(stim)` does the same thing as `exp.Show(stim)` — clear → draw → flip — and additionally returns the nanosecond timestamp of the flip.
- `GetKeyEventTS` returns the SDL3 event timestamp (not a polling delta), so subtracting any previously recorded `ShowTS` onset gives a physically meaningful RT.
- Both timestamps are in SDL nanoseconds (divide by `1e6` for milliseconds). Storing raw nanoseconds in the data file and converting offline is the recommended practice.
- `GetPressEventTS` provides the same capability for mouse responses.

5.5 Tutorial 5 — Test AI Coding

1. Take the PDF of a scientific paper about a psychology experiment and ask Gemini or Claude to read it and generate a detailed description (stimuli, design, timings) to give to n ai coder to replicate it. Save the answer in a `description.md` file.
2. Open Gemini or Claude (command-line tool), and ask it to implement the experiment detailed in `description.md`, using the goxpyriment library (provide the path of your local clone of the repository) and taking inspiration from the examples provided in the `examples/` folder.

5.6 Next Steps

- Browse the `examples/` folder for complete, documented paradigms (Stroop, Simon, Posner, Sperling, QUEST threshold estimation, and more).
- Read the User Manual for a deeper explanation of every concept, or the API Reference for complete function signatures.
- Report bugs and suggestions at https://github.com/chrplr/goxpyriment/issues.
- Happy experimenting!

Chapter 6

Goxpyriment Example Experiments

6.1 Psychological Experiments

Attentional-BlinkRSVP stream; participant detects two targets embedded in a stream of distractors — the second target is often missed within ~500 ms of the first*Raymond et al. (1992)*▶ Run in browser	**Change-Blindness**Flicker paradigm: alternating original and modified scenes separated by blanks; participant detects what changed*Rensink et al. (1997)*▶ Run in browser	**choice-reaction-times**Press F or J according to the colour (red/green) of a central square; the colour-key mapping alternates across 8 blocks of 24 trials (set with -blocks and -trials), with SOA distributions matched across colours*Donders (1868/1969); Hick (1952)*▶ Run in browser
Classification-Posner-Mitchell Classify letter pairs at three levels (physical, name, rule identity); RT increases with depth of processing required*Posner & Mitchell (1967)*▶ Run in browser	**Contrast-Detection-QUEST**2-IFC adaptive staircase estimating the contrast detection threshold for a Gabor patch; converges on the 82 % correct point*Watson & Pelli (1983)*▶ Run in browser	**Finger-Tapping**Patterned finger-tapping: memorise a key sequence then reproduce it 6 times consecutively as fast as possible; only error-free runs recorded*Povel & Collard (1982)*▶ Run in browser

Finger-Tracking Number-to-position task: drag the finger (or mouse, button held) from a bottom box up to an unmarked 0-40 number line, aiming at a target number; the full trajectory, endpoint, bias, and movement time are recorded*Dotan & Dehaene (2013)*▶ Run in browser

Geometry-consistency Two concurrent streams of geometric figures test 7-month-olds' detection of shape changes (relative length, angle) over variation in size, position, orientation and sense; experimenter codes looking direction in real time*Dillon, Izard & Spelke (2020)*▶ Run in browser

Geometry-LoT1 Geometric intruder task — click the odd one out among six quadrilaterals (five copies of one of 11 shapes from square to irregular, plus one with a displaced vertex); error rates track geometric regularity*Sablé-Meyer, Fagot, Caparos, van Kerkoerle, Amalric & Dehaene (2021), Experiment 2*▶ Run in browser

Geometry-Triangle-Completion Two-task study of intuitive Euclidean geometry — verbal judgements about the missing corner of a fragmented triangle, then mouse localization of that corner; yields the scaling exponent of visual extrapolation*Hart, Mahadevan & Dillon (2022)*▶ Run in browser

Go-NoGo Stop-signal task: respond to letters on go-trials; withhold response when a stop-signal tone is played at variable delays*Logan et al. (1984)*▶ Run in browser

Hemispheric-differences-word-processing Continuous recognition memory with lateralised study words (LVF/RVF) and central old/new probes, at nine study-test lags*Federmeier & Benjamin (2005)*▶ Run in browser

Language-Localizer Bilingual auditory language localizer for fMRI; spoken sentences in French, Mandarin, and Wolof are played on a fixed onset schedule started by the scanner trigger, while the participant fixates and listens*Lin, E. (2016), after Pallier et al. (2011) auditory language localizer*▶ Run in browser	**Language-Localizer-French-audio**auditory language localizer; spoken sentences in French, and their temporally reversed versions, are played on a fixed onset schedule started by the scanner trigger, while the participant fixates and listens*Pallier C. (unpublished)*	**Language-Localizer-Reading-English**EvLab visual language localizer for fMRI: the participant reads 12-word sentences and matched nonword sequences presented word by word (RSVP, 450 ms per word, no gap) on a fixed onset schedule started by the scanner ('t') trigger. 48 trials in 16 blocks of 3 (8 sentence, 8 nonword), separated by five 14 s fixation periods; a disc at the end of each trial prompts a button press that keeps the participant alert. The sentences > nonwords contrast localizes the high-level language network. Port of the Psychtoolbox evlab_langloc_2conds.m, using its stimulus tables unchanged (2 runs x 5 sets, selected with -run and -set).*Fedorenko, E., Hsieh, P.-J., Nieto-Castañón, A., Whitfield-Gabrieli, S., & Kanwisher, N. (2010). New method for fMRI investigations of language: defining ROIs functionally in individual subjects. Journal of Neurophysiology, 104(2), 1177-1194. Psychtoolbox implementation by T. Scott (MIT, 2013).*▶ Run in browser

Letter-size-illusion Compare heights of letters vs. mirror/pseudo-letters; replicates the letter height superiority illusion (two experiments)*New et al. (2015)*▶ Run in browser

Magnitude-Estimation-Luminosity Stevens' magnitude estimation of luminance: assign a number to perceived brightness of grey disks*Stevens (1957)*▶ Run in browser

Memory-for-binary-sequences Memory and reproduction of auditory binary sequences of varying complexity*Planton et al. (2021)*▶ Run in browser

Memory_span Adaptive staircase measuring immediate serial recall span for digits, letters, or words▶ Run in browser

Mental-Rotation-3D Decide whether two 3D figures (procedurally generated assemblies of cubes) are identical or mirror images; RT increases linearly with angular disparity.*Shepard & Metzler (1971)*

lexical_decision Decide whether a letter string is a word or a non-word (F / J keys); stimuli loaded from a CSV file▶ Run in browser

MEG-localizer Multimodal MEG localizer — table-driven RSVP of faces, houses, words, consonant strings, equations, retinotopic wedges/rings/disks, and natural sounds▶ Run in browser

Memory-Iconic-Sperling Partial-report procedure measuring capacity and duration of iconic (visual sensory) memory*Sperling (1960)*▶ Run in browser

Mental-Logic-Card-Game Mental logic and inference task using a card-game paradigm▶ Run in browser

Mouse-tracking Mouse-trajectory paradigm (written-word adaptation of Spivey et al.): click the labelled box matching a written target word that appears 500 ms after onset, while the cursor path is sampled at ~36 Hz, revealing continuous attraction toward phonological cohort competitors*Spivey et al. (2005)*▶ Run in browser

LoT-geometry Comprehension of geometric primitives and rules; reproduces Amalric et al. (2017)*Amalric et al. (2017)*▶ Run in browser

MEG-localizer2 Finger and tone MEG localizer — 4 hand pictures with one finger coloured red, an empty (fixation-only) trial, and 5 tones one octave apart, one trial per second (SOA jittered ±100 ms), 160 trials▶ Run in browser

Memory-Scanning Hold a set of digits in memory; decide whether a probe was in the set — RT scales with set size*Sternberg (1966)*▶ Run in browser

Mental-Rotation-2D Decide whether two 3-D figures are identical or mirror images; RT increases linearly with angular disparity*Shepard & Metzler (1971)*▶ Run in browser

Multiple-Object-Tracking Track a subset of identical moving targets among distractors; evidence for a parallel tracking mechanism*Pylyshyn & Storm (1988)*▶ Run in browser

Naming-Stroop Vocal-response Stroop task — a colour word is named aloud while a microphone voice key measures RT from word onset; per-trial WAV files are saved for offline verification*Stroop (1935)*▶ Run in browser

Number-Double-Digits-Comparison Compare two-digit numbers against a fixed standard (55 or 65); two experiments with different response-key mappings*Dehaene et al. (1990)*▶ Run in browser

picture_naming Vocal naming-latency task: a picture is named aloud while a microphone voice key measures RT from image onset; per-trial WAV files are saved for offline verification▶ Run in browser

Number-Change-Detection Two concurrent dot-array streams (5 vs 20 dots) test infants' preference for numerosity change; experimenter codes looking direction in real time*Decarli, Piazza & Izard (2023)*▶ Run in browser

parity_decision Classify single digits (0–9) as even or odd (F / J keys)▶ Run in browser

Posner-ANT Attention Network Task (vertical variant): flanker arrows above/below fixation measure alerting, orienting, and executive attention networks*Fan et al. (2009)*▶ Run in browser

Number-Comparison Compare numerical magnitudes of digits and dot patterns; stimulus group (digits / regular / irregular / random) selected via GetParticipantInfo UI*Buckley & Gillman (1974)*▶ Run in browser

Perception-of-Temporal-Patterns Reproduction of isochronous and non-isochronous rhythmic patterns; tests internal clock induction and coding complexity*Povel & Essens (1985)*▶ Run in browser

Psychological-Refractory-Period Two tasks presented in rapid succession; the second response is delayed when the SOA is short*Welford (1952)*▶ Run in browser

Reading-1 Lexical decision on a 5-letter string flashed for 200 ms in four 50 ms windows, with each letter's contrast sampled independently in every window (5 positions x 4 windows = 20 randomised visibility values per trial)▶ Run in browser	**Reasoning-ConceptARC**ConceptARC (the concept-organised ARC benchmark) as a self-paced grid-editing task: each trial shows the demonstrations of one ARC task and a test input, and the participant paints the output grid with the mouse — palette, click/drag, resize, copy-input, reset — and submits it, three attempts per test input. The default session follows the paper's human study (2 training + 3 minimal attention-check + 12 corpus tasks drawn from 12 concept groups, one random test input each, seeded by subject ID); -concept runs one whole concept group, -tests 3 presents all three test inputs, -explain asks for a typed explanation after each task. Every palette choice, cell edit, resize and submission is logged with the full grid, so each answer's editing history can be replayed. The 176 tasks (16 concepts x 10 + 16 minimal) are embedded*Moskvichev, Odouard & Mitchell (2023), The ConceptARC Benchmark, arXiv:2305.07141*▶ Run in browser	**Retinotopy**HCP retinotopic mapping paradigm (ported from Python); flickering wedge/ring/bar stimuli for visual cortex mapping; run type selected via GetParticipantInfo UI▶ Run in browser

Retinotopy-multifocal Multifocal retinotopic mapping — 24 visual-field regions driven by orthogonal quadratic-residue sequences, pattern-onset with a linear contrast fade; MEG/fMRI sync by flag*Kurki, Hyvärinen, Henriksson & Vanni (2022), NeuroImage 263:119643*▶ Run in browser	**RSVP-Images**RSVP oddball-detection task replicating Hebart et al. (2023, THINGS-data). Object images are shown in rapid serial visual presentation on a mid-gray background with a central black fixation dot; the participant presses SPACE whenever a pixelated oddball appears. Timings are read from a design CSV (onset,duration,trial_type,file_path); companion tool cmd/gen-design produces the MEG (SOA 1500±200 ms) and fMRI (SOA 4.5 s) designs. Oddballs are pixelated at runtime; results add a reaction_time column.*Hebart, M. N. et al. (2023). THINGS-data, a multimodal collection of large-scale datasets for investigating object representations in human brain and behavior. eLife 12:e82580.*	**RSVP-Images-OneBack**One-back RSVP picture task: images from images/ flash by in random order (200 ms on, 100 ms blank); ~10% are immediate repeats and the participant presses SPACE on repeats, with a buzzer on misses. VSYNC-locked, with per-image onset/offset, response, and hit/false-alarm logging

RSVP-MathLang MathLang RSVP paradigm for fMRI: sentences are presented word by word (300 ms per word, no blank between words) on a fixed onset schedule started by the scanner ('t') trigger. Nine sentence types (pseudoword lists, word lists, meaningless sentences, factual/contextual knowledge, theory of mind, arithmetic principles, calculation, geometry) in true and false variants contrast the language and mathematical reasoning networks. The participant judges each statement's veracity (1 = vrai, 2 = faux); percent correct and median RT are reported per condition at the end of the bloc. Onsets are read from the per-subject table stim_lists/sub-NNN_task-MathLang.csv; each of the 5 blocs is a separate run, selected with -b.*Amalric, M. & Dehaene, S. (2016). Origins of the brain networks for advanced mathematics in expert mathematicians. PNAS 113(18), 4909-4917.*▶ Run in browser

RSVP-multimodal Multimodal fMRI localizer driven by a stimulus table: rapid serial visual presentation of words, flashing checkerboards, and spoken sentences are presented on a fixed absolute onset schedule started by the scanner ('t') trigger. The bundled protocols implement the French Pinel functional localizer — reading/listening to sentences, reading/listening to mentally-solved subtractions, reading/hearing an instruction to press the left or right button three times, and passively viewing horizontal/vertical checkerboards — contrasting the language, number, motor and visual networks in one five-minute run. The schedule is not hardcoded: protocols/*.tsv give onset_time, duration, type (TEXT, BOX, IMAGE, SOUND, TEXT_STREAM, IMAGE_STREAM, SOUND_STREAM), cond and stimuli, so the same program plays any multimodal script. Select a run with -p (instructions, run1..run4). Scheduled and achieved onsets are both written to the data file.*Pinel, P., Thirion, B., Meriaux, S., Jobert, A., Serres, J., Le Bihan, D., Poline, J.-B., & Dehaene, S. (2007). Fast reproducible identification and large-scale databasing of individual functional cognitive networks. BMC

Rush-Hour Rush Hour sliding-block puzzle as a problem-solving task: on each trial the participant frees the red car from a 6x6 traffic jam by clicking the blocking vehicles one cell at a time, and every mouse click is logged, so the whole solution path — moves, dead ends, and latencies — can be reconstructed offline. The embedded library holds 49 boards ordered from 3 to 51 moves, each carrying the exact length of its shortest solution, so excess moves over the optimum is a per-trial measure; -n limits a session to the first N puzzles*Fogleman (2018), Rush Hour puzzle database; ThinkFun "Rush Hour" (Nob Yoshigahara, 1996)*▶ Run in browser

Sensory-Threshold-Estimation-Auditory1-up/2-down adaptive staircase with 2-IFC to estimate pure-tone hearing thresholds across multiple frequencies*Levitt (1971)*▶ Run in browser

simple_reaction_times20-trial simple RT task: press any key as quickly as possible when a target appears▶ Run in browser

shadowingVocal shadowing-latency task: repeat a played sound aloud while a microphone voice key measures the delay between audio onset and speech onset▶ Run in browser

SNARC-effect Parity judgment on digits 0–9 with reversed key mappings across blocks; demonstrates the SNARC effect*Dehaene, Bossini & Giraux (1993)*▶ Run in browser

Simon_task Identify colour of a square regardless of its screen position; congruent trials are faster*Simon (1969)*▶ Run in browser

Statistical-Learning-Auditory Statistical learning of tone sequences: exposure to a continuous tone stream with structured transitional probabilities, probed with 2AFC or head-turn preference*Saffran et al. (1999)*▶ Run in browser

Statistical-Learning-Community-Structure Implicit learning of community structure in a continuous visual sequence, with random-walk exposure and spacebar-based event segmentation*Schapiro et al. (2013)*▶ Run in browser

Statistical-Learning-Visual Implicit learning of statistical regularities in a shape stream, probed with forced-choice and RT tests*Turk-Browne et al. (2005)*▶ Run in browser

Stroop_task Name the ink colour of colour words; incongruent trials (e.g. RED in blue ink) are slower*Stroop (1935)*▶ Run in browser

Subliminal-Priming Masked word priming: words rendered invisible by surrounding masks still influence processing*Dehaene et al. (2004)*▶ Run in browser

Temporal-Integration-Word-Recognition Alternating odd/even letter components at variable SOA; Exp 1 (subjective report: 0/1/2 words perceived) and Exp 2 (lexical decision with RT); experiment selected via GetParticipantInfo UI*Forget et al. (2010)*▶ Run in browser

Trubutschek_Unconscious_Working_Memory Probe access to briefly presented stimuli below and above the threshold of consciousness*Trübutschek et al. (2017)*▶ Run in browser

Typing-Speed Typing-performance task: on each trial a target sentence is shown and the participant copies it into a framed input box below (with a blinking cursor). Every keystroke onset is recorded with a hardware-precision timestamp — its time from target onset, the interval since the previous key, and one of three categories (correct character, incorrect character, or movement/editing key). A trial advances only on an exact copy followed by ENTER. At the end, percent correct keystrokes and typing speed in characters/second (mean, P10, P50, P90) for correct keystrokes are reported. Built-in sentences can be overridden with -file.▶ Run in browser **Wisconsin-Card-Sorting-Task**Sort cards under four key cards by an unannounced rule (colour, shape or number) that changes without warning; measures set shifting and perseveration*Berg (1948); Grant & Berg (1948); Heaton et al. (1993)*▶ Run in browser	**Visual-Illusion-Lilac-Chaser**Lilac chaser illusion: a ring of disappearing disks produces a rotating green afterimage▶ Run in browser	**Visual-Search**Feature vs. conjunction visual search across set sizes (4/12/24), measuring how reaction time scales with the number of distractors*Treisman & Gelade (1980)*▶ Run in browser

Programs that open a **GetParticipantInfo** dialog collect all setup interactively (subject ID, monitor dimensions, fullscreen toggle, and any experiment-specific options). Pass `-headless` on the command line to skip the dialog and use field defaults — useful for scripted runs and automated testing. Programs that do not use the dialog still accept `-w` for windowed mode, `-d N` to select a monitor, and `-s <id>` for a subject ID.

Note: These experiments record and save behavioural data to an `.csv` file in `goxpy_`

`data/`.

6.2 Demonstrations

Visual illusions, interactive showcases, and minimal templates. Most do not write a data file.

demo_bouncing_gv_movies Two .gv movies bounce around the screen, additively blend on overlap, fire beep tones at frame markers, and log full state to the data file on every key press; demonstrates every runtime mutator media.Movie exposes (Pause, PauseWithLoop, SetRate, SeekFrame, SetPosition, SetSize, SetBlendMode) with the multi-movie synchrony invariant preserved via BeginBurst/EndBurst	**demo_canvas**Drawing on an off-screen `Canvas` surface before presenting it in one frame▶ Run in browser	**demo_follow_mouse**A white dot follows the mouse cursor every frame — minimal real-time input loop▶ Run in browser
demo_getinfo Demonstrates the `GetParticipantInfo` dialog: collects participant demographics and monitor characteristics before the experiment window opens▶ Run in browser	**demo_hello_world**Simplest possible goxpyriment program — good starting point for new users▶ Run in browser	**demo_Hermann_grid**Hermann grid illusion: gray smudges appear at the gap intersections between black squares▶ Run in browser
demo_images Loads every PNG file in the current directory and displays each one centered for one second	**demo_images_embedded**Displays embeded images sequentially, each one centered for one second▶ Run in browser	**demo_joystick**Move a cursor with a controller's analog stick (gamepad API with raw-joystick fallback, dead zone, live axis readout) — analog input handling▶ Run in browser

demo_keyboard Demonstrates every keyboard input method provided by the framework, section by section▶ Run in browser

demo_menu Demonstrates the stimuli.Menu widget▶ Run in browser

demo_Motion-Blur Motion blur vs. phantom array demo: animated bar demonstrates retinal blur and the strobe effect at 60 Hz▶ Run in browser

demo_mouse_audio_feedback Left/right mouse clicks trigger ping/buzzer audio; useful for testing sound output▶ Run in browser

demo_MovingGrating Presents 10 drifting sinusoidal gratings, orientation +45°/-45° in random order▶ Run in browser

demo_naming Vocal digit-naming task (“one”..”ten”) with a live VU-meter microphone calibration screen that auto-sets the voice-key threshold from observed min/max levels▶ Run in browser

demo_play_gv_videos Plays a single .gv video through the control facade

demo_play_two_gvvideos Plays two .gv videos side by side, synchronised, logging keypresses with their time relative to video onset

demo_play_two_videos Plays two MPEG videos side by side, synchronised, logging keypresses with their time relative to video onset

demo_play_videos Plays a sequence of MPEG video files found in the assets folder

demo_random-dot-stereogram Random-dot stereogram that reveals a 3-D shape when fused binocularly▶ Run in browser

demo_simple-rt-example Minimal 10-trial reaction-time loop using ShowTS / GetKeyEventTS for hardware-timestamped RT▶ Run in browser

demo_simple_audio_visual_stim Simultaneous tone and rectangle with a keypress response — shows audio preloading and the silent-tone sink warm-up▶ Run in browser

demo_spritesheet Cut one image into a grid of stimuli sharing a single GPU texture, drawn by source rectangle (Psychtoolbox’s DrawTexture sourceRect)▶ Run in browser

demo_stimuli_extras Showcase of advanced stimuli: visual mask, Gabor patch, dot cloud, stimulus circle, thermometer▶ Run in browser

demo_stream_callback Per-frame FrameCallback for stream overlays and real-time feedback (PresentStreamOfStimuli-Func)▶ Run in browser

demo_stream_images High-precision RSVP loop streaming a sequence of images (PresentStreamOfImages)

demo_stream_mixed Heterogeneous stream mixing visual and audio stimuli via PresentStreamOfStimuli▶ Run in browser

demo_stream_text High-precision RSVP loop streaming a sequence of text strings▶ Run in browser	**demo_stream_tones**Sequential auditory stream playing a regular sequence of pure tones with timing logs▶ Run in browser	**demo_sync_two_gv_movies**Two .gv movies played side by side under a shared MasterClock; demonstrates media.MovieManager multi-movie sync, burst-pause, Movie[At]/Movie[AtDisplay], and Display[Onset/Offset] events with optional TTL-trigger wiring
demo_text_input Demonstration of the `TextInput` stimulus collecting free-text keyboard input▶ Run in browser	**demo_Visual-Angle-Calibration**Draws concentric rings at 2°, 5°, and 10° of visual angle for a quick sanity-check of the `units.Monitor` calibration▶ Run in browser	**demo_Visual-Illusion-Ebbginghaus**Animated Ebbinghaus (Titchener circles) size-contrast illusion▶ Run in browser
demo_Visual-Illusion-Kanizsa Kanizsa illusory-contour square: a square is perceived where none is drawn▶ Run in browser	**demo_Visual-Persistence**Persistence-of-vision illusion: an image seen only through moving vertical slits integrates into a whole when the eyes track a moving dot▶ Run in browser	

6.3 Technical Tests

Hardware, timing, and feature tests live in the `tests/` directory at the repository root (separate Go module). Build them with `make tests`.

GvFilesPlays sequences of .gv movies while flashing a white square and pulsing DLP-IO8 line 0 at frame markers, for photodiode+oscilloscope display/TTL synchrony checks (port of PsyScope WhiteSquareSync)	**set_fullscreen**Minimal SDL program that toggles exclusive fullscreen mode — sanity check for the fullscreen video path	**test_audio_drift**Measures the audio clock's rate error against the system clock in ppm, by regressing frames consumed on monotonic time over a long tone

test_av_sync Verifies PlaySyncedWithFlip aligns audio onset with the VSYNC flip (icon flash + 1 kHz tone each second)	**test_clear_only_frames**Presents frames whose only content is a clear (no draw calls) to verify the library defence against the compositor clear-only presentation bug	**test_dlpio20**Exercises a DLP-IO20 as an 8-bit TTL trigger device (output + input loopback), with static hold/set modes for multimeter probing
test_dlpio8 Square-wave output on a DLP-IO8-G pin with rising/falling edge jitter statistics; characterises the trigger box against an oscilloscope	**test_eyelink**Checks an SR Research EyeLink through the Python bridge, and measures what a bridge message costs against a TTL fired at the same flip	**test_ft232h**Exercises an Adafruit FT232H breakout as an 8-bit TTL trigger device (output + input loopback)
test_fullscreen Sanity check for fullscreen rendering through the control facade	**test_gv_sync**Plays a synthesised .gv flash train with a TTL pulse at each bright onset; measures whether PlayGvFunc puts frames on screen when it claims to	**test_labjackt4**Exercises a LabJack T4 as an 8-bit TTL trigger device (output + input loopback)
test_linuxgpio Exercises a Linux GPIO character-device (v2) trigger as an 8-bit TTL device (output + input loopback)	**test_megttlbox**Exercises the NeuroSpin Arduino Mega TTL box — firmware identification, TTL outputs on D30–D37, response inputs on D22–D29, and an optional jumpered loopback check.	**test_mmbts**Exercises a NEUROSPEC MMBT-S interface box as an 8-bit TTL trigger device over USB-CDC serial — whole-port patterns, per-line writes, pulses, and a square wave.
test_netstation Exercises an EGI/NetStation EEG host over TCP/IP (ECI) — connect, synchronize, record, and send event markers	**test_onset_marker**Onset marker (photodiode square + TTL code) checked in software — square read back from each frame, code and clear timed against the flips with a logging TTL device	**test_parallel_port**Reads and writes a Linux LPT parallel port via the ppdev kernel interface

test_photodiode_latency Measures the delay between the timestamp ShowTS returns and light actually leaving the screen, by firing a DLP-IO8 TTL synchronously on the flip thread and letting an AD3 or BBTK time the TTL against a photodiode	**test_stream_trigger**Post-flip onset hooks fire a per-stimulus TTL aligned to TimingLog.OnsetNS (PresentStreamOfStimuli-Hooks + PlayStreamOfSound-sHook)	**test_tearing**Full-height white bar sweeping horizontally to reveal screen tearing; prints frame-interval statistics on exit
test_tobii Checks a Tobii Pro tracker through the Python bridge, and measures the coordinate origin, sample rate and clock offset	**test_triggers**Fires the same pulse train on two or more TTL output devices at once, on one absolute schedule, so an oscilloscope (or a MEG TTL box acting as witness) can measure the skew between their edges	**test_vblank_drift**Measures whether Screen.FlipTS drifts against the display, using the kernel's DRM vblank timestamps as a reference instead of a photodiode
test_videorecorder Exercises the BEL_video networked video recorder over TCP/IP — start/stop recording and send overlay labels	**test_vsync_blocking**Reports nominal, unaided and paced frame periods to show whether the platform/driver blocks on VSYNC or drops frames	**Timing-Tests**Hardware timing verification suite: six sub-tests selected with -test ; av (visual+audio+TTL, five photodiode squares) is the default

6.4 Binaries

If you have followed the Full Installation instructions, running the script `build-all.sh` has created binaries for all examples in the subfolder `_build`. You can just open this folder and launch the executables.

Else, you can download pre-built executable programs for all examples.

Alternatively, you can read the next section if you want to compile these experiments on your computer.

6.5 Running the examples from source

If you have installed go on your computer, you can run any example directly from a local clone of the goxpyriment repository:

```
go run ./examples/parity_decision/ -w -s 1
```

Or build and run from inside the example directory:

```
cd examples/hello_world
go run .            # fullscreen by default
go run . -w         # windowed 1024×768
go run . -w -s 1    # windowed, subject ID = 1
go run . -d 1       # fullscreen on monitor 1
go run . -w -d 1    # windowed on monitor 1
go build .          # build a standalone binary
```

To build all examples (and tests) at once (binaries go to `_build/`):

```
./build-all.sh      # from the repo root — works on Linux, macOS, and
↪  Windows (Git Bash)
```

(On Linux/macOS you can also use `make examples` or `make all`.)

Chapter 7

Pre-built binaries of examples and tests for goxpyriment

If want to run the examples directly, without compiling them, you download *pre-built apps* and run them on your computer. There are three ways to get them:

- **Run it in your browser** — no download, no install: 86 of the examples run directly from **downloads.pallier.org/builds/latest/** via WebAssembly. Follow the **Run** link beside a program. Use a focused, foreground tab; your results arrive as a single `.zip` when the session ends. Paradigms needing sub-millisecond stimulus onset (rapid RSVP, subliminal priming) should still be run natively.
- **One program at a time** — browse **downloads.pallier.org/builds/latest/**, find the program you want, and download just that one (a few MB) for your platform. Every experiment, demonstration and technical test is listed there with its description.
- **The whole collection** — download one archive holding every program for your platform, using the links below. These come from the GitHub releases page.

The download site keeps only the two most recent builds, so a commit-pinned link such as `downloads.pallier.org/builds/<commit>/` eventually stops working — always link `builds/latest/`. The GitHub release archives linked below are kept indefinitely.

! WARNING These binaries are unsigned. macOS Gatekeeper and Windows Defender will show security warnings or worse, *misleading messages* such as 'this program is damaged'. Your antivirus may quarantine the files. Don't let them intimidate you. These warnings pop out because I am not willing to pay third parties to sign the executables. The instructions below explain how to run these programs anyway.

7.0.1 Windows (x86-64)

1. Download `goxpyriment-examples-windows-x86_64.zip`;

2. Before unzipping it: right-click the downloaded `.zip`, click on **Properties** → at the bottom of the **General** tab tick **Unblock** → **OK**. (You can achieve the same thing in PowerShell with `Unblock-File .\goxpyriment-examples-windows-x86_`

64.zip.)
3. Unzip it. You should be able to run the `.exe` files directly, either by clicking on them, or typing their name from a command line.

If a security dialog window pops out, you have forgotten to unblock the zip file before unzipping. You can still run the executable by clicking on **More info**, then **Run anyway**. Note: You can unblock all the `.exe` files after unzipping with `Get-ChildItem -Recurse <folder> | Unblock-File`.

7.0.2 macOS (Apple Silicon)

1. Download goxpyriment-examples-macos-arm64.zip, unzip it and move the folder to the location of your choice, e.g., `$HOME/goxpy`
2. Open a Terminal at that location, move the files out of quarantine and set the execute bit, with the following commandes::

```
cd $HOME/goxpy   # <- change to the location where you move the
↪  content of the unzipped folder
xattr -rd com.apple.quarantine .
chmod -R +x *
```

   (See macOS installation and security.)

7.0.3 Linux

- **(x86-64):** Download goxpyriment-examples-linux-x86_64.tar.gz, extract the binaries with `tar xzf`, and run them directly (potentially, you may need to do `chmod +x *` to set their execute permission bit).
- **(arm64 / Raspberry Pi):** Download goxpyriment-examples-linux-arm64.tar.gz, extract the binaries with `tar xzf`, and run the binaries directly.

Note: if you do not find a version for your hardware (e.g. Mac/Intel or Windows/ARM), do not despair: it is very easy to compile these examples
following the instructions in Installation.md *** Good programs to start: `Memory_span`, `Change-Blindness`, `Simon_task`, `LoT_geometry`, `Retinotopy`...

When launched from the command line, most programs accept `-w` (windowed 1024×768 mode), `-d N` (open on monitor N, 0 = primary), and `-s <id>` (subject ID written to the `.csv` data file).

Results are saved in a folder `goxpy_data` in your Home (User) folder

Chapter 8

goxpyriment User's Manual

This manual explains the key concepts of the library. It assumes you have read the Getting Started guide and want to understand the framework well enough to write experiments confidently.

8.1 Table of Contents

1. The Experiment Object
2. Collecting Participant Information
3. The Run Loop and Error Handling
4. The Coordinate System
5. The Rendering Model
6. Timing Architecture — frame cadence, what `Update()` does with a frame, the two clocks, GC, nanosecond RT, VRR
7. Input Handling
8. Data Collection
9. Stimuli: Lifecycle and Preloading
10. High-Precision Streams (RSVP)
11. Audio
12. Experimental Design and Randomization
13. Embedding Stimuli and Trial Lists in the Executable
14. Animated Stimuli
15. Video — `.gv` for measured stimuli, MPEG-1 for convenience
16. Gamma Correction and Luminance Linearity
17. Hardware Triggers and TTL Devices
18. Display Compositor Bypass
19. Variable Refresh Rate (VRR) Stimulus Presentation
20. Putting It All Together

8.2 1. The Experiment Object

Every experiment revolves around a single `*control.Experiment` value. It is a façade that owns the SDL window, renderer, default font, audio device, keyboard and mouse handlers, and data file. You never create these separately.

```
exp := control.NewExperimentFromFlags("My Experiment", control.Black,
 ↪  control.White, 32)
defer exp.End()
```

`NewExperimentFromFlags` parses these optional command-line flags:

Flag	Effect
`-w`	Windowed mode: opens a 1024×768 window instead of going fullscreen
`-d N`	Display ID: open on monitor N (-1 = primary display)
`-s N`	Set subject ID to N (integer); written to the data file automatically
`-headless`	Skip the setup dialog (below) and use field defaults — for batch/automated runs

8.2.1 Automatic setup dialog (launch without `-s`)

If `-s` is **not** passed — most importantly when the binary is launched by **double-clicking its icon** rather than from a terminal — a small setup dialog opens before the experiment starts, letting the experimenter enter the **subject code** and confirm the **display**, **fullscreen/windowed** mode, and **results folder**. It seeds its defaults from any `-w`/`-d` you did pass and remembers the display/fullscreen/folder choices across sessions (the subject code is always asked fresh). Cancelling the dialog exits the program.

The dialog is skipped when `-s N` is given (that value is used directly, as before), when `-headless` is passed, or when the program already collected participant information via `GetParticipantInfo` itself — so no program is ever prompted twice.

`defer exp.End()` must appear immediately after construction. It saves the data file, releases fonts, destroys the window, and shuts down SDL — even if the experiment panics or returns early.

8.2.2 Key fields

```
exp.Screen       *apparatus.Screen           // window + renderer
exp.Keyboard     *apparatus.Keyboard         // keyboard input
exp.Mouse        *apparatus.Mouse            // mouse input
exp.AudioDevice  sdl.AudioDeviceID    // SDL audio device for Sound/Tone
exp.Audio        *control.AudioManager // high-level audio helpers
exp.Data         *results.DataFile         // output data file
```

```
exp.Design       *design.Experiment   // block/trial structure
exp.SubjectID    int
exp.DefaultFont  *ttf.Font
exp.Info         map[string]string    // values collected by
 ↪  GetParticipantInfo, if used
```

8.3 2. Collecting Participant Information

Before the experiment window opens, you can display a graphical setup dialog that lets the experimenter fill in participant demographics, monitor characteristics, and display mode. The dialog returns the collected values as a `map[string]string`.

```
fields := append(control.StandardFields, control.FullscreenField)
info, err := control.GetParticipantInfo("My Experiment", fields)
if err != nil {
    log.Fatalf("setup cancelled: %v", err)
}
```

Call `GetParticipantInfo` **before** `NewExperiment` or `NewExperimentFromFlags`. It initializes SDL internally, shows the window, and shuts SDL down again before returning, so the subsequent experiment initialization starts from a clean state.

8.3.1 Pre-built field sets

Variable	Fields included
`control.ParticipantFields`	`subject_id`, `age`, `gender`, `handedness`
`control.MonitorFields`	`screen_width_cm`, `viewing_distance_cm`, `refresh_rate_hz`
`control.StandardFields`	`ParticipantFields` + `MonitorFields` combined
`control.FullscreenField`	Checkbox: `fullscreen` (`"true"` / `"false"`)

8.3.2 Defining custom fields

```
fields := []control.InfoField{
    {Name: "subject_id", Label: "Subject ID",      Default: ""},
    {Name: "session",    Label: "Session (1/2/3)", Default: "1"},
    {Name: "room",       Label: "Testing room",    Default: "Lab A"},
    {Name: "fullscreen", Label: "Fullscreen mode", Default: "true",
     Type: control.FieldCheckbox},
}
info, err := control.GetParticipantInfo("My Experiment", fields)
```

Fields of type `FieldText` (the default) render as text input boxes. Fields of type `FieldCheckbox` render as tick-boxes; their value is always `"true"` or `"false"`.

8.3.3 Dialog interaction

Action	Effect
Click a field	Focus it
Type	Append text to the focused field
Backspace	Delete last character
Tab / Shift-Tab	Move focus to next / previous text field
Enter	Confirm (same as clicking OK)
Escape / close window	Cancel — `ErrCancelled` is returned

8.3.4 Session persistence

All values except `subject_id` are saved to `~/.cache/goxpyriment/last_session.json` when the experimenter confirms. They are pre-filled on the next run. `subject_id` is always reset — the experimenter must enter it fresh each session.

8.3.5 Using the results

```
info, err := control.GetParticipantInfo("My Experiment", fields)
if err != nil {
    log.Fatalf("setup cancelled: %v", err)
}

// Use the fullscreen checkbox to choose the window mode
fullscreen := info["fullscreen"] == "true"
width, height := 0, 0
if !fullscreen {
    width, height = 1024, 768
}

exp := control.NewExperiment("My Experiment", width, height, fullscreen,
    control.Black, control.White, 32)

// Set Info and SubjectID BEFORE Initialize so they are written to the .csv
 ↪  header
exp.SubjectID, _ = strconv.Atoi(info["subject_id"])
exp.Info = info

if err := exp.Initialize(); err != nil {
    log.Fatalf("failed to initialize: %v", err)
}
defer exp.End()
```

When `exp.Info` is non-nil at the time `Initialize()` is called, the collected key/value pairs are automatically written as a `--PARTICIPANT INFO` block in the `.csv` metadata header — no extra call is required.

> **Note:** When using `GetParticipantInfo` you will typically call the lower-level `NewExperiment` + `Initialize()` instead of `NewExperimentFromFlags`, so you can pass the fullscreen/windowed choice from the dialog directly.

8.4 3. The Run Loop and Error Handling

8.4.1 The `exp.Run` wrapper

All experiment logic runs inside a callback passed to `exp.Run`:

```
err := exp.Run(func() error {
    // your experiment here
    return control.EndLoop
})
if err != nil && !control.IsEndLoop(err) {
    log.Fatalf("experiment error: %v", err)
}
```

`exp.Run` does two important things:

1. **Ensures everything runs on the SDL main thread.** SDL requires that all rendering and event calls happen on the thread that created the window. `exp.Run` guarantees this.
2. **Installs a panic/recover guard.** When the participant presses ESC, the library immediately aborts the trial loop by calling `panic` with an internal sentinel value. `exp.Run` catches this panic, converts it back to an error, and returns it cleanly. Your data is saved.

8.4.2 Returning from the callback

Return value	Meaning
`control.EndLoop`	Normal exit; experiment is done
`nil`	Continue — call the callback again on the next frame
any other `error`	Propagated as the return value of `exp.Run`

In a typical experiment you run the full trial loop inside a single callback invocation and return `control.EndLoop` at the end. The `nil` / "keep looping" pattern is used for frame-by-frame animation; in most cases you will never return `nil`.

8.4.3 You don't need to check every error

Because pressing ESC triggers the panic/recover mechanism, any call that would have returned an error (e.g., `exp.Show`, `exp.Wait`, `exp.Keyboard.WaitKey`) will instead abort the loop immediately. The error never reaches your code.

This means the following two styles are both correct:

```
// Style A — check errors explicitly (safer for library/production code)
if err := exp.Show(fixation); err != nil {
    return err
}
if err := exp.Wait(500); err != nil {
    return err
}

// Style B — omit error checks inside exp.Run (fine for experiment scripts)
exp.ShowTimed(fixation, 500)
```

Style B works because if something goes wrong (ESC pressed, window closed), the panic/recover mechanism unwinds the call stack before your code can observe an error. Use Style A if you have cleanup that must run on abort; otherwise Style B keeps experiment scripts readable.

8.5 4. The Coordinate System

All stimulus positions use a **center-origin** system:

```
            +Y (up)
             |
  -X --------+-------- +X
             |
            -Y (down)
```

- `(0, 0)` is the screen center.
- Positive Y is **up** (opposite to SDL's default, which is top-down).
- Units are pixels at the logical resolution.

```
// Center of screen
stimuli.NewTextLine("Hello", 0, 0, control.White)

// 200 px to the right
stimuli.NewCircle(30, control.Red).SetPosition(control.Point(200, 0))

// Bottom-left area
stimuli.NewTextLine("Score", -400, -250, control.White)
```

The conversion to SDL coordinates (top-left origin) is handled internally by `screen.CenterToSDL(x, y)`. You never call this yourself unless you are writing a custom stimulus

type — in which case `screen.CenteredRect(pos, w, h)` returns the SDL destination rectangle for a `w×h` texture centered at `pos`, factoring out the convert-and-offset step.

8.5.1 Logical size

If you call `exp.SetLogicalSize(width, height)`, the coordinate system scales to that virtual resolution regardless of the actual window or screen size. This is useful for making experiments resolution-independent:

```
if err := exp.SetLogicalSize(1920, 1080); err != nil {
    log.Printf("warning: %v", err)
}
// Now (960, 540) is the center-right area, regardless of actual screen
 ↪  size.
```

8.5.2 Display scaling (HiDPI / fractional scaling)

> **Recommendation: set your OS display scaling to 100% before running experiments.**

Most desktop environments (GNOME, KDE, Windows, macOS) allow the user to scale the entire UI — e.g. 125%, 133%, 150% — to make text and icons larger on high-density screens. Goxpyriment does not currently compensate for fractional OS-level scaling. Running with a scale factor other than 100% can cause:

- stimulus positions and sizes to be wrong (coordinates are in logical pixels, but the OS maps them to physical pixels at the scale factor),
- video playback and canvas rendering to appear cropped or misaligned,
- the window to not fill the screen correctly in fullscreen mode.

To disable scaling before running an experiment:

- **GNOME (Ubuntu):** *Settings → Displays → Scale → 100%*
- **KDE:** *System Settings → Display and Monitor → Scale → 100%*
- **Windows:** *Settings → Display → Scale → 100%*
- **macOS:** *System Settings → Displays → Resolution → Default* (or choose a non-HiDPI mode)

You can restore your preferred scaling after the session.

8.6 5. The Rendering Model

SDL uses a **double-buffered** rendering model. There is an off-screen backbuffer where you draw, and the visible display. You draw to the backbuffer; calling `screen.Update()` (also called a "flip") presents it to the screen, synchronized to the vertical retrace (VSYNC).

The three-step cycle for showing one stimulus is:

```
exp.Screen.Clear()       // fill backbuffer with background color
myStim.Draw(exp.Screen)  // draw stimulus onto backbuffer
exp.Screen.Update()      // present to display (VSYNC-blocks)
```

`exp.Show(stim)` does all three in one call and is the right choice for single-stimulus presentations. For cases where you need to draw multiple stimuli simultaneously — so they appear on screen at the same time — call each `Draw` separately before the single `Update`:

> **Triple/mailbox buffering — handled for you.** On many systems (NVIDIA + compositor, Wayland, and more besides) `SDL_RenderPresent` does **not** block until the retrace: it queues the frame in a non-blocking mode — mailbox ("triple buffering"), where a newer frame replaces the pending one — or hands the buffer to a compositor that accepts it rather than blocking on scanout. The display stays tear-free, but the *call* no longer paces one frame per call. A per-frame loop then completes without consuming wall-clock time and the stimulus is replaced too early, its on-screen duration unpredictably short (often, but not always, zero frames).
>
> `screen.Update()` therefore presents **and then holds to the frame boundary**, so one call always occupies one display frame. You do not have to select a special method or detect the platform: where the driver blocks correctly the wait runs zero iterations and costs nothing. This is why there is no `PacedFlip` — it was removed once pacing became unconditional.
>
> The one thing pacing cannot do is recover a frame that was *dropped* on the way to the panel: it enforces a minimum frame time, not a maximum. See §Calibration below.

```
// Show fixation cross and stimulus simultaneously
exp.Screen.Clear()
fixation.Draw(exp.Screen)
targetCircle.Draw(exp.Screen)
exp.Screen.Update()
```

8.6.1 The blank screen

`exp.Blank(ms)` clears the screen, flips it (showing a blank), and then waits:

```
exp.Blank(1000) // 1-second blank inter-trial interval
```

This is equivalent to `exp.Screen.Clear() + exp.Screen.Update() + exp.Wait(ms)`.

8.6.2 Never draw outside exp.Run

All rendering must happen inside the `exp.Run` callback — equivalently, on the SDL main thread. Drawing from a goroutine will silently do nothing or crash.

8.7 6. Timing Architecture

8.7.1 Frame cadence

`screen.Update()` returns at the display's vertical retrace (VSYNC). On a 60 Hz monitor this is ~16.67 ms; on a 120 Hz monitor ~8.33 ms. This is the fundamental clock of the visual display: every stimulus change is aligned to a frame boundary automatically.

On some systems (NVIDIA proprietary driver + compositor, Wayland) the system selects a non-blocking presentation mode (mailbox/"triple buffering") or routes the swap through a compositor, so the underlying `SDL_RenderPresent` returns before the frame is scanned out. The display is still tear-free, but the call cannot be relied on to block for one frame. (This is not VSYNC being disabled: a blocking mode such as FIFO exists; the system just isn't guaranteed to use it. Wayland paces via frame callbacks rather than a blocking swap.)

`screen.Update()` closes this gap itself by holding to the frame boundary after presenting, so a tight frame loop is correct on every driver without you choosing anything. Measured on Intel i915 + Wayland with a 120 Hz panel, unaided presents returned as little as 6.95 ms apart against an 8.33 ms frame; through `Update` the same loop held 8.333 ms with SD 0.06 ms.

To know your frame duration at runtime:

```
frameDur := exp.Screen.FrameDuration()  // e.g., 16.666ms on a 60 Hz display
fmt.Printf("Frame: %.2f ms\n", frameDur.Seconds()*1000)
```

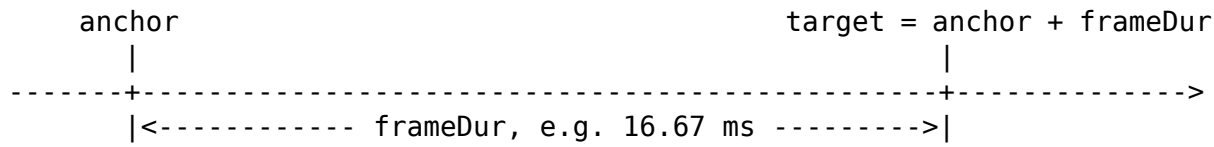
8.7.2 What `Update()` actually does with a frame

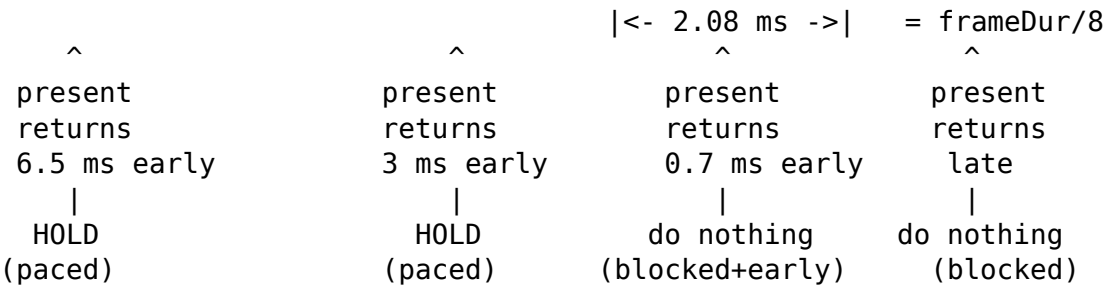
You rarely need this, but when a duration comes out one frame long, or you are deciding whether to trust an onset timestamp, this is the machinery underneath. Each `Update()` does four things in order (sketch, not compilable Go):

```
anchor := previous frame's onset      1. sampled BEFORE this frame
 ↪  overwrites it
SDL_RenderPresent(); presentNS = now  2. submit, and stamp the return
paceToFrame(anchor)                   3. maybe hold to the frame boundary
recordOnset()                         4. decide this frame's onset
```

The deadline is measured from the previous frame's *onset*, not from the previous `Update()` call. That onset comes from the best source available, in this order: the kernel's vblank timestamp (`GOXPY_VBLANK=on`), the scheduled boundary if pacing held the frame, or the moment `SDL_RenderPresent` returned. Whichever it was, `target = anchor + FrameDuration()`, and the run's `onset_source` column records which.

Now let `now` be the clock read the instant `SDL_RenderPresent` returned. Where it falls decides everything:





Early by more than an eighth of a frame — hold. The driver queued the frame and returned (triple/mailbox buffering). `Update` sleeps until 2 ms before the boundary, then spins the last 2 ms, and reports the *scheduled* boundary as the onset. It is split that way for a reason in each direction: spinning the whole wait puts a real-time thread at 100 % duty and trips the kernel's throttle (measured: 24 stalls of 51 ms in 25 s, three dropped frames a second at 60 Hz), while sleeping the whole wait overshoots the boundary by up to 0.83 ms and misses the frame.

Early by an eighth of a frame or less — do nothing. This is a driver that *did* block on the retrace and simply came back a little inside the *nominal* boundary, because the panel's grid and the nominal grid sit at a fixed phase offset. Its return is a hardware instant, and holding would throw that away and substitute a synthesised time — on one machine that happened on 98.8 % of frames. The two cases are far apart in practice, which is what makes a threshold workable: a blocking present came back 0.68 ms early on average and 1.14 ms at worst, against 6.5–15 ms for a non-blocking one. Nothing has been observed in between.

Late — also nothing, and the missed time is not made up. If `now` is already past `target`, the target is *abandoned*, not chased. `Update` never shortens a later frame to compensate, and the next deadline is measured from the present that just returned, so the schedule quietly re-phases onto wherever the display actually is. The consequence is the one you see in the data: a loop of twelve `Update()` calls that hits one stall takes thirteen frames, and its duration column reads one frame long. That is the honest report — a compressed frame afterwards would be worse — but it does mean a stall shows up as a long interval, never as a short one.

Two things follow that are worth knowing:

- A late present is tallied as `blocked`, the same as a healthy blocking one, so the pacing counters alone do not reveal lateness. What reveals it is the measured-vs-nominal comparison in Startup calibration below, and `sequence_gaps` when the vblank clock is enabled.
- Pacing is skipped entirely when VSync is off, on the grounds that a caller who turned it off wants frames as fast as the GPU can make them.

The tallies land in the run's `-info.txt`, and `early` is a **subset** of `blocked` rather than a fourth category:

```
# sys pacing: presents=30000 blocked=30000 early=29610 paced=0
 ↪  vblank_held=0 … class=blocking
```

So `presents = blocked + paced + vblank_held`. `blocked` with a high `early` and a sub-millisecond `early_mean` is a healthy blocking driver; `paced` with a multi-millisecond

`wait_mean` is a driver that does not block, being held; `vblank_held` is the kernel-vblank path. Timing Tests reads these out in full.

8.7.3 Startup calibration

`FrameDuration()` is the *nominal* period — what SDL derives from the display mode. It is not proof that your frames actually took that long. `Initialize()` therefore measures the display over 60 frames before the experiment starts and records both numbers in the `-info.txt` file:

```
# sys refresh_nominal_hz: 120.0000
# sys refresh_measured_hz: 119.9820
```

If the two disagree by more than 10%, a warning is logged. Take it seriously — it means this session's stimulus durations were not what your analysis will assume. The two directions mean different things:

Measured vs nominal	Meaning	Does pacing help?
≈ **nominal**	Normal.	—
Faster than nominal	`SDL_RenderPresent` is not blocking to the retrace (triple/mailbox buffering).	Yes — `Update`'s pacing is exactly this case.
Slower than nominal	Frames are being *dropped* before reaching the panel: a compositor throttling an unfocused or occluded window, or a GPU that cannot keep up.	**No.** Pacing enforces a minimum frame time, not a maximum.

The third row is the one that catches people out. A compositor will happily throttle a windowed experiment to a fraction of the refresh rate while every timestamp your program records still looks plausible. Run fullscreen, keep the window focused, and check the measured rate in the info file afterwards.

To measure it yourself at any point — it bypasses the pacing, so it reports the unaided driver behaviour:

```
measured, err := exp.Screen.CalibrateRefresh(120)  // median over 120 frames
```

`tests/test_vsync_blocking` reports nominal, unaided and paced side by side, with a verdict; run it once on any machine you plan to collect data on.

8.7.4 `exp.Wait` vs `clock.Wait`

Function	Pumps SDL events?	Detects ESC?	Use when
`exp.Wait(ms)`	Yes	Yes	Inside `exp.Run`, between stimuli
`clock.Wait(ms)`	No	No	Short busy-waits, inside animation loops

Always use `exp.Wait` for inter-trial and inter-stimulus intervals — it keeps the OS responsive and responds to ESC. Use `clock.Wait` only inside VSYNC-locked loops that handle events themselves (streams, animation loops).

8.7.5 Sub-millisecond timing is not guaranteed for `exp.Wait`

`exp.Wait` sleeps in 1 ms increments. For coarse delays (ISIs, fixation durations) this is perfectly fine. For frame-accurate stimulus timing — e.g., showing a stimulus for exactly 2 frames — use the stream functions described in Section 10.

8.7.6 Nanosecond event timestamps

SDL3 timestamps every input event **at hardware-interrupt time**, before any application code runs. The timestamp is stored with the event in the SDL event queue, where it remains untouched until your code reads it.

This has a crucial consequence: **it does not matter when you call `GetKeyEventTS`**. Any keypress that occurred — even during `exp.Wait`, between two `ShowTS` calls, or while other computation was running — is waiting in the queue with its exact hardware timestamp. Nothing is lost. This is fundamentally different from a polling-based approach, where a keypress can only be measured from the moment the polling function is called, and any code running before that call inflates the apparent RT.

`exp.ShowTS(stim)` records the SDL nanosecond time of the VSYNC flip. `GetKeyEventTS` retrieves the event's own hardware timestamp. Subtracting the two gives hardware-precision RT regardless of how much code ran in between:

```
onset, _ := exp.ShowTS(stim1)   // record flip timestamp
exp.Wait(500)                    // keypress here is NOT lost
exp.ShowTS(stim2)                // keypress here is also NOT lost
key, keyTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
rtToStim1 := int64(keyTS - onset)   // correct even if pressed during Wait
```

This is especially useful when you need RT relative to a specific stimulus in a multi-stimulus sequence — something that `WaitKeysRT` cannot express directly, since it measures from the call site rather than from a recorded onset.

8.7.7 Two clocks: the Go clock and the SDL clock

goxpyriment exposes time through **two independent clocks with different zero points**. A value read from one must never be subtracted from a value read from the other — the difference would silently include the unknown, machine-dependent offset between their origins.

Clock	Read it via	Zero point	Use it for
Go monotonic clock	`clock.GetTime()`, `clock.GetTimeNS()`, `Clock.Now()`, `Clock.NowMillis()`, `Clock.NowNanos()`	First use of the `clock` package, or `clock.NewClock()`	*Scheduling and logging in your own code*: inter-trial and inter-stimulus intervals, drift-free trial pacing (`SleepUntil`), and "time since start of block" columns in your data file
SDL event clock	`exp.ShowTS()`, `Screen.FlipTS()`, `Keyboard.GetKeyEventTS()`, `WaitAnyEventTS()`, and the `Timestamp` field of any SDL event (all on `sdl.TicksNS()`)	SDL initialisation	**Reaction-time measurement**: whenever you subtract a stimulus onset from a response, *both* values must come from this clock

The rule. Measure reaction times entirely on the SDL clock — subtract a `FlipTS`/`ShowTS` onset from a `GetKeyEventTS` response. Use the Go clock only for things you *schedule* rather than *measure*: ISIs, fixation durations, block pacing, and log timestamps. Crossing the two — e.g. `keyTS - clock.GetTimeNS()` — is meaningless.

Why two clocks? The SDL clock is the one SDL stamps onto hardware input events at interrupt time (see the previous subsection), so it is the only clock that can measure RT without inflation from intervening code. The Go clock needs no SDL context, is what `exp.Wait`, `clock.Wait`, and `Clock.SleepUntil` are built on, and is the natural choice for scheduling and for log timestamps that just need to be internally consistent.

The two roles coexist cleanly in a single trial loop — Go clock for pacing and logging, SDL clock for the RT:

```
c := clock.NewClock()                                    // Go clock: pacing +
 ↪  logging
for i, trial := range block.Trials {
    c.SleepUntil(time.Duration(i) * 2 * time.Second)   // Go clock:
 ↪  drift-free onset cadence
```

```
    onset, _ := exp.ShowTS(trial.Stimulus)            // SDL clock:
↪   stimulus onset
    key, keyTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
    rtMs := float64(keyTS-onset) / 1e6                 // SDL − SDL  ✓
↪   valid RT
    exp.Data.Add(trial.Label, key, rtMs, c.NowMillis())// log time on the Go
↪   clock
}
```

Here `keyTS - onset` stays within the SDL clock (a real RT), while `c.SleepUntil` and `c.NowMillis()` stay within the Go clock (scheduling and a log column). The two are never subtracted from each other.

8.7.8 Disabling garbage collection

Go's garbage collector can pause execution for several milliseconds at unpredictable times. The stream and animation functions disable it during the critical loop:

```
old := debug.SetGCPercent(-1)
defer debug.SetGCPercent(old)
```

You do not need to do this yourself for ordinary trial loops; only for high-speed RSVP or animation. The stream and animation functions handle it automatically.

8.7.9 Choosing a timing approach

Use this decision guide before committing to a timing strategy.

Situation	Recommended approach
Coarse ISIs and fixation durations ($\geq$ 100 ms)	`exp.Wait(ms)` — adequate precision, keeps the event loop alive
Show stimulus + timed hold (fixation, cue, mask)	`exp.ShowTimed(stim, ms)` — Show + Wait in one call
Single-stimulus trial, RT relative to onset	`key, rt, err := exp.ShowAndGetRT(stim, keys, timeout)` — hardware-precise RT, clears stale events automatically
Stimulus duration measured in frames (e.g. 2-frame mask)	Per-frame loop with `screen.Flip()` (paced, safe on all drivers), or `PresentStreamOfImages` (Section 10)
Multi-stimulus sequence, RT relative to a specific stimulus	`exp.ShowTS(stim)` + `exp.Keyboard.GetKeyEventTS(...)` — nanosecond timestamps on the same SDL clock; subtract directly
RSVP or rapid animation (frame-accurate presentation of many items)	`PresentStreamOfImages` / `PresentStreamOfText` (Section 10) — GC disabled, VSYNC-locked, full timing log

Situation	Recommended approach
EEG/MEG synchronisation required	The onset marker: `control.MarkerFlags() + exp.Marker.Arm(code)` before the onset, which gives a photodiode square and a TTL code on the same flip (Section 17)
Stimulus duration not a multiple of the frame period (e.g. 10 ms, 17 ms)	Variable Refresh Rate mode — see below

OS considerations. Presentation latency depends heavily on whether the display compositor is active — see the next section. For EEG work, always verify onset timing with a photodiode regardless of OS.

When in doubt, use `ShowAndGetRT`. It clears stale events, records the VSYNC flip timestamp, waits for the key with hardware-precision timing, and returns `(key, rtMs, error)` — the canonical single-stimulus RT call. Use raw `ShowTS` + `GetKeyEventTS` only when composing multiple stimuli before a single response.

`ShowTS` vs `FlipTS`. `exp.ShowTS(stim)` is the standard high-level call: it clears the screen, draws the stimulus, flips, and returns the flip timestamp — one line for the common case. `exp.Screen.FlipTS()` is the lower-level primitive that only flips and timestamps, leaving clearing and drawing to you. Both hold to the frame boundary, so both are safe in a per-frame loop; there is no third variant to choose between.

8.7.10 Display compositor bypass

A compositing window manager blends every application frame with other on-screen elements before sending the result to the display. This adds latency and jitter that can be significant for millisecond-accurate stimulus presentation. How much overhead you incur — and whether you can avoid it — depends on the operating system.

Linux / X11 (fullscreen). SDL3 automatically sets the `_NET_WM_BYPASS_COMPOSITOR` window property when the application enters fullscreen mode. Compliant compositing WMs (KWin, Mutter, Compton) respond by *unredirecting* the fullscreen window: the application's framebuffer is routed directly to the display scan-out pipeline without compositing. Nothing special needs to be done in goxpyriment; running fullscreen (omit `-w`) is sufficient.

This removes the compositor's *variance* but not the pipeline's latency. Measured with a photodiode on a bare Xorg session with openbox, trigger-to-light was **35.74 ms with a standard deviation of 0.083 ms** — the steadiest configuration measured, and also the latest, one whole frame behind KMS/DRM. Do not assume "bypass" means sub-millisecond latency; it does not.

Linux / Wayland (fullscreen). The Wayland compositor is always the intermediary for frame delivery, and modern compositors (KWin 5.21+, GNOME Mutter 44+) support *direct scan-out* for fullscreen applications. In practice this is the **worst** of the three Linux configurations for timing: 21.75 ms with a standard deviation of **1.344**

ms, sixteen times the scatter of a compositor-free session, with 7 % of flips landing more than 1 ms off the frame grid. See Timing tests.

Linux / KMS–DRM (no display server). The lowest-latency configuration on Linux is to run without any display server at all, from a virtual terminal (`Ctrl+Alt+F2`). Setting the environment variable `SDL_VIDEODRIVER=kmsdrm` before launching the experiment directs SDL3 to use the Linux kernel's KMS/DRM subsystem directly, bypassing both X11 and Wayland entirely:

```
SDL_VIDEODRIVER=kmsdrm go run myexperiment/main.go
```

This is the recommended configuration for the most demanding timing requirements, such as single-frame subliminal stimulation or high-frequency RSVP.

Windows (fullscreen). SDL3's fullscreen mode creates an exclusive fullscreen surface that bypasses the Desktop Window Manager (DWM), yielding frame-interval jitter typically below 0.3 ms (one standard deviation), comparable to Linux.

macOS. Apple's Metal pipeline always delivers frames through the WindowServer compositor, and no third-party application can bypass it — even in fullscreen.

These platform differences can be measured empirically with the `display` and `av` subtests of the bundled `Timing-Tests` suite (see TimingTests.md).

8.7.11 Variable Refresh Rate (VRR) — arbitrary stimulus durations

On a standard 60 Hz monitor, every stimulus duration must be a multiple of 16.67 ms. You cannot present a stimulus for 10 ms or 20 ms — the display simply cannot change in between frame boundaries. Variable Refresh Rate technology (AMD FreeSync, NVIDIA G-Sync, VESA Adaptive-Sync) removes this constraint: the panel holds each frame for exactly as long as the software asks, refreshing only when the next `Present()` call arrives.

8.7.11.1 Enabling VRR in goxpyriment

goxpyriment opens the renderer with VSYNC enabled by default, which locks every flip to a frame boundary. To enter VRR mode, disable VSYNC before the timing-critical section and restore it afterwards:

```
// Switch to VRR mode — Present() now returns immediately.
if err := exp.Screen.SetVSync(0); err != nil {
    log.Fatalf("VRR not supported: %v", err)
}
defer exp.Screen.SetVSync(1) // always restore

// Pre-load stimulus textures before disabling GC.
stimuli.PreloadVisualOnScreen(exp.Screen, myStim)

old := debug.SetGCPercent(-1)
defer debug.SetGCPercent(old)
```

```
// Draw stimulus and present (returns immediately with VSync=0).
exp.Screen.Renderer.SetDrawColor(0, 0, 0, 255)
exp.Screen.Clear()
myStim.Draw(exp.Screen)
onsetNS, _ := exp.Screen.FlipTS()

// Hold for exactly targetDur using a busy-wait (sub-ms precision).
deadline := time.Now().Add(targetDur)
for time.Now().Before(deadline) { /* spin */ }

// Present blank — the display switches immediately on VRR.
exp.Screen.Renderer.SetDrawColor(0, 0, 0, 255)
exp.Screen.Clear()
offsetNS, _ := exp.Screen.FlipTS()

actualDurMs := float64(offsetNS-onsetNS) / 1e6
```

`onsetNS` and `offsetNS` are both `sdl.TicksNS()` timestamps on the SDL nanosecond clock, captured right after each `Present()` returns. Their difference measures the software-controlled stimulus duration. On a VRR display this equals the actual on-screen duration plus a small, constant hardware pipeline latency (GPU scan-out + panel response, typically 1–5 ms) that can be measured independently with the `frames` test and a photodiode.

8.7.11.2 VRR range and self-diagnosis

Every VRR panel has a supported refresh-rate range (e.g. 48–144 Hz on a typical gaming monitor). Requesting a duration shorter than `1000/max_Hz` ms or longer than `1000/min_Hz` ms takes the panel outside its VRR window; the display then falls back to fixed-rate behaviour and durations are again quantised to frame multiples.

You can characterise your panel's VRR window empirically with the `Timing-Tests` suite:

```
go run ./tests/Timing-Tests -test vrr -vrr-max-ms 50 -cycles 5
```

This sweeps target durations from 1 ms to 50 ms in 1 ms steps and reports the actual vs. target duration at each step. Duration errors below 0.5 ms across the sweep confirm that VRR is working; large periodic errors confirm that VRR is absent or the requested duration is outside the supported range. See TimingTests.md — VRR section for detailed interpretation, including how to read the VRR boundary from the output and how to enable FreeSync on Linux.

8.7.11.3 Requirements

- A VRR-capable monitor (FreeSync, G-Sync Compatible, or Adaptive-Sync).
- VRR enabled in the OS display settings (Linux: DRM/KMS or compositor; Windows: Display Settings → Advanced display → Variable refresh rate).
- The application does not need any special SDL hints; `SetVSync(0)` is sufficient.

8.7.11.4 Note on tearing

On a monitor without VRR, `SetVSync(0)` disables the frame-boundary synchronisation entirely and the GPU writes to the framebuffer while the display is reading it, producing a horizontal tear line. On a VRR monitor this does not happen: the display waits for the GPU's signal before starting each new refresh. If your VRR test shows tearing artefacts, VRR is either not enabled in your OS or not supported by your hardware.

8.8 7. Input Handling

8.8.1 Keyboard

The `exp.Keyboard` object provides blocking and non-blocking access:

```
// Block until any key — returns the keycode
key, err := exp.Keyboard.Wait()

// Block until a specific key
err := exp.Keyboard.WaitKey(control.K_SPACE)

// Block until one of several keys, with optional timeout (-1 = no timeout)
// Returns 0 on timeout, sdl.EndLoop on ESC/quit
key, err := exp.Keyboard.WaitKeys([]control.Keycode{control.K_F, control.K_
 ↪  J}, 3000)

// Same, but also returns reaction time in milliseconds from the call site
key, rt, err := exp.Keyboard.WaitKeysRT([]control.Keycode{control.K_F,
 ↪  control.K_J}, 3000)

// Hardware-precision version: returns the SDL event's nanosecond timestamp
key, eventTS, err :=
 ↪  exp.Keyboard.GetKeyEventTS([]control.Keycode{control.K_F, control.K_J},
 ↪  3000)

// Non-blocking poll — returns 0 if nothing pressed
key, err := exp.Keyboard.Check()

// Query whether a key is physically held down right now (no event queue
 ↪  involvement)
held := exp.Keyboard.IsPressed(control.K_SPACE)

// Wait for KEY_UP on a specific key; returns its SDL hardware timestamp
 ↪  (nanoseconds)
upTS, err := exp.Keyboard.WaitKeyReleaseTS(key, -1)

// Drain all pending key events (use before a new trial to discard stale
 ↪  presses)
```

```
exp.Keyboard.Clear()
```

`GetKeyEventTS` "get" semantics. Unlike the `Wait*` functions, `GetKeyEventTS` reads from the SDL event queue and returns *immediately* if a matching event is already there — it only blocks when the queue has no matching event. This means a keypress that happened during `exp.Wait` or any other intervening code is consumed instantly on the next `GetKeyEventTS` call, with its original hardware timestamp intact.

When to call `Clear()`. Despite living on `exp.Keyboard`, `Clear()` drains the **entire** SDL event queue — keyboard, mouse, gamepad, and everything else — because SDL uses a single shared queue. Call it at the start of a trial (before `ShowTS`) to discard stale events from any device. Do **not** call it after `ShowTS`: the participant may have already responded, and `Clear()` would silently discard that event before `GetKeyEventTS` or `GetPressEventTS` ever sees it.

`WaitKeysRT` vs `GetKeyEventTS`:

`WaitKeysRT` measures elapsed time from the moment the call is made (using `sdl.Ticks()`, millisecond granularity). In a single-stimulus trial this is a good approximation of RT from stimulus onset, but it accumulates any delay between `Show()` returning and `WaitKeysRT` being called.

`GetKeyEventTS` returns the SDL3 event's own `Timestamp` field — the nanosecond time at which the hardware key-down event was generated. Combined with `exp.ShowTS(stim)`, which records the nanosecond time of the VSYNC flip, reaction time is simply:

```
key, rtMs, _ := exp.ShowAndGetRT(target, responseKeys, 3000)
```

Internally `ShowAndGetRT` calls `exp.Keyboard.Clear()`, then `exp.ShowTS` to get the VSYNC flip timestamp, then `GetKeyEventTS`, and returns the difference in milliseconds. When you need the raw nanosecond timestamps (e.g. for EEG trigger alignment), you can call the primitives directly:

```
onset, _ := exp.ShowTS(target)   // nanoseconds at VSYNC flip
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, 3000)
rtNS := int64(eventTS - onset)   // nanoseconds; divide by 1e6 for ms
```

This form is also the only correct approach when multiple stimuli are presented in sequence and you need RT relative to a specific one:

```
onset1, _ := exp.ShowTS(prime)   // prime appears
exp.Wait(500)
exp.ShowTS(target)               // target appears 500 ms later
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, 3000)
rtToPrime := int64(eventTS - onset1)  // RT from prime onset
```

Collecting multiple simultaneous responses — `GetKeyEventsTS`:

In rare cases you may want to capture *all* key events, not just the first. `GetKeyEventsTS` is designed for detecting bilateral responses (e.g. both hands pressing at once) and

works in two phases:

1. **Phase 1** — blocks until the first matching key arrives, respecting the timeout exactly like `GetKeyEventTS`.
2. **Phase 2** — waits up to 50 ms for any additional matching keys.

The second phase is necessary because human "simultaneous" presses are rarely truly simultaneous — the two KEY_DOWN events typically arrive 10–50 ms apart. Without this window, a non-blocking drain after the first key would miss the second key almost every time.

```
events, err := exp.Keyboard.GetKeyEventsTS(responseKeys, 3000)
if len(events) > 0 {
    firstKey := events[0].Key
    firstTS  := events[0].TimestampNS
    rtNS     := int64(firstTS - onset)
}
if len(events) == 2 {
    lagNS := int64(events[1].TimestampNS - events[0].TimestampNS)
}
```

In the common single-response case only one event is returned, at the cost of at most 50 ms of extra latency before the function returns. For the typical single-response trial, `GetKeyEventTS` avoids that overhead entirely.

Recording all presses over a fixed duration — `CollectKeyEventsTS`:

When the goal is to record *everything* the participant presses within a known time window (finger-tapping, free-response periods, stimulus-stream logging), use `CollectKeyEventsTS`. Unlike `GetKeyEventsTS`, it always runs for the full duration — it does not return early on the first keypress:

```
// Record every tap of F or J over 5 seconds
exp.Keyboard.Clear()
onset, _ := exp.ShowTS(stim)
events, err := exp.Keyboard.CollectKeyEventsTS(
    []control.Keycode{control.K_F, control.K_J}, 5000)
for _, ev := range events {
    rtNS := int64(ev.TimestampNS - onset)
    fmt.Printf("key %s at %d ms\n", ev.Key.KeyName(), rtNS/1_000_000)
}
```

Pass `keys = nil` to capture any key. Pass `durationMS = 0` to do a non-blocking drain of whatever is already in the queue. Returns an empty (non-nil) slice if nothing was pressed.

Querying whether a key is currently held — `IsPressed`:

`IsPressed` queries SDL's internal scancode state array and returns `true` if the key is physically depressed at the instant of the call, regardless of the event queue. It is useful in animation loops that need to react continuously to a held key, or to verify that the participant has released a key before starting a new trial:

```
for exp.Keyboard.IsPressed(control.K_SPACE) {
    // do something while SPACE is held
    time.Sleep(10 * time.Millisecond)
}
```

Because `IsPressed` does not consume queue events, it can be called freely alongside `GetKeyEventTS` without interfering with the event stream.

Measuring keypress duration — `WaitKeyReleaseTS`:

`WaitKeyReleaseTS` blocks until a KEY_UP event arrives for the specified key and returns its SDL3 hardware timestamp in nanoseconds. Together with the KEY_DOWN timestamp from `GetKeyEventTS`, this gives nanosecond-precision keypress duration:

```
key, downTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
// ... optionally update display while key is held ...
upTS, _ := exp.Keyboard.WaitKeyReleaseTS(key, 5000)
durationNS := upTS - downTS           // nanoseconds
durationMS := durationNS / 1_000_000  // milliseconds
```

This is the recommended approach for paradigms where press duration is a dependent variable (e.g., force-choice hold-to-respond, finger-tapping, or hold-triggered responses).

8.8.2 Mouse

```
// Block until any button
btn, err := exp.Mouse.WaitPress()

// RT in milliseconds from call site (mirrors Keyboard.WaitKeysRT)
btn, rt, err := exp.Mouse.WaitPressRT(3000)

// Hardware-precision version: returns the SDL event's nanosecond timestamp
btn, eventTS, err := exp.Mouse.GetPressEventTS(3000)

// Non-blocking poll
btn, err := exp.Mouse.Check()

// Query whether a button is physically held down right now
held := exp.Mouse.IsPressed(sdl.BUTTON_LEFT)

// Wait for MOUSE_BUTTON_UP; returns its SDL hardware timestamp
 ↪  (nanoseconds)
upTS, err := exp.Mouse.WaitButtonReleaseTS(btn, 5000)

// Current cursor position in center-based coordinates
x, y := exp.Mouse.Position()

// Show / hide cursor
```

```
exp.Mouse.ShowCursor(false)
```

The cursor is hidden by default. `Initialize()` (and therefore `NewExperimentFromFlags`) hides the mouse pointer, so it never sits on top of your stimuli. Mouse-driven paradigms must ask for it back, right after the experiment is created:

```
exp := control.NewExperimentFromFlags("Mouse Tracking", control.Black,
 ↪  control.White, 36)
defer exp.End()
exp.ShowCursor()   // participants need to see what they are pointing at
```

Setting `exp.CursorVisible = true` before `Initialize()` does the same thing. The participant-info dialog always shows the cursor while it is open, whatever the experiment's setting, since it is clicked.

Button values: `sdl.BUTTON_LEFT`, `sdl.BUTTON_MIDDLE`, `sdl.BUTTON_RIGHT`, `sdl.BUTTON_X1`, `sdl.BUTTON_X2`.

`IsPressed` and `WaitButtonReleaseTS` mirror the keyboard's `IsPressed` and `WaitKeyReleaseTS` and can be used to measure click-hold duration:

```
btn, downTS, _ := exp.Mouse.GetPressEventTS(-1)
upTS, _        := exp.Mouse.WaitButtonReleaseTS(btn, 5000)
durationMS     := int64(upTS-downTS) / 1_000_000
```

8.8.3 Multi-device input — `WaitAnyEventTS`

If you want to accept a response from *either* the keyboard or the mouse (or both), use `exp.WaitAnyEventTS` instead of calling `Keyboard` and `Mouse` separately:

```
onset, _ := exp.ShowTS(stim)

// Accept F or J key, or any mouse button click, up to 3 s
ev, err := exp.WaitAnyEventTS(
    []control.Keycode{control.K_F, control.K_J},
    true,   // catchMouse
    3000,
)

rtNS := int64(ev.TimestampNS - onset)  // hardware-precision RT, nanoseconds
rtMs := rtNS / 1_000_000

switch ev.Device {
case apparatus.DeviceKeyboard:
    fmt.Printf("key %d, RT %d ms\n", ev.Key, rtMs)
case apparatus.DeviceMouse:
    fmt.Printf("mouse button %d, RT %d ms\n", ev.Button, rtMs)
}
```

Pass `keys = nil` to accept any key. Pass `catchMouse = false` to ignore the mouse. On timeout, returns a zero `InputEvent` with `nil` error.

8.8.4 Key codes

Key codes are re-exported in `control` so experiment code never needs to import `go-sdl3` just to name a key. Coverage includes the full alphabet (`K_A` … `K_Z`), the digit row (`K_0` … `K_9`), the numeric keypad (`K_KP_0` … `K_KP_9`, `K_KP_ENTER`, `K_KP_PLUS`, `K_KP_MINUS`), navigation/control keys (`K_SPACE`, `K_ESCAPE`, `K_RETURN`, `K_BACKSPACE`, `K_TAB`, the arrow keys), and common punctuation (`K_MINUS`, `K_PLUS`, `K_EQUALS`, `K_LEFTBRACKET`, `K_RIGHTBRACKET`):

```
key, _ := exp.Keyboard.Wait()
if key == control.K_A { ... }
```

For a rarely-used code not in `defaults.go`, fall back to `go-sdl3/sdl` directly (e.g. `sdl.K_SEMICOLON`).

8.8.5 Response Devices

All of the input methods above are device-specific: `GetKeyEventTS` for keyboards, `GetPressEventTS` for the mouse, and so on. If the experiment should run equally well with different input hardware — a keyboard in one lab, a response box in another — device-specific calls scatter conditional logic throughout the trial loop.

`ResponseDevice` is a single interface that abstracts over all of them:

```
// in package io
type ResponseDevice interface {
    WaitResponse(ctx context.Context) (Response, error)
    DrainResponses(ctx context.Context) error
    Close() error
}
```

`WaitResponse` blocks until a response is detected and returns a `Response`:

```
type Response struct {
    Source  apparatus.DeviceKind  // which device fired
    Code    uint32         // key code, button index, or TTL bitmask
    RT      time.Duration  // elapsed from WaitResponse call
    Precise bool           // timing quality (see below)
}
```

Wrap any input device with the matching adapter:

```
// Keyboard (SDL hardware timestamps → Precise: true)
var rd apparatus.ResponseDevice = &apparatus.KeyboardResponseDevice{KB:
 ↪  exp.Keyboard}

// Mouse (SDL hardware timestamps → Precise: true)
```

```
var rd apparatus.ResponseDevice = &apparatus.MouseResponseDevice{M:
 ↪  exp.Mouse}

// TTL response box, e.g. MEGTTLBox (software poll → Precise: false)
box, _ := triggers.NewMEGTTLBox("/dev/ttyACM0")
var rd apparatus.ResponseDevice = apparatus.NewTTLResponseDevice(box,
 ↪  5*time.Millisecond)
```

All three look identical inside a trial loop:

```
onset, _ := exp.ShowTS(stim)
_ = rd.DrainResponses(ctx)        // clear stale presses from previous trial
resp, err := rd.WaitResponse(ctx)
if err != nil { /* ESC, timeout, etc. */ }

rtMs := resp.RT.Milliseconds()     // always valid
```

8.8.5.1 Timing precision and the `Precise` flag

The `Precise` field tells you whether `RT` came from a nanosecond hardware event timestamp or a software poll:

Device	`Precise`	RT accuracy
Keyboard, Mouse, Gamepad	`true`	SDL3 hardware event timestamp, nanosecond resolution — identical to using `GetKeyEventTS` directly
MEGTTLBox, DLPIO8	`false`	`time.Now()` at poll detection; accuracy bounded by poll interval (default 5 ms)

When `Precise` is `true`, the RT in the `Response` is computed from `sdl.TicksNS()` captured at the `WaitResponse` call versus the SDL3 hardware event timestamp — the same nanosecond clock used by `Screen.FlipTS`. It is therefore suitable for stimulus-onset-locked RT:

```
onset, _ := exp.ShowTS(stim)
resp, _ := rd.WaitResponse(ctx)
if resp.Precise {
    // resp.RT was computed from SDL hardware timestamps: nanosecond
     ↪  accuracy
    rtFromOnset := time.Duration(int64(onset) + resp.RT.Nanoseconds())  //
 ↪  approximate
}
```

When `Precise` is `false`, `RT` is still a valid elapsed duration — it just has ~poll-interval jitter (5 ms by default). For typical behavioral RT tasks this is acceptable; for EEG/MEG

experiments requiring sub-millisecond synchrony, use the TTL output path for stimulus marking and treat RT as an approximate measure or use a hardware response box with sub-ms timestamping.

8.9 8. Data Collection

8.9.1 Output files

When `exp.End()` is called, two files are written to `~/goxpy_data/`:

File	Contents
`<expName>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>.csv`	Pure CSV data rows — directly importable by Excel, R, or pandas
`<expName>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>-info.txt`	Session metadata: start/end time, hostname, framework version, participant info, the machine and OS it ran on (model, kernel, desktop/compositor, CPU, every GPU and its driver, sound server and version), and the display and audio configuration SDL actually opened

Both files open automatically on `Initialize()` and are flushed to disk when `exp.End()` is called. Any spaces in `<expName>` are replaced by `-` so the resulting filenames never contain whitespace.

8.9.2 Choosing the output directory

By default, data files are written to `~/goxpy_data/` (the folder name is the `results.DataFileDirectory` constant, appended to the user's home directory). To write them elsewhere, call `exp.SetOutputDirectory()` **before** `Initialize()` opens the files:

```
exp := control.NewExperiment("My Experiment", control.Black, control.White,
 ↪  32)
exp.SetOutputDirectory("/path/to/mydata")  // must come before Initialize()
if err := exp.Initialize(); err != nil { log.Fatal(err) }
defer exp.End()
```

The directory is created if it does not exist.

Because `NewExperimentFromFlags` calls `Initialize()` for you, overriding the directory requires the lower-level `NewExperiment(...) + Initialize()` path shown above rather than `NewExperimentFromFlags`.

8.9.3 Declaring column names

Call this once, before the trial loop:

```
exp.AddDataVariableNames([]string{"condition", "response", "rt_ms",
 ↪  "correct"})
```

Subject ID is always prepended automatically — do not include it in the name list.

8.9.4 Adding rows

```
exp.Data.Add(condition, response, rt, correct)
```

Fields are written in the same order as the variable names. Any type that has a meaningful `fmt.Sprint` representation works: `int`, `float64`, `bool`, `string`. Fields containing commas or quotes are escaped per RFC 4180.

8.9.5 Example CSV output

```
subject_id,condition,response,rt_ms,correct
3,"congruent","F",412,true
3,"incongruent","J",538,false
```

Numbers and booleans are unquoted; strings are always double-quoted.

8.9.6 Saving mid-experiment

For long experiments it is good practice to call `exp.Data.Save()` after each block. This flushes both files to disk so that data up to that point is not lost if the experiment crashes.

8.10 9. Stimuli: Lifecycle and Preloading

8.10.1 GPU textures are lazily allocated

Creating a stimulus (e.g., `stimuli.NewTextLine(...)`) is cheap — it allocates a Go struct and stores the parameters, but does no GPU work. The SDL texture is created on the **first call to `Draw`**. This means:

- You can safely create all your stimuli before the run loop.
- The first presentation of each stimulus will be slightly slower due to texture upload.

For timing-sensitive presentations, preload explicitly:

```
stimuli.PreloadVisualOnScreen(exp.Screen, myStim)
// or, for a slice:
stimuli.PreloadAllVisual(exp.Screen, []stimuli.VisualStimulus{stim1, stim2,
 ↪  stim3})
```

Call this after `exp.Run` starts (the renderer is ready), but before the critical section:

```
err := exp.Run(func() error {
    // Preload everything while showing instructions
    exp.ShowInstructions("Loading, please wait...")
    stimuli.PreloadAllVisual(exp.Screen, stimSlice)

    // Now timing-sensitive trials
    for _, stim := range stimSlice { ... }
    return control.EndLoop
})
```

8.10.2 Releasing textures

If you generate many stimuli dynamically (e.g., hundreds of unique text strings), call `stim.Unload()` after each trial to free the GPU memory:

```
for _, word := range wordList {
    stim := stimuli.NewTextLine(word, 0, 0, control.White)
    exp.Show(stim)
    exp.Keyboard.Wait()
    stim.Unload()  // release GPU texture
}
```

For a fixed, small set of stimuli created once and reused throughout the experiment, you never need to call `Unload` — `exp.End()` handles cleanup.

8.10.3 Writing a custom stimulus

Any type that implements `VisualStimulus` can be passed to `exp.Show`:

```
type VisualStimulus interface {
    Present(screen *apparatus.Screen, clear, update bool) error
    Preload() error
    Unload() error
    Draw(screen *apparatus.Screen) error
    GetPosition() sdl.FPoint
    SetPosition(pos sdl.FPoint)
}
```

Embed `stimuli.BaseVisual` to get the position methods and no-op `Preload`/`Unload` for free, then implement only `Draw` and `Present`:

```
type MyStimulus struct {
    stimuli.BaseVisual
    // your fields
}

func (m *MyStimulus) Draw(screen *apparatus.Screen) error {
    // draw using screen.Renderer SDL calls
    return nil
```

```
}

func (m *MyStimulus) Present(screen *apparatus.Screen, clear, update bool)
 ↪  error {
    return stimuli.PresentDrawable(m, screen, clear, update)
}
```

8.10.4 Interactive widgets

Two stimuli present a UI and collect a structured response from the participant in a blocking loop. Neither is timing-critical, so they use `sdl.WaitEvent` rather than VSYNC polling.

`Menu` — a numbered, keyboard-navigable list:

```
m := stimuli.NewMenu([]string{"Beginner", "Expert", "Practice run"})
// Optional customisation:
m.HighlightColor = sdl.Color{R: 255, G: 220, B: 0, A: 255}

idx, err := m.Get(exp.Screen, exp.Keyboard, 0)
// idx is 0-based; -1 + sdl.EndLoop on ESC/quit
```

The selected item is shown with a `>` prefix in `HighlightColor`; all others use `TextColor`. Navigation: UP/DOWN arrows move the highlight; ENTER or SPACE confirms; digit keys 1–9 (0 for tenth) select directly without requiring confirmation.

`ChoiceGrid` — a grid of labelled buttons, activated by mouse click or matching key:

```
cg := stimuli.NewChoiceGrid([]string{"B","C","D","F","G"}, 3, "Pick 3
 ↪  letters:")
selections, err := cg.Get(exp.Screen, exp.Keyboard)
// selections is []string in press order
```

8.11 10. High-Precision Streams (RSVP)

For paradigms that present stimuli at high speed — RSVP, attentional blink, priming — the standard `exp.Show` / `exp.Wait` cycle is not precise enough. The stream functions provide frame-accurate presentation:

- GC is **disabled** for the duration of the stream.
- Every onset and offset is aligned to a **VSYNC boundary**.
- A `TimingLog` records the predicted and actual onset for each stimulus.
- All keyboard and mouse events that occur during the stream are captured.

8.11.1 Text stream

The simplest entry point:

```
words := []string{"CHAIR", "RIVER", "TIGER", "CLOCK", "STONE"}
on  := 150 * time.Millisecond
off :=  50 * time.Millisecond

events, logs, err := stimuli.PresentStreamOfText(
    exp.Screen, words, on, off,
    0, 0,           // center of screen
    control.White,
)
```

8.11.2 Image / mixed stimulus stream

For images or any `VisualStimulus`:

```
stims := []stimuli.VisualStimulus{pic1, fixation, pic2, fixation}

// Regular cadence (all stimuli share the same on/off duration)
elements := stimuli.MakeRegularVisualStream(stims, 100*time.Millisecond, 0)

// Irregular cadence (individual onset times and durations, in ms)
elements, err := stimuli.MakeVisualStream(stims, onsetMs, durationMs)

events, logs, err := stimuli.PresentStreamOfImages(exp.Screen, elements, 0,
 ↪  0)
```

8.11.3 Reading events from the stream

Each `UserEvent` carries two timestamps:

- `Timestamp` — a `time.Duration` relative to stream start, Go monotonic clock, millisecond precision. Useful for quick inspection.
- `TimestampNS` — the SDL3 hardware event timestamp in nanoseconds, same clock as `Screen.FlipTS`. Use this for sub-millisecond RT computation.

```
for _, ev := range events {
    if ev.Event.Type == sdl.EVENT_KEY_DOWN {
        key := ev.Event.KeyboardEvent().Key
        t   := ev.Timestamp.Milliseconds()  // ms from stream start (coarse)
        fmt.Printf("key %d pressed at %d ms\n", key, t)
    }
}
```

Use `stimuli.FirstKeyPress` to find the first matching key-down event without writing a loop:

```
if ev, ok := stimuli.FirstKeyPress(events, sdl.K_SPACE); ok {
    fmt.Printf("Space pressed at %d ms\n", ev.Timestamp.Milliseconds())
}
```

Both `ev.Timestamp` (Go clock, ms precision) and `ev.TimestampNS` (SDL3 hardware, nanoseconds) are available on the returned `UserEvent`.

8.11.4 Computing RT from stream events

Because `UserEvent.TimestampNS` and `TimingLog.OnsetNS` are on the same SDL nanosecond clock, reaction time from a specific stimulus is exact:

```
for _, ev := range events {
    if ev.Event.Type == sdl.EVENT_KEY_DOWN {
        for _, l := range logs {
            if ev.TimestampNS >= l.OnsetNS && (l.OffsetNS == 0 ||
             ↪  ev.TimestampNS < l.OffsetNS) {
                rtNS := int64(ev.TimestampNS - l.OnsetNS)
                fmt.Printf("RT from stimulus %d: %d ms\n", l.Index, rtNS/1_
↪  000_000)
            }
        }
    }
}
```

8.11.5 Reading the timing log

```
for i, l := range logs {
    // Read the authoritative SDL-clock fields, not the Go-clock
     ↪  ActualOnset/
    // ActualOffset diagnostics. Achieved on-screen duration vs target =
     ↪  jitter.
    shownMs := int64(l.OffsetNS-l.OnsetNS) / 1_000_000
    jitter := shownMs - l.TargetOn.Milliseconds()
    fmt.Printf("stimulus %d: target %d ms, shown %d ms, jitter %d ms\n",
        i, l.TargetOn.Milliseconds(), shownMs, jitter)
}
```

`OnsetNS` and `OffsetNS` give the SDL3 nanosecond timestamps of the actual VSYNC flips that turned each stimulus on and off — the authoritative timing record, on the same clock as input events. `ActualOnset`/`ActualOffset` are Go-clock diagnostics on a different timebase; never subtract them from an event timestamp or use them as onsets. `OnsetNS` is zero only for a stimulus that was never displayed (zero on-frames); `OffsetNS` is the takedown flip, synthesised from the onset for the final element of a contiguous stream (which has no ISI flip of its own) so it stays on the SDL clock rather than reading a frame early.

Jitter below ±1 frame (±8–17 ms depending on monitor) is normal and expected. Larger jitter indicates system load or GPU driver issues.

8.11.6 Platform timing notes

`PresentStreamOfImages` provides the best timing accuracy that the operating system and display driver allow. The characteristics differ by platform:

Linux

- Without a compositing window manager, SDL3's `Present()` blocks until the next VSYNC boundary. Measured trigger-to-light scatter: **0.083 ms** on bare Xorg with openbox, **0.113 ms** on a KMS/DRM console (Precision 5490, 60 Hz, ~590 trials each, 2026-08-08).
- With a Wayland compositor, the compositor controls buffer swaps. Measured on the same machine the same night: **1.344 ms** — sixteen times worse — with 7 % of flips more than 1 ms off the frame grid.
- For the most reliable timing on Linux, disable the compositor or use a plain X11 session. This is the single largest effect available to you; see Timing tests.

macOS (Metal)

- The macOS WindowServer compositor is **always active** — exclusive fullscreen does not bypass it. SDL3 submits each frame to the compositor, which forwards it to the display at the next VSYNC.
- `FlipTS` captures `sdl.TicksNS()` immediately after `Present()` returns; this reflects when the frame was *submitted to the compositor*, not when photons reached the screen. The additional compositor latency is typically **1–2 frames** (~17-33 ms at 60 Hz).

Windows

- In **exclusive fullscreen** mode (`fullscreen=true`), the DWM (Desktop Window Manager) compositor is bypassed. Timing behaviour is similar to Linux without a compositor — jitter is typically **< 1 ms**.
- In **windowed mode**, DWM is always active and adds approximately one frame of compositor latency. Jitter is typically **1–3 ms**.
- For the most reliable timing on Windows, always run in fullscreen mode.

What `OnsetNS` measures

`TimingLog.OnsetNS` is recorded immediately after `screen.FlipTS()` returns, not at the moment photons reach the screen. On top of the compositor latency noted above, there is additional hardware pipeline latency (GPU scan-out, cable propagation, display panel response) of typically **0–2 frames**. This latency is constant across trials and does not affect within-experiment RT precision, but it must be accounted for if absolute (photodiode-verified) onset times are required.

Minimum duration

Because presentation is VSYNC-locked, durations shorter than one frame period (e.g. < 16.7 ms at 60 Hz) are rounded up to the nearest whole frame. A stimulus requested for 50 ms on a 60 Hz display is shown for exactly **3 frames (50.0 ms)**; one requested for 60 ms is shown for **4 frames (66.7 ms)**.

8.11.7 Audio stream

The same model applies to sounds:

```
// All tones must be pre-loaded before the stream
for _, t := range tones {
    t.PreloadDevice(exp.AudioDevice)
}

elements := stimuli.MakeRegularSoundStream(tones, 200*time.Millisecond,
 ↪  100*time.Millisecond)
events, logs, err := stimuli.PlayStreamOfSounds(elements)
```

8.11.8 Mixed stream

`PresentStreamOfStimuli` presents a single **sequential** stream that mixes visual and audio stimulus types. It reuses the VSYNC-locked visual loop, so visual elements are frame-accurate; audio elements are triggered once at the slot's first flip and the previous frame is **held** for the slot (the last stream visual is re-rendered every frame, so it stays rock-steady rather than relying on GPU buffer persistence) — place a visual just before a sound to keep it visible while the sound plays.

```
tone.PreloadDevice(exp.AudioDevice) // audio elements must be device-bound
 ↪  first

elements := []stimuli.StreamElement{
    {Stimulus: stimuli.NewTextLine("Ready?", 0, 0, control.White),
     ↪  DurationOn: 600 * time.Millisecond, DurationOff: 200 *
     ↪  time.Millisecond},
    {Stimulus: pic,  DurationOn: 800 * time.Millisecond}, // stays on screen
     ↪  during the tone
    {Stimulus: tone, DurationOn: 400 * time.Millisecond, DurationOff: 200 *
     ↪  time.Millisecond},
}
events, logs, err := stimuli.PresentStreamOfStimuli(exp.Screen, elements,
 ↪  0, 0)
```

Concurrent audio-visual overlap (a sound and an animating visual at the same instant) is not yet supported; the stream is strictly one slot at a time.

8.11.9 Per-stimulus onset triggers

To emit a hardware TTL — or run any action — aligned to each stimulus onset, use `PresentStreamOfStimuliHooks(screen, elements, x, y, onFrame, onOnset)`. The `onOnset` `OnsetCallback` (`func(index int, onsetNS uint64) error`) fires once per element **immediately after** its onset VSYNC flip, with `onsetNS == TimingLog[index].OnsetNS`. This is the *post-flip* counterpart to `FrameCallback`, which runs one frame earlier (pre-flip): a trigger emitted from `onOnset` coincides with the logged onset instead of leading it by a frame — the right hook for EEG/MEG synchronisation. Keep it short

and non-blocking (emit an edge, clear it from a later `FrameCallback`; never sleep or call a blocking `Pulse`). The pure-audio equivalent is `PlayStreamOfSoundsHook(elements, onOnset)`, firing at the `Play()` instant. See the API reference and the `test_stream_trigger` example.

For the usual case — a photodiode square and a TTL code at each onset — arm the library's onset marker from the `FrameCallback` instead of writing to a device yourself: `if ctx.OnPhase && ctx.Frame == 0 { exp.Marker.Arm(code) }`. The marker writes the code right after the onset flip and clears it on a timer (see Onset markers in Section 17).

8.12 11. Audio

8.12.1 Sounds from files or embedded bytes

```
// From a file
snd := stimuli.NewSound("ping.wav")
snd.PreloadDevice(exp.AudioDevice)
snd.Play()
snd.Wait()   // block until playback finishes

// From embedded bytes (go:embed)
//go:embed assets/beep.wav
var beepWav []byte

snd := stimuli.NewSoundFromMemory(beepWav)
snd.PreloadDevice(exp.AudioDevice)
snd.Play()
```

8.12.2 Procedural tones

```
tone := stimuli.NewTone(1000.0, 200*time.Millisecond, 200) // 1 kHz, 200
 ↪  ms, medium volume
tone.PreloadDevice(exp.AudioDevice)
tone.Play()
```

Volume is 0–255. Use `PreloadDevice` once; then call `Play` repeatedly.

8.12.3 Segment playback with fade

Play only part of a longer sound, with smooth fade-in and fade-out:

```
snd.PlaySegment(1.5, 3.0, 0.05) // play 1.5–3.0 s, 50 ms ramps
```

This is useful for stimuli like vowels extracted from longer recordings.

8.12.4 Built-in feedback sounds

```
exp.Audio.PlayBuzzer()   // play the embedded error/incorrect sound
 ↪  asynchronously
exp.Audio.PlayCorrect()  // play the embedded correct/reward sound
 ↪  asynchronously
```

Both return immediately; the sound plays in the background.

8.12.5 Synchronous vs asynchronous

`snd.Play()` starts playback and returns immediately. `snd.Wait()` blocks until it finishes. For trial timing where you need to know when a sound ended, call both:

```
snd.Play()
doSomethingElse()
snd.Wait()  // wait here if needed
```

For fire-and-forget feedback sounds, call `snd.Play()` without `snd.Wait()`.

8.12.6 Microphone recording

goxpyriment can capture audio from the system microphone via SDL3's recording API. The primary use cases are:

- **Picture naming** — measure the latency from image onset to the participant's first vocalization.
- **Vocal shadowing** — measure the latency between a spoken or tonal stimulus and the participant's repetition of it.

Open the microphone once, before the trial loop:

```
// nil uses the default recording spec: F32LE mono 44100 Hz
if err := exp.OpenMicrophone(nil); err != nil {
    log.Fatalf("microphone unavailable: %v", err)
}
// exp.Microphone is now ready; it is closed automatically by exp.End()
```

To select a custom format (e.g. higher sample rate or stereo):

```
spec := &sdl.AudioSpec{Format: sdl.AUDIO_F32LE, Channels: 1, Freq: 48000}
exp.OpenMicrophone(spec)
```

To list all connected recording devices and open a specific one:

```
devices, _ := apparatus.GetRecordingDevices()
for _, d := range devices {
    name, _ := d.Name()
    fmt.Println(d, name)
}
mic, _ := apparatus.NewMicrophoneFromDevice(devices[0], nil)
```

```
exp.Microphone = mic
```

8.12.7 Voice key — detecting vocal onset

A `VoiceKey` monitors the microphone and fires when the amplitude exceeds a threshold. It is the goxpyriment equivalent of a hardware voice key or PsychoPy's `OnsetVoiceKey`.

```
vk := apparatus.NewVoiceKey(exp.Microphone, 0.03, 128)
// threshold = 0.03 (RMS amplitude 0–1 for F32LE)
// windowSz  = 128 samples ≈ 2.9 ms at 44100 Hz
```

Per-trial pattern:

```
// 1. Arm: flush the mic buffer and start capturing.
//    Returns the SDL3 nanosecond timestamp of capture start.
captureStartNS, _ := vk.Arm()

// 2. Present the stimulus.
imageOnsetNS, _ := exp.ShowTS(picture)   // picture naming
// – or –
stimulusNS, _ := tone.PlaySyncedWithFlip(exp.Screen)  // shadowing

// 3. Wait for the vocal response (2-second window).
onsetNS, pcm, err := vk.WaitOnset(captureStartNS, 2000)
if errors.Is(err, apparatus.ErrVoiceTimeout) {
    // participant did not respond in time
}

// 4. Compute RT (both timestamps are on the SDL3 nanosecond clock).
rtMs := int64(onsetNS - imageOnsetNS) / 1_000_000
```

`WaitOnset` also returns `pcm` — the raw PCM bytes captured during the trial. Save them as WAV for offline verification (see below).

Threshold calibration. A value of 0.02–0.05 works in a quiet room. If you observe:

- **Spuriously short RTs (< 100 ms)** — the threshold is too low; it is triggering on breath noise, lip smacks, or microphone bleed from the speakers. Raise the threshold or use headphones.
- **`detected = false` (missed responses)** — the threshold is too high for soft or breathy voices. Lower it.

Always verify using the saved WAV files rather than adjusting the threshold blindly.

8.12.8 Timing: microphone and screen on the same clock

Both `imageOnsetNS` (from `exp.ShowTS` or `screen.FlipTS`) and the voice onset computed by `WaitOnset` use `sdl.TicksNS()` — SDL3's monotonic nanosecond clock. No cross-clock conversion is needed:

```
vk.Arm()  ──────────────────────────────── captureStartNS  (sdl.TicksNS())
                │
                │  exp.ShowTS(picture)
                │  ↳ VSYNC flip ──────── imageOnsetNS   (sdl.TicksNS())
                │
                │  [participant starts speaking at sample N]
                │
                └── onsetNS = captureStartNS + N × 1e9 / sampleRate
                                                    (sdl.TicksNS() domain)

RT = (onsetNS − imageOnsetNS) / 1 000 000  [ms]
```

Call `vk.Arm()` just before showing the stimulus so the microphone is already capturing when the image appears. There is no need to arm it long in advance.

8.12.9 Saving WAV files for verification

Always save a WAV file for each trial and spot-check a random subset after the session. Voice keys can fire on breaths, lip smacks, or room noise; the WAV is the ground truth.

```
apparatus.WriteWAV(
    fmt.Sprintf("sub-%03d_trial-%02d.wav", exp.SubjectID, trial),
    exp.Microphone.Spec,
    pcm,
)
```

`WriteWAV` supports F32LE, S16LE, S32LE, and U8 formats and writes a standard RIFF/WAV file that any audio editor can open.

For post-hoc re-analysis of saved files (e.g. applying a different threshold), use `apparatus.ScanOnset`:

```
sampleIndex := apparatus.ScanOnset(pcm, newThreshold, 128)
if sampleIndex >= 0 {
    revisedOnsetNS = apparatus.SampleOnsetNS(captureStartNS, sampleIndex,
 ↪  int(exp.Microphone.Spec.Freq))
}
```

8.12.10 Acoustic feedback (shadowing and AV tasks)

When audio plays through laptop built-in speakers, the microphone picks up the playback signal and the voice key triggers on it rather than on the participant's voice. This produces spuriously short latencies (sometimes < 0 ms if the mic captures the speaker before the amp threshold is crossed).

Always use headphones for playback in any task that simultaneously records and plays audio.

If headphones are unavailable, a simple analysis-side fix is to reject any onset that falls within a guard interval after stimulus onset (e.g. discard `onsetNS - stimulusNS < 100 ms`).

8.13 12. Experimental Design and Randomization

8.13.1 When to use `design.Block` / `design.Trial`

The `design` package provides a structured `Experiment → Block → Trial` hierarchy with string-keyed factors. Use it when:

- You have a multi-block experiment and want `exp.Design.Blocks` to drive the loop.
- You want between-subjects Latin-square counterbalancing (`AddBWSFactor`).

For simple single-block experiments, a plain Go slice of a custom struct is often more readable (and type-safe). Both approaches are valid.

8.13.2 Building a design

```
block := design.NewBlock("main")
for _, cond := range []string{"congruent", "incongruent"} {
    t := design.NewTrial()
    t.SetFactor("condition", cond)
    t.SetFactor("soa", 200)
    block.AddTrial(t, 20, false)  // 20 copies, appended in order
}
block.ShuffleTrials()
exp.AddBlock(block, 1)
```

Iterate at runtime:

```
for _, blk := range exp.Design.Blocks {
    for _, trial := range blk.Trials {
        cond := trial.GetFactor("condition").(string)
        soa  := trial.GetFactor("soa").(int)
        // ...
    }
}
```

8.13.3 Randomization helpers

The `design` package provides randomization functions that work on any slice type:

```
// In-place shuffle (generic — works on any []T)
design.ShuffleList(mySlice)

// Random integer in [a, b] inclusive
design.RandInt(500, 1500)

// Random element from a slice
word := design.RandElement(wordList)
```

```
// Weighted coin flip
if design.CoinFlip(0.75) { /* 75% chance */ }

// Truncated normal sample in [a, b]
design.RandNorm(800.0, 1200.0)

// Shuffled integer range [first, last]
order := design.RandIntSequence(0, len(stimuli)-1)
```

8.13.4 Between-subjects counterbalancing

```
// Register once during setup
exp.AddBWSFactor("mapping", []interface{}{"F=left/J=right",
 ↪  "F=right/J=left"})

// At runtime — returns the condition assigned to this subject's ID
mapping := exp.GetPermutedBWSFactorCondition("mapping").(string)
```

The assignment follows a balanced Latin square so that conditions rotate across subjects (subject 1 → condition A, subject 2 → condition B, subject 3 → condition A, …).

8.13.5 Constrained shuffling

For designs where you need to prevent the same condition from appearing more than N times in a row, use block-level `AddTrial` with `randomPosition: true`, which inserts each trial at a random position rather than appending:

```
for _, cond := range conditions {
    t := design.NewTrial()
    t.SetFactor("condition", cond)
    block.AddTrial(t, 5, true)  // 5 copies each, randomly interleaved
 ↪  during construction
}
```

8.14 13. Embedding Stimuli and Trial Lists in the Executable

One of Go's most useful deployment features is `//go:embed`: a compiler directive that copies files from disk into the binary at build time. The result is a **single self-contained executable** that carries all its images, sounds, and trial-list CSV files with it — no separate `assets/` folder, no installer, no missing-file errors on lab computers.

> **When to embed.** Embedding is the right choice for stimuli that are fixed at design time: images, sounds, and pre-generated trial lists. It is not ap-

propriate for stimuli you generate procedurally at runtime (textures drawn with `Canvas`, tones created with `NewTone`, text rendered from `NewTextLine`), or for data files that are written during the experiment.

8.14.1 13.1 Project layout

Collect all static assets in a subdirectory alongside `main.go`:

```
myexperiment/
  main.go
  assets/
    stim_A.png
    stim_B.png
    feedback_correct.wav
    trials.csv
```

8.14.2 13.2 Embedding individual image and sound files

Import `_ "embed"` (blank identifier, needed to activate the directive) and declare a `[]byte` variable per file:

```go
package main

import _ "embed"

//go:embed assets/stim_A.png
var stimAImg []byte

//go:embed assets/stim_B.png
var stimBImg []byte

//go:embed assets/feedback_correct.wav
var feedbackWav []byte
```

Pass the byte slices to the `FromMemory` constructors:

```go
import "github.com/chrplr/goxpyriment/stimuli"

stimA := stimuli.NewPictureFromMemory(stimAImg, 0, 0)
stimB := stimuli.NewPictureFromMemory(stimBImg, 0, 0)
feedback := stimuli.NewSoundFromMemory(feedbackWav)
```

These constructors behave identically to `NewPicture` and `NewSound`; textures are still allocated lazily on first `Draw`, and `PreloadAllVisual` / `PreloadDevice` work the same way.

8.14.3 13.3 Embedding a whole stimulus directory

When you have many images (e.g. one per trial) it is impractical to declare a variable for each one. Embed the whole directory as an `embed.FS` and iterate over it at runtime:

```
package main

import "embed"

//go:embed assets/stimuli
var stimuliFS embed.FS
```

Load every PNG in the directory into a map:

```
import (
    "fmt"
    "github.com/chrplr/goxpyriment/stimuli"
)

pics := map[string]*stimuli.Picture{}

entries, err := stimuliFS.ReadDir("assets/stimuli")
if err != nil {
    log.Fatal(err)
}
for _, e := range entries {
    if e.IsDir() {
        continue
    }
    data, err := stimuliFS.ReadFile("assets/stimuli/" + e.Name())
    if err != nil {
        log.Fatal(err)
    }
    pics[e.Name()] = stimuli.NewPictureFromMemory(data, 0, 0)
}

// Use by filename:
exp.Show(pics["face_happy.png"])
```

Or load a specific file by name:

```
data, _ := stimuliFS.ReadFile(fmt.Sprintf("assets/stimuli/face_%s.png",
 ↪  condition))
stim := stimuli.NewPictureFromMemory(data, 0, 0)
```

8.14.4 13.4 Embedding a CSV trial list

A CSV file that defines trial order and parameters is a natural fit for embedding. The workflow is: embed → read into `[]byte` → wrap in `bytes.NewReader` → parse with `encoding/csv`.

Example `assets/trials.csv`:

```
condition,soa_ms,correct_key
congruent,200,F
incongruent,200,J
congruent,400,F
incongruent,400,J
```

Embedding and parsing:

```
package main

import (
    "bytes"
    _ "embed"
    "encoding/csv"
    "log"
    "strconv"
)

//go:embed assets/trials.csv
var trialsCSV []byte

type trial struct {
    condition  string
    soaMs      int
    correctKey string
}

func loadTrials() ([]trial, error) {
    r := csv.NewReader(bytes.NewReader(trialsCSV))
    records, err := r.ReadAll()
    if err != nil {
        return nil, err
    }
    var trials []trial
    for i, rec := range records {
        if i == 0 {
            continue // skip header row
        }
        soa, _ := strconv.Atoi(rec[1])
        trials = append(trials, trial{
            condition:  rec[0],
            soaMs:      soa,
            correctKey: rec[2],
        })
    }
    return trials, nil
}
```

For **tab-separated** files, set `r.Comma = '\t'` immediately after creating the reader.

For **semicolon-separated** files (common in European locales), set `r.Comma = ';'`.

Use the loaded trials in the experiment loop:

```
trials, err := loadTrials()
if err != nil {
    log.Fatalf("loading trials: %v", err)
}

err = exp.Run(func() error {
    exp.ShowInstructions("Press F or J to respond.")
    for _, t := range trials {
        exp.Blank(500)
        stim := stimuli.NewTextLine(t.condition, 0, 0, control.White)
        key, rt, _ := exp.ShowAndGetRT(stim, []control.Keycode{control.K_F,
↪   control.K_J}, 3000)
        correct := string(key.KeyName()) == t.correctKey
        exp.Data.Add(t.condition, t.soaMs, rt, correct)
    }
    return control.EndLoop
})
```

8.14.5 13.5 Mixing embedded and runtime-generated stimuli

Embedding is selective — embed only what is fixed, generate the rest at runtime:

```
//go:embed assets/mask.png
var maskImg []byte

// Fixed stimulus: loaded from embedded bytes
mask := stimuli.NewPictureFromMemory(maskImg, 0, 0)

// Dynamic stimulus: generated from trial parameters
target := stimuli.NewTextLine(word, 0, 0, control.White)
```

8.14.6 13.6 Build and deployment

Nothing changes in the build command:

```
go build -o myexperiment .
```

The `assets/` directory must exist on disk at build time but is **not needed at runtime**. Copy the single binary to any lab computer running the same OS:

```
# Cross-compile for Windows from Linux
GOOS=windows GOARCH=amd64 go build -o myexperiment.exe .

# Cross-compile for macOS (Apple Silicon)
```

```
GOOS=darwin GOARCH=arm64 go build -o myexperiment-mac .
```

The embedded assets are included in all cross-compiled outputs automatically.

> **Binary size.** Each embedded file adds its uncompressed size to the binary. A typical experiment with a few dozen PNG images and WAV files will add 5–50 MB — acceptable for a self-contained research tool. For very large stimulus sets (hundreds of high-resolution images, video files), consider distributing them as a separate archive alongside the binary instead of embedding.

8.15 14. Animated Stimuli

Three functions run self-contained VSYNC-locked animation loops. All three:

- Disable GC for the duration of the loop.
- Drain the SDL event queue before the first frame.
- Return a `MotionResult` with the response key, mouse button, and reaction time.

```
type MotionResult struct {
    Key    sdl.Keycode  // interrupt key pressed (0 if none)
    Button uint8        // mouse button (0 if none)
    RTms   int64        // ms from first frame to response; total elapsed on
 ↪  timeout
}
```

8.15.1 Moving dot cloud

```
result, err := stimuli.PresentMovingDotCloud(
    exp.Screen,
    100,                          // number of dots
    3.0,                          // dot radius (px)
    150.0,                        // cloud radius (px)
    control.Origin(),             // center at (0, 0)
    150.0,                        // speed in px/sec
    5000,                         // max duration ms (0 = infinite)
    []control.Keycode{control.K_SPACE}, // interrupt keys
    false,                        // catch mouse?
    control.White,                // dot color
    control.Color{A: 0},          // background (transparent)
)
fmt.Printf("RT: %d ms\n", result.RTms)
```

8.15.2 Drifting grating

```
result, err := stimuli.PresentMovingGrating(
    exp.Screen,
    400, 400,              // width, height (px)
    control.Origin(),      // center
    0.0,                   // orientation (degrees; 0 = vertical bars
    ↪  drifting right)
    0.05,                  // spatial frequency (cycles per pixel)
    2.0,                   // temporal frequency (Hz)
    0.8,                   // contrast [0, 1]
    0.5,                   // mean luminance [0, 1]
    3000,                  // max duration ms
    []control.Keycode{control.K_SPACE},
    false,
)
```

8.15.3 Drifting Gabor patch

```
result, err := stimuli.PresentMovingGabor(
    exp.Screen,
    200,                   // bounding box size (px); at least 6×sigma
    30.0,                  // sigma (Gaussian SD in px)
    control.Origin(),      // center
    45.0,                  // orientation
    0.05,                  // spatial frequency (cycles/px)
    3.0,                   // temporal frequency (Hz)
    0.9,                   // contrast
    0.5,                   // mean luminance
    0,                     // max duration (0 = until response)
    []control.Keycode{control.K_SPACE},
    false,
)
```

8.16 15. Video

goxpyriment plays video in two formats, and the choice is a deliberate trade between **timing determinism** and **file size / convenience**. Neither is a general-purpose media player: both exist to put moving images on screen at a known time.

	`.gv`	MPEG-1 (`.mpg`)
Type	`stimuli.GvVideo`, `stimuli.PlayGv`, `media.Movie`	`stimuli.Video`

	.gv	MPEG-1 (.mpg)
Frame cadence	VSYNC-locked, GC disabled	wall-clock, drops frames under load
Decode cost	constant per frame	varies with I/P/B frame type
Audio	none	MP2 soundtrack
Seeking	O(1), every frame independent	sequential only
Size (10 s, 960×400)	~41 MB	~2 MB
Use for	measured stimuli	instructions, fillers, long clips

Rule of thumb: if the onset time of a frame ends up in your data file, use `.gv`. If the participant is just watching something between trials, MPEG-1 is far more convenient.

8.16.1 Start here

Almost everyone arrives with an `.mp4` and one of two goals.

"I want to play a movie, with its sound." You need MPEG-1, because `.gv` has no audio track. Convert once, then play:

```
ffmpeg -i movie.mp4 -c:v mpeg1video -b:v 1500k \
                    -c:a mp2 -b:a 192k -ar 44100 -ac 2 \
                    -f mpeg movie.mpg
```

```
v, _ := stimuli.NewVideo(exp.Screen, "movie.mpg")
defer v.Close()
v.PreloadDevice(exp.AudioDevice)   // without this it plays silently
v.Play()
```

Full walkthrough in 15.2. Frame onsets are approximate — fine for instructions and filler, not for measured stimuli.

"I need frame-accurate timing, and maybe a TTL trigger." You need `.gv`. Convert once, then play:

```
make gv-convert
_build/gv-convert movie.mp4 movie.gv    # needs ffmpeg for non-MPEG-1 input
```

```
events, logs, err := stimuli.PlayGv(exp.Screen, "movie.gv", 0, 0)
```

`logs` tells you afterwards whether any frame was late. Full walkthrough, including how to fire a trigger on a chosen frame, in 15.1. If you also need sound, play a separate `stimuli.Sound` alongside it — which additionally gives you independent control of audio onset (see Section 11).

8.16.2 15.1 `.gv` files: the timing-critical format

`.gv` is the *Extreme GPU Friendly Video Format*, originally from ofxExtremeGpuVideo. Each frame is an independent, LZ4-compressed block of raw RGBA, followed by an index table of `(address, size)` pairs at the end of the file.

That layout is what buys the timing guarantee. There is no inter-frame prediction, so presenting frame *n* costs a seek, one LZ4 decompression, and a texture upload — the same work for every frame, whatever the content. Compare this with H.264, where an I-frame costs several times a B-frame and the resulting jitter is *periodic*, aligned to the GOP boundary. That is the pathological case for frame-accurate onsets, and it is exactly what `.gv` avoids.

You pay for it in disk space. A frame is `width × height × 4` bytes before compression; LZ4 is fast rather than thorough, so expect files roughly an order of magnitude larger than an equivalent MPEG-1. The 10 s 960×400 clip in the table above is 366 MB raw, 41 MB as `.gv`, and 2 MB as MPEG-1.

`.gv` carries **no audio track**. Play a separate `stimuli.Sound` alongside it, which also gives you independent control over audio onset (see Section 11).

8.16.2.1 Creating a `.gv` file

`cmd/gv-convert` converts a video directly:

```
make gv-convert                          # builds _build/gv-convert

gv-convert clip.mpg clip.gv              # MPEG-1 input: pure Go, no ffmpeg
gv-convert movie.mp4 movie.gv            # anything else: needs ffmpeg on
 ↪  PATH
gv-convert frames/ anim.gv               # a directory of numbered images
gv-convert -fps 30 movie.mov movie.gv    # resample to 30 fps
gv-convert -max 100 long.mp4 preview.gv  # first 100 frames only
```

MPEG-1 program streams are decoded in pure Go, so that path needs nothing installed. Every other video format is piped through `ffmpeg`, which must be on `PATH`; if it is missing, the tool says so and names both decoders it tried.

8.16.2.2 From an image sequence

Passing a **directory** converts the `.png`/`.jpg`/`.jpeg`/`.gif` files it contains. This is the right route when frames are generated programmatically (drifting gratings, custom animations) rather than filmed. Two behaviours differ from a naive image-sequence converter, both deliberately:

- **Frames are ordered numerically, not lexicographically.** With unpadded names a plain string sort puts `frame_10.png` before `frame_2.png`, silently playing the sequence out of order. `gv-convert` sorts by numeric value and prints a note when the two orders disagree, so you can see that it mattered.

- **All frames must be the same size.** A mismatch is an error, not a silent crop: a wrongly-sized stimulus is the kind of fault that surfaces only at analysis. Pass `-force-size` to crop/pad against the first frame instead.

A directory has no inherent frame rate, so `-fps` sets it (default 30).

The standalone images2gv does the same job without requiring the goxpyriment tree, and targets other `.gv` players such as `ofxExtremeGpuVideo` and ebiten.

8.16.2.3 Inspecting a `.gv` file

`cmd/gv-getinfo` prints what a `.gv` file actually contains — useful to confirm a conversion produced the frame rate and dimensions you expected, and to check that a file will play at all:

```
make gv-getinfo                  # builds _build/gv-getinfo

$ gv-getinfo stim.gv
file        : stim.gv
frame size  : 960 x 400 px  (1.5 MB raw per frame)
frame count : 250
frame rate  : 25.000 fps   (duration 10.00 s)
pixel format: raw RGBA
file size   : 40.8 MB
frame data  : 40.8 MB compressed from 366.2 MB  (9.0x, 11.2% of raw)
frame sizes : min 5.9 KB, median 104.3 KB, max 611.2 KB
```

`-frames` lists every frame's offset and compressed size; `-q` prints one tab-separated line per file for scripting. The tool also cross-checks the header against the index table and warns about inconsistencies — a `.gv` whose header disagrees with its index loads but plays incorrectly, which is otherwise hard to diagnose from a black screen.

Files written by other tools may use DXT1/DXT3/DXT5 or BC7 block compression instead of raw RGBA. `gv-getinfo` flags these: goxpyriment's reader LZ4-decompresses straight into a texture and never DXT-decodes, so only `raw RGBA` is playable.

8.16.2.4 Playing a `.gv` file

One-shot, for when the video *is* the trial:

```
events, logs, err := stimuli.PlayGv(exp.Screen, "stim.gv", 0, 0)
```

`PlayGv` disables GC, locks to VSYNC, and returns a per-frame `TimingLog` you can write to your data file to verify that no frame was late. It exits on ESC.

8.16.2.4.1 The clip's frame rate must divide the refresh rate

`PlayGv` plays at the rate stored in the `.gv` header, holding each video frame for however many refreshes that takes — a 30 fps clip on a 60 Hz display is shown for 2 refreshes per frame. Both rates are rounded first, so a display reporting 59.94 Hz and a clip authored at 29.97 fps behave as 60 and 30.

Only exact integer ratios are supported. A 24 fps clip on a 60 Hz display would need 2.5 refreshes per frame, so `PlayGv` refuses it rather than play it at the wrong speed or with uneven onsets:

```
display refresh 60 Hz is not an integer multiple of the video's 24 fps
(2.500 frames per refresh); re-encode the clip at a divisor of 60 fps,
e.g. gv-convert -fps 20
```

The 3:2 pulldown that would otherwise be required makes frame onsets uneven, which defeats the reason for using `.gv` at all. **Choose the clip's frame rate to divide the refresh rate of the machine it will run on** — at 60 Hz that means 60, 30, 20, 15, 12, 10 ...; `gv-convert -fps` sets it, and `gv-getinfo` reports what a file actually claims.

Interactive, when you need to draw around the video or respond mid-clip:

```
v, err := stimuli.NewGvVideo("stim.gv")
if err != nil {
    log.Fatal(err)
}
defer v.Close()

v.Play()
exp.Run(func() error {
    if err := v.Update(exp.Screen); err == io.EOF {
        return control.EndLoop
    }
    exp.Screen.Clear()
    v.DrawAt(exp.Screen, &control.FRect{X: 100, Y: 50, W: 640, H: 480})
    return exp.Screen.Flip()   // presents and holds one frame: see Section 5
})
```

`Flip` is an alias for `Update`; both hold one frame. On drivers that present without blocking (triple/mailbox buffering), `Update` returns immediately and the loop runs faster than the display, swallowing frames.

`Width`, `Height`, `FrameCount` and `FPS` are read from the header at construction; the GPU texture is allocated lazily on the first `Update` or `Draw`. For playing several clips back to back without a gap, use `media.MovieManager` (see `docs/MediaMovies.md`).

8.16.2.5 Marking chosen frames for a recording

To mark the onset of particular video frames with the onset marker (photodiode square and TTL code — see Section 17), list them after the callback:

```
control.MarkerFlags()     // before NewExperimentFromFlags: -photodiode,
 ↪  -ttl, ...
...
events, logs, err := stimuli.PlayGvFunc(exp.Screen, "movie.gv", 0, 0, nil,
    stimuli.GvMark{Frame: 0, Code: 1},    // movie onset
    stimuli.GvMark{Frame: 186, Code: 2})  // the event inside the movie
```

Each mark is armed before the frame's **first** refresh, so the square turns white and the code is written on the flip logged as `logs[frame].FlipNS`, however many refreshes each video frame is held for. Code 0 marks a frame with the square only. A frame outside the video, or listed twice, is an error before playback starts; without `-photodiode/-ttl` the marks do nothing. If a mark near the end leaves the square still white when the video stops, the last frame is held, redrawn, until it is black (at most the marker's frame count).

Checked in software by `tests/test_onset_marker` (a 30 fps clip on a 60 Hz display, so each frame is held for 2 refreshes; frames 0, 10 and 32 marked): each code was written 0.8–3.6 µs after its frame's logged flip, and the square was white on exactly the 3 refreshes from each marked frame's first one.

8.16.2.6 Firing a trigger on a chosen frame

`PlayGvFunc` is `PlayGv` with a per-frame callback, invoked immediately after the flip that puts each frame on screen:

```
events, logs, err := stimuli.PlayGvFunc(exp.Screen, "stim.gv", 0, 0,
    func(ctx stimuli.GvFrameContext) error {
        if ctx.Frame == onsetFrame {
            trig.SetHigh(0)              // rising edge tied to this flip
        }
        if ctx.Frame == onsetFrame+3 {   // ~50 ms later at 60 Hz
            trig.SetLow(0)
        }
        return nil                       // control.EndLoop to stop early
    })
```

`ctx.OnsetNS` is the SDL timestamp of that frame's **first** flip, in the same reference frame as event timestamps. `ctx.Frame` is the 0-based video frame index and `ctx.Hold` the number of refreshes it is shown for. The callback runs inside the GC-disabled VSYNC loop, so it must not allocate heavily or sleep — see the warning below.

`tests/test_gv_sync` is a complete worked example: it plays a generated flash train and fires a TTL pulse at each bright onset for photodiode verification.

Because the callback runs *after* the frame's first flip, arming the onset marker from it (`exp.Marker.Arm`) marks the flip after that one — a hold refresh of the same frame when the video is slower than the display. Use the `GvMark` list above to mark frames; use the callback for logging, aborting, or a bare TTL (`exp.Marker.Fire`, written right after the frame's flip but with no square).

If you need to draw around the video as well, drive the loop yourself instead. `CurrentFrame()` returns the frame now in the GPU texture — the one the next flip will show — so raise the line immediately *after* the flip that displays it:

```
const onsetFrame = 120        // the frame whose onset you want marked
const pulseFrames = 3         // ~50 ms at 60 Hz

// Falls back to a no-op device when no hardware is present, so this code
```

```
// still runs on a machine without a trigger box.
trig, _, err := triggers.AutoDetectDLPIO8()
if err != nil {
    log.Fatal(err)
}
defer trig.Close()

lowAt := -1
v.Play()
exp.Run(func() error {
    if err := v.Update(exp.Screen); err == io.EOF {
        return control.EndLoop
    }
    exp.Screen.Clear()
    v.Draw(exp.Screen)

    flipNS, err := exp.Screen.FlipTS()
    if err != nil {
        return err
    }
    if v.CurrentFrame() == onsetFrame {
        trig.SetHigh(0)                       // rising edge, right after the
 ↪  flip
        lowAt = v.CurrentFrame() + pulseFrames
        exp.Data.Add(trialNo, onsetFrame, flipNS)   // trialNo: your own
 ↪  counter
    }
    if v.CurrentFrame() == lowAt {
        trig.SetLow(0)
    }
    return nil
})
```

Do not call `trig.Pulse` here. `Pulse` is `SetHigh`, `time.Sleep`, `SetLow` — it blocks for the whole pulse width, which inside a VSYNC-locked loop costs you frames. Splitting it across iterations as above sleeps not at all, and ties the rising edge (the thing your amplifier timestamps) to a known flip. See Section 17 for why wrapping `Pulse` in a goroutine is not the fix.

`flipNS` shares its reference frame with SDL event timestamps, so it subtracts directly from a keypress timestamp to give a hardware-precision RT relative to the video frame (see Section 6).

8.16.2.7 Verifying that no frame was late

`PlayGv` returns a `[]VideoFrameLog`, one entry per frame:

```
type VideoFrameLog struct {
    Frame         int    // 0-based video frame index
```

```
    FlipNS        uint64 // SDL nanosecond timestamp after the flip
    SkippedFrames int    // display frames late (0 = on time)
}
```

Scan `SkippedFrames` after a trial and record the worst value, or abort the session if it exceeds what your paradigm tolerates. A clip that drops frames on the experiment machine is a data-quality problem you want to detect during piloting, not discover in the analysis.

8.16.3 15.2 MPEG-1 files: the convenient format

`stimuli.Video` decodes MPEG-1 in pure Go via gen2brain/mpeg. No ffmpeg, no cgo, no conversion step — you point it at a `.mpg` and it plays, with sound.

The cost is that playback is driven by the wall clock, not by VSYNC. `Update` computes the target frame from elapsed time and *drops* frames to catch up if decoding falls behind. Playback stays the right length, but the onset of any particular frame is only approximate. **Do not put a frame onset from `stimuli.Video` in a data file** — use `.gv` for that.

8.16.3.1 Converting an mp4 (or anything else) to MPEG-1

`stimuli.Video` plays only MPEG-1, so anything else has to be converted once, up front. This is the command:

```
ffmpeg -i input.mp4 -c:v mpeg1video -b:v 1500k \
                    -c:a mp2 -b:a 192k -ar 44100 -ac 2 \
                    -f mpeg output.mpg
```

Reading it piece by piece:

Part	Why
`-c:v mpeg1video`	MPEG-**1** video. MPEG-2 in a `.mpg` will not play
`-b:v 1500k`	video bitrate; raise for large frames, lower for smaller files
`-c:a mp2`	MP2 is the only audio codec an MPEG-1 program stream carries
`-ar 44100 -ac 2`	44.1 kHz stereo; safe defaults
`-f mpeg`	**the load-bearing flag** — selects the MPEG-1 system muxer

`-f mpeg` is the one people get wrong. `-f vob` looks equivalent and produces an MPEG-2 program stream that is rejected at load. If you omit `-f` entirely, ffmpeg guesses from the `.mpg` extension and usually gets it right, but being explicit costs nothing.

Drop the three audio flags for a silent clip. To check the result before wiring it into an experiment, see the next section.

8.16.3.2 Format requirements

The decoder is strict, and the two most common failures are worth knowing:

- It must be an **MPEG-1 program stream** — the file starts with the pack header `00 00 01 BA`. A raw elementary stream (starting `00 00 01 B3`) is rejected with `invalid MPEG-PS`. An elementary stream also cannot carry audio at all, whatever the filename suggests.
- The video must be **MPEG-1**, not MPEG-2. A `.mpg` containing MPEG-2 is out of scope for the library even though the extension is the same.

Check a file before blaming your code:

```
file clip.mpg     # want: "MPEG sequence, v1, system multiplex"
ffprobe clip.mpg  # want: format_name=mpeg, codec_name=mpeg1video (+ mp2
 ↪  audio)
```

If `file` says *"MPEG sequence, v2"* the video is MPEG-2; if it does not say *"system multiplex"* you have an elementary stream. Either way, re-run the conversion above. A file with only one stream listed by `ffprobe` has no soundtrack, whatever its name promises.

8.16.3.3 Playing an MPEG-1 file

```
v, err := stimuli.NewVideo(exp.Screen, "clip.mpg")
if err != nil {
    log.Fatal(err)
}
defer v.Close()

// Enable the MP2 soundtrack. Safe to call unconditionally: it is a no-op
// returning nil when the file has no audio stream.
if err := v.PreloadDevice(exp.AudioDevice); err != nil {
    log.Printf("audio unavailable: %v", err)
}

v.Play()
exp.Run(func() error {
    if err := v.Update(); err == io.EOF {   // note: no screen argument
        return control.EndLoop
    }
    exp.Screen.Clear()
    v.Draw(exp.Screen, 0, 0)
    exp.Screen.Update()
    return nil
})
```

Audio is off until `PreloadDevice` is called. Without it the video plays silently, and audio packets are discarded at the demuxer — leaving decoding enabled with nothing

draining the buffer would grow it for the length of the clip. Check `v.HasAudio` if you need to know whether a soundtrack exists, and use `v.SetVolume(gain)` to scale it.

`Update` keeps 100 ms of decoded audio queued on the SDL device. Audio is free-running once queued, so it stays aligned with the video to within that depth. Two consequences: `Pause` clears the queue so sound stops with the image (resuming leaves audio up to 100 ms ahead), and `Rewind` clears it too so a restarted clip does not play over its own tail.

8.17 16. Gamma Correction and Luminance Linearity

8.17.1 The gamma problem

Standard monitors apply a power-law transfer function when converting digital RGB values to physical luminance:

$L(V) = k \cdot (V/255)^{\gamma}$ $\gamma \approx 2.2$ for sRGB/LCD

This means equal steps in RGB values do **not** produce equal steps in physical luminance (cd/m²). For example, with $\gamma = 2.2$:

RGB value	Physical luminance (% of max)
0	0%
64	4.9%
128	21.6%
192	52.2%
255	100%

This matters in psychophysics experiments where luminance values are specified as physical quantities (e.g. contrast masks, threshold estimation, magnitude estimation).

8.17.2 Inverse-gamma LUT

goxpyriment corrects for this by pre-computing a 256-entry look-up table (LUT) per channel. For each desired linear luminance value v (0–255), the LUT stores the digital value required to achieve it:

$LUT[v] = 255 \cdot (v/255)^{(1/\gamma)}$

Applying the LUT before sending a color to the renderer means the monitor's forward gamma converts it back to the intended linear value.

8.17.3 Enabling gamma correction

```
// After exp.Initialize() and before the trial loop:
exp.SetGamma(2.2)   // typical sRGB monitor
```

```
// Or with per-channel calibration (from photometer measurements):
exp.GammaCorrector = apparatus.NewGammaCorrector(2.1, 2.2, 2.3)
```

Then wrap every color with `exp.CorrectColor`:

```
// Specify colors in linear luminance space (0–255).
// exp.CorrectColor maps them to the physical digital values.
disk := stimuli.NewCircle(radius, exp.CorrectColor(control.RGB(128, 128,
↪  128)))
```

When `SetGamma` has not been called, `CorrectColor` is a no-op that returns the color unchanged, so it is safe to always call it.

8.17.4 Measuring your monitor's gamma

The most accurate approach is photometer-based: measure luminance at several RGB levels, fit the model L(V) = k·(V/255)^γ, and pass the resulting γ to `SetGamma`. Without a photometer, $\gamma = 2.2$ is a reasonable default for modern sRGB displays.

8.17.5 Practical example

The `examples/Magnitude-Estimation-Luminosity` example accepts a `-gamma` flag:

```
go run main.go -gamma 2.2
```

With the flag set, the 7 luminance levels (10, 25, 50, 100, 150, 200, 255) are treated as linear targets and corrected before display, so they produce physically uniform luminance steps.

8.18 17. Hardware Triggers and TTL Devices

EEG and MEG recordings require a synchronisation signal — a short TTL pulse sent at the exact moment a stimulus is presented — so that electrophysiological data can be time-locked to experimental events. The `triggers` package provides this, along with the ability to read TTL inputs from response hardware such as fiber-optic response pads.

8.18.1 Concepts

All trigger devices in goxpyriment share two interfaces:

- **`OutputTTLDevice`** — send trigger codes (set lines HIGH/LOW, generate pulses)
- **`InputTTLDevice`** — read response button states (poll or block)

Lines are **0-indexed (0–7)**. Bit N of a bitmask drives line N.

8.18.2 Onset markers: photodiode square and TTL codes

For marking stimulus onsets in a recording, use the library's **onset marker** rather than driving a device yourself. At each marked onset it does two things on the same flip:

- a square in the top-left corner of the drawing area turns white for 3 frames (black on every other frame), for a photodiode taped to the screen;
- a TTL code is written to the trigger device right after the flip, and the lines are cleared 3 frame periods later (50 ms at 60 Hz).

```
func main() {
    control.MarkerFlags()       // -photodiode, -photodiode-size,
 ↪  -marker-frames, -ttl, -ttl-ms
    exp := control.NewExperimentFromFlags("My task", control.Gray,
 ↪  control.White, 32)
    defer exp.End()
    ...
    err := exp.Run(func() error {
        for _, t := range trials {
            exp.Marker.Arm(t.code)            // the next flip is an onset
            onset, _ := exp.ShowTS(t.stim)    // square + code go out with
 ↪  this flip
            key, keyTS, _ := exp.Keyboard.GetKeyEventTS(keys, -1)
            exp.Marker.Fire(responseCode)     // a response: code now, no
 ↪  square
            ...
        }
        return control.EndLoop
    })
    ...
}
```

```
go run . -s 1 -photodiode -ttl megttlbox:/dev/ttyACM0
go run . -s 1 -photodiode -ttl dlpio8          # auto-detected
go run . -s 1 -ttl null                        # no hardware: exercises the
 ↪  code path
```

- **The flags are opt-in.** Only programs that call `control.MarkerFlags()` (before `NewExperimentFromFlags`) have them. A paradigm can change the defaults, e.g. `control.MarkerFlags(control.MarkerConfig{Photodiode: true})`. Programs built with `NewExperiment` set `exp.MarkerConfig` before `Initialize`.
- **`-ttl` names a device in one string**, `KIND[:PORT]`: `dlpio8[:PORT|auto]`, `dlpio20`, `megttlbox:/dev/ttyACM0`, `mmbts:/dev/ttyACM0`, `parallel[:/dev/parport0]`, `gpio`, `ft232h`, `labjackt4:HOST`, `null` (see `triggers.DeviceSpecUsage`). A device that cannot be opened stops the program before any window opens. Hardware triggers are desktop-only; the browser build refuses `-ttl`, while `-photodiode` works there.
- **Any presentation path is covered.** The marker is driven from inside

`Screen.Update`, which every flip goes through: `Show`/`ShowTS`, streams (arm from the `FrameCallback` on `ctx.OnPhase && ctx.Frame == 0`), `.gv` video (list the frames to mark: `PlayGvFunc(..., stimuli.GvMark{Frame, Code})`, Section 15), and `PlaySyncedWithFlip` (arm before the call). For a sound, the marker marks the flip that starts it; the audio output latency is in neither signal.

- **`ShowTS` holds an armed onset** until the square is black again (4 flips by default), so a key wait after it cannot leave the square white; it still returns the first flip's timestamp, and reaction times are unaffected. `ShowFrames`/`BlankFrames` never hold. Wrap a composite scene in `stimuli.DrawFunc` to present it through `ShowTS`.
- **`Fire(code)`** writes a code immediately with no square — for events that are not a flip: a response once detected, a sound started with `Play`.
- **Recorded in the `-info.txt`**: the square and the device used, and at the end `marker stats:` — onsets, codes, clears, write errors, overlaps (a code written while the previous one was still on the lines) and merged flashes (onsets less than 4 frames apart). A failed write is counted and logged once; it does not end the session.

One line per event type is the safe choice on the DLP boxes. Their `Send` writes the eight lines one command at a time — about 0.6 ms on the DLP-IO8, 3.5 ms on the DLP-IO20 — and a recording sampling during that window can latch a code that was never sent. A code with a single bit set (1, 2, 4, … 128) avoids it: the marker writes it as one `SetHigh` and clears it with one `SetLow`. Multi-bit codes are fine on the MEG TTL box when its firmware advertises the atomic-port command (the `-info.txt` records the firmware); older firmware leaves a narrow window too.

Two more limits: onsets closer than the TTL duration overlap on the lines (lower `-ttl-ms`), and a multi-bit code on a slow device delays the write by the times above. The offset between the code and the photons is the display's latency (one to three frames): measure it once per rig with the photodiode and subtract it in analysis.

8.18.3 Sending triggers (EEG event codes)

```
import (
    "github.com/chrplr/goxpyriment/triggers"
    "time"
)

// Auto-detect a DLP-IO8-G; falls back to NullOutputTTLDevice (no-op) if
 ↪  absent.
out, _, err := triggers.AutoDetectDLPIO8()
if err != nil { log.Fatal(err) }
defer out.Close()

// In the trial loop: raise line 0 right after the flip; only the falling
// edge is deferred (see "Pulse blocks" below).
onset, _ := exp.ShowTS(stim)
triggers.FireTriggerSync(out, 0, 10*time.Millisecond)
```

For the NeuroSpin MEGTTLBox:

```
box, err := triggers.NewMEGTTLBox("/dev/ttyACM0")
defer box.Close()

box.Pulse(0, 5*time.Millisecond)   // single line
box.PulseMask(0b00000011, 5*time.Millisecond)  // lines 0 and 1
 ↪  simultaneously
```

8.18.4 Timing advice

The output pulse should be sent as close as possible to `exp.ShowTS(stim)`. Because both `Screen.FlipTS` and the trigger output use the system's real-time clock, the latency between the VSYNC flip and the TTL edge is typically under 1 ms. The EEG amplifier records this edge, and the exact onset can be recovered by subtracting `int64(triggerEdgeNS - onsetNS)` if the amplifier's sample clock is synchronised.

8.18.5 `Pulse` blocks — and a goroutine is not the fix

`Pulse(line, d)` is `SetHigh`, `time.Sleep(d)`, `SetLow`. It blocks for the full width. That is fine in an ordinary trial loop, where the sleep overlaps a stimulus you were going to display anyway — but inside a per-frame loop (`.gv` playback, `PresentStreamOfStimuliFunc`, any VSYNC-locked animation) it costs you frames.

The tempting fix is `go trig.Pulse(0, 5*time.Millisecond)`. Don't:

- **It races.** `ParallelPort.SetHigh` is an unsynchronised read-modify-write on a shadow register, so overlapping calls can lose an update and leave a line stuck HIGH or send a wrong code. `LabJackT4` shares one Modbus TCP connection; the serial devices give no ordering guarantee across goroutines. Only `FT232HTrigger` and `LinuxGPIOTrigger` hold a mutex. These failures are silent and intermittent, and they corrupt the event record in your EEG file.
- **It moves the edge onto the Go scheduler.** The amplifier timestamps the *rising* edge — that is your event marker. Blocking `Pulse` raises the line immediately and only the *width* costs time; `go Pulse` defers the `SetHigh` to whenever the scheduler runs it, adding jitter to the one edge that must not have any.
- **It outlives its device.** If the trial ends and the device closes while the goroutine sleeps, `SetLow` fires on a closed port.

In a per-frame loop, split the pulse across iterations instead — no sleeping at all, and the rising edge is locked to a known flip:

```
if thisIsTheOnsetFrame {
    trig.SetHigh(0)              // rising edge, right after the flip
    lowAt = frameIndex + 3       // ~50 ms at 60 Hz
}
if frameIndex == lowAt {
    trig.SetLow(0)
}
```

The width quantises to whole frames, which is well above the ~1 ms most amplifiers need. Section 15 shows this applied to `.gv` playback.

8.18.6 Reading response inputs (FORP pads)

The MEGTTLBox wires a fiber-optic response pad (fORP) to its 8 TTL input lines. Use `WaitForInput` directly or via the `ResponseDevice` wrapper (section 7):

```
// Via InputTTLDevice directly
_ = box.DrainInputs(ctx)                    // clear stale presses
mask, rt, err := box.WaitForInput(ctx)
buttons := triggers.DecodeMask(mask)
for _, b := range buttons {
    fmt.Println(b)  // e.g. "left blue", "right red"
}

// Via ResponseDevice
rd := apparatus.NewTTLResponseDevice(box, 5*time.Millisecond)
_ = rd.DrainResponses(ctx)
resp, _ := rd.WaitResponse(ctx)
// resp.Code == bitmask, resp.Precise == false (software poll)
```

fORP button mapping (NeuroSpin wiring):

FORPButton constant	Line	Arduino pin	STI channel
`FORPLeftBlue`	0	D22	STI007
`FORPLeftYellow`	1	D23	STI008
`FORPLeftGreen`	2	D24	STI009
`FORPLeftRed`	3	D25	STI010
`FORPRightBlue`	4	D26	STI012
`FORPRightYellow`	5	D27	STI013
`FORPRightGreen`	6	D28	STI014
`FORPRightRed`	7	D29	STI015

8.18.7 FT232H (Adafruit FT232H breakout, Linux)

A USB-to-MPSSE bridge. AD0–AD7 are TTL output lines; AC0–AC7 are TTL input lines. No libftdi or D2XX installation needed — the driver uses Linux usbfs directly.

```
box, err := triggers.NewFT232H()
defer box.Close()

box.Pulse(0, 5*time.Millisecond)
mask, rt, _ := box.WaitForInput(ctx)

// Auto-detect (returns NullOutputTTLDevice if no device found):
out, _ := triggers.AutoDetectFT232H()
```

Prerequisites: unload the `ftdi_sio` kernel module (`sudo rmmod ftdi_sio`) and ensure the user has rw access to the USB device node (udev rule or `plugdev` group membership).

8.18.8 GPIO (Raspberry Pi and other Linux SBCs)

Drives any 8 GPIO pins as TTL outputs and reads any other 8 pins as TTL inputs, using the Linux GPIO character device v2 API. Works on Raspberry Pi (BCM pin numbers), Rock Pi, BeagleBone, Jetson, and any Linux SBC with kernel $\geq$ 5.10.

```
box, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
    triggers.WithGPIOInputPins([8]int{12, 16, 20, 21, 4, 25, 24, 23}),
)
defer box.Close()

box.Pulse(0, 5*time.Millisecond)
mask, rt, _ := box.WaitForInput(ctx)
```

Output-only and input-only configurations are both valid. Prerequisites: kernel $\geq$ 5.10; user in the `gpio` group.

8.18.9 LabJack T4 (Modbus TCP)

Communicates over Ethernet via Modbus TCP — no LabJack SDK or driver required. The eight output lines are DIO4–DIO11 (FIO4–FIO7 + EIO0–EIO3) and the eight input lines are DIO12–DIO19 (EIO4–EIO7 + CIO0–CIO3); `WithT4OutputBase` / `WithT4InputBase` move those groups. DIO0–DIO3 are the T4's dedicated analog inputs and are unavailable as digital lines.

```
box, err := triggers.NewLabJackT4("192.168.1.100")
defer box.Close()

box.Pulse(0, 5*time.Millisecond)
mask, rt, _ := box.WaitForInput(ctx)
```

The device must be reachable on the local network. Default Modbus port 502 is used; append `:port` to the host string to override.

8.18.10 DLP-IO20 (USB-CDC serial)

A 20-channel USB module, 5 V logic. It has more digital channels (17) than the 8-bit TTL interfaces can express, so interface lines 0–7 address a *window* of 8 channels — `AN0–AN7` for output and `AN8–AN13` + `RB6/RB7` for input by default. Every channel remains reachable through the channel-level methods.

```
d, err := triggers.NewDLPIO20("/dev/ttyUSB0")   // or
↪  triggers.AutoDetectDLPIO20()
defer d.Close()
```

```
d.Send(0b00000101)                    // lines 0 and 2 → AN0, AN2
d.SetChannelHigh(triggers.IO20_RA4)   // a channel outside the window
v, _ := d.ReadChannel(triggers.IO20_AN12)
```

Remap the windows with `WithIO20OutputChannels` / `WithIO20InputChannels` (8 channels each, no duplicates, no overlap between the two groups).

Two caveats. The device has no write-all command, so `Send` issues one packet per line and the 8 lines change over ~3.5 ms rather than together — use `SetHigh` on a single line when trigger onset matters. And the driver is **untested on real hardware**: it was written from the datasheet, so verify it with `tests/test_dlpio20` before an experiment depends on it.

8.18.11 NEUROSPEC MMBT-S (USB-CDC serial)

An output-only box: 8 TTL lines on a D-Sub 25 socket with the standard LPT pinout (bit N → pin N+2, ground on pins 20–25, 5 V HIGH). It is a USB virtual COM port, and one byte at 9600 baud sets all 8 lines at once.

```
box, err := triggers.NewMMBTS("/dev/ttyACM0")   // mode p, the factory
 ↪  setting
defer box.Close()

box.Send(0b00000011)
box.Pulse(0, 5*time.Millisecond)
```

Two things about this box do not follow the pattern of the other devices.

Its **runtime mode is a hardware switch**, marked `P/S` next to the USB-C socket, read at reset and invisible to software. In `P` — the factory setting, and the driver's default — the firmware clears the port 8 ms after each byte, so **every trigger is 8 ms wide** whatever duration `Pulse` or `FireTriggerSync` is given, and codes issued closer together than 8 ms are queued and delayed. Set the switch to `S` and pass `triggers.WithMMBTSMode(triggers.MMBTSSimpleMode)` to time the width from the host. If the option and the switch disagree, nothing fails — the lines simply behave differently from the code.

And there is **no auto-detection**: the box never answers on the serial line, and it enumerates as a generic Arduino Micro, so it cannot be distinguished from any other Arduino-based box. Name the port. The green LED beside the socket follows bit 1 and lights on odd codes, which is enough to confirm the link without an amplifier; `tests/test_mmbts` drives the patterns to watch it with.

8.18.12 Supported devices

Device	Type	Output	Input	Connection	Notes
DLP-IO8-G	`DLPIO8`	✓	✓	USB-CDC serial	ASCII protocol
DLP-IO20	`DLPIO20`	✓	✓	USB-CDC serial	Packet protocol; 17 digital channels via 8-line windows; 5 V; untested on hardware
NeuroSpin MEGTTL-Box	`MEGTTLBox`	✓	✓	USB serial	Arduino Mega; binary protocol; fORP input
NEUROSPEC MMBT-S	`MMBTS`	✓	—	USB-CDC serial	D-Sub 25 LPT pinout; 9600 baud only; 8 ms fixed pulse in the factory Pulse mode
Adafruit FT232H	`FT232HTrigger`	✓	✓	USB (usbfs)	Linux only; no libftdi needed
Linux GPIO	`LinuxGPIOTrigger`	✓	✓	GPIO pins	Any Linux SBC; kernel ≥ 5.10
LabJack T4	`LabJackT4`	✓	✓	Ethernet (Modbus TCP)	No SDK needed
LPT parallel port	`ParallelPort`	✓	—	LPT	Linux only; ppdev ioctl
None / fallback	`NullOutputTTLDevice`	✓ (no-op)	—	—	Returned by `AutoDetectDLPIO8`, `AutoDetectFT232H`

8.18.13 Networked recording control (non-TTL)

The `triggers` package also hosts two devices that are **not** TTL bitmask hardware: they mark events in — and control — an *external recorder* over TCP/IP. They share the package because their purpose is the same (annotate a recording so it can be aligned with your paradigm), but they implement neither `OutputTTLDevice` nor `InputTTLDevice`; each has its own richer API. `SerialPort` (a generic UART wrapper) is the third non-TTL member.

8.18.13.1 NetStation (EGI EEG event markers over TCP/IP)

`NetStation` talks to an EGI / Philips NetStation EEG host using the ECI (Experiment Control Interface) protocol. Instead of an electrical pulse it sends *named events* — a 4-character code with an onset time, a duration, and optional key/value payloads — directly into the EEG stream, and it starts/stops recording and synchronises the host clock remotely.

```
import "github.com/chrplr/goxpyriment/triggers"

ns, err := triggers.NewNetStation("134.225.198.12") // default ECI port
 ↪  55513
if err != nil { log.Fatal(err) }
defer func() {
    // Always check this: a failed stop can leave an unreadable .mff on the
     ↪  host.
    if err := ns.Close(); err != nil { log.Printf("netstation: %v", err) }
}()                          // stops recording + disconnects (blocks ~2 s to
 ↪  flush)

ns.Synchronize()            // align the host clock to ours (call once after
 ↪  connect)
ns.StartRecording()

// In the trial loop — send the marker right after the VSYNC flip:
onset, _ := exp.ShowTS(stim)
ns.SendEvent("STIM")        // now, 1 ms, no keys

// Full form: explicit onset, duration and key/value metadata:
ns.SendEventFull(triggers.Event{
    Code:     "RESP",
    Start:    time.Now(),   // zero value = now; pass the flip time to mark
 ↪  true onset
    Duration: 2 * time.Millisecond,
    Keys:     []triggers.EventKey{{Code: "corr", Value: 1}, {Code: "rt",
 ↪  Value: 423}},
})

ns.StopRecording()
```

Timing note: a network event marker is convenient (it carries a rich label and needs no wiring) but its latency is bounded by the TCP round-trip, not the sub-millisecond edge of a TTL pulse. For hard sub-millisecond EEG synchrony, prefer a `Pulse` on a TTL device for the onset marker and use NetStation events for the accompanying metadata. The driver always advertises the `QNTEL` (Intel/little-endian) handshake and encodes every field little-endian, so it is portable regardless of the machine it runs on. See `tests/test_netstation` for a runnable session.

Never leave a recording open. The output `.mff` is written by NetStation Acquisition — no ECI command names it, chooses its format or finalizes it, so nothing here can change the file's format. What *does* depend on this client is whether the recording is closed properly. A bundle that contains `Acquiring.xml` and no `info.xml`, whose events and EEG signal are both unreadable, is EGI's documented signature of an acquisition that never completed its recording. Two rules follow:

- Check the error from `Close` (as above). It reports a refused stop instead of swallowing it, and every ECI command's acknowledgement is now validated — a host that answers `F` (refused) or `R` (not ready to record) produces an error rather than a silent success.
- Do not call `log.Fatal` between `StartRecording` and `Close`. `log.Fatal` exits without running deferred functions, so the host keeps acquiring. Put the session in a `run() error` function and fail from `main` after the deferred `Close` has run — `tests/test_netstation/main.go` shows the pattern, including a Ctrl-C trap.

If a recording was left open, EGI's File Validator utility (shipped with Net Station 5.4, under Applications ▸ EGI ▸ Utilities) removes `Acquiring.xml` and makes the file openable again.

8.18.13.2 BEL video recorder (labelled camera over TCP/IP)

`VideoRecorder` is the client for the NeuroSpin EEG-room **behavioral video recorder** ("BEL_video"): a camera PC films the participant, burns event labels (subject id, trial, condition, ...) into the footage, and saves an AVI. This type starts/stops that recording and pushes the labels over TCP — the capture and encoding stay on the camera PC. It is not an eye tracker: there is no gaze, pupil, or calibration data, only a labelled video.

```
vr, err := triggers.NewVideoRecorder("192.168.8.212") // default port 55113
if err != nil { log.Fatal(err) }
defer vr.Close()            // stops recording + disconnects

vr.Start()
vr.SetSubject("bb0012025")  // "NIP:<id>" — names the output file
vr.Label("TRL", "001")      // "KEY:VALUE" overlay, shown until the next
 ↪  label / timeout
vr.Label("CND", "007")
vr.Stop()
```

Unlike NetStation, the recorder's protocol has **no framing and no acknowledgement** — it is fire-and-forget, and the server reads one socket buffer per captured

video frame. Two messages sent too close together can therefore arrive coalesced and be mis-parsed, so `VideoRecorder` waits a short `SendGap` (default 50 ms; tune with `triggers.WithVRSendGap`, or set 0 to disable) after each message. This makes the labels unsuitable for precise stimulus timing — treat them as monitoring/annotation metadata, not event markers. See `tests/test_videorecorder` for a runnable session.

Device	Type	Role	Connection	Ack?
EGI NetStation	`NetStation`	EEG event markers + recording control + clock sync	TCP/IP (ECI, port 55513)	yes (1 byte/command)
BEL_video	`VideoRecorder`	Labelled video recording control	TCP/IP (port 55113)	no (fire-and-forget)

8.19 18. Display Compositor Bypass

A compositing window manager (compositor) intercepts every application frame and blends it with other on-screen elements before sending the result to the display. This introduces additional latency and jitter that are problematic for millisecond-accurate stimulus presentation. The degree to which `goxpyriment` can avoid this overhead depends on the operating system.

On **Linux with X11**, SDL3 automatically sets the `_NET_WM_BYPASS_COMPOSITOR` window property when the application enters fullscreen mode. Compliant compositors (KWin, Mutter, Compton) respond by *unredirecting* the fullscreen window — routing its framebuffer directly to the display scan-out pipeline without compositing. On **Linux with Wayland**, the compositor is always the intermediary for frame delivery, though modern compositors (KWin 5.21+, GNOME Mutter 44+) support *direct scan-out* for fullscreen applications. The best achievable timing on Linux is obtained by running without any display server: setting the environment variable `SDL_VIDEODRIVER=kmsdrm` before launching the experiment directs SDL3 to communicate with the Linux KMS/DRM subsystem directly, bypassing X11 and Wayland entirely. This configuration — typically used from a virtual terminal (`Ctrl+Alt+F2`) — is recommended for the most demanding timing requirements such as single-frame subliminal stimulation or high-frequency RSVP.

The difference between these three is large and has been measured: onset scatter of 0.113 ms on KMS/DRM, 0.083 ms on bare Xorg, and 1.344 ms in a Wayland session, on one machine on one night. Bare Xorg is the steadiest and also the latest, one whole frame behind KMS/DRM. See Timing tests.

On **Windows**, SDL3's fullscreen mode creates an exclusive fullscreen surface that bypasses the Desktop Window Manager (DWM). We have no photodiode measurements

on Windows; run the `display` and `av` sub-tests on your own machine rather than assuming it behaves like Linux.

macOS is the exception. Apple's Metal rendering pipeline always delivers frames to the display through the WindowServer compositor, and no third-party application can bypass it — even in fullscreen.

These platform differences can be quantified empirically with the `display` and `av` sub-tests of the bundled `Timing-Tests` suite.

8.20 19. Variable Refresh Rate (VRR) Stimulus Presentation

Standard displays refresh at a fixed rate (e.g., 60 Hz), which quantises every stimulus duration to multiples of the frame period (16.67 ms at 60 Hz). A researcher wishing to present a stimulus for 10 ms, 17 ms, or 23 ms cannot do so on such a display without relying on temporal dithering or hardware frame-blending techniques that introduce their own artefacts. Variable Refresh Rate (VRR) technology — marketed as AMD FreeSync, NVIDIA G-Sync, or the VESA Adaptive-Sync standard — removes this constraint by allowing the display to hold each frame for an arbitrary duration and refresh only when instructed to do so by the GPU.

`goxpyriment` exposes VRR through a single API call: `exp.Screen.SetVSync(0)` disables the VSYNC block, so that every subsequent `SDL_RenderPresent` call returns immediately. The following pattern then achieves arbitrary stimulus durations with duration error typically below 0.5 ms (one standard deviation):

1. Draw the stimulus and call `screen.FlipTS()`, which presents the frame and records `onsetNS` (an SDL nanosecond timestamp) immediately after `Present()` returns.
2. Busy-wait for the desired duration d: spin until `time.Now() >= onset + d`. The last ~500 µs use a CPU spin loop (already used in the trigger timing routines) to eliminate OS scheduling jitter.
3. Draw the blank screen, call `FlipTS` again to record `offsetNS`.

The actual software-controlled duration is `(offsetNS - onsetNS) / 1e6` ms. On a VRR-capable display this equals the true on-screen duration plus a small, constant hardware pipeline latency (GPU scan-out and panel response, typically 1–5 ms) that can be calibrated once with a photodiode using the `frames` timing sub-test of the `Timing-Tests` suite.

Every VRR panel supports VRR only within a finite refresh-rate window (e.g., 48–144 Hz, corresponding to frame durations of 6.9–20.8 ms for a typical gaming monitor). Durations outside this window cause the panel to revert to fixed-rate behaviour, reintroducing quantisation. The `Timing-Tests` suite includes a dedicated `vrr` sub-test that sweeps target durations from 1 ms to a user-specified maximum in 1 ms steps

and reports duration errors at each step, directly revealing the panel's VRR window in the output data.

On a display without VRR support, disabling VSYNC produces frame tearing. The `vrr` sub-test detects this automatically: on a non-VRR display the duration errors cluster at multiples of the frame period rather than being uniformly small, providing an unambiguous diagnostic.

8.21 20. Putting It All Together

Here is a skeleton that illustrates how the concepts compose in a realistic experiment:

```
package main

import (
    "fmt"
    "log"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

const (
    NReps           = 10
    FixationDuration = 500
    ResponseTimeout  = 3000
)

type trialDef struct {
    word  string
    color control.Color
    name  string
}

func main() {
    exp := control.NewExperimentFromFlags("Color Word Task", control.Black,
↪   control.White, 36)
    defer exp.End()

    exp.AddDataVariableNames([]string{"trial", "word", "ink", "key", "rt_
↪   ms", "correct"})

    // Build trial list
    words  := []string{"RED", "GREEN", "BLUE"}
    colors := []control.Color{control.Red, control.Green, control.Blue}
    names  := []string{"red", "green", "blue"}
```

```
    keys   := []control.Keycode{control.K_R, control.K_G, control.K_B}

    var trials []trialDef
    for _, word := range words {
        for j := range colors {
            trials = append(trials, trialDef{word, colors[j], names[j]})
        }
    }
    for i := 0; i < NReps-1; i++ {
        trials = append(trials, trials[:9]...)
    }
    design.ShuffleList(trials)

    // Preload stimuli
    fixation := stimuli.NewFixCross(20, 2, control.White)

    err := exp.Run(func() error {
        // Preload fixation cross (it will be used every trial)
        stimuli.PreloadVisualOnScreen(exp.Screen, fixation)

        exp.ShowInstructions(
            "Name the INK COLOR of each word.\n\n" +
            "R = Red   G = Green   B = Blue\n\n" +
            "Press SPACE to start.",
        )

        for i, t := range trials {
            // Fixation
            exp.ShowTimed(fixation, FixationDuration)

            // Stimulus + Response
            stim := stimuli.NewTextLine(t.word, 0, 0, t.color)
            key, rt, _ := exp.ShowAndGetRT(stim, keys, ResponseTimeout)

            // Score
            correct := false
            for j, k := range keys {
                if key == k && names[j] == t.name {
                    correct = true
                    break
                }
            }

            exp.Data.Add(i, t.word, t.name, key, rt, correct)
            fmt.Printf("trial %3d  %s/%s  rt=%dms  %v\n", i, t.word, t.name,
↪   rt, correct)

            if !correct {
```

```
                exp.Audio.PlayBuzzer()
            }

            // Release texture (new TextLine each trial)
            stim.Unload()

            exp.Blank(500)
        }

        return control.EndLoop
    })

    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```

Key patterns illustrated:

- `NewExperimentFromFlags` + `defer exp.End()` at the top.
- `exp.AddDataVariableNames` before the run loop.
- `design.ShuffleList` for trial randomization.
- `stimuli.PreloadVisualOnScreen` inside `exp.Run` for timing-sensitive stimuli.
- `exp.Show` for single stimuli; `exp.Blank` for ITIs.
- `exp.Keyboard.WaitKeysRT` for response + RT.
- `stim.Unload()` for dynamically created stimuli.
- `exp.Audio.PlayBuzzer()` for feedback.
- `return control.EndLoop` at the end.

For the complete function signatures and type definitions, see the API Reference. For more worked examples, browse the `examples/` directory.

Chapter 9

goxpyriment API Reference

This guide documents the complete public API of the `goxpyriment` framework, organized by package.

Note that you can access the source code documentation by running `pkgsite` and opening http://localhost:8080. To install pkgsite:

```
go install golang.org/x/pkgsite/cmd/pkgsite@latest
```

See Viewing the documentation locally for details (and for serving this site offline with zensical).

9.1 Package Overview

```
control/      ← experiment lifecycle and orchestration (start here)
stimuli/      ← visual and audio stimulus objects
media/        ← multi-movie playback with hardware-verified display onsets
apparatus/    ← SDL window/renderer, keyboard, mouse, gamepad, gamma
 ↪  corrector
results/      ← experiment data file and output file
design/       ← trial/block structure and randomization
clock/        ← timing utilities
geometry/     ← coordinate conversion helpers
triggers/     ← hardware trigger devices (EEG sync, etc.)
staircase/    ← adaptive threshold estimation (up/down, QUEST)
units/        ← pixels ↔ degrees ↔ cm through a Monitor
assets_embed/ ← embedded default font and sounds
```

Two further packages are infrastructure rather than experiment API and are not documented here: `sysinfo/` (the machine snapshot printed by `-sysinfo` and written into every `-info.txt`) and `vblank/` (per-platform vblank clocks used to anchor flip timestamps; see Timing tests).

9.2 Package `control`

Import: `github.com/chrplr/goxpyriment/control`

9.2.1 Boilerplate

Every experiment starts the same way:

```
exp := control.NewExperimentFromFlags("My Experiment", control.Black,
 ↪  control.White, 32)
defer exp.End()

err := exp.Run(func() error {
    // trial loop
    return control.EndLoop
})
if err != nil && !control.IsEndLoop(err) {
    log.Fatalf("experiment error: %v", err)
}
```

9.2.2 Pre-experiment Setup Dialog

`GetParticipantInfo` opens a graphical SDL window **before** the experiment starts to collect participant demographics, monitor properties, and display preferences. Call it before `NewExperiment` / `NewExperimentFromFlags`.

```
fields := append(control.StandardFields, control.FullscreenField)
info, err := control.GetParticipantInfo("My Experiment", fields)
if err != nil {
    log.Fatalf("setup cancelled: %v", err) // user pressed Escape or closed
 ↪  window
}
```

9.2.2.1 Types

```
// FieldType selects how a field is rendered.
type FieldType int
const (
    FieldText     FieldType = iota // text input box (default)
    FieldCheckbox                  // tick-box; value is "true" or "false"
)

// InfoField describes one entry in the dialog.
type InfoField struct {
    Name    string    // key in the returned map
    Label   string    // displayed label
    Default string    // initial value
    Type    FieldType // FieldText (default) or FieldCheckbox
```

```
}
```

9.2.2.2 Pre-built field sets

Variable	Fields
`control.ParticipantFields`	`subject_id`, `age`, `gender`, `handedness`
`control.MonitorFields`	`screen_width_cm`, `viewing_distance_cm`, `refresh_rate_hz`
`control.StandardFields`	`ParticipantFields` + `MonitorFields`
`control.FullscreenField`	Checkbox: `fullscreen` (`"true"` / `"false"`)

9.2.2.3 Function

Function	Description
`GetParticipantInfo(title string, fields []InfoField) (map[string]string, error)`	Shows the dialog and returns collected values. Returns `ErrCancelled` if the user closes or presses Escape without confirming.

9.2.2.4 Session persistence

All values except `subject_id` are saved to `~/.cache/goxpyriment/last_session.json` on OK and pre-filled on the next run. `subject_id` is always reset.

9.2.2.5 Using the fullscreen checkbox and persisting to the data file

```
info, err := control.GetParticipantInfo("My Experiment", fields)
// ...
fullscreen := info["fullscreen"] == "true"
width, height := 0, 0
if !fullscreen {
    width, height = 1024, 768
}
exp := control.NewExperiment("My Experiment", width, height, fullscreen,
    control.Black, control.White, 32)

// Set Info (and SubjectID) BEFORE Initialize — they are written to the info
 ↪  file automatically
exp.SubjectID, _ = strconv.Atoi(info["subject_id"])
exp.Info = info

if err := exp.Initialize(); err != nil { log.Fatal(err) }
defer exp.End()
```

`Initialize()` writes a `--PARTICIPANT INFO` block to the companion `-info.txt` file whenever `exp.Info` is non-nil at that point. No explicit call to `WriteParticipantInfo` is needed.

9.2.2.6 Sentinel error

```
control.ErrCancelled  // returned when the user cancels the dialog
```

9.2.3 Constructor Functions

Function	Description
`NewExperimentFromFlags(name string, bg, fg Color, fontSize float32, extra ...InfoField) *Experiment`	Creates and fully initializes an experiment from `-w` (windowed 1024×768), `-d N` (display index, -1 = primary), and `-s N` (subject ID) command-line flags. Calls `log.Fatal` on error. **This is the preferred entry point.** Any `extra` fields are appended to the session-setup dialog that opens when `-s` is absent, and read back from `exp.Info` (nil when the dialog does not open, so keep a flag as the fallback); an extra field named like one of the four built-ins replaces it.
`NewExperiment(name string, width, height int, fullscreen bool, bg, fg Color, fontSize float32) *Experiment`	Lower-level constructor; call `Initialize()` before use.

9.2.4 Lifecycle Methods

Method	Description
`exp.Initialize() error`	Initializes SDL, audio, window, renderer, font, and data file.
`exp.End()`	Cleans up all resources. Always `defer exp.End()` immediately after construction.
`exp.Run(logic func() error) error`	Runs the main trial loop on the SDL main thread. Return `control.EndLoop` to exit cleanly.
`exp.HideCursor() error`	Hides the mouse cursor. `Initialize()` already does this — only needed to hide it again after a `ShowCursor` call.

Method	Description
`exp.ShowCursor() error`	Makes the mouse cursor visible. Call it after `Initialize()` in mouse-driven paradigms; the cursor is hidden by default so it never sits over the stimuli. Equivalent to setting `exp.CursorVisible = true` before `Initialize()`. `GetParticipantInfo` shows the cursor for its own dialog regardless.

9.2.5 Presentation Methods

Method	Description
`exp.Show(stim VisualStimulus) error`	Clear → draw → flip. The standard one-call stimulus presentation.
`exp.ShowTS(stim VisualStimulus) (uint64, error)`	Clear → draw → flip, and return the SDL nanosecond timestamp captured immediately after the VSYNC flip. Use with `GetKeyEventTS` for hardware-precision RT measurement. With an armed onset marker it keeps the stimulus up (redrawn) until the photodiode square is black again — see *Onset markers* below.
`exp.ShowTimed(stim VisualStimulus, durationMs int) error`	`Show(stim)` + `Wait` for what remains of `durationMs` since the onset flip, in one call. For fixation crosses, cues, and passive stimulus viewing.
`exp.ShowFrames(stim VisualStimulus, n int) (uint64, error)`	Hold the stimulus for exactly `n` display frames, returning the onset timestamp of the first flip. The stimulus is redrawn every frame — see *Holding a stimulus for a fixed number of frames* below.
`exp.BlankFrames(n int) (uint64, error)`	Frame-locked counterpart of `Blank(ms)`: clear and hold blank for exactly `n` frames, returning the timestamp of the first flip (i.e. the previous stimulus's offset).
`exp.ShowAndGetRT(stim VisualStimulus, keys []Keycode, timeoutMs int) (Keycode, int64, error)`	Clears stale keyboard events, shows stim with hardware-precise onset timing, waits for a key, and returns `(key, rtMs, error)`. Pass `timeoutMs = -1` for no timeout; returns `(0, 0, nil)` on timeout. This is the canonical single-stimulus RT measurement call.

Method	Description
`exp.ShowEndMessage(message string) error`	Display a centered completion message and wait for any key. For end-of-experiment screens. Laid out by `FittedTextBox`.
`exp.ShowInstructions(text string) error`	Display centered text and wait for spacebar. Laid out by `FittedTextBox`.
`exp.FittedTextBox(text string) *stimuli.TextBox`	A centered `TextBox` wrapped to the drawing area and rendered at the largest point size — never above the default — at which the whole block fits. See *Laying out a block of text* below.
`exp.DrawArea() (w, h float32)`	Size of the coordinate space stimuli are drawn in (the logical resolution). What layout code should measure against.
`exp.Blank(ms int) error`	Clear and flip screen, then wait until `ms` milliseconds have passed since that flip.
`exp.Wait(ms int) error`	Wait `ms` ms while pumping SDL events (ESC-abortable).
`exp.ShowSplash(waitForKey bool) error`	Show experiment name + version splash.
`exp.Flip() error`	Present the backbuffer and hold to the frame boundary (one call = one display frame).

9.2.6 Input

Method	Description
`exp.Keyboard`	`*apparatus.Keyboard` — see Keyboard section
`exp.Mouse`	`*apparatus.Mouse` — see Mouse section
`exp.PollEvents(handle func(sdl.Event) bool) EventState`	Process all pending SDL events; optionally forward to a handler. Returns `EventState` including nanosecond timestamps.
`exp.HandleEvents() (Keycode, uint32, error)`	Convenience wrapper: returns `(key, mouseButton, error)`.

`EventState` now includes SDL event timestamps:

```
type EventState struct {
    LastKey            sdl.Keycode
    LastMouseButton    uint32
    LastKeyTimestamp   uint64  // SDL nanosecond timestamp of the last key
↪   event
    LastMouseTimestamp uint64  // SDL nanosecond timestamp of the last mouse
↪   event
```

```
    QuitRequested      bool
}
```

9.2.7 Design and Data

Method	Description
`exp.AddDataVariableNames(names []string)`	Register CSV column names for the data file.
`exp.Data.Add(values ...interface{})`	Append a data row. Subject ID is prepended automatically.
`exp.AddBlock(b *design.Block, copies int)`	Add trial blocks to the experiment.
`exp.ShuffleBlocks()`	Randomize block presentation order.
`exp.AddBWSFactor(name string, conditions []interface{})`	Register a between-subjects factor for Latin-square counterbalancing.
`exp.GetPermutedBWSFactorCondition(name string) interface{}`	Return this subject's condition for a BWS factor.
`exp.Design`	`*design.Experiment` — full design object
`exp.Info`	`map[string]string` — values from `GetParticipantInfo`; set before `Initialize()` to persist them automatically to the `-info.txt` file
`exp.CursorVisible`	`bool` — whether the mouse pointer is shown over the experiment window. Defaults to `false`: `Initialize()` hides the cursor. Set it to `true` before `Initialize()` (or call `ShowCursor()` after) for mouse-driven paradigms.

9.2.8 Font and Display

Method	Description
`exp.LoadFont(path string, size float32) error`	Load a TTF font from file.
`exp.LoadFontFromMemory(data []byte, size float32) error`	Load a TTF font from a byte slice.
`exp.SetVSync(vsync int) error`	Toggle vertical sync (1 = on, 0 = off).
`exp.SetLogicalSize(w, h int32) error`	Set device-independent logical resolution.
`exp.SetOutputDirectory(dir string)`	Override default data file directory (`~/goxpy_data`).

9.2.9 Gamma Correction

Standard monitors apply a power-law transfer function L(V) = k·(V/255)^γ (γ ≈ 2.2 for sRGB displays). Equal steps in RGB values do **not** produce equal steps in physical luminance. Use `SetGamma` to enable inverse-gamma correction.

Method	Description
`exp.SetGamma(gamma float64)`	Install a uniform inverse-gamma corrector. Call once after `Initialize()`.
`exp.CorrectColor(c sdl.Color) sdl.Color`	Apply gamma correction to a color. Returns `c` unchanged when no corrector is set.
`exp.GammaCorrector`	`*apparatus.GammaCorrector` — set directly for per-channel calibration.

```
// Uniform gamma (typical sRGB monitor)
exp.SetGamma(2.2)

// Per-channel gamma (from photometer measurements)
exp.GammaCorrector = apparatus.NewGammaCorrector(2.1, 2.2, 2.3)

// Use in trial loop — specify colors in linear luminance space (0–255)
disk := stimuli.NewCircle(radius, exp.CorrectColor(control.RGB(128, 128,
 ↪  128)))
```

The `apparatus.GammaCorrector` type is also available directly:

```
gc := apparatus.NewGammaCorrectorUniform(2.2)
corrected := gc.CorrectColor(sdl.Color{R: 128, G: 128, B: 128, A: 255})
// corrected.R ≈ 186 — the physical digital value for 50% luminance on γ=2.2
```

9.2.10 Colors, Types, and Constants

```
// Named colors
control.Black, White, Red, Green, Blue, Yellow, Magenta, Cyan
control.Gray, DarkGray, LightGray

// Type aliases (so you only need to import "control")
type Color   = sdl.Color
type Keycode = sdl.Keycode
type FPoint  = sdl.FPoint
type FRect   = sdl.FRect

// Constructors
control.RGB(r, g, b uint8) Color
control.RGBA(r, g, b, a uint8) Color
control.Point(x, y float32) FPoint
```

```
control.Origin() FPoint  // returns (0, 0)

// Font helpers
control.FontFromFile(path string, size float32) (*ttf.Font, error)
control.FontFromMemory(data []byte, size float32) (*ttf.Font, error)

// Loop control
control.EndLoop          // sentinel error: return from Run callback to exit
control.IsEndLoop(err)   // test whether an error is the EndLoop sentinel

// Keyboard codes
control.K_SPACE, K_ESCAPE, K_RETURN, K_BACKSPACE, K_TAB
control.K_UP, K_DOWN, K_LEFT, K_RIGHT
control.K_A … K_Z                                    // full alphabet
control.K_0 … K_9                                    // digit row
control.K_KP_0 … K_KP_9, K_KP_ENTER, K_KP_PLUS, K_KP_MINUS   // numeric
 ↪  keypad
control.K_MINUS, K_PLUS, K_EQUALS, K_LEFTBRACKET, K_RIGHTBRACKET  //
 ↪  punctuation

// Mouse buttons
control.BUTTON_LEFT, BUTTON_RIGHT
```

> **Tip:** The full alphabet, digit row, keypad, and common punctuation keys are re-exported, so experiment code never needs to import `go-sdl3` just to name a key. For a rarely-used code not listed in `defaults.go`, import `go-sdl3/sdl` directly and use `sdl.K_…`.

9.2.11 Onset markers (photodiode square and TTL codes)

```
control.MarkerFlags(defaults ...control.MarkerConfig) // before
 ↪  NewExperimentFromFlags: registers
                                                      // -photodiode
                                                       ↪  -photodiode-size
                                                       ↪  -marker-frames
                                                       ↪  -ttl -ttl-ms

type control.MarkerConfig struct {
    Photodiode     bool
    PhotodiodeSize float32                // drawing-space units; 0 =
 ↪  DefaultPhotodiodeSize (100)
    Frames         int                    // white frames per onset; 0 =
 ↪  DefaultMarkerFrames (3)
    TTL            string                 // device spec for triggers.Open;
 ↪  "" = no TTL
    TTLDevice      apparatus.MarkerOutput // an already-open output, used
 ↪  instead of TTL
    TTLDuration    time.Duration          // 0 = Frames frame periods
```

```
}
exp.MarkerConfig                          // set before Initialize when not
 ↪  using flags
exp.Marker *apparatus.Marker              // never nil after Initialize
 ↪  (does nothing if unconfigured)

exp.Marker.Arm(code byte)                 // the next flip is an onset:
 ↪  square white, code written after it (0 = square only)
exp.Marker.Fire(code byte) uint64         // write a code now, no square
 ↪  (responses, Play); returns sdl.TicksNS before the write
exp.Marker.Busy() bool                    // the square still needs flips
 ↪  before it is black
exp.Marker.PhotodiodeEnabled() bool; TTLEnabled() bool; Size() float32;
 ↪  Frames() int
exp.Marker.TTLDuration() time.Duration; OutputDesc() string; LastOnsetNS()
 ↪  uint64
exp.Marker.Stats() apparatus.MarkerStats  // Onsets Codes Fires Clears
 ↪  WriteErrors Overlaps Merged
```

A code with a single bit set is written as one `SetHigh(line)` and cleared with one `SetLow(line)` (on the DLP boxes `Send` writes the 8 lines one command at a time); other codes use `Send`.

The marker is drawn and fired from inside `Screen.Update`, so it works with every presentation path. A device that cannot be opened makes `Initialize` fail before any window opens; the browser build refuses `-ttl`. The configuration and final stats are written to the `-info.txt`. See UserManual Section 17.

9.2.12 Audio

```
exp.AudioDevice  // sdl.AudioDeviceID — pass to Sound.PreloadDevice()

// Top-level helper (call before NewExperiment)
control.SetAudioSampleFrames(frames int)  // set audio buffer size
 ↪  (256–2048)
```

9.2.13 Microphone

```
exp.Microphone  // *apparatus.Microphone — nil until OpenMicrophone is
 ↪  called

// Open the default recording device (nil = F32LE mono 44100 Hz)
exp.OpenMicrophone(spec *sdl.AudioSpec) error

// Closed automatically by exp.End()
```

9.3 Package `stimuli`

Import: `github.com/chrplr/goxpyriment/stimuli`

9.3.1 Interfaces

```
type Stimulus interface {
    Present(screen *apparatus.Screen, clear, update bool) error
    Preload() error
    Unload() error
}

type VisualStimulus interface {
    Stimulus
    Draw(screen *apparatus.Screen) error
    GetPosition() sdl.FPoint
    SetPosition(pos sdl.FPoint)
}
```

`stimuli.DrawFunc(func(screen *apparatus.Screen) error)` turns a function that draws a whole scene into a `VisualStimulus` (no position, no resources of its own), so a composite display can go through `exp.Show` / `exp.ShowTS`. The function may run once per held frame.

GPU textures are **lazily allocated** on the first `Draw`. To force early allocation (for timing-sensitive code), use:

```
stimuli.PreloadVisualOnScreen(screen, stim)     // single stimulus
stimuli.PreloadAllVisual(screen, []VisualStimulus{...})  // slice
```

9.3.2 Visual Stimuli

9.3.2.1 Text

Constructor	Description
`NewTextLine(text string, x, y float32, color Color) *TextLine`	Single line of text.
`NewTextBox(text string, width int32, pos FPoint, color Color) *TextBox`	Word-wrapped multi-line text.

Both support a `Font *ttf.Font` field — set it to override the screen default.

9.3.2.2 Shapes

Constructor	Description
`NewFixCross(size, lineWidth float32, color Color) *FixCross`	Fixation cross centered at (0, 0).
`NewCircle(radius float32, color Color) *Circle`	Filled circle.
`NewRectangle(cx, cy, w, h float32, color Color) *Rectangle`	Filled rectangle.
`NewLine(x1, y1, x2, y2 float32, color Color) *Line`	Line segment.

9.3.2.3 Images and Video

Constructor/Function	Description
`NewPicture(filePath string, x, y float32) *Picture`	Image loaded from file (PNG, JPG, BMP…).
`NewPictureFromMemory(data []byte, x, y float32) *Picture`	Image loaded from embedded bytes.
`NewSpriteSheet(filePath string) *SpriteSheet`	One image holding many stimuli, sharing a single GPU texture.
`NewSpriteSheetFromMemory(data []byte) *SpriteSheet`	Same, from embedded bytes (required for the browser build).
`(*SpriteSheet) Grid(screen, cols, rows int) ([]*Sprite, error)`	Cut into `cols*rows` equal cells, row-major.
`(*SpriteSheet) GridWithSpacing(screen, cols, rows int, margin, spacing float32) ([]*Sprite, error)`	As `Grid`, for sheets with a border or gutters.
`(*SpriteSheet) Sprites(screen, clips []sdl.FRect) ([]*Sprite, error)`	Explicit source rectangles, for irregular sheets.
`(*SpriteSheet) Unload() error`	Destroys the shared texture. `Sprite.Unload` is a no-op by design.
`PlayGv(screen, path string, x, y float32) ([]UserEvent, []VideoFrameLog, error)`	Play a `.gv` (LZ4-compressed RGBA) video file, VSYNC-locked.
`PlayGvFunc(screen, path string, x, y float32, onFrame GvFrameCallback, marks ...GvMark) ([]UserEvent, []VideoFrameLog, error)`	`PlayGv` with a post-flip per-frame callback (may be nil), and optional `GvMark{Frame int; Code byte}` values: each listed frame's onset is marked with the screen's onset marker (photodiode square + TTL code, code 0 = square only) on that frame's first refresh.
`NewGvVideo(path string) (*GvVideo, error)`	Open a `.gv` file for frame-by-frame access.

9.3.2.4 Psychophysics Stimuli

Constructor	Description
`NewGaborPatch(sigma, theta, lambda, phase, psi, gamma float64, bgColor Color, size float32) *GaborPatch`	Static Gabor patch. `theta` in degrees, `lambda` = spatial wavelength in pixels.
`NewDotCloud(radius float32, bgColor, dotColor Color) *DotCloud`	Static random-dot cloud. Call `Make(nDots, dotRadius, gap)` to populate.
`NewRDS(imgSize, innerSize [2]int, shift, gap, scale int) *RDS`	Random-dot stereogram (side-by-side pair).
`NewVisualMask(w, h, dotW, dotH float32, bgColor, dotColor Color, pct int) *VisualMask`	Random-dot masking stimulus. `pct` = dot fill percentage 0–100.

9.3.2.5 Composite / Interactive

Constructor	Description
`NewThermometerDisplay(size FPoint, nSegments int, state, goal float32) *ThermometerDisplay`	Segmented progress bar. `State` and `Goal` in 0–100.
`NewChoiceGrid(choices []string, maxSelect int, prompt string) *ChoiceGrid`	Multiple-choice button grid (mouse + keyboard). See below.
`NewTextInput(msg string, pos FPoint, boxW float32, bgColor, frameColor, textColor Color) *TextInput`	Free-text keyboard input box. Call `ti.Get(screen, keyboard)`.
`NewMenu(items []string) *Menu`	Numbered keyboard-navigable list. Call `m.Get(screen, keyboard, initialSel)`.

9.3.3 ChoiceGrid

```
cg := stimuli.NewChoiceGrid(choices, maxSelect, prompt)
cg.Cols = 7       // optional: set column count (0 = auto)

selections, err := cg.Get(exp.Screen, exp.Keyboard)
// selections is a []string preserving selection order
```

- `MaxSelect > 0`: auto-submits after N selections.
- `MaxSelect == 0`: participant presses ENTER or SPACE to submit.
- BACKSPACE removes the last selection.
- Both mouse click and matching keypress (single-char labels) activate buttons.

9.3.4 Menu

```
m := stimuli.NewMenu([]string{"Option A", "Option B", "Option C"})
m.Pos = sdl.FPoint{X: 0, Y: 0}    // optional: reposition (default = screen
 ↪  center)
```

```
m.HighlightColor = control.Yellow  // optional: override highlight color

idx, err := m.Get(exp.Screen, exp.Keyboard, 0)  // 0 = initially highlight
 ↪  first item
// idx is 0-based; -1 + sdl.EndLoop on ESC/quit
```

Navigation: UP/DOWN arrows move the highlight; ENTER or SPACE confirms; number keys 1–9 (0 for tenth) select and confirm directly. The selected item is shown in `HighlightColor` with a > prefix; others use `TextColor`. `LineSpacing` controls vertical item spacing (0 = auto from font height).

9.3.5 Animated / VSYNC-locked Loops

All three functions disable GC, lock to VSYNC, and return `(MotionResult, error)`.

```
type MotionResult struct {
    Key    sdl.Keycode // interrupt key pressed (0 if none)
    Button uint8       // mouse button pressed (0 if none)
    RTms   int64       // ms from first frame to response (or total duration
 ↪  on timeout)
}
```

Function	Description
`PresentMovingDotCloud(screen, nDots int, dotRadius, cloudRadius float32, center FPoint, speedPxPerSec float32, maxDurationMs int64, interruptKeys []Keycode, catchMouse bool, dotColor, bgColor Color) (MotionResult, error)`	Animated random-dot cloud. Each dot moves at a fixed speed and respawns when it exits the boundary.
`PresentMovingGrating(screen, width, height float32, center FPoint, orientation, spatialFreq, temporalFreq, contrast, bgLuminance float64, maxDurationMs int64, interruptKeys []Keycode, catchMouse bool) (MotionResult, error)`	Drifting sinusoidal grating in a rectangular aperture.
`PresentMovingGabor(screen, size float32, sigma float64, center FPoint, orientation, spatialFreq, temporalFreq, contrast, bgLuminance float64, maxDurationMs int64, interruptKeys []Keycode, catchMouse bool) (MotionResult, error)`	Drifting Gabor patch with Gaussian envelope (alpha-blended edges).

Spatial frequency is in **cycles per pixel** (e.g. `0.05` = one cycle every 20 px). Temporal frequency is in **Hz**. Orientation is in **degrees from horizontal** (0° = vertical bars drifting right).

9.3.6 Stimulus Streams (High-Precision RSVP)

Stream functions disable GC, lock every onset and offset to a VSYNC boundary, and return `([]UserEvent, []TimingLog, error)`.

```
type UserEvent struct {
    Event       sdl.Event     // raw SDL event (KeyboardEvent,
 ↪  MouseButtonEvent, …)
    Timestamp   time.Duration // time relative to stream start (Go clock, ms
 ↪  precision)
    TimestampNS uint64        // SDL3 hardware event timestamp, nanoseconds
 ↪  (same clock as Screen.FlipTS)
}

type TimingLog struct {
    Index        int
    TargetOn     time.Duration
    ActualOnset  time.Duration // DIAGNOSTIC (Go clock): first-frame draw,
 ↪  stream-relative; not for RT
    ActualOffset time.Duration // DIAGNOSTIC (Go clock): after last
 ↪  on-frame, stream-relative; not for RT
    OnsetNS      uint64        // AUTHORITATIVE: SDL3 ns timestamp
 ↪  (sdl.TicksNS clock) of the stimulus onset
    OffsetNS     uint64        // AUTHORITATIVE: SDL3 ns timestamp
 ↪  (sdl.TicksNS clock) of the stimulus offset
}
```

The `OnsetNS`/`OffsetNS` pair is the authoritative timing record: both are on the SDL3 nanosecond clock (`sdl.TicksNS()`) — the same clock that stamps input events (`UserEvent.TimestampNS`, `Keyboard.GetKeyEventTS`) and VSYNC flips (`Screen.FlipTS`). A reaction time or a displayed duration is a plain subtraction within this one clock: `int64(event.TimestampNS - l.OnsetNS)`, or `int64(l.OffsetNS - l.OnsetNS)`.

`ActualOnset`/`ActualOffset` are Go-clock (`time.Now`) **diagnostics only**, kept for coarse pacing checks. They live on a different timebase (different origin) from the NS fields and from input events, so they must never be subtracted from an event timestamp or logged as a canonical onset/offset. Use the NS fields for any real timing.

`OnsetNS` is zero only when a stimulus was never displayed (zero on-frames); `OffsetNS` is the VSYNC timestamp of the flip that takes the stimulus down, and for the final element of a contiguous stream (which has no ISI flip of its own) it is synthesised from the onset plus the on-frame count so that it, too, is on the SDL clock rather than one frame early.

`UserEvent.TimestampNS` and `TimingLog.OnsetNS`/`OffsetNS` are all on the SDL3 nanosecond clock, so reaction times measured during a stream can be computed with full hardware precision:

```
for _, ev := range events {
    if ev.Event.Type == sdl.EVENT_KEY_DOWN {
        // Find the stimulus that was on-screen when the key was pressed
```

```
        for _, l := range logs {
            if ev.TimestampNS >= l.OnsetNS && ev.TimestampNS < l.OffsetNS {
                rtNS := int64(ev.TimestampNS - l.OnsetNS)
                fmt.Printf("RT from stimulus %d: %d ms\n", l.Index, rtNS/1_
↪  000_000)
            }
        }
    }
}
```

9.3.6.1 Searching event lists

Function	Description
`FirstKeyPress(events []UserEvent, ke y sdl.Keycode) (UserEvent, bool)`	Returns the first `KEY_DOWN` event matching key from the slice, plus a found flag.

```
if ev, ok := stimuli.FirstKeyPress(events, sdl.K_SPACE); ok {
    fmt.Printf("Space pressed at %d ms\n", ev.Timestamp.Milliseconds())
}
```

9.3.6.2 Visual Streams

```
// RSVP text stream — simplest entry point
events, logs, err := stimuli.PresentStreamOfText(
    exp.Screen, words, durationOn, durationOff, x, y, color,
)

// Image/mixed stream
elements := stimuli.MakeRegularVisualStream(stims, durationOn, durationOff)
events, logs, err := stimuli.PresentStreamOfImages(exp.Screen, elements, x,
↪  y)

// Irregular timing
elements, err := stimuli.MakeVisualStream(stims, onsetMs, durationMs)
events, logs, err := stimuli.PresentStreamOfImages(exp.Screen, elements, x,
↪  y)
```

9.3.6.3 Audio Streams

```
// Regular timing
elements := stimuli.MakeRegularSoundStream(sounds, durationOn, durationOff)
events, logs, err := stimuli.PlayStreamOfSounds(elements)
```

```
// Irregular timing
elements, err := stimuli.MakeSoundStream(sounds, onsetMs, durationMs)
events, logs, err := stimuli.PlayStreamOfSounds(elements)
```

`sounds` is `[]stimuli.AudioPlayable` — satisfied by both `*Sound` and `*Tone`. The returned `TimingLog` carries `OnsetNS`/`OffsetNS` on the SDL3 clock (captured at the `Play()` instant and the end of the on-phase), so audio-stream onsets are directly comparable with input-event timestamps, same as visual streams.

9.3.6.4 Mixed Streams

`PresentStreamOfStimuli` presents a heterogeneous, **sequential** stream that freely mixes visual and audio stimulus types (anything satisfying the broad `Stimulus` interface). It uses the same VSYNC-locked, GC-disabled loop as `PresentStreamOfImages` (which now delegates to it).

```
elements := []stimuli.StreamElement{
    {Stimulus: stimuli.NewTextLine("Ready?", 0, 0, color), DurationOn: 600 *
     ↪  time.Millisecond, DurationOff: 200 * time.Millisecond},
    {Stimulus: pic,  DurationOn: 800 * time.Millisecond}, // held while the
     ↪  tone plays
    {Stimulus: tone, DurationOn: 400 * time.Millisecond, DurationOff: 200 *
     ↪  time.Millisecond},
}
events, logs, err := stimuli.PresentStreamOfStimuli(exp.Screen, elements,
 ↪  x, y)

// Builders mirror the visual/sound ones but take []stimuli.Stimulus:
elements = stimuli.MakeRegularStream(stims, durationOn, durationOff)
elements, err = stimuli.MakeStream(stims, onsetMs, durationMs)
```

Per-element semantics:

- **Visual** elements (those satisfying `VisualStimulus`) are centered on `(x, y)` and redrawn every frame for `DurationOn`, then blanked for `DurationOff`.
- **Audio / non-visual** elements (and a `nil` Stimulus) are triggered once, right after the slot's first VSYNC flip, and **do not clear the screen** — the previous frame is held for the whole slot. Place a visual just before a sound to keep it on screen while the sound plays. `TimingLog.OnsetNS` is the VSYNC reference at which the audio was triggered.

Audio elements must already be bound to the device via `PreloadDevice(exp.AudioDevice)` before the call (same precondition as `PlayStreamOfSounds`). For pure-audio streams prefer `PlayStreamOfSounds`, whose sub-frame sleep-polling gives finer timing than 60 Hz frame quantization.

9.3.6.5 Per-frame callback

`PresentStreamOfStimuliFunc` adds an optional `FrameCallback` invoked once per frame, **just before each flip**. Use it to draw a persistent overlay (trial counter, frame border, fixation) on top of the stimulus, or to run real-time logic such as firing feedback the instant a response window lapses — things the plain stream functions cannot express.

```
header := stimuli.NewTextLine("3 / 40", 0, -300, control.White)
cb := func(ctx stimuli.FrameContext) error {
    _ = header.Draw(ctx.Screen)           // overlay, rendered over the
 ↪  stimulus
    if ctx.OnPhase && ctx.FirstFrame {    // onset of element ctx.Index
        // real-time feedback using ctx.NowNS / ctx.Events;
        // return sdl.EndLoop to stop early, any other error to abort
    }
    return nil
}
events, logs, err := stimuli.PresentStreamOfStimuliFunc(exp.Screen,
 ↪  elements, x, y, cb)
```

`FrameContext` fields: `Screen`, `Index`, `Frame`, `OnPhase`, `FirstFrame`, `NowNS` (pre-flip `sdl.TicksNS()`), `Elapsed`, `Events` (through the previous frame). Passing `nil` for the callback is identical to `PresentStreamOfStimuli`.

9.3.6.6 Per-stimulus onset trigger (post-flip hook)

To emit a hardware TTL — or run any action — aligned to each stimulus **onset**, use `PresentStreamOfStimuliHooks`, which adds an optional `OnsetCallback` to the per-frame callback:

```
func PresentStreamOfStimuliHooks(
    screen *apparatus.Screen, elements []StreamElement, x, y float32,
    onFrame FrameCallback, onOnset OnsetCallback,
) ([]UserEvent, []TimingLog, error)

type OnsetCallback func(index int, onsetNS uint64) error
```

`onOnset` fires once per element with a stimulus onset, **immediately after** the VSYNC flip that turns it on (for a held audio element whose `DurationOn` rounds to zero frames, after the first-off-frame flip that triggers it). `index` matches the returned `TimingLog` slice and `onsetNS` equals `TimingLog[index].OnsetNS`.

This is the **post-flip** counterpart to `FrameCallback`: `FrameCallback` runs one frame earlier (pre-flip, at GPU-submission time), whereas `OnsetCallback` runs on the photon side of the frame boundary, so a trigger emitted here coincides with the logged onset instead of leading it by a frame.

```
dev := triggers.NewParallelPort("/dev/parport0") // any
 ↪  triggers.OutputTTLDevice
if err := dev.Open(); err != nil {
```

```
    log.Fatal(err)
}
defer dev.Close()

onset := func(index int, onsetNS uint64) error {
    return dev.Send(codes[index])        // per-stimulus code, at the
     ↪  displayed onset
}
clear := func(ctx stimuli.FrameContext) error {
    if !ctx.OnPhase && ctx.FirstFrame {  // drop the lines at the start of
     ↪  each ISI
        return dev.AllLow()
    }
    return nil
}
events, logs, err := stimuli.PresentStreamOfStimuliHooks(exp.Screen,
 ↪  elements, 0, 0, clear, onset)
```

The hook runs on the timing-critical path with GC disabled, between the onset flip and the next frame, so it **must be short and non-blocking**: emit an edge (`SetHigh` / `Send`) here and clear it from a later `FrameCallback` or after the stream — never `time.Sleep` or a blocking `Pulse`, or a frame is missed. Elements with a `nil` Stimulus (pure delay / hold slots) have no onset and do not fire it. Either callback may be `nil`; `PresentStreamOfStimuliFunc` is exactly `PresentStreamOfStimuliHooks(..., onFrame, nil)`.

For pure-audio streams the equivalent is `PlayStreamOfSoundsHook(elements, onOnset)`, which fires `onOnset` once per non-nil `Sound`, immediately after `Play()`, with the same SDL-clock `onsetNS` (`== TimingLog[index].OnsetNS`). `PlayStreamOfSounds` is `PlayStreamOfSoundsHook(elements, nil)`.

9.3.7 Audio Stimuli

```
// WAV file
snd := stimuli.NewSound(filePath)
snd.PreloadDevice(exp.AudioDevice)
snd.Play()
snd.Wait()                                      // block until done
snd.PlaySegment(onset, offset, rampSec)        // time-delimited segment with
 ↪  optional fade

// Embedded WAV
snd := stimuli.NewSoundFromMemory(data)

// Procedural tone
tone := stimuli.NewTone(frequency, duration, volume)   // duration:
 ↪  time.Duration; volume: 0–255
tone.PreloadDevice(exp.AudioDevice)
tone.Play()
```

```
// One-shot helper (no preload needed)
stimuli.PlaySoundFromMemory(exp.AudioDevice, data)

// Embedded feedback sounds (via assets_embed)
import "github.com/chrplr/goxpyriment/assets_embed"
stimuli.PlaySoundFromMemory(exp.AudioDevice, assets_embed.BuzzerWav)
stimuli.PlaySoundFromMemory(exp.AudioDevice, assets_embed.CorrectWav)
```

9.4 Package apparatus and results

Import: `github.com/chrplr/goxpyriment/apparatus` (screen, input, gamma) Import: `github.com/chrplr/goxpyriment/results` (data file)

In normal experiments you access `apparatus` types through `exp.Screen`, `exp.Keyboard`, `exp.Mouse`, and `exp.Data`. Direct use of `apparatus` is only needed when writing custom stimulus types.

9.4.1 Screen

All stimulus positions use a **center-origin coordinate system**: `(0, 0)` is the screen center; positive Y is upward.

`screen.Width` and `screen.Height` are the **logical** drawing space — the coordinate system stimuli are positioned in — and move with `SetLogicalSize`. They are what layout code should measure against; a width computed from anything else lands somewhere other than where `CenterToSDL` puts it. For *physical* pixels, as in a degrees-of-visual-angle calculation, ask `screen.Renderer.CurrentOutputSize()` instead. In fullscreen with no logical size set the two are the same, which is what makes the distinction easy to miss.

```
screen.CenterToSDL(x, y float32) (float32, float32)  // convert to SDL
 ↪  top-left coords
screen.CenteredRect(pos FPoint, w, h float32) *FRect  // SDL dest rect for a
 ↪  w×h texture centered at pos
screen.MousePosition() (float32, float32)              // current cursor in
 ↪  center coords
screen.Clear() error                                   // fill with
 ↪  background color
screen.Update() error                                  // present + hold to
 ↪  the frame boundary
screen.Flip() error                                    // alias for Update
screen.FlipTS() (uint64, error)                        // present + return
 ↪  SDL nanosecond timestamp after flip
screen.FrameDuration() time.Duration                   // nominal frame
 ↪  duration (falls back to 60 Hz)
```

```
screen.CalibrateRefresh(n int) (time.Duration, error)  // measured frame
 ↪  period, pacing bypassed
screen.SetLogicalSize(w, h int32) error                // also updates
 ↪  screen.Width/Height
screen.SetVSync(vsync int) error
screen.DisplayInfo() apparatus.DisplayInfo                    // monitor
 ↪  properties
screen.SetMarker(m *apparatus.Marker)                  // drive an onset
 ↪  marker from Update (nil detaches)
screen.Marker() *apparatus.Marker
screen.SetPrePresentHook(f func() error)               // run f after
 ↪  drawing, just before present (frame read-back)
screen.Destroy()
```

`apparatus.NewMarker(apparatus.MarkerOptions{Photodiode, Size, Frames, Output, OutputDesc, TTLDuration})` builds the onset marker that `control` sets up from `MarkerConfig`; `apparatus.MarkerOutput` (`Send`, `AllLow`, `Close`) is satisfied by `triggers.OutputTTLDevice`.

9.4.2 Laying out a block of text

`ShowInstructions`, `ShowEndMessage` and `FittedTextBox` size a text screen by measuring it, not by a fixed fraction of the window. Two things go wrong when a wrap width is written as a constant share of the screen:

- **The column count moves with the display.** A wrap width in pixels divided by a font in points is a number of characters, and it changes per machine: at 28 pt in the default monospace font, 80 % of a 1024-pixel-wide screen is 48 characters and 80 % of a 1920-pixel one is 90. Instruction text hand-wrapped in the source at the usual 55–70 columns therefore survives on the author's screen and is re-broken on a narrower one, each over-long line becoming a full line plus a short orphan.
- **Nothing checks the height.** A screen that has grown a few lines simply runs off the bottom edge, with no error and nothing in the log.

`FittedTextBox` wraps at `0.92 × DrawArea().w` and then picks the largest point size, up to `exp.DefaultFontSize`, at which the rendered block fits within `0.90 × DrawArea().h`. Where it can do so without shrinking below `keepBreaksFloor` (three quarters) of that size, it prefers a size at which no line the author wrote has to be re-wrapped — so hand-wrapped text keeps its own breaks on any display, while a paragraph written as one long line still wraps, as intended. Below `control.MinTextFontSize` (11 pt) it stops and logs a warning: a screen that will not fit at 11 pt is too long, not too big.

The fitted font belongs to the experiment and is closed by `End()`; a caller of `FittedTextBox` only has to `Unload()` the box.

`FlipTS` returns `sdl.TicksNS()` captured immediately after the flip. This timestamp is on the same nanosecond clock as SDL3 event timestamps, so `int64(event.Timestamp - onsetNS)` gives hardware-precision reaction time without any polling latency.

`Update` (and therefore `Flip`, `FlipTS`, `Show`, `ShowTS`) presents **and then holds to the frame boundary**, so one call always occupies exactly one display frame. This is unconditional because `SDL_RenderPresent` cannot be trusted to block until the retrace — under triple/mailbox buffering it returns immediately — and the wait costs nothing where the driver does block. Pacing is skipped when VSync is off.

`CalibrateRefresh` bypasses that pacing and presents directly, so it reports the *unaided* driver behaviour; compare it against `FrameDuration`. `Initialize` runs it over 60 frames at startup and writes `sys.refresh_nominal_hz` / `sys.refresh_measured_hz` into the session's `-info.txt`, warning if they differ by more than 10%. Measured slower than nominal means frames are being dropped before the panel, which pacing cannot fix.

9.4.2.1 Holding a stimulus for a fixed number of frames

There is no "wait for *n* VSYNC edges" call, and there cannot be a useful one: SDL3 exposes no `SDL_WaitVBlank`, so the only way to stay locked to the display is to present a frame — and a frame carrying no draw calls is not reliably scanned out under a compositor (see *Never present a frame with no draw calls* in `apparatus/CLAUDE.md`). A hold must therefore **redraw its content once per frame**:

```
for i := 0; i < n; i++ {
    screen.Clear()
    stim.Draw(screen)
    screen.Flip()
}
```

`exp.ShowFrames(stim, n)` and `exp.BlankFrames(n)` wrap that loop and return the onset timestamp of the first flip:

```
onsetNS, _ := exp.ShowFrames(rect, 10)   // 10 frames on screen
offsetNS, _ := exp.BlankFrames(10)       // 10 frames blank
exp.Data.Add(onsetNS, offsetNS)
```

There is no "wait for n frames" call that skips the redraw. A hold must redraw its content once per frame: SDL invalidates the backbuffer on every present, so re-clearing with whatever draw colour was last set paints the screen in that colour instead of holding the stimulus. Use the two calls above.

9.4.3 Keyboard

```
key, err := exp.Keyboard.Wait()                                  // any key
key, err := exp.Keyboard.WaitKey(control.K_SPACE)               // specific
 ↪ key
key, err := exp.Keyboard.WaitKeys(keys, timeoutMS)               // first
 ↪ of several keys (-1 = no timeout)
key, rt, err := exp.Keyboard.WaitKeysRT(keys, timeoutMS)         // with RT
 ↪ in ms from call site
key, ts, err := exp.Keyboard.GetKeyEventTS(keys, timeoutMS)      // with
 ↪ SDL event timestamp (nanoseconds)
```

```
events, err := exp.Keyboard.GetKeyEventsTS(keys, timeoutMS)       // first
 ↪  key + 50 ms window; for bilateral responses
events, err := exp.Keyboard.CollectKeyEventsTS(keys, durationMS)  // all
 ↪  keys during full fixed window
key, err := exp.Keyboard.Check()                                  //
 ↪  non-blocking poll
held := exp.Keyboard.IsPressed(key)                              // true if
 ↪  key is physically held now
upTS, err := exp.Keyboard.WaitKeyReleaseTS(key, timeoutMS)        // wait
 ↪  for KEY_UP; returns hardware timestamp
exp.Keyboard.Clear()                                            // drain SDL
 ↪  event queue
```

`WaitKeys` and `WaitKeysRT` return `0, nil` on timeout; return `sdl.EndLoop` on ESC or window close.

`IsPressed` queries SDL's scancode state array — no event queue involvement. `WaitKeyReleaseTS` returns the KEY_UP hardware timestamp so that `upTS - downTS` gives nanosecond-precision press duration.

`GetKeyEventsTS` — two-phase collection for bilateral responses. The timeout governs how long to wait for the *first* key. After the first key arrives, the function waits an additional 50 ms for any second key before returning. This extra window is necessary because human "simultaneous" bilateral presses (e.g. both hands at once) arrive 10–50 ms apart — a non-blocking drain after the first key would miss the second key almost every time. Use `GetKeyEventTS` for ordinary single-key trials to avoid the 50 ms overhead.

`GetKeyEventTS` returns the SDL3 `KeyboardEvent.Timestamp` field — the nanosecond time at which the hardware key-down event was generated, on the same clock as `sdl.TicksNS()` and `Screen.FlipTS()`. This allows computing reaction time from any specific stimulus onset without manual arithmetic:

```
onset, _ := exp.ShowTS(stim1)    // nanoseconds at VSYNC flip
exp.Wait(500)
exp.ShowTS(stim2)
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
rtToStim1 := int64(eventTS - onset)  // nanoseconds
```

9.4.4 Mouse

```
x, y := exp.Mouse.Position()                                    // current position
 ↪  (center coords)
btn, err := exp.Mouse.WaitPress()                                // block until
 ↪  button pressed
btn, rt, err := exp.Mouse.WaitPressRT(timeoutMS)                 // with RT in ms
 ↪  from call site
btn, ts, err := exp.Mouse.GetPressEventTS(timeoutMS)             // with SDL event
 ↪  timestamp (nanoseconds)
```

```
btn, err := exp.Mouse.Check()                              // non-blocking poll
held := exp.Mouse.IsPressed(sdl.BUTTON_LEFT)                 // true if button
 ↪  is physically held now
upTS, err := exp.Mouse.WaitButtonReleaseTS(btn, timeoutMS) // wait for
 ↪  MOUSE_BUTTON_UP; hardware timestamp
exp.Mouse.ShowCursor(show bool) error
```

`WaitPressRT` mirrors `Keyboard.WaitKeysRT`: reaction time is measured in milliseconds from the call site. `GetPressEventTS` returns the SDL3 hardware event timestamp in nanoseconds, suitable for use with `ShowTS`. `IsPressed` and `WaitButtonReleaseTS` mirror the keyboard's `IsPressed` and `WaitKeyReleaseTS`.

9.4.5 GamePad

```
pads, err := apparatus.GetGamePads()                                    //
 ↪  enumerate connected gamepads
defer pads[0].Close()

btn, err := pads[0].WaitPress()                                  // block until
 ↪  any button
btn, ts, err := pads[0].GetPressEventTS(timeoutMS)               // with SDL
 ↪  event timestamp (nanoseconds)
```

`GetPressEventTS` returns the `GamepadButtonEvent.Timestamp` field — same nanosecond clock as `Screen.FlipTS` and keyboard/mouse event timestamps.

9.4.6 Unified Input — `WaitAnyEventTS`

When the response device is not fixed in advance (keyboard *or* mouse click), use the method on `Experiment`:

```
// Accept F or J key, or any mouse button, timeout after 3 s
ev, err := exp.WaitAnyEventTS(
    []control.Keycode{control.K_F, control.K_J},
    true,   // catchMouse
    3000,
)
```

Returns an `apparatus.InputEvent`:

```
type InputEvent struct {
    Device       apparatus.DeviceKind     // DeviceKeyboard | DeviceMouse |
 ↪  DeviceGamepad
    Key          sdl.Keycode       // non-zero for keyboard events
    Button       uint32             // non-zero for mouse events
    GamepadButton sdl.GamepadButton // non-zero for gamepad events
    TimestampNS  uint64             // SDL3 hardware timestamp, nanoseconds
}
```

`TimestampNS` is on the same clock as `ShowTS`, so RT computation is identical regardless of device:

```
onset, _ := exp.ShowTS(stim)
ev, _ := exp.WaitAnyEventTS(keys, true, -1)
rtNS := int64(ev.TimestampNS - onset)
```

Pass `keys = nil` to accept any key. Pass `catchMouse = false` to ignore the mouse. On timeout, returns a zero `InputEvent` and `nil` error. On ESC or quit, returns `sdl.EndLoop`.

9.4.7 ResponseDevice

`ResponseDevice` is a unified interface over all participant-input hardware — SDL-event-driven devices (keyboard, mouse, gamepad) **and** polled TTL devices (MEGTTLBox, DLPIO8). It is the recommended abstraction when the experiment design does not commit to a specific input modality.

```
type ResponseDevice interface {
    WaitResponse(ctx context.Context) (Response, error)
    DrainResponses(ctx context.Context) error
    Close() error
}

type Response struct {
    Source  apparatus.DeviceKind  // DeviceKeyboard | DeviceMouse |
↪   DeviceGamepad | DeviceTTL
    Code    uint32         // SDL Keycode, mouse button, gamepad button, or
↪   TTL bitmask
    RT      time.Duration  // elapsed from WaitResponse call to detection
    Precise bool           // true = hardware event timestamp; false =
↪   software poll
}
```

`Response.Precise` distinguishes two timing regimes:

Device	Precise	RT origin
Keyboard, Mouse, Gamepad	`true`	SDL3 hardware event timestamp (nanosecond)
MEGTTLBox, DLPIO8	`false`	`time.Now()` at poll detection (±poll interval, ~5 ms)

Construct wrappers with the provided adapters:

```
// SDL-event-driven devices
rd := &apparatus.KeyboardResponseDevice{KB: exp.Keyboard}
rd := &apparatus.MouseResponseDevice{M: exp.Mouse}
rd := &apparatus.GamepadResponseDevice{GP: pad}
```

```
// Polled TTL device (MEGTTLBox, DLPIO8, or any type with
 ↪  ReadAll/DrainInputs)
box, _ := triggers.NewMEGTTLBox("/dev/ttyACM0")
rd := apparatus.NewTTLResponseDevice(box, 5*time.Millisecond)
```

Usage in a trial loop:

```
onset, _ := exp.ShowTS(stim)
_ = rd.DrainResponses(ctx)
resp, err := rd.WaitResponse(ctx)
// resp.RT is always valid; resp.Precise tells you whether to trust
 ↪  nanosecond accuracy
```

9.4.8 Microphone

```
// Construction
mic, err := apparatus.NewMicrophone(spec)                          // default
 ↪  recording device
mic, err := apparatus.NewMicrophoneFromDevice(devID, spec)          //
 ↪  specific device
apparatus.DefaultRecordingSpec() sdl.AudioSpec                      // F32LE
 ↪  mono 44100 Hz
devices, err := apparatus.GetRecordingDevices() []sdl.AudioDeviceID

// Fields
mic.DeviceID  sdl.AudioDeviceID
mic.Stream    *sdl.AudioStream
mic.Spec      sdl.AudioSpec

// Methods
captureStartNS, err := mic.StartCapture()    // flush buffer, resume device,
 ↪  return TicksNS()
err := mic.StopCapture()                     // pause device
n, err := mic.ReadSamples(buf []byte)       // read PCM bytes (non-blocking)
n, err := mic.Available()                    // bytes waiting to be read
mic.Close()                                  // destroy stream
```

In normal experiments, open the microphone via `exp.OpenMicrophone(spec)` (see control package); `exp.Microphone` is then available throughout the experiment and closed by `exp.End()`.

9.4.9 VoiceKey

Detects voice onset by computing per-window RMS over F32LE PCM samples.

```
vk := apparatus.NewVoiceKey(mic, threshold float32, windowSz int) *VoiceKey
// threshold: RMS amplitude (0–1). Recommended starting value: 0.02–0.05.
```

```
// windowSz:  samples per RMS window. 0 → 128 (≈ 2.9 ms at 44100 Hz).

captureStartNS, err := vk.Arm()   // flush mic buffer, start capture, return
 ↪  TicksNS()

onsetNS, pcm, err := vk.WaitOnset(captureStartNS uint64, timeoutMS int)
 ↪  (uint64, []byte, error)
// Returns: onset timestamp (sdl.TicksNS() domain), all PCM bytes captured,
 ↪  nil or ErrVoiceTimeout.
```

`onsetNS` and the `imageOnsetNS` returned by `exp.ShowTS` or `screen.FlipTS` are on the same SDL3 nanosecond clock:

```
captureStartNS, _ := vk.Arm()
imageOnsetNS, _   := exp.ShowTS(picture)
onsetNS, pcm, _   := vk.WaitOnset(captureStartNS, 2000)
rtMs := int64(onsetNS - imageOnsetNS) / 1_000_000
```

Sentinel error: `apparatus.ErrVoiceTimeout`

Post-hoc helpers (for re-analysing saved WAV files):

```
sampleIdx := apparatus.ScanOnset(pcm []byte, threshold float32, windowSz
 ↪  int) int
// Returns the sample index of the first onset window, or -1 if not found.

onsetNS := apparatus.SampleOnsetNS(captureStartNS uint64, sampleIndex,
 ↪  sampleRate int) uint64
// Converts a sample index back to a nanosecond timestamp.

names, err := apparatus.DeviceNames() map[sdl.AudioDeviceID]string
// Returns all recording devices and their human-readable names.
```

9.4.10 WriteWAV

```
apparatus.WriteWAV(path string, spec sdl.AudioSpec, data []byte) error
```

Saves raw PCM bytes as a standard RIFF/WAV file. Supports `AUDIO_F32LE`, `AUDIO_S16LE`, `AUDIO_S32LE`, and `AUDIO_U8`. Writes atomically via a temp file + rename.

9.4.11 DataFile

```
exp.Data.Add(field1, field2, ...)              // append a data row
exp.Data.AddVariableNames([]string{...})       // write column header
exp.Data.WriteDisplayInfo(info)                // append display metadata to
 ↪  the info file
```

```
exp.Data.WriteHostInfo(sysinfo.Host())        // append machine/OS metadata
 ↪  to the info file
exp.Data.WriteParticipantInfo(info)           // append --PARTICIPANT INFO
 ↪  block to the info file (called automatically by Initialize when exp.Info
 ↪  is set)
exp.Data.WriteEndTime()                       // append end time + duration
 ↪  to the info file
```

Two files are written to `~/goxpy_data/` for each session:

File	Contents
`<expname>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>.csv`	Pure CSV data rows — directly importable by Excel, R, or pandas
`<expname>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>-info.txt`	#-prefixed metadata: start/end time, hostname, OS, framework version, display and audio configuration, participant info

---

9.5 Package `media`

Import: `github.com/chrplr/goxpyriment/media`

Multi-movie playback with shared master-clock synchronisation, look-ahead frame conditions, and post-vsync display events for hardware- trigger alignment. Adds `MovieManager` (per-experiment owner of the clock and movie set), `Movie` (per-movie state with PsyScope-Tahoe semantics), and the `media/present` subpackage of OS-level vsync backends. Pure Go, `.gv` movies only, silent.

Full reference (precision contracts, platform support, mapping to PsyScope movie commands): see MediaMovies.md.

9.5.1 Quick API summary

```
mgr := media.NewMovieManager(exp.Screen)
defer mgr.Close()

gv, _ := gvvideo.LoadGVVideo("clip.gv")
mov, _ := media.NewMovie(mgr, gv,
    media.WithTag("M1"),
    media.WithRepeat(-1),
    media.WithSize(640, 360),
    media.WithPosition(sdl.FPoint{X: -200, Y: 0}),
)
defer mov.Close()
```

```
// Look-ahead — fires from inside DrawWithoutFlip, BEFORE the frame is
 ↪  presented.
mov.OnAt(media.Frame(186), func(o media.Onset) { /* ... */ })

// Hardware-verified — fires from NotifyFlipped with the OS vsync timestamp.
mov.OnAtDisplay(media.Frame(186), func(o media.Onset) {
    _ = ttl.Pulse(0, 5*time.Millisecond)
})

// Display[Onset/Offset] — fires when the named tag changes visibility.
mgr.OnDisplayOnset("M1", func(o media.Onset) { /* ... */ })

// Atomic command burst (PsyScope script-line equivalent):
mgr.BeginBurst()
movieA.Pause(); movieB.Pause(); movieC.Pause()
mgr.EndBurst()

// Per-frame, mixed compositing:
exp.Screen.Clear()
mgr.DrawWithoutFlip()           // decode + render movies
fix.Present(exp.Screen, false, false)  // overlay
ts, _ := exp.Screen.FlipTS()
mgr.NotifyFlipped(ts)
```

9.5.2 Onset precision summary

Source	Where	Meaning
`LookAhead`	`OnAt` / `OnDone` callbacks	`sdl.TicksNS()` at fire time, pre-vsync
`VsyncEstimated`	`OnAtDisplay` / `Display*` callbacks (fallback timer)	Post-`Present` `FlipTS`, ~vsync-period
`HardwareVerified`	`OnAtDisplay` / `Display*` callbacks (Stage 5 timer)	OS-measured first-pixel-visible, sub-ms

9.5.3 Platform status

Platform	Backend	Precision
macOS	`CVDisplayLink` (CoreVideo via purego)	`HardwareVerified`
Linux	`DRM_IOCTL_WAIT_VBLANK` (`/dev/dri/cardN`, `video` group)	`HardwareVerified`
Windows	(not yet — fallback)	`VsyncEstimated`
Other	(fallback)	`VsyncEstimated`

See MediaMovies.md §8 for the Windows roadmap and §7 for the trigger-synchrony contract (the wire pulse arrives ~0.5–5 ms after `Onset.TimestampNS`; calibrate the constant offset with a photodiode for sub-ms wire-side alignment).

9.6 Package `design`

Import: `github.com/chrplr/goxpyriment/design`

9.6.1 Data Structures

```
// Trial — one experimental trial
trial := design.NewTrial()
trial.SetFactor("condition", "congruent")
trial.GetFactor("condition")   // → "congruent"
trial.Copy()                   // deep copy

// Block — a sequence of trials
block := design.NewBlock("Practice")
block.SetFactor("type", "practice")
block.AddTrial(trial, copies, randomPosition)
block.ShuffleTrials()

// Experiment design (separate from control.Experiment)
exp.Design  // *design.Experiment — contains Blocks, DataVariableNames, etc.
```

9.6.2 Randomization

```
design.RandInt(a, b int) int                          // random int in [a, b]
design.RandElement(list []T) T                        // random element
 ↪  (generic)
design.CoinFlip(headBias float64) bool                // weighted coin flip
design.RandNorm(a, b float64) float64                 // truncated normal in
 ↪  [a, b]
design.ShuffleList(list []T)                         // in-place Fisher-Yates
 ↪  shuffle (generic)
design.MakeMultipliedShuffledList(list []T, n int) []T  // n shuffled
 ↪  copies concatenated
design.RandIntSequence(first, last int) []int         // shuffled range
 ↪  [first, last]
```

9.6.3 Latin Square (Between-Subjects Counterbalancing)

```
// Registration
```

```
exp.AddBWSFactor("handedness", []interface{}{"left", "right"})

// At runtime — returns the condition for the current subject
condition := exp.GetPermutedBWSFactorCondition("handedness")

// Low-level
square, err := design.LatinSquare(elements, design.PBalancedLatinSquare)
square, err := design.LatinSquareInts(n, design.PCycledLatinSquare)
// permutation types: design.PBalancedLatinSquare, PCycledLatinSquare,
 ↪  PRandom
```

9.7 Package `clock`

Import: `github.com/chrplr/goxpyriment/clock`

```
clock.Wait(ms int)                    // block for ms milliseconds
clock.GetTime() int64                 // ms since package first used
clock.GetTimeNS() int64              // nanoseconds since package first used

c := clock.NewClock()                 // clock relative to "now"
c.NowMillis() int64                   // ms elapsed
c.NowNanos() int64                    // nanoseconds elapsed
c.Now() time.Duration
c.Reset()                             // restart reference
c.SleepUntil(target time.Duration)    // sleep until target offset (returns
 ↪  immediately if past)
```

> **Note:** Prefer `exp.Wait(ms)` over `clock.Wait(ms)` in experiment code — `exp.Wait` pumps SDL events and detects ESC.
>
> **Clock domains:** `GetTimeNS()` and `NowNanos()` use the Go monotonic clock (`time.Since`). SDL event timestamps from `Screen.FlipTS`, `GetKeyEventTS`, `GetPressEventTS`, and `WaitAnyEventTS` use `sdl.TicksNS()`. The two clocks have different origins and **must not be subtracted from each other** for reaction-time computation. Use the SDL-based functions exclusively for RT measurement.

9.8 Package `geometry`

Import: `github.com/chrplr/goxpyriment/geometry`

```
geometry.GetDistance(p1, p2 sdl.FPoint) float32
geometry.CartesianToPolar(x, y float32) (radius, angleDeg float32)
geometry.PolarToCartesian(radius, angleDeg float32) (x, y float32)
```

```
geometry.DegreeToRadian(deg float32) float64
```

9.9 Package `triggers`

Import: `github.com/chrplr/goxpyriment/triggers`

Provides hardware TTL signal output (EEG/MEG trigger codes) and TTL input (response pads wired over serial). Lines are **0-indexed (0–7)** throughout; bit N of a bitmask corresponds to line N.

9.9.1 Interfaces

```
// OutputTTLDevice — send trigger codes to recording equipment.
type OutputTTLDevice interface {
    Send(mask byte) error                    // set all 8 lines from bitmask
    SetHigh(line int) error                  // drive line HIGH (0-indexed)
    SetLow(line int) error                   // drive line LOW  (0-indexed)
    Pulse(line int, d time.Duration) error   // HIGH for d, then LOW (blocks)
    AllLow() error                           // all lines LOW
    Close() error                            // AllLow + release port
}

// InputTTLDevice — read TTL inputs from response hardware.
type InputTTLDevice interface {
    ReadAll() (byte, error)                                              //
 ↪  bitmask of all lines
    ReadLine(line int) (byte, error)                                    // 0
 ↪  or 1 (0-indexed)
    WaitForInput(ctx context.Context) (mask byte, rt time.Duration, err
 ↪  error)
    DrainInputs(ctx context.Context) error
    Close() error
}
```

9.9.2 DLPIO8 (DLP-IO8-G, USB-CDC serial)

Implements both `OutputTTLDevice` and `InputTTLDevice`.

```
// Auto-detect (recommended): returns NullOutputTTLDevice if not found, no
 ↪  error.
out, portName, err := triggers.AutoDetectDLPIO8()
defer out.Close()
out.Pulse(0, 10*time.Millisecond)   // 10 ms pulse on line 0

// Manual:
```

```
d, err := triggers.NewDLPIO8("/dev/ttyUSB0")
defer d.Close()
d.Send(0b00000101)                  // lines 0 and 2 HIGH
mask, err := d.ReadAll()            // bitmask of all 8 input lines
mask, rt, err := d.WaitForInput(ctx)
```

9.9.3 DLPIO20 (DLP-IO20, USB-CDC serial)

Implements both `OutputTTLDevice` and `InputTTLDevice`. 5 V logic (the T4 is 3.3 V).

> **Untested on hardware** — written from the datasheet rev 1.1. Bring it up with `tests/test_dlpio20` and a multimeter before relying on it.

The device has 17 usable digital channels but the TTL interfaces are 8-bit, so interface lines 0–7 address a *window* of 8 channels — `AN0`–`AN7` for output and `AN8`–`AN13`, `RB6`, `RB7` for input by default. Channels outside the windows stay reachable through the channel-level methods.

```
// Auto-detect (recommended): returns NullOutputTTLDevice if not found, no
 ↪  error.
out, portName, err := triggers.AutoDetectDLPIO20()
defer out.Close()

// Manual, with a remapped output window:
d, err := triggers.NewDLPIO20("/dev/ttyUSB0",
    triggers.WithIO20OutputChannels(
        triggers.IO20_AN0, triggers.IO20_AN1, triggers.IO20_AN2,
 ↪  triggers.IO20_AN3,
        triggers.IO20_AN4, triggers.IO20_AN5, triggers.IO20_AN6,
 ↪  triggers.IO20_AN7),
    triggers.WithIO20PollInterval(5*time.Millisecond),
)
defer d.Close()

d.Send(0b00000101)                      // lines 0 and 2 → AN0, AN2
d.Pulse(0, 5*time.Millisecond)
mask, err := d.ReadAll()                // bitmask of the 8 input-window
 ↪  channels

// Any of the 20 channels, in or out of the windows:
d.SetChannelHigh(triggers.IO20_RA4)
v, err := d.ReadChannel(triggers.IO20_AN12)
```

Channel constants are `IO20_AN0`–`IO20_AN13`, `IO20_RA4`, `IO20_P5`–`IO20_P7` (relay drivers — not TTL, not readable), `IO20_RB6`, `IO20_RB7`.

Timing: the device has no write-all command, so `Send` issues one 5-byte packet per line — the 8 lines change over ~3.5 ms rather than simultaneously. Use `SetHigh` on a single line for trigger onsets. On Linux, lower the FTDI latency timer (16 ms default)

before timing-sensitive reads: `echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer`.

9.9.4 MEGTTLBox (NeuroSpin Arduino Mega TTL box)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Provides 8 TTL output lines (D30–D37) and 8 TTL input lines for a FORP response pad (D22–D29).

```
box, err := triggers.NewMEGTTLBox("/dev/ttyACM0",
    triggers.WithResetDelay(2*time.Second),   // wait for Arduino boot
 ↪  (default 2 s)
    triggers.WithPollInterval(5*time.Millisecond),
)
defer box.Close()

// Output
box.Pulse(0, 5*time.Millisecond)    // pulse line 0
box.PulseMask(0b00000011, 5*time.Millisecond)  // pulse lines 0 and 1
box.Send(0b00000001)                // set line 0 HIGH, all others LOW

// Input (FORP response pad)
_ = box.DrainInputs(ctx)            // clear latched presses from previous
 ↪  trial
mask, rt, err := box.WaitForInput(ctx)
buttons := triggers.DecodeMask(mask)  // []FORPButton
```

`FORPButton` constants (also serve as 0-indexed line numbers for bitmask operations):

```
triggers.FORPLeftBlue    // 0
triggers.FORPLeftYellow  // 1
triggers.FORPLeftGreen   // 2
triggers.FORPLeftRed     // 3
triggers.FORPRightBlue   // 4
triggers.FORPRightYellow // 5
triggers.FORPRightGreen  // 6
triggers.FORPRightRed    // 7
```

9.9.5 MMBTS (NEUROSPEC MMBT-S interface box)

Implements `OutputTTLDevice` only — the box has 8 TTL output lines on a D-Sub 25 socket and no inputs. It is a USB-CDC virtual COM port: one byte at 9600 baud sets all 8 lines, bit N driving line N. Bit N (0-indexed) is D-Sub 25 pin N+2; ground is pins 20–25; 5 V HIGH, 0 V LOW.

```
box, err := triggers.NewMMBTS("/dev/ttyACM0",
    triggers.WithMMBTSMode(triggers.MMBTSPulseMode),  // default: the
 ↪  factory switch setting
)
defer box.Close()
```

```
box.Send(0b00000011)                // both lines in one byte
box.SetHigh(0)                      // rewrites the whole mask from the
 ↪  driver's shadow
box.Pulse(0, 5*time.Millisecond)    // width honoured in simple mode only
```

The runtime mode is chosen by a **hardware P/S switch** on the box, read at reset and not queryable over the serial link — `WithMMBTSMode` tells the driver what the switch is set to, it does not set it:

Constant	Switch	Behaviour
`triggers.MMBTSPulseMode` (default)	`P`, the factory setting	The firmware clears the port 8 ms after each byte, so **every trigger is 8 ms wide** whatever duration is requested, and codes closer together than that are delayed.
`triggers.MMBTSSimpleMode`	`S`	A byte latches until the next is written; `0` pulls all lines LOW. The host controls the width.

```
func triggers.NewMMBTS(portPath string, opts ...triggers.MMBTSOption)
 ↪  (*triggers.MMBTS, error)
func triggers.WithMMBTSMode(m triggers.MMBTSMode) triggers.MMBTSOption
func triggers.WithMMBTSPulseWidth(d time.Duration) triggers.MMBTSOption
func (b *triggers.MMBTS) Mode() triggers.MMBTSMode
func (b *triggers.MMBTS) PulseWidth() time.Duration
```

Errors: `ErrMMBTSNotOpen`, `ErrMMBTSBadLine`, `ErrMMBTSBadMode`.

There is no auto-detection: the box never replies, and it enumerates as a generic Arduino Micro, so it cannot be told apart from another Arduino-based box. Name the port. The baud rate is fixed at 9600 and deliberately not exposed — opening the port at 1200 baud resets the box into its bootloader.

9.9.6 FT232HTrigger (Adafruit FT232H, Linux)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Pure-Go driver via Linux usbfs — no libftdi or D2XX installation required.

Wiring: AD0–AD7 (D-bus) → 8 TTL output lines; AC0–AC7 (C-bus) → 8 TTL input lines.

```
box, err := triggers.NewFT232H(
    triggers.WithFT232HPollInterval(5*time.Millisecond), // optional;
 ↪  default 5 ms
```

```
)
if err != nil { log.Fatal(err) }
defer box.Close()

box.Send(0b00000001)              // AD0 HIGH (persistent)
box.Pulse(0, 5*time.Millisecond)  // AD0: HIGH for 5 ms, then LOW
box.AllLow()

_ = box.DrainInputs(ctx)
mask, rt, err := box.WaitForInput(ctx)

// Auto-detect (falls back to NullOutputTTLDevice if no device found):
out, err := triggers.AutoDetectFT232H()
defer out.Close()
```

Prerequisites (Linux): - The `ftdi_sio` kernel module must not hold the device: `sudo rmmod ftdi_sio` - User needs rw access to `/dev/bus/usb/BBB/DDD`. Recommended udev rule: `ACTION=="add", SUBSYSTEM=="usb", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6014", MODE="0666", GROUP="plugdev"`

9.9.7 LinuxGPIOTrigger (Raspberry Pi and other Linux SBCs)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Uses the Linux GPIO character device v2 API (kernel $\geq$ 5.10) — no external libraries required.

Wiring: any 8 GPIO pins for output, any other 8 for input. Pin numbers are chip-relative offsets (= BCM numbers on Raspberry Pi).

```
box, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
    triggers.WithGPIOInputPins([8]int{12, 16, 20, 21, 4, 25, 24, 23}),
    triggers.WithGPIOChip("/dev/gpiochip0"),           // optional; this is
 ↪  the default
    triggers.WithGPIOPollInterval(5*time.Millisecond), // optional; default
 ↪  5 ms
)
if err != nil { log.Fatal(err) }
defer box.Close()

box.Send(0b00000001)              // pin 17 HIGH
box.Pulse(0, 5*time.Millisecond)  // pin 17: HIGH for 5 ms, then LOW

_ = box.DrainInputs(ctx)
mask, rt, err := box.WaitForInput(ctx)
```

Output-only and input-only configurations are both valid — omit `WithGPIOOutputPins` or `WithGPIOInputPins` as needed.

Prerequisites: - Linux kernel $\geq$ 5.10 - User in the `gpio` group: `sudo usermod -aG gp`

```
io $USER
```

9.9.8 LabJackT4 (Modbus TCP)

Implements both `OutputTTLDevice` and `InputTTLDevice`. Pure-Go Modbus TCP driver — no LabJack SDK or system library required.

Wiring: the T4's digital lines are DIO4–DIO19. Output lines 0–7 → DIO4–DIO11 (FIO4–FIO7 screw terminals + EIO0–EIO3 on the DB15); input lines 0–7 → DIO12–DIO19 (EIO4–EIO7 + CIO0–CIO3). DIO0–DIO3 are the T4's dedicated analog inputs AIN0–AIN3 and cannot be used as digital lines. DIO4–DIO11 are *flexible I/O* that power up as analog inputs; `NewLabJackT4` switches them to digital mode (`DIO_ANALOG_ENABLE`) before setting directions.

```
box, err := triggers.NewLabJackT4("192.168.1.100",
    triggers.WithT4PollInterval(5*time.Millisecond), // optional; default 5
 ↪  ms
    triggers.WithT4Timeout(1*time.Second),           // optional; default 1
 ↪  s
    triggers.WithT4UnitID(1),                        // optional; default 1
    triggers.WithT4OutputBase(4),                     // optional; DIO of
 ↪  output line 0
    triggers.WithT4InputBase(12),                     // optional; DIO of
 ↪  input line 0
)
if err != nil { log.Fatal(err) }
defer box.Close()

box.Send(0b00000001)              // output line 0 (FIO4) HIGH
box.Pulse(0, 5*time.Millisecond)  // FIO4: HIGH for 5 ms, then LOW

_ = box.DrainInputs(ctx)
mask, rt, err := box.WaitForInput(ctx)
```

The host string may include the port number (`"192.168.1.100:502"`); default Modbus port 502 is appended automatically if omitted.

9.9.9 ParallelPort (Linux LPT)

Implements `OutputTTLDevice`.

```
ports := triggers.AvailableParallelPorts()      // scans /dev/parport0..3
pp := triggers.NewParallelPort("/dev/parport0")
if err := pp.Open(); err != nil { log.Fatal(err) }
defer pp.Close()
pp.Send(0b00000111)                             // lines 0,1,2 HIGH
pp.Pulse(0, 10*time.Millisecond)
status, _ := pp.ReadStatus()                   // Linux only: status register
```

Prerequisites: `sudo modprobe ppdev`; user in the `lp` group.

9.9.10 Null devices

`NullOutputTTLDevice` and `NullInputTTLDevice` are silent no-ops, safe to call without hardware. `AutoDetectDLPIO8` and `AutoDetectFT232H` return a `NullOutputTTLDevice` when no device is found.

9.9.11 Opening a device by name

```
triggers.Open(spec string) (OutputTTLDevice, string, error)   // "" → nil,
 ↪  "", nil; the string is a description
triggers.ParseDeviceSpec(spec string) (DeviceSpec, error)
triggers.OpenDeviceSpec(spec DeviceSpec) (OpenedDevice, error) // .Device
 ↪  .Where .GPIOPins .Note .Description()
triggers.DeviceSpecUsage                                       // help text
 ↪  for the syntax
triggers.DeviceKinds                                           // dlpio8
 ↪  dlpio20 megttlbox mmbts parallel gpio ft232h labjackt4 null
```

Syntax: `KIND[:PORT][,key=value,...]` — `megttlbox:/dev/ttyACM0`, `dlpio8` (auto-detect), `mmbts:/dev/ttyACM0,mode=s`, `labjackt4:192.168.1.100`, `gpio:pins=17+27+22+5+6+13+19+26`. An open failure is an error, never a silent null device. This is what the `-ttl` flag of `control.MarkerFlags` uses.

9.10 Package `staircase`

Import: `github.com/chrplr/goxpyriment/staircase`

Adaptive threshold estimation. Both procedures satisfy one interface, so an experiment loop is written once and the method swapped:

```
type Staircase interface {
    Intensity() float64   // intensity for the next trial (stable until
 ↪  Update)
    Update(correct bool)  // record the response and advance
    Done() bool           // stopping criterion met
    Threshold() float64   // current threshold estimate
    History() []Trial     // every trial so far
}

type Trial struct {
    Intensity float64
    Correct   bool
    Reversal  bool  // always false for Quest, which has no reversals
}
```

9.10.1 `UpDown` — transformed up/down (Levitt 1971)

```
sc := staircase.NewUpDown(staircase.UpDownConfig{
    StartIntensity:         0.5,
    MinIntensity:           0.0,
    MaxIntensity:           1.0,
    StepUp:                 0.1,
    StepDown:               0.05,
    NCorrectDown:           3,    // 3-down/1-up → ~79.4 % correct
    MaxReversals:           12,
    MaxTrials:              200,
    NReversalsForThreshold: 6,    // average the last 6 reversals
})

for !sc.Done() {
    correct := presentTrial(sc.Intensity())
    sc.Update(correct)
}
fmt.Println(sc.Threshold(), sc.NReversals(), sc.Reversals())
```

`Reversals() []float64` and `NReversals() int` are `UpDown`-specific; the rest come from the interface.

9.10.2 `Quest` — Bayesian estimation (Watson & Pelli 1983)

`NewQuest` returns an error, unlike `NewUpDown`, because the prior can be mis-specified:

```
q, err := staircase.NewQuest(staircase.QuestConfig{
    TGuess:        0.3,   // prior mode
    TGuessSd:      0.5,   // prior SD
    PThreshold:    0.82,  // target proportion correct
    Beta:          3.5,   // psychometric slope
    Delta:         0.01,  // lapse rate
    Gamma:         0.5,   // chance level (0.5 for 2AFC, 0 for yes/no)
    IntensityMin:  0.0,
    IntensityMax:  1.0,
    IntensityStep: 0.01,
    MaxTrials:     40,
})
if err != nil { log.Fatal(err) }
```

`q.SD()` reports the posterior width — the usual convergence check.

9.10.3 `Runner` — interleaved staircases

```
r := staircase.NewRunner(rng, sc1, sc2, sc3)
for !r.Done() {
    sc, err := r.Next()   // picks a not-yet-finished staircase at random
```

```
    if err != nil { break }
    sc.Update(presentTrial(sc.Intensity()))
}
for _, sc := range r.All() { fmt.Println(sc.Threshold()) }
```

Interleaving prevents participants from tracking a single adaptive sequence.

9.11 Package `units`

Import: `github.com/chrplr/goxpyriment/units`

Vision-science unit conversion through a `Monitor` describing the physical geometry. All conversions need the viewing distance, so they live on the struct rather than being free functions.

```
type Monitor struct {
    WidthCm, HeightCm   float64
    WidthPx, HeightPx   int
    DistanceCm          float64  // observer's viewing distance
}

m := units.NewMonitor(52.7, 29.6, 1920, 1080, 60)          // explicit size
m := units.NewMonitorFromDiagonal(24, 1920, 1080, 60)      // from the inch
 ↪  rating
if err := m.Validate(); err != nil { log.Fatal(err) }
```

Method	Converts
`DegToPx(deg)` / `PxToDeg(px)`	visual degrees ↔ pixels (horizontal)
`CmToPx(cm)` / `PxToCm(px)`	centimetres ↔ pixels (horizontal)
`DegToCm(deg)` / `CmToDeg(cm)`	visual degrees ↔ centimetres on the screen
`DegToPxY` / `PxToDegY` / `CmToPxY` / `PxToCmY`	the same, vertically

The `…Y` variants exist because pixels are not always square. `HasSquarePixels()` reports whether they are; when it returns false the horizontal and vertical conversions genuinely differ and using the wrong one is a silent error.

Descriptive helpers: `PPcmX()`, `PPcmY()`, `PPI()`, `PPD()` (pixels per degree, i.e. `DegToPx(1)`), and `String()` for logging the geometry into a data file.

```
radiusPx := float32(m.DegToPx(2.5))            // a 2.5° stimulus
disk := stimuli.NewCircle(radiusPx, control.White)
```

9.12 Package assets_embed

Import: `github.com/chrplr/goxpyriment/assets_embed`

Provides embedded default assets as `[]byte` slices, ready for `FontFromMemory` or `PlaySoundFromMemory`:

```
assets_embed.InconsolataFont  []byte  // default monospace TTF font
assets_embed.BuzzerWav        []byte  // error/incorrect feedback sound
assets_embed.CorrectWav       []byte  // correct/reward feedback sound
```

9.13 Coordinate System

All stimulus positions are in **screen-center coordinates**:

- `(0, 0)` = screen center
- Positive X = right; positive Y = **up** (not down)
- `screen.CenterToSDL(x, y)` converts to SDL's top-left origin

```
          +Y (up)
           |
 -X ------+------ +X
           |
          -Y (down)
```

9.14 Typical Experiment Structure

```go
package main

import (
    "log"
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("My Experiment", control.Black,
 ↪  control.White, 32)
    defer exp.End()

    exp.AddDataVariableNames([]string{"block", "trial", "condition", "key",
 ↪  "rt_ms"})

    // Build design
```

```
    block := design.NewBlock("main")
    for _, cond := range []string{"left", "right"} {
        t := design.NewTrial()
        t.SetFactor("condition", cond)
        block.AddTrial(t, 10, false)
    }
    block.ShuffleTrials()
    exp.AddBlock(block, 1)

    err := exp.Run(func() error {
        exp.ShowInstructions("Press F for left, J for right.\n\nPress SPACE
↪   to start.")

        for bi, blk := range exp.Design.Blocks {
            for ti, trial := range blk.Trials {
                cond := trial.GetFactor("condition").(string)

                exp.Show(stimuli.NewFixCross(20, 2, control.White))
                exp.Wait(500)

                stim := stimuli.NewTextLine(cond, 0, 0, control.White)
                exp.Show(stim)

                key, rt, err := exp.Keyboard.WaitKeysRT(
                    []control.Keycode{control.K_F, control.K_J}, 3000,
                )
                if control.IsEndLoop(err) {
                    return control.EndLoop
                }

                exp.Data.Add(bi, ti, cond, key, rt)
                exp.Blank(500)
            }
        }
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```

Chapter 10

Migrating to goxpyriment

This guide is for researchers who already know one of the mainstream experiment libraries and want to get productive in goxpyriment quickly. Each section maps the concepts and idioms of one tool to their goxpyriment equivalents, highlights the most important differences, and provides side-by-side code for a representative trial sequence.

- From Expyriment (Python)
- From PsychoPy (Python)
- From Psychtoolbox (MATLAB)

Before reading this guide, complete the Getting Started tutorial. It introduces the coordinate system, the rendering model, and the trial loop idiom that all three sections below assume.

10.1 From Expyriment (Python)

Goxpyriment was directly inspired by Expyriment and shares its philosophy: a single experiment object owns all resources, stimuli are objects with a `present`/`Show` method, and data are written to a structured file automatically. The mapping is the closest of the three tools covered here.

10.1.1 Concept map

Expyriment (Python)	goxpyriment (Go)	Notes
`design.Experiment("name")`	`control.NewExperimentFromFlags("name", bg, fg, fontSize)`	Also parses `-w`, `-d`, `-s` flags.
`control.initialize(exp)`	*(called inside `NewExperimentFromFlags`)*	SDL, audio, font, and data file initialized automatically.

Expyriment (Python)	goxpyriment (Go)	Notes
`control.end(exp)`	`defer exp.End()`	Always defer immediately after construction.
`exp.clock.wait(1000)`	`exp.Wait(1000)`	Both pump the event loop; both abort on ESC.
`exp.clock.reset()`	`c := clock.NewClock()` then `c.Reset()`	`clock.NewClock()` starts a new reference; `c.Reset()` restarts it.
`exp.clock.time`	`c.NowMillis()`	Returns `int64` milliseconds.
`stim.present()`	`exp.Show(stim)`	Both do clear → draw → flip in one call.
`stimuli.TextLine("Hello", (0,0), exp.screen)`	`stimuli.NewTextLine("Hello", 0, 0, control.White)`	Position is center-relative in both (Expyriment center = (0,0), Y+ = up).
`stimuli.Circle(radius=50)`	`stimuli.NewCircle(50, color)`	
`stimuli.FixCross(size=30)`	`stimuli.NewFixCross(30, lineWidth, color)`	
`stimuli.Picture("img.png")`	`stimuli.NewPicture("img.png", x, y)`	
`exp.keyboard.wait()`	`exp.Keyboard.Wait()`	Returns `(Keycode, error)`.
`exp.keyboard.wait_for_keys([K_F, K_J])`	`exp.Keyboard.WaitKeys([]Keycode{K_F, K_J}, timeout)`	`timeout = -1` means no timeout.
`exp.keyboard.wait_for_keys(...)` RT	`exp.Keyboard.WaitKeysRT(keys, timeout)`	Returns `(key, rtMs, error)`.
`exp.data_variable_names = [...]`	`exp.AddDataVariableNames([]string{...})`	`subject_id` is prepended automatically — do not include it.
`exp.data.add([v1, v2, ...])`	`exp.Data.Add(v1, v2, ...)`	
`design.Block()`	`design.NewBlock("name")`	
`design.Trial()`	`design.NewTrial()`	
`trial.set_factor("cond", "target")`	`trial.SetFactor("cond", "target")`	
`trial.get_factor("cond")`	`trial.GetFactor("cond")`	
`block.add_trials([t1, t2])`	`block.AddTrial(t, copies, randomPosition)`	
`block.shuffle_trials()`	`block.ShuffleTrials()`	

`io.DataFile`	`exp.Data (*io.DataFile)`	Opened automatically; output to `~/goxpy_data/`. `.csv` is the same CSV-with-comments format.

10.1.2 Side-by-side: simple RT trial

Expyriment

```
import expyriment
from expyriment import design, control, stimuli, io

exp = design.Experiment("SimpleRT")
control.initialize(exp)

fix   = stimuli.FixCross()
target = stimuli.Circle(radius=30, colour=(255,255,255))

exp.data_variable_names = ["rt"]

fix.present()
exp.clock.wait(500)
target.present()
key, rt = exp.keyboard.wait()
exp.data.add([rt])

control.end(exp)
```

goxpyriment

```
package main

import (
    "log"
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("SimpleRT", control.Black,
 ↪  control.White, 32)
    defer exp.End()

    fix    := stimuli.NewFixCross(30, 3, control.White)
    target := stimuli.NewCircle(30, control.White)

    exp.AddDataVariableNames([]string{"rt"})
```

```
    err := exp.Run(func() error {
        exp.ShowTimed(fix, 500)
        exp.Show(target)
        _, rt, err := exp.Keyboard.WaitKeysRT(nil, -1)
        if err != nil {
            return err
        }
        exp.Data.Add(rt)
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatal(err)
    }
}
```

10.1.3 Key differences

The run loop. In Expyriment the trial logic runs in `main()` directly. In goxpyriment all SDL calls must run on the thread that owns the window; `exp.Run(func() error { ...})` enforces this. Return `control.EndLoop` to exit cleanly.

Clock domains. Expyriment's `exp.clock.time` measures from `control.initialize`. `clock.NewClock()` measures from whenever you create it. For stimulus-onset-locked RT, use `exp.ShowTS` + `exp.Keyboard.GetKeyEventTS` (SDL nanosecond timestamps) rather than the `clock` package — see UserManual §6.

`.csv` files. goxpyriment keeps the metadata out of the data file: the `.csv` holds a header row and data rows only, and the `#`-prefixed session metadata goes to a companion `-info.txt` with the same basename. Existing Python scripts that read Expyriment data files with `pandas.read_csv(..., comment='#')` still work unchanged — there is simply nothing left for `comment='#'` to skip — and the file now also imports directly into a spreadsheet.

Adaptive staircases. Expyriment has no built-in staircase. Goxpyriment provides `staircase.NewUpDown` (Levitt 1971) and `staircase.NewQuest` (Watson & Pelli 1983), including a `Runner` for interleaved designs. Import `github.com/chrplr/goxpyriment/staircase`.

RSVP streams. Expyriment's `stimuli.extras.StreamingTextDisplay` is replaced by `stimuli.PresentStreamOfText` / `stimuli.PresentStreamOfImages`, which are VSYNC-locked, disable GC, and return a full `TimingLog` per item with nanosecond onset/offset timestamps.

10.2 From PsychoPy (Python)

PsychoPy Coder mode and goxpyriment are structurally similar: both give you a window object, stimulus objects, and explicit flip calls. The main adjustments are coordi-

nate convention, the rendering model, and how timing is measured.

10.2.1 Concept map

PsychoPy (Python)	goxpyriment (Go)	Notes
`visual.Window(size=[1024,768], units='pix')`	`control.NewExperimentFromFlags(...)` with `-w` flag	Pass `-w` for a 1024×768 window; default is fullscreen.
`win.flip()`	`exp.Flip()`	Both are VSYNC-locked. Usually use `exp.Show(stim)` instead.
`win.color = (-1,-1,-1)`	`control.Black` (background passed to constructor)	
`core.wait(0.5)`	`exp.Wait(500)`	PsychoPy uses seconds; goxpyriment uses milliseconds.
`core.Clock()` / `clock.getTime()`	`clock.NewClock()` / `c.NowMillis()`	See clock-domain note below.
`core.CountdownTimer(t)`	`c.SleepUntil(target)`	
`visual.TextStim(win, text="Hello")`	`stimuli.NewTextLine("Hello", x, y, color)`	
`visual.TextStim(..., wrapWidth=800)`	`stimuli.NewTextBox(text, width, pos, color)`	Word-wrapped.
`visual.Circle(win, radius=0.5, units='pix')`	`stimuli.NewCircle(radius, color)`	
`visual.Rect(win, width=100, height=50)`	`stimuli.NewRectangle(cx, cy, w, h, color)`	
`visual.ImageStim(win, image='img.png')`	`stimuli.NewPicture("img.png", x, y)`	
`visual.GratingStim(win, sf=0.05, ori=45)`	`stimuli.NewGaborPatch(sigma, theta, lambda, ...)`	See spatial/temporal frequency note below.
`stim.draw(); win.flip()`	`exp.Show(stim)`	
`stim.setPos((x, y))`	`stim.SetPosition(control.Point(x, y))`	
`event.waitKeys(keyList=['f','j'])`	`exp.Keyboard.WaitKeys(keys, timeout)`	
`event.waitKeys(maxWait=3)`	`exp.Keyboard.WaitKeys(keys, 3000)`	Timeout in ms.
`clock.getTime()` for RT	`exp.Keyboard.WaitKeysRT(keys, timeout)`	Returns RT in ms from call site.
`win.callOnFlip(clock.reset)` + `event.waitKeys`	`exp.ShowAndGetRT(stim, keys, timeout)`	Hardware-precise RT in ms; no callOnFlip needed.
`data.TrialHandler(trialList, nReps)`	`design.NewBlock(...)` + `block.AddTrial(t, copies, true)`	

PsychoPy (Python)	goxpyriment (Go)	Notes
`data.ExperimentHandler(...)`	`exp.Data`	
`thisExp.addData("rt", rt)`	`exp.Data.Add(rt)`	
`data.QuestHandler(startVal, ...)`	`staircase.NewQuest(cfg)`	
`data.StairHandler(startVal, ...)`	`staircase.NewUpDown(cfg)`	
`sound.Sound('A', secs=0.5)`	`stimuli.NewTone(frequency, duration, volume)`	
`sound.Sound('beep.wav')`	`stimuli.NewSound("beep.wav")`	

10.2.2 Side-by-side: simple RT trial

PsychoPy

```
from psychopy import visual, core, event

win   = visual.Window([1024,768], color='black', units='pix')
fix   = visual.TextStim(win, text='+', height=30, color='white')
target = visual.Circle(win, radius=30, fillColor='white')

trial_clock = core.Clock()

fix.draw(); win.flip()
core.wait(0.5)

trial_clock.reset()
target.draw(); win.flip()
keys = event.waitKeys(keyList=['f','j'], timeStamped=trial_clock)
key, rt = keys[0]  # rt is in seconds

win.close()
```

goxpyriment

```
package main

import (
    "log"
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("SimpleRT", control.Black,
 ↪  control.White, 32)
```

```
    defer exp.End()

    fix    := stimuli.NewFixCross(30, 3, control.White)
    target := stimuli.NewCircle(30, control.White)

    err := exp.Run(func() error {
        exp.ShowTimed(fix, 500)
        key, rtMs, _ := exp.ShowAndGetRT(target,
            []control.Keycode{control.K_F, control.K_J}, -1,
        )
        _ = key
        _ = rtMs
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatal(err)
    }
}
```

10.2.3 Key differences

Units and coordinate system. PsychoPy supports multiple unit systems (norm, pix, deg, cm). Goxpyriment always uses pixels, with (0, 0) at the screen center, +X right, +Y **up** (same as PsychoPy's `units='pix'`). For visual-angle calculations, use the `units` package: `units.NewMonitor(widthCm, distanceCm, widthPx)` then `mon.DegToPx(deg)`.

Time units. PsychoPy uses seconds throughout (`core.wait(0.5)`, `clock.getTime()` returns float seconds). Goxpyriment uses **milliseconds** for `exp.Wait` and `WaitKeysRT`, and **nanoseconds** for hardware event timestamps (`ShowTS`, `GetKeyEventTS`). Divide nanosecond differences by `1_000_000` to get milliseconds.

`callOnFlip` is not needed. PsychoPy's `win.callOnFlip(clock.reset)` is a workaround for the fact that `flip()` and the clock reset are separate operations. `exp.ShowTS(stim)` captures the SDL nanosecond timestamp atomically at the VSYNC flip — no workaround needed.

Builder vs Coder. PsychoPy Builder generates Python Coder scripts. There is no Builder equivalent in goxpyriment; all experiments are written in code.

Spatial frequency. PsychoPy's `GratingStim` spatial frequency is in cycles/degree (when `units='deg'`). Goxpyriment's `spatialFreq` is in **cycles per pixel**. Convert: `sf_cpx = sf_cpd / mon.DegToPx(1)`.

TrialHandler nReps. PsychoPy's `TrialHandler` accepts a list of condition dicts and repeats them. The equivalent in goxpyriment is `block.AddTrial(t, nReps, randomPosition)` where `randomPosition=true` shuffles. For multiple condition lists, build one block per block and call `exp.ShuffleBlocks()` if needed.

Psychtoolbox-style timestamps. PsychoPy's `logging.flush()` and `win.recordFrameIntervals` give post-hoc timing diagnostics. In goxpyriment, `PresentStreamOfIm-`

`ages` returns a `[]TimingLog` per item. Use the SDL-clock `OnsetNS`/`OffsetNS` for timing analysis (they share the clock of input-event timestamps); `ActualOnset`/`ActualOffset` are Go-clock diagnostics only and must not be subtracted from event timestamps.

10.3 From Psychtoolbox (MATLAB)

Psychtoolbox-3 (PTB) and goxpyriment share a lower-level philosophy: you open a window, draw into it, flip explicitly, and get a VBL timestamp back. The key differences are language ergonomics, the data-file model, and the adaptive staircase API.

10.3.1 Concept map

Psychtoolbox (MATLAB)	goxpyriment (Go)	Notes
`Screen('OpenWindow', screenNum, bgColor)`	`control.NewExperimentFromFlags(...)`	Also initializes audio, font, and data file.
`Screen('CloseAll')`	`defer exp.End()`	
`Screen('Flip', win)`	`exp.Flip()`	Both return a timestamp.
`Screen('Flip', win)` → VBL timestamp	`exp.ShowTS(stim)` → `uint64` nanoseconds	PTB returns seconds (64-bit float); goxpyriment returns nanoseconds (uint64).
`Screen('FillRect', win, color, rect)`	`stimuli.NewRectangle(cx, cy, w, h, color)` then `exp.Show(rect)`	
`Screen('DrawText', win, text, x, y, color)`	`stimuli.NewTextLine(text, x, y, color)` then `exp.Show(tl)`	
`Screen('MakeTexture', win, img)`	`stimuli.NewPicture("path", x, y)`	Texture is lazily uploaded on first `Draw`.
`Screen('DrawTexture', win, tex)`	`exp.Show(pic)`	
`Screen('MakeTexture', win, sheet)` (one texture for many stimuli)	`stimuli.NewSpriteSheet("sheet.png")` or `NewSpriteSheetFromMemory(data)`	The PTB idiom of packing a condition into one image.
`Screen('DrawTexture', win, tex, sourceRect)`	`sprites, _ := sheet.Grid(exp.Screen, cols, rows)` then `exp.Show(sprites[i])`	`sourceRect` becomes the sprite's `Clip`. One upload for the whole set.
`Screen('DrawLine', ...)`	`stimuli.NewLine(x1, y1, x2, y2, color)` then `exp.Show(line)`	
`DrawFormattedText(win, text, 'center', 'center', color)`	`exp.ShowInstructions(text)`	Centered, waits for spacebar.

Psychtoolbox (MATLAB)	goxpyriment (Go)	Notes
`WaitSecs(secs)`	`exp.Wait(ms)`	PTB uses seconds; goxpyriment uses **milliseconds**.
`GetSecs()`	`clock.GetTime()` (ms) or `clock.GetTimeNS()` (ns)	See clock-domain note below.
`KbWait`	`exp.Keyboard.Wait()`	
`KbCheck`	`exp.Keyboard.Check()`	Non-blocking poll.
`KbStrokeWait` / `KbName`	`exp.Keyboard.WaitKeys(keys, timeout)`	Pass `nil` for any key.
RT via `GetSecs` before/after `KbWait`	`exp.Keyboard.WaitKeysRT(keys, timeout)` (ms)	Or `GetKeyEventTS` for nanosecond accuracy.
`PsychPortAudio('Open', ...)` / `'Start'`	`stimuli.NewSound(path)` + `snd.Play()`	
`MakeBeep(freq, dur, rate)`	`stimuli.NewTone(freq, duration, volume)`	
`Snd('Play', ...)`	`tone.Play()`	
`Quest('init', ...)`	`staircase.NewQuest(cfg)`	
`QuestUpdate(q, x, response)`	`sc.Update(correct)`	
`QuestQuantile(q)`	`sc.Intensity()`	
`QuestMean(q)`	`sc.Threshold()`	
Custom up-down staircase	`staircase.NewUpDown(cfg)`	Levitt (1971); 2-down-1-up built in.

10.3.2 Side-by-side: simple RT trial

Psychtoolbox

```
screenNum = max(Screen('Screens'));
[win, rect] = Screen('OpenWindow', screenNum, [0 0 0]);

cx = rect(3)/2;  cy = rect(4)/2;

% Fixation cross
Screen('DrawLine', win, [255 255 255], cx-15, cy, cx+15, cy, 3);
Screen('DrawLine', win, [255 255 255], cx, cy-15, cx, cy+15, 3);
Screen('Flip', win);
WaitSecs(0.5);

% Target circle
Screen('FrameOval', win, [255 255 255], [cx-30, cy-30, cx+30, cy+30], 3);
t0 = Screen('Flip', win);   % VBL timestamp in seconds

[~, t1, ~] = KbWait([], 2);
rt = (t1 - t0) * 1000;  % milliseconds
```

```
Screen('CloseAll');
```

goxpyriment

```
package main

import (
    "log"
    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/stimuli"
)

func main() {
    exp := control.NewExperimentFromFlags("SimpleRT", control.Black,
 ↪  control.White, 32)
    defer exp.End()

    fix    := stimuli.NewFixCross(30, 3, control.White)
    target := stimuli.NewCircle(30, control.White)

    err := exp.Run(func() error {
        exp.ShowTimed(fix, 500)
        _, rtMs, _ := exp.ShowAndGetRT(target, nil, -1)
        _ = rtMs
        return control.EndLoop
    })
    if err != nil && !control.IsEndLoop(err) {
        log.Fatal(err)
    }
}
```

10.3.3 Key differences

Time units. PTB uses **seconds** (double-precision float) throughout. Goxpyriment uses **milliseconds** for `exp.Wait` / `WaitKeysRT`, and **nanoseconds** for hardware event timestamps. To convert: multiply PTB seconds by 1000 to get ms, or 1,000,000,000 to get ns.

VBL timestamps. PTB's `Screen('Flip')` returns the VBL timestamp as `GetSecs` seconds. `exp.ShowTS(stim)` returns `sdl.TicksNS()` nanoseconds captured immediately after the VSYNC flip — a different clock origin. Do not mix SDL timestamps with `clock.GetTimeNS()` for RT calculation; use SDL timestamps exclusively (see UserManual §6).

Coordinate system. PTB uses a top-left origin (+Y down), consistent with screen pixel conventions. Goxpyriment uses a **center origin (+Y up)**, same as PsychoPy `pix` units. A stimulus at (0, 0) appears at screen center; a stimulus at (0, 200) appears 200 pixels **above** center.

Drawing model. PTB requires explicit `DrawLine`/`FillRect`/etc calls per frame and a separate `Flip`. Goxpyriment encapsulates each stimulus as an object; `exp.Show(stim)` does clear → draw → flip in one call. For multi-stimulus frames, draw each one (`stim.Draw(exp.Screen)`) then call `exp.Flip()` explicitly.

Data files. PTB has no built-in data file; researchers typically write custom CSV writers or use `fopen`/`fprintf`. Goxpyriment writes a structured `.csv` file automatically. Declare column names once with `exp.AddDataVariableNames`; append rows with `exp.Data.Add(...)`.

Quest staircase. PTB's `Quest` functions are a set of global procedures (`QuestCreate`, `QuestUpdate`, `QuestMean`). `staircase.NewQuest(cfg)` is an object that implements the same Bayesian update; call `sc.Intensity()`, `sc.Update(correct)`, `sc.Threshold()`. The parameters (`tGuess`, `tGuessSd`, `pThreshold`, `beta`, `delta`, `gamma`) map directly.

EEG triggers. PTB sends triggers via `lptwrite` (Windows parallel port). Goxpyriment provides `triggers.NewDLPIO8`, `triggers.NewMEGTTLBox`, `triggers.NewMMBTS` (the NEUROSPEC MMBT-S), and `triggers.NewParallelPort`, all implementing the same `OutputTTLDevice` interface with `Send(mask)`, `Pulse(line, duration)`, and `AllLow()`. For labs that mark the EEG stream over the network instead of with a TTL line, `triggers.NewNetStation(host)` speaks the EGI NetStation ECI protocol (`Synchronize`, `StartRecording`, `SendEvent("STIM")`); and `triggers.NewVideoRecorder(host)` drives the "BEL_video" participant-video recorder (`Start`, `Label("TRL", "001")`, `Stop`). Both are TCP clients — richer than a TTL pulse but bounded by network latency, so keep a `Pulse` for sub-millisecond onset markers. See UserManual §17.

No Flip scheduling. PTB's `Screen('Flip', win, when)` allows scheduling a flip at a specific VBL. Goxpyriment does not support scheduled flips; instead, call `SleepUntil(target)` on a `clock.Clock` (there is no package-level `clock.SleepUntil`) before `exp.Show` to achieve frame-accurate onset scheduling, or use the stream functions (`PresentStreamOfImages`) which handle VSYNC-locked scheduling internally.

10.4 Common gotchas (all three tools)

ESC handling. Goxpyriment treats ESC as a universal experiment abort. `exp.Wait`, `exp.Keyboard.WaitKeys`, and similar functions return `control.EndLoop` when ESC is pressed. Propagate this error upward from `exp.Run`'s callback; `control.IsEndLoop(err)` tests for it. There is no equivalent of PTB's `ListenChar`/`CharAvail` or PsychoPy's `event.clearEvents()` — use `exp.Keyboard.Clear()` to flush stale key presses before a new trial.

GPU texture allocation. The first time any visual stimulus is drawn, its SDL texture is allocated on the GPU. In all three tools the first presentation can be slower than subsequent ones. In goxpyriment, call `stimuli.PreloadVisualOnScreen(exp.Screen, stim)` (or `stimuli.PreloadAllVisual`) before the critical section to force allocation during an instruction screen, not during a timed trial.

RSVP / rapid stimulus sequences. Go's garbage collector can pause execution mid-sequence. The stream functions (`PresentStreamOfImages`, `PresentStreamOfText`)

disable GC automatically for the duration of the stream. Do not implement your own VSYNC-locked loop without also disabling GC (`debug.SetGCPercent(-1)`).

Single binary distribution. Unlike Python (Expyriment, PsychoPy) and MATLAB (PTB), goxpyriment experiments compile to a single self-contained binary. Run `go build .` in the experiment directory; the result runs on any machine with the same OS and architecture without any runtime installation. For cross-platform distribution, use `GOOS=windows GOARCH=amd64 go build .` etc. — see Installation.

Chapter 11

goxpyriment compared with PsychoPy

PsychoPy is the established, most widely used platform for creating and running psychological experiments. This page compares it with goxpyriment feature by feature, so that you can judge whether goxpyriment fits your paradigm — and, where it does not, what you would be giving up.

The comparison is based on the PsychoPy documentation sources (psychopy/psychopy-docs, `release` branch) and on the current state of this repository.

Summary in one sentence: PsychoPy optimises for *accessibility and breadth* (a GUI builder, online deployment, support for nearly every laboratory device); goxpyriment optimises for *deployability and timing determinism* (one static binary, no interpreter, no garbage collection inside the presentation loop).

If you are porting existing PsychoPy code, read the Migration Guide — it gives a concept map and side-by-side code for a representative trial sequence.

11.1 The structural difference

PsychoPy is three layers stacked on top of each other:

1. **Builder** — a graphical editor in which an experiment is assembled from routines, loops, and ~45 drag-and-drop components. It compiles to Python *or* to JavaScript.
2. **Coder** — the `psychopy.*` Python library, which the Builder generates calls into and which you can use directly.
3. **PsychoJS / Pavlovia** — a JavaScript runtime plus a hosting platform, with integrations for Prolific, MTurk, Qualtrics, and Sona.

goxpyriment is only the middle layer: a Go library. There is no graphical builder and no hosting platform. That single fact explains most of the gaps listed below — and most of the advantages.

11.2 Where goxpyriment matches or has an edge

Aspect	PsychoPy	goxpyriment
Deployment	Python environment plus wheels, or Pavlovia hosting	A single self-contained binary. SDL3, SDL3_ttf and SDL3_image are embedded as compressed blobs and loaded at startup — nothing to install on the target machine
Presentation loop	Python, subject to the GIL and to interpreter overhead; Psychtoolbox/ioHub back-ends are used for precision	VSYNC-locked loops with the garbage collector disabled (`debug.SetGCPercent(-1)`) for the duration of the critical section
Clock	`psychopy.clock`, `core.getTime`	`clock` package built on SDL's high-resolution performance counter
RSVP and rapid streams	Written by hand as a frame-by-frame loop	A dedicated high-precision path, `stimuli.PresentStreamOfImages`, with variants for images, text, tones, mixed streams, and per-frame callbacks
Adaptive procedures	`StairHandler`, `QuestHandler`	`staircase.UpDown` (Levitt 1971) and `staircase.Quest` (Watson & Pelli 1983) — the same two families
Unit conversion	`psychopy.monitors`, `deg`/`cm`/`norm` units	`units` package: pixels ↔ degrees ↔ cm through a `Monitor` struct
Counterbalancing	`CounterbalanceRoutine` (a recent addition)	`design.AddBWSFactor` / `GetPermutedBWSFactorCondition` — Latin-square between-subject assignment, present from the start
Gamma correction	`monitors` calibration	`apparatus.GammaCorrector` — per-channel exponents, `exp.SetGamma` / `exp.CorrectColor`

Aspect	PsychoPy	goxpyriment
Data output	`.csv` / `.psydat` / Excel	Plain `.csv` (no comment lines, imports straight into a spreadsheet) plus a companion `-info.txt` holding the session metadata; both written through a buffered output file
System reporting	`psychopy.info`	`sysinfo` package: CPU, GPU, audio, memory, machine — including a WMI path on Windows
Hardware triggers	Parallel, serial, LabJack, Cedrus, EGI, BrainProducts, TriggerBox	Parallel port (Linux LPT), DLP-IO8, FT232H, Linux GPIO (Raspberry Pi and other SBCs), LabJack T4 over Modbus TCP, NeuroSpin MEG TTL box, NEUROSPEC MMBT-S, generic serial
Eye tracking	ioHub: EyeLink, Tobii, GazePoint, Pupil Labs; calibration/validation routines, ROI component	`eyetracker` package: vendor-neutral `Tracker` interface, EyeLink and Tobii Pro via a Python bridge process, null and mouse-simulated trackers, calibration through `exp.CalibrateTracker`
Networked recording control	Egi/`iohub`, `emulator`, MQTT/OSC components	EGI NetStation event markers over TCP/IP (ECI), plus a “BEL_video” participant-video recorder client
Worked examples	A demos folder	over 50 complete paradigms in `examples/` (plus ~30 feature demos), plus a `tests/` suite of programs whose output is analysed to verify timing and hardware behaviour

The core of an experiment — window, text, images, shapes, gratings, dot fields, keyboard and mouse input, sound, trial structure, randomisation, staircases, data files — is covered by both.

11.3 Where PsychoPy is ahead

11.3.1 1. Eye tracking

PsychoPy's **ioHub** provides a vendor-neutral API over EyeLink, Tobii, GazePoint and Pupil Labs, with Builder routines for calibration and validation and a region-of-interest component.

goxpyriment's `eyetracker` package covers the first two vendors: a vendor-neutral `Tracker` interface (open, calibrate, record, mark, latest sample, drained samples and fixation/saccade/blink events) with an EyeLink and a Tobii Pro back end, a null tracker, and a mouse-driven simulator. The vendor SDK runs in a small Python bridge process rather than being linked in, so the Go build stays pure and cross-compilable — the same split ioHub makes, for a different reason. Calibration is one call, `exp.CalibrateTracker`, drawn by the EyeLink's own routine or, for the Tobii, by goxpyriment itself. What PsychoPy still has and goxpyriment does not: GazePoint and Pupil Labs back ends, validation routines, a region-of-interest component, and Builder integration. Both have been run against real hardware (EyeLink on a MEG rig, Tobii on Windows); see `tests/test_eyelink` and `tests/test_tobii`.

11.3.2 2. Breadth of visual stimuli

PsychoPy's `visual` module documents 42 classes. Notable ones without a goxpyriment equivalent:

- **`ElementArrayStim`** — thousands of elements drawn in a single call. Required for Glass patterns, large-scale multiple-object tracking, and texture-defined stimuli. Probably the most consequential omission.
- **`Aperture`** — restricting drawing to an arbitrary region.
- **`RadialStim`**, **`NoiseStim`**, **`SecondOrder`** — specialised psychophysical textures.
- **`Form`**, **`Slider`**, **`RatingScale`** — questionnaire and rating widgets. goxpyriment offers `stimuli.ChoiceGrid`, `stimuli.ThermometerDisplay` and `stimuli.TextInput`, which cover some but not all of this ground.
- **`BufferImageStim`** — snapshotting a rendered screen for fast redisplay.
- **`windowWarp`** — spherical and cylindrical warping for domes and curved screens.
- The entire 3D/VR branch: `rift`, `SphereStim`, `ObjMeshStim`, `PhongMaterial`, `LightSource`, `SceneSkybox`.

11.3.3 3. Online experiments

PsychoPy's `online/` documentation covers Pavlovia synchronisation, participant-recruitment integrations (Prolific, MTurk, Qualtrics, Sona), credit and statistics dashboards, and compatibility checking.

goxpyriment compiles to WebAssembly and runs in the browser (see WASM.md), so the *runtime* half of this exists. What does not exist is the surrounding service: hosting, recruitment, and server-side data collection are left to you.

11.3.4 4. Video

PsychoPy plays whatever ffmpeg can decode. goxpyriment plays MPEG-1 directly (pure Go, with MP2 audio) and requires conversion to `.gv` for timing-critical clips — `cmd/gv-convert` does this, needing no external tools for MPEG-1 input and ffmpeg for anything else. See UserManual §15.

The `.gv` requirement is a deliberate trade: independent per-frame blocks remove the decode-cost asymmetry between I- and B-frames, which is what makes H.264 onset timing jitter periodically. But it is still friction, the files are roughly an order of magnitude larger, and it rules out playing arbitrary participant-supplied media at measured onsets.

11.3.5 5. Ecosystem and tooling

- A **plugin architecture** for third-party components.
- The **`Session`** API for running several experiments under one process.
- **Alerts and linting** that catch common design errors before a run.
- A **logging framework** with configurable levels.
- A substantial **teaching corpus** and community.

11.3.6 6. Camera and speech

PsychoPy has a `CameraComponent`, Google Speech integration, and audio/visual validator routines. goxpyriment provides `apparatus.Microphone`, `apparatus.VoiceKey` and WAV writing, but no camera capture and no speech recognition.

11.3.7 7. The Builder itself

For readers who do not program, the Builder is not a convenience — it is the only way in. goxpyriment requires writing Go.

11.4 Choosing between them

Reach for PsychoPy when you need a GazePoint or Pupil Labs tracker, or gaze validation and region-of-interest components; when the paradigm depends on large element arrays, apertures, or 3D/VR; when the study runs online with recruitment-platform integration; when arbitrary video must be played; or when the person building the experiment does not program.

Reach for goxpyriment when the experiment must be distributed as a binary that runs on a machine you do not control and cannot install software on; when timing determinism matters more than stimulus breadth (rapid serial presentation, tight audio-visual synchronisation, hardware-triggered acquisition); when Python environment drift has been a recurring problem; or when you are already comfortable in Go and want the compiler to catch design errors before the participant arrives.

The two are not mutually exclusive. A common pattern is to prototype in PsychoPy and re-implement the timing-critical parts in goxpyriment once the design is settled.

11.5 See also

- Migration Guide — concept map and side-by-side code for Expyriment, PsychoPy and Psychtoolbox
- Timing Tests — measured presentation timing on real hardware
- Gallery of Examples — the full catalogue of examples and tests
- WASM build — running experiments in the browser

Chapter 12

A discussion between Gemini and Me about Go vs. Python (March 2026)

Me: Is it fair to say that Go is simpler than Python?

Gemini:

The short answer is **yes**, but it depends on how you define "simple." In the programming world, there is a famous distinction between **Easy** and **Simple**.

Python is generally considered **easier** (more intuitive for humans), while Go is arguably **simpler** (fewer moving parts and more predictable).

12.0.1 1. Minimalism: The "One Way" Philosophy

Go was designed by Google to be a "small" language. The entire language specification is roughly 50 pages, whereas Python's documentation is massive. * **Go:** Deliberately omits features like classes, inheritance, method overloading, and traditional exceptions. There is usually only one way to do something, which reduces "cognitive load" when reading someone else's code. * **Python:** Offers multiple paradigms (Object-Oriented, Functional, Imperative). While "The Zen of Python" says there should be one obvious way, in practice, you have list comprehensions, `map/filter`, decorators, and multiple ways to handle strings or concurrency.

12.0.2 2. Explicit vs. Implicit

Go favors being explicit, which makes the code more "simple" to reason about because there is no "magic" happening behind the scenes. * **Error Handling:** In Python, an exception might bubble up from a deep library call, and you might not know it's coming. In Go, you must explicitly check `if err != nil`. It's more verbose, but it's simpler to track the flow of data. * **Typing:** Python is **dynamically typed**, which is "easy" to write but can be "complex" to debug when a variable unexpectedly changes

from an integer to a string. Go is **statically typed**, catching these errors before the code even runs.

12.0.3 3. Tooling and Deployment

Go simplifies the "meta" part of programming—getting your code to actually run on a server. * **The Binary:** Go compiles into a single static binary. You don't need to worry if the server has the right version of Python, `pip`, or a `virtualenv` installed. You just drop the file and run it. * **Standard Library:** Go's standard library is powerful enough that you rarely need external dependencies for web servers or JSON processing. This avoids the "dependency hell" often found in Python or Node.js.

12.0.4 Comparison at a Glance

Feature	Python (Easy)	Go (Simple)
Syntax	Reads like English; very high-level.	C-like but cleaned up; very few keywords.
Learning Curve	Extremely low; great for first-time coders.	Slightly steeper at first, then plateaus quickly.
Concurrency	Complex (Asyncio, GIL, Threads).	Built-in and easy (Goroutines, Channels).
Maintenance	Can become "messy" in very large teams.	Highly consistent; easy to read others' code.

Which of Go and Python easier for AI coding agents?

In 2026, the answer depends on whether you are measuring "ease" by how fast a **human** can write the code or how effectively an **AI agent** can generate and maintain it.

While **Python** remains the gateway for beginners due to its massive ecosystem, **Go (Golang)** has emerged as the surprise winner for "agentic" workflows—where AI agents are the ones doing the heavy lifting.

12.0.5 Comparison at a Glance

Feature	**Python** (Human-First Ease)	**Go** (Agent-First Ease)
Learning Curve	Very Low (Reads like English)	Moderate (Strict but simple)
Ecosystem	**Dominant** (PyTorch, LangChain, etc.)	Growing (LangGraph-Go, Genkit)
Concurrency	Difficult (GIL limitations)	**Native** (Goroutines/Channels)

Feature	**Python** (Human-First Ease)	**Go** (Agent-First Ease)
Agent Reliability	Medium (Dynamic typing risks)	**High** (Strict types, clear errors)
Deployment	Complex (Virtual environments/Docker)	**Seamless** (Single binary)

12.1 1. Why Python is Easier for *Humans*

Python's "ease" comes from its **abstractions**. It allows you to write high-level logic without worrying about memory or types. * **The Ecosystem Advantage:** If you want to use a new model from Hugging Face or a specific RAG (Retrieval-Augmented Generation) technique, the Python library exists today. * **Rapid Prototyping:** You can go from an idea to a working "toy" agent in 10 lines of code. * **Community:** Almost every AI tutorial on the planet is written in Python, making troubleshooting a matter of a quick search.

12.2 2. Why Go is Easier for *AI Agents*

Recent data from 2025–2026 suggests that LLMs (like Claude or GPT) actually produce **fewer bugs** when writing Go compared to Python. * **Predictability:** Go has a "one right way" philosophy. There are fewer ways to do the same thing, which reduces the "entropy" in AI-generated code. * **Explicit Error Handling:** In Python, agents often "bury" errors in deep `try-except` blocks. In Go, the `if err != nil` pattern forces the agent to handle issues exactly where they happen. * **Type Safety:** Go's compiler catches "hallucinations" (like calling a function that doesn't exist or passing a string instead of an int) before the code ever runs. * **Concurrency for Multi-Agent Systems:** Modern AI setups often involve many agents talking at once. Go's **Goroutines** make managing these simultaneous "thoughts" much simpler and more performant than Python's `asyncio`.

Peer Tip: If you are just starting and want to see results in 5 minutes, go with **Python**. If you are building a production-grade system where an AI agent will help you write the code, **Go** will save you months of "debugging hell" later on.

Chapter 13

Timing tests

This document does two things.

Part 1 shows you how to characterise the timing behaviour of *your* machine with the bundled `Timing-Tests` program — both the checks that need no equipment and the photodiode/TTL measurement that is the real answer — and how to read each of them.

Part 2 summarises what we have measured on our own machines, so you have reference points to compare against. Every figure there names the machine, the configuration, the date and the sample size. Nothing is quoted that we cannot point at a capture for.

> **The one-sentence version of Part 2:** on Linux the display stack dominates everything else by more than an order of magnitude — a compositor costs about sixteen times the onset jitter of a bare KMS/DRM console, and no amount of care in the experiment code recovers it.

Companion documents. `tests/Timing-Tests/README.md` is the flag-by-flag reference — invocations, defaults, equipment, wiring. This page explains what the tests *mean*. For the EEG/MEG question specifically, see Minimising trigger-to-stimulus jitter, and for the comparison against published packages, the mega-study comparison.

13.1 Part 1 — Measuring your own machine

13.1.1 Read this before you trust any number

This suite produces **two kinds of output, and they are not equally trustworthy.** Keeping them apart is the single most important thing on this page.

- **What the program prints to the console** — frame intervals, bright-phase durations, drain times, SOAs — comes entirely from software timestamps, in all six sub-tests without exception. It records when goxpyriment *believes* a flip happened, not what reached the panel.

- **What an instrument records while `av` runs** — a photodiode on the screen, the TTL line on a second channel, a microphone if you are checking audio — is a measurement of the physical event. This is ground truth.

So the suite *does* measure your hardware properly: that is what `av` is for, and `run-timing-tests.sh` can wrap those steps in a `bbtk-capture` recording so the stimulus always falls inside the capture window. What it cannot do is tell you any of it from the console. **Never read the printed statistics as if they were the instrument's.**

This is not a theoretical caveat. A presentation bug on a GNOME/Wayland machine once left the display showing stale frames for *seconds at a time* while the program's own numbers stayed textbook-perfect: bright-phase duration 199.915 ± 0.16 ms against a 200 ms target. Nothing in the console output hinted at a problem. It was visible only with a photodiode. (The cause and fix are in `apparatus/CLAUDE.md`; the regression test is `tests/test_clear_only_frames`.)

None of which makes the console output useless — it is the right tool for a different job. Software statistics genuinely detect dropped frames, scheduler jitter and audio-buffer effects, and `check`, `display` and `latency` need no equipment at all, so run them first. They just cannot confirm that light changed when you said it would.

If you are validating a rig for publication, take the hardware path: `av` plus a photodiode and a trigger box, then `timing-drift` to check the two against each other.

13.1.2 What the display actually does

Most monitors refresh at a fixed rate — 60 Hz (a new frame every 16.67 ms), 120 Hz, or 144 Hz. The driver synchronises presentation to that cycle (VSYNC), with two consequences:

- **Durations are quantised to multiples of the frame period.** At 60 Hz you can show a stimulus for 16.67, 33.33 or 50 ms — but not 25 ms. Plan accordingly, or see the `vrr` test.
- **Onset is not when you call the flip; it is the next VSYNC**, anywhere from 0 to 16.67 ms later. goxpyriment holds that offset constant once the pipeline is warm, which is why the first frames must be excluded — `-warmup` does this.

13.1.2.1 The screen does not change all at once

An LCD paints top to bottom, so a square at the bottom of the screen lights measurably later than one at the top — on a 60 Hz panel, approaching a full frame period. A stimulus at screen centre appears roughly half a frame after one at the top.

This matters more than it sounds: if your photodiode sits in a corner but your stimulus is central, your measured onset carries a systematic bias larger than most of the jitter people worry about.

The gradient is a property of your panel, and you have to measure it. The `av` test draws five squares — four corners plus centre — precisely so you can. Put a photodiode on a top square and another on a bottom one; the difference is your scan-out gradient. Two squares on the same row are only microseconds apart and serve as

a sanity check. Each display needs its own figure.

13.1.3 The six tests

```
No hardware needed
  check    Go/no-go: can this machine display and make noise at all?
  display  What is my true refresh rate, and how stable are frame intervals?
  latency  How long does SDL hold audio before it sounds?

Photodiode and/or trigger box
  av       THE stimulus timing test — visual + audio + TTL, every cycle
  vrr      Can I present arbitrary durations, not just multiples of a frame?
  rt       How precise is my reaction-time measurement?
```

`av` is the default: running with no `-test` flag runs it.

Test	What it measures	Reads its answer from
`check`	Nothing. Flashes white, plays a buzzer and a ping.	Your eyes and ears
`display`	Frame-interval distribution over `-duration-s`; estimated true refresh rate; the frame-pacing verdict	Console
`latency`	How long SDL holds PCM beyond a tone's nominal duration — the software audio pipeline delay	Console
`av`	Bright-phase duration, cycle period, software-side audio-vs-visual SOA — **and drives the photodiode/TTL capture that is the real measurement**	A recorder
`vrr`	Achieved-vs-target duration across a 1 ms-step sweep with VSync off	Console, then a photodiode
`rt`	SDL3 hardware event timestamp against the flip timestamp	Console

Note that `-test latency` labels its own output `drain:` and writes `test=drain` into the info file — the drain time *is* the measurement.

Two related tests live in their own directories: `tests/test_gv_sync` (`.gv` playback synchronisation) and `tests/test_dlpio8` (DLP-IO8-G square-wave characterisation).

Build it once:

```
go build -o Timing-Tests ./tests/Timing-Tests
```

Run every test fullscreen — that is the default. `-w` exists for debugging the tests themselves; a windowed run measures your compositor, not goxpyriment.

13.1.4 Recording the machine (`-sysinfo`)

Before running anything, capture a snapshot. `-sysinfo` prints it and exits — it opens no window, though it initialises SDL briefly to enumerate displays and audio devices:

```
Timing-Tests -sysinfo > sysinfo-$(hostname)-$(date +%Y%m%d).txt
```

A real capture (`tests/Timing-Tests/report-dell-precision-5490-laptop-kmsdrm/sysinfo.txt`):

```
Machine:    product: Precision 5490  System: Dell Inc.  Type: laptop
System:     Host: is158520  Kernel: 7.0.0-29-generic x86_64  Uptime: 32m
            OS: Ubuntu 26.04 LTS  Shell: bash 5.3.9
CPU:        Model: Intel(R) Core(TM) Ultra 7 165H  Info: 16 cores / 22
 ↪  threads  Speed: 1076 MHz (min: 400 / max: 4700)
Memory:     RAM: total: 30.04 GiB  used: 2.12 GiB (7.0%)  Swap: total: 8.00
 ↪  GiB  used: 0 KiB (0.0%)
Graphics:   Card: Intel Corporation Meteor Lake-P [Intel Arc Graphics]
 ↪  Driver: i915
            Card: NVIDIA Corporation AD107GLM [RTX 2000 Ada Generation
             ↪  Laptop GPU]  Driver: nvidia
Audio:      Card: sof-soundwire  Driver: snd_soc_sof_sdw
            Server: PipeWire  v: 1.6.2  ALSA: k7.0.0-29-generic
Sched:      policy: SCHED_OTHER  nice: 0  (real-time available up to 50, not
 ↪  used)
Displays:   [0] eDP-1                  2560x1600  60.040 Hz  bounds 0,0
 ↪  2560x1600  [primary]
            [1] DP-5                   2560x1440  59.950 Hz  bounds 2560,0
             ↪  2560x1440
            video driver: kmsdrm   (the [N] above is the -d N value)
Audio out:  [0] Dummy Output
            driver: pulseaudio
```

Reviewers routinely ask for CPU model, OS version, graphics driver and audio server. Two lines repay a second look:

- **`Sched:`** records the policy the run actually got, not the one it asked for. See Setting priority under Linux.
- **`Displays:`** — on a multi-monitor machine the heads differ in resolution, refresh rate and pixel density, and every calibration is per-display. Each data file's `-info.txt` records **which display the window actually opened on**; read it from there rather than assuming.

`-sysinfo` also prints a `Vblank:` line. By default, onsets are timestamped when `SDL_`

`RenderPresent` returns; `GOXPY_VBLANK=on` instead anchors every frame on the kernel's DRM vblank stamps. **It is off by default and there is currently no reason to turn it on** — measured against a photodiode the two are indistinguishable, both flat to well under a ppm. The case it exists for is a host whose nominal refresh rate is badly wrong. If you do enable it, read `sys vblank_resolution:` in the run's `-info.txt`: **if it ever says `WRONG DISPLAY`, the onsets in that file are on another monitor's frame grid and must not be used.** The mechanism lives in the `vblank/` package.

13.1.5 The `Frame pacing` block

`-test display` and `-test av` both end with a block like this:

```
── Frame pacing ──────────────────────────────
  presents: 588   (frame = 16.656 ms)
  blocked : 566 (96.3 %)  — present carried the frame; its return is the
   ↪  onset
            of which 281 came back inside the nominal boundary by mean
            0.285 ms, max 1.861 ms — the phase offset between the nominal
            frame grid and the panel's, not a wait.
  paced   : 22 (3.7 %)  — returned early; Update held to the schedule
            wait mean 2.496 ms  max 2.696 ms
  verdict : the driver blocks. Flip timestamps carry the display's own
            instant, and cannot drift against the panel.
```

It does not report dropped or missed frames. Every present is counted here; a frame that never reached the panel shows up in the interval statistics above it, not in this block.

It answers a different question: *what were this run's flip timestamps anchored to?* `SDL_RenderPresent` is supposed to block until the retrace, but under triple or mailbox buffering it queues the frame and returns immediately. When it does, `Screen.Update` holds the frame to the boundary itself — and the onset it reports is then a *scheduled* time rather than a hardware one.

line	meaning	the onset `FlipTS` reports
`blocked`	present covered the frame and returned at, or just inside, the boundary	the present's own return — a hardware instant
`early`	the part of `blocked` that returned *just* inside the nominal boundary	unchanged: still the present's return
`paced`	present came back with most of the frame left, so `Update` held it	the scheduled boundary — synthesised
`vblank`	a kernel vblank timestamp was available (`GOXPY_VBLANK=on`)	the vblank — measured

A large `early` count is not a warning. A blocking present returns one *panel* period

after the last one, while the boundary it is compared against is one *nominal* period after — and the two grids are never in exact phase. It is a constant offset, re-established by the hardware every frame, so it cannot accumulate, and it cancels out of any duration or reaction time (both are *differences* between timestamps).

Read the **verdict**, not the percentages: it is computed from the time held per present across the whole run, precisely because a driver that blocks but sits a fraction of a millisecond off the nominal grid can take the held branch on nearly every frame while blocking for 96 % of each one.

- **`the driver blocks`** — nothing to do. Onsets are hardware-anchored.
- **`onsets come from the kernel's vblank timestamp`** — nothing to do; the most accurate configuration available.
- **`the driver does NOT block`** — your frames are still correctly paced and your *durations* are fine, but the onsets are stamped with a schedule running at the nominal refresh rate. If that rate is wrong, absolute onsets slide against the panel over long blocks. Compare the estimated refresh rate printed above against the nominal one — the gap between them *is* the drift rate — and run `timing-drift` on a photodiode capture before quoting absolute onsets.
- **`MIXED`** — some frames took each path. Worth a photodiode check before quoting absolute onsets.

A mostly-paced machine is not broken, and that is exactly the problem: **pacing makes a non-blocking driver produce software statistics indistinguishable from a blocking one.** What it cannot do is keep the schedule and the panel in step forever.

Only the third verdict asks anything of you, and even then it bounds a *slow slide* over minutes, not per-trial error.

The same counts are written to every run's `-info.txt`, so you can tell after the fact which branch a capture took:

```
# sys pacing: presents=30000 blocked=30000 early=29610 paced=0
 ↪  vblank_held=0 (0.0 % paced) … class=blocking
# sys pacing: presents=30000 blocked=0 early=0 paced=30000 (100.0 % paced)
 ↪  wait_mean=16.141 ms … class=not-blocking
```

A drift-free result means two different things in those two cases. `class=blocking` means every frame re-anchored to the hardware, so the run says little about how accurate the pacing schedule is. `class=not-blocking` means the schedule *was* what advanced the clock, and a flat result there is evidence about the schedule itself. Without this line the two are indistinguishable afterwards, and a set of captures can end up unable to answer the question it was run for.

13.1.6 `av` — the stimulus timing test

This is the one that matters. Every cycle presents all three modalities at once:

- **Visual** — five squares (four corners + centre) bright for `-frames-on` frames, then dark for `-frames-off` frames
- **Audio** — a tone lasting `frames-on × frame-period`, synchronised to the visual onset or offset from it by `-soa-ms`

- **TTL** — a `-trigger-ms` pulse at the visual onset, on the device chosen by `-trigger-device`

```
Timing-Tests -test av -cycles 100          # 200 ms on, 300 ms off at 60 Hz
Timing-Tests -test av -no-sound -no-ttl    # visual only, no hardware at all
Timing-Tests -test av -soa-ms -50         # audio leading the flash by 50 ms
```

The frame period is read from the display, not passed in; `-frames-on 12` is 200 ms only if the panel really runs at 60 Hz, which is what `-test display` tells you.

`-no-sound` and `-no-ttl` drop a modality, which is how one test covers what used to be three.

What to record. A photodiode on one square, the TTL line on a second channel, a microphone if you are checking audio. The quantity of interest is the *offset* between the TTL edge and the luminance step, and how stable it is across cycles. A constant offset is a calibration you subtract; a varying one is jitter you cannot.

What the console tells you. Bright-phase duration, cycle period, and (with sound) audio-queued minus visual onset. All software-side — cross-checks, not ground truth.

Expect a warm-up transient. The first several cycles run long and variable, and they dominate the summary statistics if left in. `-warmup` (default 10) excludes them from the test's own statistics, but offline analysis tools do not know about it — quote steady-state numbers, and say which cycles you used.

Audio onsets are quantised by the buffer. With `-audio-frames 256` at 44100 Hz, tone onsets land on 5.8 ms steps. This appears as a lattice in the inter-onset intervals and is not jitter — it is the buffer. See Audio: the buffer size is the whole story before choosing a value; smaller is emphatically not better.

Choosing a TTL output device. `-trigger-device` selects among `dlpio8` (USB serial, 5 V), `parallel` (ppdev `ioctl`, 5 V), `gpio` (GPIO chip `ioctl`, 3.3 V), `ft232h` (USB bulk, 3.3 V) and `labjackt4` (Modbus TCP, 3.3 V). They are not interchangeable — the trigger fires immediately after the flip returns, so whatever the write costs lands in the trial-to-trial *spread* of the flip-to-edge interval. The full table, the per-device meaning of `-trigger-pin`, and the Linux prerequisites are in `tests/Timing-Tests/README.md`, under "Choosing a TTL output device"; what the differences actually cost is in How much the trigger device costs.

> **A device that fails to open no longer silently continues.** `av` refuses to start if the chosen trigger device could not be opened, unless you passed `-no-ttl`. Still watch the start-up lines — they echo the device, the resolved pin and the pulse width.

Running a whole session. `tests/Timing-Tests/run-timing-tests.sh` drives the standard sequence (`sysinfo check display display-gc av av-gc av-visual latency`) and can wrap the photodiode steps in a `bbtk-capture` recording so the stimulus always falls inside the capture window. Set `BBTK_CAPTURE=1` and `BBTK_CAPTURE_BIN`; `BBTK_MARGIN_S` (default 8) sets how much is recorded either side. Its environment variables are documented in its own header and in `tests/Timing-Tests/README.md`.

If you are recording audio with a BBTK microphone, max the output volume

and place the microphone within a few millimetres of the speaker membrane. Anything less and events are silently dropped. Two checks on the resulting `-events.csv`: `N(Mic1)` should equal `N(TTLin1)`, and the Mic duration should be close to `frames-on × frame period`. A 197-cycle run yielding 6 Mic events is a placement problem, not a timing one.

13.1.7 `vrr`: arbitrary stimulus durations

On a 60 Hz monitor every duration is a multiple of 16.67 ms. At 120 Hz the quantum shrinks to 8.33 ms — better, but still coarse for subliminal work where you may want 10, 12 or 15 ms.

Variable Refresh Rate — AMD FreeSync, NVIDIA G-Sync, or the underlying VESA Adaptive-Sync standard — removes the quantisation: the display holds the current frame for exactly as long as you ask. Durations become controlled by your software timer rather than the display clock.

```
Timing-Tests -test vrr -vrr-max-ms 50 -vrr-reps 5
```

The test switches the renderer to vsync=0 for its duration (restored on exit), sweeps target durations from 1 ms to `-vrr-max-ms` (default 20) in 1 ms steps, `-vrr-reps` times each, holding with a busy-wait and recording what was achieved. It prints one line per target plus an aggregate over the sweep.

- **On a working VRR display:** duration error stays small and roughly constant across the whole sweep.
- **On a fixed-refresh display:** the error is *periodic* — near zero at multiples of the frame period, rising in between. A sawtooth means you do not have VRR, whatever the monitor's box claimed.

On Linux, check the connector first (`sudo cat /sys/kernel/debug/dri/0/*/vrr_capable`, looking for `vrr_capable: 1`) — then confirm with the test rather than trusting the setting.

We have no photodiode capture of a VRR run. The console numbers describe what the busy-wait achieved, not what the panel did.

13.1.8 `rt` — reaction-time timestamp precision

```
Timing-Tests -test rt -cycles 50
```

Flashes the screen at a jittered interval and waits for a key press, recording the SDL3 hardware event timestamp against the flip timestamp. This characterises the *input* path: how much delay and jitter your keyboard, USB stack and event queue add.

Press as fast as you can, or better, use a solenoid — you are characterising the machine, and human variability swamps it. The useful number is the SD, not the mean.

13.1.9 `timing-drift`: is the flip timestamp tracking the panel?

The two av output files each look fine on their own; a drift failure exists only *between* them. This tool joins them and reports the slope first:

```
make timing-drift
./_build/timing-drift bbtk-av-001-events.csv Timing-Tests_sub-000_
 ↪  date-….csv
```

Options: `-ttl` (channel name, default `TTLin1`), `-light` (default `Opto1`), `-hz` (0 = read the run's sibling `-info.txt`), `-warmup` (10), `-out`.

It prints, for TTL→light and then for flip-timestamp→light:

- the **slope**, in µs/cycle and ppm — the number that matters;
- the **de-trended SD**, the real trial-to-trial scatter once the ramp is gone;
- how much of the variance the ramp accounts for;
- one-frame jumps, and the panel's **true frame period** fitted from the photon train, scored against both rates the framework recorded.

The host and the instrument run off different crystals — tens of ppm apart before anything interesting has happened — so the tool fits that clock ratio out of the trigger-versus-flip relation rather than assuming it away. Events are paired by nearest neighbour within half a cycle, not by index, because a single spurious detection shifts every later pair by a whole cycle.

Every slope is fitted over the longest stretch containing no dropped frame, and the `fitted over` line says which cycles those were. This is not tidying: a frame-sized step at index k of n points biases a least-squares slope by $6 \cdot h \cdot m \cdot k/n^3$ (with $m = n-k$), which at the midpoint is $1.5 \cdot h/n$. Over a 1000-cycle run at 60 Hz that is 25 µs/cycle — **50 ppm from a single dropped frame in eight minutes**, ten times the effect these captures exist to resolve, and it does not average away with more cycles because the lever arm grows with n too. On a short capture the bias is larger still: 20 cycles with one drop puts it above 2000 ppm.

The `one-frame jumps` count is printed separately, and a run with more than a few of them is not a usable capture however clean the surviving stretch looks.

Read the verdict line. `DRIFT DOMINATES` means the flip timestamps and the panel are on different clocks and no absolute onset from that run can be trusted; `NO DRIFT, bu t … scatter` means the timestamps track the panel and the spread is genuine.

13.1.10 Rules of thumb for your own numbers

These thresholds are judgement, not measurement. No capture in this repository established them; they are a starting point for a reader with no reference at all. For real reference points — machines, dates, sample sizes — use Part 2.

Metric	Excellent	Acceptable	Problematic
Frame-interval SD (`display`)	< 0.1 ms	< 0.5 ms	> 1 ms

Metric	Excellent	Acceptable	Problematic
Frames > 0.5 ms late (`display`)	< 1 %	< 5 %	> 10 %
Audio pipeline latency (`latency`)	< 15 ms	< 30 ms	> 50 ms
Bright-phase duration SD (`av`)	< 0.2 ms	< 1 ms	> 2 ms
TTL→photodiode SD (`av`, hardware)	< 0.5 ms	< 2 ms	> 5 ms
Dropped frames per 100 cycles (`av`)	0	< 5	> 10
VRR duration error SD (`vrr`)	< 0.1 ms	< 0.5 ms	> 1 ms (or periodic → no VRR)
RT SD (`rt`)	< 3 ms	< 10 ms	> 20 ms
Pacing verdict (`display`)	vblank, or blocks	MIXED	does NOT block

The hardware rows are the ones worth quoting in a methods section.

The pacing row grades *what the onsets are anchored to*, not frame quality, and it is the one row where "problematic" still leaves durations and reaction times intact — read The `Frame pacing` block before acting on it.

An SD alone cannot grade a TTL→photodiode series, and that row is the one most likely to mislead. A delay that slides steadily across a run has an SD like any other, and a large one — yet nothing about it is jitter, it cannot be averaged away, and every within-channel statistic in a BBTK report looks perfect while it happens. Measured on a Raspberry Pi 4: SD 4.07 ms, of which **99.8 % was a monotonic 13.9 ms ramp** and 0.18 ms was real scatter. Run `timing-drift` before reading this row.

13.1.11 Loading the data in Python

Each run writes two files to `~/goxpy_data/`, or to `-outdir` if given:

- `Timing-Tests_sub-000_date-<YYYYMMDD>-<HHMMSS>.csv` — data rows, no comments
- `...-info.txt` — session metadata: times, host, OS, and the display and audio configuration actually in use

```
import pandas as pd

df = pd.read_csv("~/goxpy_data/Timing-Tests_sub-000_
 ↪  date-20260817-091541.csv")

# av: drop the warm-up cycles before quoting anything
steady = df[df.cycle >= 10]
print(steady.bright_duration_ms.describe())
print(steady.period_ms.std())
```

Every CSV starts with `subject_id`. After that:

Test	Columns
`av`	`cycle`, `t_visual_before_ms`, `t_visual_after_ms`, `bright_duration_ms`, `period_ms`, `t_audio_queued_ms`, `soa_intended_ms`, `soa_actual_ms`, `onset_source`
`display`	`frame`, `t_before_ms`, `t_after_ms`, `interval_ms`
`rt`	`trial`, `onset_ns`, `event_ts_ns`, `rt_ns`, `rt_ms`
`vrr`	`target_ms`, `rep`, `actual_ms`, `duration_error_ms`, `onset_ns`, `offset_ns`, `trigger`
`latency`	`duration_ms`, `rep`, `drain_ms`, `overhead_ms`
`check`	none — the CSV is empty; only the `-info.txt` is meaningful

`onset_source` records how each onset was anchored: `hardware-verified`, `present-return`, `pacing-schedule`, `vsync-estimated` or `unknown`. Captures made before this column existed do not have it.

Always read display parameters from the run's own `-info.txt` rather than assuming — on a multi-monitor machine the heads differ in every relevant parameter, and the header records which one the window opened on.

13.1.12 Improving timing on your system

If you need a sub-millisecond spec, in order of what it is worth. Every figure is measured on this page or in Minimising trigger jitter:

#	do this	worth
1	Get off the compositor — `SDL_VIDEODRIVER=kmsdrm` from a VT, or a non-compositing WM with exclusive fullscreen	**16×** (sd 1.344 → 0.083–0.113 ms)
2	Run at `SCHED_FIFO` 50	**1.8×** (sd 2.342 → 1.320 ms)
3	CPU governor `performance`	drift +11.8 → **+0.41 ppm**
4	Count frames, never sleep, for stimulus durations and ISIs (`ShowFrames`/`BlankFrames`)	trigger→light spread 14.1 → **3.4 ms**

#	do this	worth
5	Raise the trigger synchronously on the flip thread (`triggers.FireTriggerSync`), not `go FireTrigger(...)`	0.73 ms → **34 µs**
6	Discard warm-up trials — the display settles	sd 1.127 → **0.384 ms** over the first 10
7	Measure the flip→photons offset once on this rig, record it, subtract it in analysis	it is constant to 83–113 µs; see below

Steps 1–3 are the display and the scheduler, 4–6 are how the experiment is written, and 7 is the one thing no setting can remove.

Linux

- **Get off the compositor.** By far the largest single improvement, and it is not close — see Part 2. Start from a virtual terminal (`Ctrl+Alt+F2`) with the window system stopped (`systemctl stop gdm`) and `SDL_VIDEODRIVER=kmsdrm`. Failing that, a plain non-compositing window manager (openbox, i3) with exclusive fullscreen is the other measured-good configuration. See Linux virtual console for the SDL driver selection details.
- **Run at real-time scheduling.** Worth a measured factor of 1.8 in onset jitter. Grant your user the privilege once; after that goxpyriment programs — `Timing-Tests` included — request priority 50 at startup on their own:

  ```
  Timing-Tests -test display -duration-s 30    # asks for real-time itself
  Timing-Tests -realtime-priority 0 -test display   # opt out
  ```

  The `Sched:` line in the system report records which policy it actually got. The full procedure — and the traps, including that the limit is inherited from your login session and a new terminal will not pick it up — is in Setting priority under Linux.
- Disable CPU frequency scaling: `cpupower frequency-set -g performance`.
- Consider a real-time kernel — see https://ubuntu.com/blog/enable-real-time-ubuntu.
- **Send triggers off the USB bus if you can.** `-trigger-device parallel` on a machine with an LPT port, or `-trigger-device gpio` on a single-board computer; both write via one `ioctl`. Before assuming `/dev/gpiochip0` owns the header, ask the system (`gpiodetect`, `gpioinfo`) — which chip carries the 40-pin header varies by board and kernel, and another chip's lines may drive real board hardware. Verify the wiring with `tests/test_linuxgpio` or `tests/test_parallel_port` before spending a long capture.

- **Lower the FTDI latency timer** if you *read* responses over a USB serial device. It does nothing for *sending* triggers — see the DLP-IO8 appendix.

Windows

- Disable "Hardware-Accelerated GPU Scheduling" in Display Settings if you see high frame jitter. See Setting priority under Windows.

13.2 Part 2 — What we found

Every figure below names the machine, the stack, the date and *n*. The full workings live in Minimising trigger-to-stimulus jitter and the mega-study comparison; this is the summary.

13.2.1 The display stack is the dominant term

Three runs differing only in the display stack — same binary, same machine, same panel, same night. Dell Precision 5490, Intel Arc (Meteor Lake), 60.04 Hz, `SCHED_FIFO 50`, synchronous trigger, DLP-IO8 and photodiode captured in one Analog Discovery 3 acquisition at 200 kS/s so the instrument's clock cancels. Onset is the photodiode's 10 % crossing. **2026-08-08**, ~590 trials per arm:

stack	mean	**sd**	full range	flips > 1 ms off the frame grid
Wayland session	21.75 ms	1.344 ms	18.83–36.74	7.1 %
KMS/DRM, no display server	18.91 ms	**0.113 ms**	18.58–19.13	**0 of 590**
Bare Xorg + openbox, exclusive fullscreen	35.74 ms	**0.083 ms**	**35.52–35.95**	**0 of 580**

Sixteen times better, and the mechanism is gone rather than reduced. Off a compositor, not one flip in ~590 lands more than 1 ms off the frame grid; under Wayland one trial in fifteen does. An entire five-minute Xorg run fits inside a 0.43 ms band. By comparison, real-time priority on the same rig is worth 1.8× (sd 2.342 → 1.320 ms).

The result is three-way, not "kmsdrm beats Wayland". Bare Xorg is the *steadiest* and the *latest*, which is not a contradiction — they are different quantities. The gap is exactly one frame:

```
Xorg − KMS/DRM = 16.826 ms = 1.010 frames
```

One more buffer in the pipeline, dead constant. For EEG and MEG the distinction is the whole point: **a constant 19 or 36 ms is subtracted in analysis and costs**

nothing; 1.3 ms of scatter around it cannot be. Measure your offset once per rig, record it, subtract it.

Set against the fourteen lab configurations of Bridges et al. (2020) Table 2, the bare-Xorg run has the best onset precision in the table by a factor of two, and the Wayland run is worse than thirteen of the fourteen — *the same binary on the same machine*. See the mega-study comparison.

13.2.1.1 The display stack is worth two frames of latency and five times the jitter

A second, independent campaign found the same effect on different hardware. **2026-08-17**, BBTK v3, a Radeon Pro W5700 (radeonsi) driving a DLP-IO8, one variable changed:

W5700, everything else identical	onset latency	scatter	n
X11, exclusive fullscreen	54.88 ms	0.296 ms	481
kmsdrm, bare console	**22.55 ms**	**0.057 ms**	286

32.4 ms is almost exactly two frames — an X11 Present path plus a driver swapchain queueing ahead, where on kmsdrm the application page-flips straight to the CRTC.

This is a different machine and a different campaign from the table above, which is why its X11 scatter (0.296 ms) is not the 0.083 ms measured on the laptop's bare Xorg. That configuration was openbox with no compositor at all; this one was a full X session. The two are not the same condition, and the comparison to draw from each is the within-campaign one.

The panel is not involved: its 10–90 % transition measured 16.29 ms on a Pi 4 and 17.28 ms here, with the same asymmetry between halves, so the same monitor behaves the same way on both machines and the latency is upstream of it.

None of this is visible from inside the program. Both configurations report `class=blocking`, sub-ppm drift against the panel, and frame intervals with sub-millisecond scatter. An experiment quoting an absolute onset would be 32 ms wrong under X11 with no way to know.

13.2.2 What a returning `SDL_RenderPresent` actually means

The tables above raise an obvious question: if the present blocks on VSYNC, why are the photons one to three frames later?

Because it does not block on *your* frame. `SDL_RenderPresent` returns when the driver will accept the **next** frame — i.e. when a buffer slot frees, which happens as an *earlier* page flip completes. The frame just drawn becomes the scanout source at a later vblank. This is true even on bare KMS/DRM with no display server; it is a property of the swap chain, not of the compositor.

Psychtoolbox gets scanout semantics deliberately: it schedules the swap against a specific future vblank with `glXSwapBuffersMscOML`, then blocks in `glXWaitForSbcOML`

until *that* swap has completed. Hence its 4.53 ms in Bridges et al. (2020) against our 18.91 ms on the equivalent stack.

This also explains an apparent contradiction in the numbers above. The pacing block reports `class=blocking`, with present covering 15.99 ms of a 16.66 ms frame. That is real — it is simply blocking on the *previous* frame's completion. Nothing inside the process can tell the two cases apart, which is why an experiment quoting an absolute onset can be 32 ms wrong with no indication.

So a correct cadence and a multi-frame absolute offset are not in conflict. The flip timestamps track the panel to sub-ppm (next section) *and* lead the photons by one to three frames. The first makes durations and intervals exact; the second is a constant that has to be measured per rig and subtracted. Both are true at once, and only the second needs a photodiode.

The arithmetic closes: subtracting the queue depth from the archived flip→photons figures leaves 4.9–6.0 ms on one panel across two machines and two stacks — inside the 2.35–7.10 ms band Bridges et al. report for every Linux and Windows package they tested. The workings, and the limitation that the diode position was not recorded, are in `TODO.md` under "Presentation latency".

13.2.3 Flip timestamps track the panel

Two machines, BBTK v3 photodiode, **1010 cycles each, 2026-08-17**, after the flip-timestamp anchoring was corrected:

	Raspberry Pi 4 (V3D/kmsdrm)	Radeon Pro W5700 (radeonsi/X11)
flip → photons, slope	**+0.01 ppm**	**+0.13 ppm**
total drift over the run	0.006 ms / 8.3 min	0.066 ms / 8.3 min
de-trended scatter	0.128 ms	0.278 ms
one-frame jumps	0	0
TTL → light	23.43 ms, sd 0.075 ms (GPIO, kmsdrm)	54.88 ms, sd 0.296 ms (DLP-IO8, X11)

The W5700 is the informative case for drift. Its GPU and CPU keep independent crystals, so its loop cadence runs **−4.20 ppm** off nominal; before the anchoring was corrected, its flip timestamps advanced on the CPU clock at the nominal rate while the panel ran on the GPU clock, and that difference showed up as a measured **−4.73 ppm** of drift. The two numbers are the same quantity. That the cadence is still −4.20 ppm while the drift is now +0.13 ppm is the evidence that onsets follow the panel rather than the schedule.

For a sense of what a healthy `display` run looks like on a good stack, the laptop above on kmsdrm (2026-08-09, `tests/Timing-Tests/report-dell-precision-5490-laptop-kmsdrm/display-gc-off.txt`) estimated 60.037 Hz with frame intervals **mean 16.656 ms, SD 0.008 ms** over n=677, and not one interval more than 0.5 ms late.

13.2.4 Audio: the buffer size is the whole story

Measured on a Raspberry Pi 4 (PipeWire, 48 kHz) on **2026-08-17**, capturing the line output directly with an Analog Discovery 3 at 100 kS/s and taking the onset from a running-RMS envelope:

driver	`-audio-frames`	period	n	median AV lag	SD	`period/√12`	torn tones
PipeWire	2048	42.67 ms	60	101.50 ms	12.28 ms	12.32 ms	none
PipeWire	**1024**	21.33 ms	481	**49.88 ms**	**6.17 ms**	6.16 ms	**none**
PipeWire	512	10.67 ms	500	27.30 ms	7.28 ms*	3.08 ms	10 (2.0 %)
PipeWire	256	5.33 ms	483	9.43 ms	2.36 ms	1.54 ms	100 (20.7 %)
ALSA direct	512	10.67 ms	463	4.95 ms	6.74 ms	3.08 ms	**463 (100 %)**

* inflated by a mid-run escalation, below; its first half scatters by 3.10 ms.

1024 is the recommended setting on this hardware — the only one that is both clean and stable, and what `run-timing-tests.sh` now uses.

A large buffer costs jitter, and the jitter is a beat, not noise. The tone is handed over on time and then waits for the queue, so the audio onset scatters across one buffer period — matching `period/√12` to within 0.2 % at 1024 and 2048. But it is deterministic: the tone's position in the buffer grid advances by `cycle mod period` every trial and wraps. At 512, a 499.830 ms cycle predicts +1.503 ms per trial with a −9.163 ms wrap; observed, +1.500 ms (n=161) and −9.170 ms (n=58) — agreement to 7 µs. So a per-trial lag can be **predicted** rather than merely bounded, and a cycle chosen as a whole multiple of the buffer period removes the beat altogether.

A small buffer costs glitches, and PipeWire moves the goalposts. At 512, ten of the first 245 tones were torn (gaps to 37.7 ms); then at trial 245 the lag stepped by **+10.85 ms — one buffer period — and the tearing stopped dead**, with no glitch in the remaining 255 tones. That is the audio server adding a period to the graph after repeated underruns. A naive linear fit through that step reports a 15 ms "drift" that does not exist.

The floor is the hardware, not the audio server. Bypassing PipeWire entirely (`SDL_AUDIODRIVER=alsa` on `hw:2,0`, server stopped) gave the lowest latency of anything measured — median 4.95 ms, some trials negative — and was completely unusable: **every one of 463 tones was torn**, typically five times each. So the Pi cannot sustain a 10.7 ms buffer, and PipeWire's mid-run escalation was it discovering the same floor and working around it. Sit above the floor deliberately rather than removing the thing that was hiding it.

Scheduling priority is not involved. The 2×2 of 512/2048 against `-realtime-priority 50/0` scratched at 512 under both policies and was clean at 2048 under

both.

None of this appears in what the host prints. In these same runs the software-side SOA read **0.080 ± 0.035 ms** — handing the tone over on time is the part the software does correctly. Check underruns at the source instead: `pw-top` over ssh, watching the **ERR** column.

One methodological note worth keeping. The 512 case was first measured through a BBTK microphone channel, which reported 23 % of tones torn by gaps whose median was 22.3 ms; the electrical capture found the tearing real but affecting 2.0 % of tones, mostly by about 2 ms. A threshold detector watching an acoustic envelope needs the signal to recover past its threshold before it calls the tone present again, so short electrical glitches read as long acoustic gaps. **A second instrument measuring by a different route is the only way to learn what the first one is adding** — and prefer the route with fewer transducers when the question is about software.

13.2.5 How much the trigger device costs

Two campaigns on **2026-08-21** put trigger devices on one schedule and one timebase, an Analog Discovery 3 at 1 MS/s:

comparison	machine	result
DLP-IO8 vs FT232H	Precision 5490, `SCHED_FIFO 50`, 1.5 V threshold	DLP-IO8 lags by **179.4 µs**, sd 5.7, n=322 over three 60 s runs
DLP-IO8 vs parallel port	Precision 3650 Tower, 2.5 V threshold	DLP-IO8 follows by **185.9 µs**, sd 12.6, n=900; worst pulse 254.8 µs

The host's own contribution — flip return to trigger write — was **0.03 µs median**. So the ~180 µs is the USB serial path, it is consistent across two machines and two comparison devices, and against a 1 ms trigger budget it leaves a 3.9× margin.

The practical reading: the DLP-IO8 is fine for a millisecond-scale budget, and the `ioctl` paths (`parallel`, `gpio`) are measurably tighter if you have them. Full data in `tests/test_triggers/report-is158520/` and `tests/test_triggers/report-precision3650/`.

13.2.6 What we have not measured

Stated so the next reader knows where the edges are:

- **The browser.** WASM frame pacing has been characterised in software (16.666 ms, SD 0.11–0.12 ms over 299 frames, headless Chrome, 2026-07-13) but there is **no photodiode validation of actual pixel onset** — see WASM.md.
- **VRR.** No photodiode capture of a `vrr` sweep exists. The console numbers describe the busy-wait, not the panel.
- **Reaction time.** No solenoid or hardware-actuator characterisation of the `rt` input path.

- **Scan-out gradient.** No current report directory carries a top-to-bottom gradient measurement, which is why Part 1 tells you to measure your own.
- **macOS and Windows.** Every figure on this page is from Linux.

13.3 Related tests

- `tests/test_gv_sync` — `.gv` video playback: does `PlayGvFunc` put a frame on screen when it says it does?
- `tests/test_linuxgpio` — GPIO character-device output/input, used to verify header wiring before a session.
- `tests/test_parallel_port` — LPT data lines via ppdev, the equivalent check for a parallel-port trigger.
- `tests/test_dlpio8` — square-wave output for characterising the trigger box itself against an oscilloscope.
- `tests/test_triggers` — two trigger devices against each other on one timebase; the source of the 180 µs figure above.
- `tests/test_clear_only_frames` — regression test for the compositor presentation bug described at the top of this document.
- `tests/test_vsync_blocking` — does `SDL_RenderPresent` block on VSYNC on this platform, or return immediately?

13.4 Contributing a report

If you measure your own rig, we would like to see it — good results help people choose hardware, and bad ones tell us what to fix. See `performance-reports/README.md` for what to include.

Chapter 14

goxpyriment against the timing mega-study

Bridges D, Pitiot A, MacAskill MR, Peirce JW (2020). *The timing mega-study: comparing a range of experiment generators, both lab-based and online.* PeerJ 8:e9414. DOI 10.7717/peerj.9414

Their Table 2 gives visual onset, visual duration, audio onset and audiovisual synchrony for PsychoPy, PsychToolBox, Presentation, E-Prime, OpenSesame and Expyriment across three operating systems. This page puts goxpyriment's measured numbers next to it, on the visual measures.

Short answer: the display stack decides this, not the package. In a Wayland session goxpyriment is worse than thirteen of their fourteen lab configurations. On bare Xorg — the stack they actually measured — it is better than all fourteen, by a factor of two, at sd 0.083 ms. The same binary, the same machine, the same night.

Caveat: we ran our tests on a different machine from the one the mega-study was run on, so, like the paper's authors, we decline to generalise across machines.

The protocol mapping — which flags reproduce their trial structure — lives in `paper/megastudy/megastudy_timing_tests.md`.

See also Minimising trigger-to-stimulus jitter, which is where the mechanism is worked out in detail.

14.1 Conditions

Built-in 2560×1600 laptop panel at **60.0400 Hz** (SDL) / **60.0395 Hz** (measured from the photodiode train — 8 ppm apart), Mesa, Intel Arc (MTL). 12 frames on, 18 off, 640 cycles, fullscreen, photodiode on the top-left patch. An Analog Discovery 3 at 200 kS/s recorded the TTL and the photodiode in **one acquisition**, so the instrument's clock cancels; a Black Box ToolKit v3 was on the same trigger line throughout. `onset` is the photodiode's 10 % crossing.

Five runs. A→C vary the software while holding the stack fixed; C, D and E vary only the stack.

	harness / stack	n	**sd, AD3**	mean, AD3	steps > 1 ms	TTL > 1 ms off the frame grid
A	Wayland, normal priority, goroutine trigger	598	2.342 ms	21.18	82	13.07 %
B	Wayland, FIFO 50, goroutine	590	1.320 ms	20.96	37	6.28 %
C	Wayland, FIFO 50, syn-chronous	590	1.344 ms	21.75	43	7.13 %
D	**KMS/DRM**, no display server	591	**0.113 ms**	18.91	**0**	**0.00 %**
E	**Bare Xorg + openbox**, exclusive fullscreen	581	**0.083 ms**	35.74	**0**	**0.00 %**

Photons were more than 1 ms off a whole frame period in **0.00 %** of intervals in all five runs — median error 6 µs, worst case 169 µs, 2951 trials. The panel was never the variable.

Three software hypotheses were tested against the fixed Wayland stack. Real-time priority is worth 1.8×. Firing the trigger synchronously rather than from a goroutine is worth **nothing** — 1.344 against 1.320, in the wrong direction; `test_photodiode_latency` logs its own flip-to-trigger gap and puts the entire host-side path at **median 11 µs, max 38 µs**. Changing the display stack is worth **16×**.

14.2 The stack decides it, and it is not a single axis

only the stack differs (all FIFO 50, synchronous trigger)	mean	sd	full range
Wayland session	21.75 ms	1.344 ms	18.83–36.74

only the stack differs (all FIFO 50, synchronous trigger)	mean	sd	full range
KMS/DRM, no display server	18.91 ms	0.113 ms	18.58–19.13
Bare Xorg + openbox	**35.74 ms**	**0.083 ms**	**35.52–35.95**

Xorg is the **steadiest and the latest**. The lag difference is not approximate:

```
Xorg − KMS/DRM = 16.826 ms = 1.010 frames
```

One whole extra buffer in the pipeline, and dead constant. Neither Xorg nor KMS/DRM puts a single flip more than 1 ms off the frame grid, in 580 and 590 intervals respectively; Wayland misses on one trial in fifteen.

So the answer to "where does X11 fall between the other two" is: neither between nor beyond. It is a different trade — best-in-class stability bought with an extra frame of latency.

14.3 Against Table 2

Their "visual onset" is *"the difference between the occurrence of the trigger pulse and the pixels changing on the LCD screen"* — the same quantity, their monitor is also 60 Hz, and **bare Xorg is the stack they ran**, which makes E the honest row to compare.

configuration	onset Var (ms)
goxpyriment — bare Xorg (E)	**0.083**
goxpyriment — KMS/DRM console (D)	**0.113**
PsychToolBox Ubuntu · E-Prime Win10	0.18
PsychToolBox Win10 · Expyriment Win10	0.19
PsychoPy Ubuntu · Presentation Win10	0.34
PsychoPy Win10	0.35
PsychToolBox macOS	0.41
OpenSesame Ubuntu	0.50
PsychoPy macOS	0.55
OpenSesame Win10 · Expyriment Ubuntu	0.72 / 0.73
OpenSesame macOS	0.79
goxpyriment — Wayland (B, C)	**1.32 / 1.34**
goxpyriment — Wayland, normal priority (A)	**2.34**
Expyriment macOS	4.82

On the stack they measured, goxpyriment has **the best onset precision in the table**, twice as good as the best published figure. In a Wayland session the same binary is

worse than thirteen of the fourteen. Nobody's row in that column is a property of their package alone.

The lag tells the opposite story and should be reported alongside:

	visual onset lag
PsychToolBox / PsychoPy / E-Prime, Linux & Windows	2.35 – 7.10 ms
PsychoPy macOS · PsychToolBox macOS (the 10.13 buffering bug)	18.24 / 21.52 ms
Expyriment macOS — their worst	29.02 ms
goxpyriment, bare Xorg	**35.74 ms**

PsychToolBox's flip returns essentially *at* scanout. goxpyriment's returns two frames early, and on KMS/DRM one frame early. **That is a software property, not a hardware one, and it is the one number here that looks reducible.** It costs nothing scientifically — a constant offset subtracts out of any analysis — but it is a real difference in how deep the swap chain runs. See `TODO.md`.

Visual duration Var: 3.30 / 3.25 ms under Wayland, from 2.6–3.0 % of trials running exactly one frame long and nothing else; excluding those, 0.15 ms. Against their Ubuntu rows (PTB 0.15, PsychoPy 1.19, Expyriment 8.31, OpenSesame 9.16) that is mid-pack under a compositor.

14.4 Threshold sensitivity, and where the second instrument stops being usable

The BBTK's Opto1 threshold was left at its default and never calibrated against this panel. Sweeping the AD3's onset level across the panel's whole rise shows what a threshold can and cannot affect:

onset level	Wayland (C): mean / sd	KMS/DRM (D): mean / sd
5 %	21.14 / 1.344	18.31 / 0.116
10 %	21.75 / 1.344	18.91 / 0.113
50 %	24.70 / 1.344	21.86 / 0.102
90 %	27.22 / 1.345	24.38 / 0.094

A 6 ms sweep moves the **mean by 6 ms and the sd by at most 0.02 ms**. So every precision figure here is threshold-insensitive and every lag figure is not — quote a lag only with its level attached, which is why `extract-onsets.py` records it in the file.

For A–D the BBTK independently reproduced the AD3's sd to three significant figures (2.33/2.342, 1.32/1.320, 1.35/1.344, 0.17/0.113 — the last at its 250 µs quantisation floor). **For E it does not, and should not be quoted.** It reports sd 1.55 ms against the AD3's 0.083, because seven trials read 24–28 ms where the AD3 saw nothing below 35.52 ms in any of 581. An uncalibrated threshold sitting high on a 5.5 ms

ramp crosses erratically when the final luminance wobbles, and this is the condition where that finally showed. Calibrating Opto1 against this panel would be needed before the BBTK can corroborate E, and before any BBTK lag here can be set beside a published BBTK lag.

14.5 What this licenses saying

That goxpyriment's own timing code is not a limiting factor anywhere: the host-side trigger path is under 40 µs in every run, and on the stack the published study used, the trigger-to-photon interval is stable to **83 µs over five minutes** — better than any configuration in that study.

That a Wayland session costs a factor of sixteen in onset precision on this machine, and is the dominant term for anyone doing EEG or MEG with a compositor running, whatever package they use.

That goxpyriment carries one to two frames more presentation latency than PsychToolBox does. Constant, correctable, and worth fixing anyway.

One machine, one panel. The study's own authors decline to generalise across machines, and so should this.

Chapter 15

frame-syncing

To ensure your software has perfect pacing on as many systems as possible, you should follow this hierarchy of detection in your Vulkan renderer: * Priority Method Hardware/OS Support * Primary VK_EXT_present_timing Best for 2026+ systems (NVIDIA, AMD, Intel). * Secondary VK_KHR_present_waitGood for simpler “wait until displayed” logic; widely supported. * Legacy VK_GOOGLE_display_timingKeep as a fallback for older Android-based or mobile Linux setups. * System Wayland wp_ presentationIf running natively on Wayland, use the presentation-time protocol.

Chapter 16

Minimising trigger-to-stimulus jitter (EEG / MEG)

In EEG and MEG the TTL trigger is the timestamp. Everything downstream — epoching, averaging, latency measurement — is referred to it. So the question is not how quickly the stimulus appears after the trigger, but **how *consistently* it does**.

That distinction decides what you have to fix:

- **A constant offset is harmless.** If the screen always lights 22 ms after the trigger, subtract 22 ms in the analysis and nothing is lost.
- **Jitter is not recoverable.** A trigger-to-stimulus delay that varies from trial to trial smears every average by that amount, and no analysis can undo it because nothing recorded says which trial was late.

Everything below is aimed at the second quantity.

16.1 The short version

0. **Get off the compositor.** Bare Xorg with a plain non-compositing window manager measured **sixteen times** better than the same binary in a Wayland session — 0.083 ms against 1.34 — and a bare KMS/DRM console 0.113 ms. Nothing else here comes close.
1. **Never sleep for an inter-trial interval. Count frames.**
2. **Fire the trigger synchronously, on the flip thread, immediately after the flip returns** — not from a goroutine.
3. **Discard the first several trials.** They are measurably different.
4. **Run at real-time priority.** Measured worth: it halves the jitter, 2.34 ms to 1.32 ms over five minutes. Read the warning about busy-waits in Setting priority under Linux.
5. **One TTL line per event type.** A multi-bit code on a DLP-IO8 takes ~610 µs to settle and will be sampled mid-transition.
6. **Measure it on your own rig.** The numbers below are one host and two panels; yours will differ.

Done properly, at real-time priority and **without a compositor**, this gives **sd 0.083 ms over a five-minute run** on the hardware tested — an order of magnitude better than the best figure in the published cross-package comparison. In a Wayland session the same binary gives 1.34 ms, and at normal priority 2.34 ms. None of it is ever the display, which put every one of 2951 stimuli measured on an exact frame boundary. See “The jitter is in the timestamp, not in the display”.

16.2 Where the time actually goes

Measured on a Linux 7.0 laptop, 60 Hz panel, DLP-IO8 trigger and a photodiode on the screen, both recorded in one acquisition on an Analog Discovery 3 so the instrument’s clock cancels:

stage	contribution
`ShowTS` returns → TTL write issued	**25 µs** (p95 33 µs)
DLP-IO8 host write → edge on the wire	tens of µs (bounded ≤ 0.79 ms)
flip → photons on the panel	**20–40 ms**, and this is where all the variance is
panel black→white transition (10–90%)	**5.5–6.5 ms**

The software path is three orders of magnitude below the display. Nothing in your experiment code, the Go runtime, or the trigger box is what limits you.

16.3 Why sleeping is the mistake

An inter-trial interval written as `time.Sleep(300 * time.Millisecond)` is not a whole number of frames: at 60 Hz it is 18.01 of them. Each trial therefore starts at a slightly different point in the frame cycle, and the stimulus flip lands at a drifting phase. Measured over 35 trials with a sleep-based ITI:

	sleep ITI	frame-counted ITI
trigger→light spread	**14.1 ms**	**3.4 ms**
sd	2.80 ms	1.13 ms
host clock vs display clock	drifts 0.27 ms per trial	—

(Both from 35-trial runs. Over five minutes the frame-counted figure is worse — sd 2.3 ms — for the reason in “Measure over a realistic run length” below. The comparison between the two ITI styles stands; the absolute value does not.)

The failure mode is worth recognising because it does **not** look like noise. The trigger-to-light delay walks smoothly across a run — 27 ms at the start, 41 ms in the middle, back to 37 ms — as the host’s trial period slides past the display’s frame period. Two things follow:

- **A short pilot will not reveal it.** Ten trials sample a small arc of the walk and look tight.
- **Within-channel checks will not reveal it either.** Trigger-to-trigger and flash-to-flash intervals are both perfectly stable while their *relative* phase drifts. If your timing check measures only interval regularity, it will pass.

Count frames instead:

```
// Inter-trial interval as 18 blank frames -- exactly 300 ms at 60 Hz, and
// exactly a whole number of frames at any refresh rate.
for f := 0; f < 18; f++ {
    exp.Screen.ClearAndUpdate()   // Update blocks on VSYNC
}
```

This keeps the display pipeline continuously fed, so the stimulus flip stays vsync-locked. `tests/Timing-Tests` has always done this; it is why its numbers have always been good.

16.4 Fire the trigger synchronously

```
onset, err := exp.ShowTS(stim)   // presents and timestamps the flip
if err != nil { return err }
triggers.FireTriggerSync(trig, line, 5*time.Millisecond)
```

`FireTriggerSync` raises on the calling goroutine — the next statement, same thread, nothing between — and defers only the falling edge, which carries no information. Measured this way the gap between the flip timestamp and the trigger write is **25 µs** (p50 34 µs, max 37 µs at `SCHED_FIFO` 50 in a later campaign).

Do not reach for `go triggers.FireTrigger(...)` instead. `FireTrigger` blocks for the whole pulse, so a frame loop can only call it from a goroutine — and that hands the *rising* edge to the Go scheduler, which at normal priority under CPU load has been measured at **+0.73 ms with about 1 ms of spread**. Forty times the synchronous figure, for no benefit at the scale that matters here. `FireTrigger` is still the right call for a marker whose timing is not the measurement: a block boundary, an end-of-run flag.

`tests/Timing-Tests` splits the edges exactly this way in its `av` loop, for exactly this reason; its `rt` loop still uses the goroutine form, where the trigger is not the quantity being measured.

16.5 What is left, and why it is quantised

After the two fixes above, the residual is not a smear — it is **whole frames**. Measured over 34 consecutive intervals in a frame-counted run, every single one was an exact multiple of the frame period: mostly 20 frames, occasionally 21. None were in between.

So the display is genuinely vsync-locked, and the remaining variability is a trial occasionally slipping one frame later. At 60 Hz that is 16.7 ms when it happens; at 120 Hz, 8.3 ms.

This matters because a quantised error is **detectable**, and a smeared one is not. `ShowTS` returns the flip timestamp, so an experiment can log it and find the slips afterwards:

```
onset, _ := exp.ShowTS(stim)
if prev != 0 {
    frames := float64(onset-prev) / float64(frameDur)
    if math.Abs(frames-math.Round(frames)) > 0.25 || math.Round(frames) !=
     ↪  expected {
        // this trial's stimulus was a frame late: mark it, or drop it
    }
}
prev = onset
```

For EEG and MEG this is the difference between an unknown error and a known one. Log the flip timestamp on every trial and put it in the data file.

16.6 Measure over a realistic run length, not a pilot

A short pilot can tell you the timing is excellent, and it will be wrong.: Measure over the length of a real block (5~10min).

16.7 The jitter is in the timestamp, not in the display

This is the most useful thing on this page, and it took a five-minute run to see. Recording the trigger and the photodiode on one instrument gives three event trains, and asking whether each one falls on a whole number of frame periods separates them completely:

train, intervals between consecutive trials	off a whole frame by > 1 ms
photons on the panel	**0 of 597 (0.0 %)**
TTL edge on the wire	~5 % of trials, up to 6.8 ms
the host's own `ShowTS` timestamp	**82 of 638 (12.9 %)**, up to 6.7 ms

The panel is exact. Photodiode intervals sit on a whole number of frames with a median error of **5 µs** and a worst case of 169 µs, across 553 intervals of exactly 30 frames, 37 of 31 and 7 of 32. The implied frame period, 16.6557 ms, is **60.0395 Hz** against the 60.0400 Hz SDL reports for the panel — agreement to 8 ppm, from an instrument that knows nothing about the display.

What wanders is the moment the software believes the flip happened. And it wanders in one direction: across the 594 intervals that were exactly 30 frames, the host times-

tamp was never early (minimum −0.01 ms) and was late by as much as **+6.12 ms**. `Update()` returned, `ShowTS` stamped the clock, and the photons had already been on their way for several milliseconds.

The trigger inherits that error, because it is fired off the flip's return. So the 2.3 ms of trigger-to-stimulus jitter is not the display and not the trigger box:

> **The stimulus appears on an exact frame boundary every time. The TTL that is supposed to mark it is occasionally several milliseconds late.**

Two things follow, and they point in opposite directions from the usual advice:

- **Do not treat `ShowTS`'s return as a photon timestamp.** It is right to within microseconds most of the time and several milliseconds wrong about one trial in eight, with no indication of which.
- **The error is bounded and reconstructible.** Because the photons are on an exact grid, the true onsets of a whole block can be recovered by fitting that grid to the flip timestamps, which is not possible for genuinely random noise.

16.7.1 The display stack is worth sixteen times more than anything else

Three runs differing only in the display stack — same binary, same `chrt -f 50`, same synchronous trigger, same night:

	mean	**sd**	full range, ~590 trials	flips > 1 ms off the frame grid
Wayland session	21.75 ms	1.344 ms	18.83–36.74	7.1 %
KMS/DRM, no display server	18.91 ms	**0.113 ms**	18.58–19.13	**0 of 590**
Bare Xorg + openbox, exclusive fullscreen	35.74 ms	**0.083 ms**	**35.52–35.95**	**0 of 580**

Sixteen times better, and the mechanism is gone rather than reduced. Off a compositor, not one flip in ~590 lands more than 1 ms off the frame grid; in a Wayland session one trial in fifteen does. An entire five-minute Xorg run fits inside a 0.43 ms band.

This is the single largest effect on this page. Real-time priority is worth 1.8×; the display stack is worth 16×.

Xorg is the steadiest and the latest, which is not a contradiction — the two are different quantities. The gap is exactly one frame:

```
Xorg − KMS/DRM = 16.826 ms = 1.010 frames
```

One more buffer in the pipeline, dead constant. So goxpyriment emits its trigger one frame before the photons on bare hardware and two frames before them under X, and neither is the compositor's doing — what the compositor added was the *variance* around it. That is consistent with `Update()` returning when a page flip is queued at one vblank while the content becomes visible at a later one.

For EEG and MEG the distinction is the whole point: **a constant 19 or 36 ms is subtracted in analysis and costs nothing; 1.3 ms of scatter around it cannot be.** Measure your offset once per rig, record it, subtract it.

A worthwhile aside on instruments. Across the Wayland and KMS/DRM runs the BBTK reproduced the AD3's sd to three significant figures. On the Xorg run it does not — 1.55 ms against 0.083 — because seven trials read 24–28 ms where the AD3 saw nothing below 35.52 in any of 581. Its optical threshold was never calibrated against this panel, and a threshold high on a 5.5 ms ramp crosses erratically when the final luminance wobbles. Agreement between instruments is worth a great deal right up to the point where one of them is misconfigured, and that point is not announced.

16.7.2 Removing the compositor is worth twelve times more than anything else

The same protocol again, in a bare Linux console with `SDL_VIDEODRIVER=kmsdrm` and no display server at all:

five minutes each	Wayland, FIFO 50	**KMS/DRM console, FIFO 50**
trigger→light sd, AD3	1.344 ms	**0.113 ms**
trigger→light sd, BBTK	1.35 ms	**0.17 ms**
p05–p95	3.12 ms	**0.38 ms**
full range over 590 trials	18.8–36.7 ms	**18.58–19.13 ms**
largest trial-to-trial step	16.7 ms	**0.275 ms**
`ShowTS` > 1 ms off the frame grid	6.4 %	**0 of 640**

Twelve times better, and the mechanism is gone rather than reduced: not one flip in 640 lands more than 1 ms off the frame grid, where a Wayland session misses on one trial in fifteen. The whole five-minute run fits in a 0.55 ms band.

This is the single largest effect on this page. Real-time priority is worth 1.8×; the compositor is worth 12×.

It also means the earlier sections diagnosed the mechanism correctly and blamed the wrong layer for the offset. Removing the compositor removed the *variance* around the one-frame lag but not the lag itself, which only fell 2.8 ms, from 21.75 to 18.91. goxpyriment emits its trigger about one frame before the photons either way. That residual is consistent with `Update()` returning when the flip is queued at one vblank while the content appears at the next — 16.66 ms plus about 2 ms of scanout down to the patch.

For EEG and MEG the distinction is the whole point: a constant 19 ms is subtracted in analysis and costs nothing, and 1.3 ms of scatter around it cannot be.

See the mega-study comparison — on a bare console this beats every configuration in the published table; under Wayland it is near the bottom of it.

16.7.3 Real-time priority halves it — and that is measured, not assumed

A one-sided, several-millisecond lateness in returning from a vsync wait is what preemption looks like. The run above was at normal priority, so the same protocol was repeated changing nothing but `chrt -f 50`:

five minutes, same rig, same night	SCHED_OTHER	SCHED_FIFO 50
trigger→light sd, **AD3**	2.342 ms	**1.320 ms**
trigger→light sd, **BBTK v3**	2.33 ms	**1.32 ms**
p05–p95 spread (BBTK)	7.2 ms	**3.0 ms**
`ShowTS` timestamps > 1 ms off the frame grid	12.85 %	**6.43 %**
TTL edges > 1 ms off the frame grid	13.07 %	**6.28 %**
photons > 1 ms off the frame grid	**0.00 %**	**0.00 %**
trial-to-trial steps > 3 ms	30 of 597	**10 of 589**
mean delay	21.18 ms	20.96 ms

Real-time scheduling **halves the jitter** and leaves the mean where it was, which is the signature of removing a delay that only ever happened sometimes. Both instruments agree on the improved figure to three significant figures, as they did on the worse one.

The off-grid fractions for `ShowTS` and for the TTL fall together — 12.85 → 6.43 and 13.07 → 6.28 — which confirms the trigger is simply following the flip timestamp and contributes nothing of its own. And the photons stay exactly where they were: perfectly frame-locked in both conditions. Scheduling never touched the display, only the software's knowledge of it.

It is not a complete fix. 6.4 % of flips are still late, and 1.32 ms is not 1.32 µs. Whatever remains is not answered here.

`tests/Timing-Tests` requests `SCHED_FIFO 50` at startup like any other goxpyriment program; `-realtime-priority 0` opts out, which is how the SCHED_OTHER column above was produced. Check `sched_policy` in your data file's header rather than assuming either way, and see Setting priority under Linux.

16.8 Discard the first trials — they are genuinely different

The first few trials after a run starts have a longer and more variable delay than everything that follows, and they dominate the summary statistics if left in. Same recording, the only difference being how many leading trials are excluded:

	all trials	discarding 10
trigger→light sd	1.127 ms	**0.384 ms**
spread	3.44 ms	**1.15 ms**

Three times the sd, from the first handful of trials. On the BBTK recording of the same test the raw sequence starts at 32.25 ms and settles to about 25 ms within four trials, so the transient is real and not an artifact of one instrument.

`tests/Timing-Tests` already discards ten cycles (`-warmup`). An experiment should do the equivalent: present some warm-up trials before the first one that counts, or mark the early trials in the data so the analysis can drop them.

16.9 Two instruments agree

Everything above was measured with an Analog Discovery 3. The same `Timing-Tests` run was also recorded with a Black Box ToolKit v3 — a different sensor, a different front end and its own clock — with both instruments on the same trigger line:

five-minute run, ~598 trials	AD3 (1 µs resolution)	BBTK v3 (250 µs)
trigger→light **sd**	**2.340 ms**	**2.327 ms**
spread	24.74 ms	24.75 ms
p05 – p95	17.1 – 24.5 ms	19.8 – 27.0 ms
mean	21.18 ms	23.84 ms

The two instruments agree on the variability to three decimal places while differing by 2.7 ms in the mean. That is exactly the expected pattern: the offset depends on where each instrument's threshold sits on the panel's 5.5 ms ramp, and is constant; the jitter does not depend on the threshold at all. It is also the strongest evidence available that the number is real, since the two share nothing but the signal.

Over a short window the BBTK looks worse than the AD3 (sd 0.79 against 0.38 ms across thirteen trials) because a 0.43 ms mean step is only 1.7 of its 250 µs quanta. Over a realistic run that resolution difference is irrelevant — the quantity being measured is ten times its resolution.

16.10 How this compares with other packages

Bridges et al. (2020) measured this same quantity — trigger pulse to pixels changing — for PsychoPy, PsychToolBox, Presentation, E-Prime, OpenSesame and Expyriment, on a 60 Hz panel. Their best is 0.18 ms and their worst is 4.82 ms.

goxpyriment measures **0.083 ms on bare Xorg** — the stack they actually ran, and twice as good as the best figure in their table — and **1.32 ms under Wayland**, worse

than thirteen of the fourteen. Same binary, same machine, same night. That column describes a display stack at least as much as it describes a package.

The lag runs the other way and belongs in the same breath: 35.7 ms on bare Xorg against 2.35–7.10 ms for every Linux and Windows package they measured. PsychToolBox's flip returns at scanout; goxpyriment's returns two frames early. Constant, so correctable, but real. See the mega-study comparison.

16.11 The floor you cannot code around

Frame quantisation. A stimulus can only appear when the panel scans it out. The only way to reduce this is a faster panel: 16.7 ms of quantisation at 60 Hz becomes 4.2 ms at 240 Hz.

Panel transition time. Measured 10–90% on two unrelated LCDs, one external 4K and one laptop panel: **6500 µs and 5571 µs**. A multi-millisecond black-to-white transition appears to be a property of the technology, not of one bad monitor. It is also why quoting an onset to better than a millisecond is not meaningful without saying which point on the ramp you mean.

Scanout position. The top of the panel lights nearly a frame before the bottom. Put the photodiode — and the stimulus that matters — near the top, and record where it was.

16.12 The honest assessment for EEG / MEG

With a frame-counted ITI, a synchronous trigger and warm-up trials discarded, the trigger-to-stimulus delay on the hardware tested has **sd 2.3 ms across a five-minute block**, on a mean of about 21 ms that is panel-specific. Two independent instruments agree on that figure.

Where that jitter lives matters more than its size. The panel puts the stimulus on an exact frame boundary on every one of 1188 trials measured; the software is late to notice on 6 % of them at real-time priority and 13 % at normal priority. The jitter is in the timestamp, not in the photons.

Whether this is good enough depends on the paradigm. For ERP components with latencies of tens of milliseconds, averaged over many trials, 1.3 ms of onset jitter is a small smear. For anything resolving fine temporal structure — early auditory components, phase measures, single-trial latency — measure it on your own rig before relying on it.

Unlike a genuinely noisy display, this one has somewhere to go. Real-time priority is worth a factor of two and is one flag. Beyond that, the frame grid is exact enough — 6 µs — to reconstruct the true onsets after the fact, which is possible precisely because the residual is not random.

The three ways to make it worse are all in this page and all avoidable: sleeping for the ITI costs a factor of twelve in spread, firing the trigger from a goroutine costs a

factor of forty in the host term, and keeping the warm-up trials costs a factor of three in sd. None of them announce themselves in the data.

The robust answer for both cases is the same one used in MEG labs generally: **record a photodiode alongside the TTL** and use the photodiode as the onset in analysis. That converts every term on this page — offset, quantisation, panel rise — into a measured per-trial quantity rather than an assumption.

16.13 Verify it on your own hardware

```
# Does this display block on VSYNC, and at what rate?
go run ./tests/test_vsync_blocking

# Do the flip timestamps drift against the display? (no photodiode needed)
go run ./tests/test_vblank_drift

# Trigger against a photodiode, one AD3 acquisition, per-trial gap logged
go run ./tests/test_photodiode_latency -s 1 -isi-frames 18 -diode all

# The same with the established harness
go run ./tests/Timing-Tests -test av -no-sound

# Two trigger devices against each other, on one schedule and one timebase
go run ./tests/test_triggers -device parallel:pin=1 -device dlpio8:pin=1
```

`tests/test_vblank_drift` is the one to run first, because it needs no hardware at all. It compares `FlipTS` against the kernel's own DRM vblank timestamps — an independent clock on the display, playing a photodiode's role minus the photons — and reports the drift in ppm. It cannot measure the *offset* between the flip and the photons (scanout position and panel rise are outside it), but it can tell you whether that offset is constant, which is the part no host-side statistic can otherwise reach. On a Precision 5490 its refresh estimate agreed with a BBTK photodiode to 1.3 ppm.

`tests/test_photodiode_latency` logs the flip timestamp, the trigger timestamp and the gap between them for every trial, so the host-side contribution is in the data rather than assumed. Its README works through the arithmetic of converting an instrument's trigger-to-light interval into the quantity you actually want.

`tests/test_triggers` answers a different question: *which* trigger device. The figures above come from one device at a time, and comparing runs made hours apart on different clocks is exactly the comparison this document warns about everywhere else. That test pulses two or more devices on **one absolute schedule** — a locked thread each, all waiting on the same deadline — so an oscilloscope reads the difference between their edges directly and its own clock cancels. It records when the host issued each write as well, which is what separates a device that is slow from a program that was late. Its `-sequential` mode measures the other thing worth knowing: what a blocking USB write costs the device queued behind it, which is what experiment code pulsing two boxes in a row actually does.

16.14 Appendix: the DLP-IO8-G

Driver: `triggers/dlpio8.go`. Full protocol notes and raw data at https://github.com/chrplr/dlp-io8-g; the figures below were measured there with a Siglent SDS1104X-E and are repeated because they change how you should use the device. For how it compares with the other trigger back-ends on one timebase, see `tests/test_triggers/` and Timing tests.

16.14.1 Lower the FTDI latency timer before *reading* anything

The DLP-IO8 is an FTDI device, and the `ftdi_sio` driver defaults to a **16 ms latency timer**: the chip holds a partly-filled buffer that long before sending it to the host. A poll the module answers instantly still takes 16 ms to come back.

Measured, n=300 per setting, for an 8-channel read:

`latency_timer`	round trip	poll rate
16 (default)	**15.98 ms**	**63 Hz**
4	3.99 ms	251 Hz
1	1.01 ms	995 Hz

The relationship is exactly `round trip = latency_timer`, so the module's own processing is negligible and the whole cost is driver batching. A polling loop gets the worst case rather than the average: waiting for each reply synchronises the loop to the timer and pays the full 16 ms every iteration.

```
echo 1 | sudo tee /sys/bus/usb-serial/devices/ttyUSB0/latency_timer
```

That reverts on replug. To make it stick:

```
# /etc/udev/rules.d/99-ftdi-latency.rules
SUBSYSTEM=="usb-serial", DRIVERS=="ftdi_sio", ATTR{latency_timer}="1"
```

The rule applies to every FTDI serial device on the machine, not just this one.

It does nothing for sending triggers. Output latency is governed by USB frame scheduling. Do not expect this setting to make a trigger arrive sooner.

16.14.2 A multi-bit code is not atomic

There is no multi-channel command: every command is one ASCII byte affecting one line. `Send(mask)` therefore emits eight bytes which the module acts on as they arrive, and **the port takes ~610 µs to settle**, showing partly-updated values throughout. Measured n=99: 86.2 µs per byte, 609.5 µs from the first line to the eighth.

Against a system sampling at 1 kHz that is about 61 % of a sample period, so a code change is sampled mid-transition roughly three times in five and recorded as a value that was never sent.

Use one line per event type, pulsed. A single line is one command byte, so there is no skew at all, and eight lines still distinguish eight event types. Reserve `Send` for a multi-bit code where the acquisition reads the code milliseconds after the onset edge rather than latching it at the edge, or where a strobe line is raised last once the code has settled.

16.14.3 Pulse width is only as good as your scheduling

The device has no pulse timer, so a pulse is two host writes and the width inherits host scheduling in full. Measured n=50 per width:

host state	median error	spread
idle	−10 to −20 µs	≤ 120 µs
under CPU load	up to +1.85 ms	up to 4.75 ms

The host's own busy-wait interval degrades by the same amount, tracking the wire to within 80 µs — so the cause is the scheduler descheduling the process, not the USB path or the device. See Setting priority under Linux; that is the fix, and it is a different mechanism from the latency timer above.

16.14.4 Parallel port and GPIO alternatives

Both avoid the USB path entirely: a write is one `ioctl`, not a USB serial transaction subject to frame scheduling. On hardware that offers either, this is the cheapest available improvement to onset-vs-TTL precision.

In the timing tests, select them with `-trigger-device parallel` or `-trigger-device gpio`. In your own experiment code:

```
// Parallel port (LPT), 5 V. Needs `sudo modprobe ppdev` and the `lp` group.
p := triggers.NewParallelPort("/dev/parport0")
if err := p.Open(); err != nil { log.Fatal(err) }
defer p.Close()
p.Send(0x01)                    // all 8 data lines at once, D0 = DB25 pin 2

// GPIO character device (Raspberry Pi, Rock Pi, …), 3.3 V. Needs kernel >=
 ↪  5.10
// and the `gpio` group. Check which chip owns the header with `gpiodetect`.
g, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOChip("/dev/gpiochip0"),
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
)
if err != nil { log.Fatal(err) }
defer g.Close()
```

```
g.Pulse(0, 5*time.Millisecond)    // line 0 = the first pin in the array =
 ↪  BCM 17
```

`Send(mask)` sets all 8 lines simultaneously on both — unlike the DLP-IO8, where a multi-bit code is written one byte per line and takes ~610 µs to settle.

Note the voltage difference: parallel is 5 V, GPIO is 3.3 V. Confirm your acquisition system latches at 3.3 V before relying on the GPIO path.

Here are the step-by-step instructions to set up the `goxpyriment` group with those high-priority privileges.

Steps 1-5 cover real-time priority. Step 6 covers the `input` and `video` groups, which a machine needs before it can read keyboards or drive the display from a bare console — the configuration the timing measurements recommend, and the one where their absence first shows — and `lp`, for a machine that triggers through a parallel port. Running from a virtual console covers that configuration itself: getting there, the display mode you actually get, and what a second lit monitor does to the timing.

If you are here because of EEG or MEG trigger timing, read Minimising trigger-to-stimulus jitter as well: real-time priority is necessary but nowhere near sufficient, and the dominant term is not the one most people expect.

On another platform? See Setting priority under Windows or Setting priority under macOS. Both use a different mechanism, and neither has been measured — this page is the only one whose figures come from real runs, and most of its *reasoning* transfers even where the commands do not.

Chapter 17

Step 1: Create the Group

First, you need to create the group in the system database.

```
sudo groupadd goxpyriment
```

17.0.1 Step 2: Create the Limits Configuration

Linux stores these specific "privilege" rules in `/etc/security/limits.d/`. You should create a new file specifically for your group so it doesn't mess with other system settings.

1. **Open a new config file:**

   ```
   sudo nano /etc/security/limits.d/99-goxpyriment.conf
   ```

2. **Paste the following lines into the file:**

   ```
   @goxpyriment - nice -20
   @goxpyriment - rtprio 50
   @goxpyriment - memlock unlimited
   ```

3. **Save and Exit:** Press `Ctrl + O`, then `Enter` to save, and `Ctrl + X` to exit.

4. **Check the text actually landed:**

   ```
   grep rtprio /etc/security/limits.d/*.conf
   ```

   Worth the two seconds. An editor opened *without* `sudo` — or a graphical editor that cannot write to `/etc` — may fail to save without making it obvious. The only later symptom is `ulimit -r` still returning `0` after a re-login, which is easy to misread as "the limits system doesn't work" rather than "the file was never written".

   Why a new file rather than `/etc/security/limits.conf`? That file belongs to the `libpam-modules` package and can be replaced on upgrade, silently taking your setting with it. Anything you add under `/etc/security/limits.d/` is yours and survives.

Why not simply join an existing group such as `audio`? Tempting, because on many systems `audio` already carries an rtprio grant. But that grant is installed by the jackd package and can be revoked by `dpkg-reconfigure -p high jackd2`, and it hands out far more than you need (`rtprio 95`, `memlock unlimited`). The real objection is subtler: an experiment that has quietly lost real-time priority behaves exactly like one that still has it, right up until you look at the timing data. A group of your own cannot be switched off by another package's maintainer script.

17.0.2 Step 3: Add Yourself (and others) to the Group

Simply creating the group isn't enough; you have to tell Linux which users belong to it. Replace `$USER` with a specific username if you are adding someone else.

```
sudo usermod -aG goxpyriment $USER
```

17.0.3 Step 4: Apply the Changes

Important: Linux only checks group memberships and limits when a user **logs in**.

- You **must** log out of your Linux session and log back in.
- Alternatively, you can run `su - $USER` in your terminal to start a sub-shell with the new permissions for testing.

17.0.4 Step 5: Actually Use It

The grant only makes real-time priority *available* — nothing runs at it until something asks.

goxpyriment programs ask for themselves. `Experiment.Initialize()` requests priority 50 at startup, so once Steps 1-4 are done there is nothing further to do — including when the program is launched by clicking its icon, where no command-line prefix is possible. If the grant is not in place it says so and continues at normal priority rather than refusing to run:

```
real-time scheduling not obtained, continuing at normal priority: real-time
scheduling is not permitted for this user (RLIMIT_RTPRIO is 0). ...
```

Two flags control it:

```
./my-experiment -no-realtime                # do not ask at all
./my-experiment -realtime-priority 20      # ask for something other than 50
```

A program with no flags — one built with `NewExperiment` + `Initialize` rather than `NewExperimentFromFlags` — sets the field directly instead:

```
exp := control.NewExperiment(...)
exp.RealTimePriority = 0     // decline; the -no-realtime equivalent
```

Every run records what it ended up with, as `sys sched_policy` in its `-info.txt`. Read it from there rather than assuming: a run at `SCHED_OTHER` and one at `SCHED_FIFO` are

not comparable, so a study should not mix them.

For anything else, or to override, use `chrt`:

```
chrt -f 50 ./some-other-program
```

`-f` selects SCHED_FIFO and `50` is the priority. **It must not exceed the `rtprio` value granted in Step 2** — asking for more fails with "Operation not permitted", which is the same error you get when the grant is missing entirely, so it is easy to misdiagnose. Keep the two numbers equal unless you have a reason not to.

The policy is inherited by child processes, so one `chrt` covers the whole run. It does not persist to your shell or to the next command; that is deliberate. You *can* make a shell real-time with `chrt -f -p 50 $$`, but do not: every command you then type runs above most system threads, and a mistyped one is very hard to interrupt.

17.0.5 Step 6: The other groups the same machine usually needs

Real-time priority is not the only permission a stimulus machine wants, and the other two bite in exactly the configuration the timing measurements recommend: running without a display server, from a virtual console.

```
sudo usermod -aG input $USER      # /dev/input/event*  — keyboards, mice,
↪  gamepads
sudo usermod -aG video $USER      # /dev/dri/card*     — KMS/DRM output and
↪  vblank
```

Both need a full logout and login, the same as Step 4.

Why it only shows up on the console. In a desktop session `systemd-logind` attaches an ACL granting the active user access to that seat's devices — the `+` at the end of the permission bits is the ACL:

```
crw-rw----+ 1 root video  226,   0 /dev/dri/card0
crw-rw---- 1 root input   13,  64 /dev/input/event0
```

So everything works in a desktop session and the same program run from a bare VT finds the nodes closed to it. Group membership is not seat-dependent and covers both.

`input` — without it SDL prints a line per device it cannot open while it enumerates for joysticks:

```
Error: could not open /dev/input/event3
```

Harmless in itself, and a visual-only run is unaffected. It matters for **reaction times**: with evdev unavailable SDL falls back to reading the console tty, which is not the same input path as the one a desktop run uses. We have not measured the difference — which is the reason to remove the ambiguity rather than to reason about it. Never compare RT distributions between a console run and a desktop run without checking both used the same path.

> **!** A user in `input` can read every keystroke on the machine, in every session, including other users'. On a dedicated stimulus box that is a fair trade; on

a shared machine it is not, and the alternative is to make sure the VT login creates a proper logind session so the ACLs are applied (`loginctl session-status` should show it active on `seat0`).

`video` — needed to open `/dev/dri/card*`, which is both how SDL drives the display under `kmsdrm` and how the vblank backend reads hardware timestamps (`GOXPY_VBLANK=on`, see `vblank/drm_linux.go`). If GPU rendering then fails while output works, add `render` as well: the two are separate nodes with separate groups.

```
crw-rw----+ 1 root video  226,   0 /dev/dri/card0        # video
crw-rw----+ 1 root render 226, 128 /dev/dri/renderD128   # render
```

`audio` — usually **not** needed, despite looking like the same case. The nodes are protected identically:

```
crw-rw----+ 1 root audio  116,   8 /dev/snd/pcmC0D0p
```

but the resemblance stops there. SDL does not normally open them: it talks to PipeWire or PulseAudio over a socket in your own runtime directory, and the sound server holds the device. So audio keeps working from a console where the display and the keyboard do not, and joining `audio` changes nothing.

It matters only if you bypass the server and open ALSA directly (`SDL_AUDIODRIVER=alsa` with no server running) — which our own measurements argue against on other grounds: on a Raspberry Pi 4 with the server stopped and the device opened directly, **every one of 463 tones was torn**. If you are in that configuration, the missing group is not your biggest problem.

Do not join `audio` to obtain real-time priority either — see the note in Step 2 on why that grant is the wrong one to rely on. PipeWire gets its own real-time threads through RealtimeKit, not through the group.

`lp` — only on a machine that triggers through a **parallel port**, and it is two separate things wearing one name. Get both, or the port is unusable in a way that is hard to read as a permissions problem:

```
sudo usermod -aG lp $USER   # the GROUP lp — rw access to /dev/parport0
sudo rmmod lp               # the MODULE lp — the parallel printer driver
```

The group is the ordinary case: without it `/dev/parport0` cannot be opened and you get a plain permission error.

The module is the one that costs an afternoon. `lp` and `ppdev` can both be registered on one port, and `ParallelPort.Open`'s `PPCLAIM` then goes through the kernel's `parport_claim_or_block`. If `lp` is *holding* the port, that ioctl blocks in **uninterruptible sleep** — the process survives Ctrl-C and `kill -9` alike, and on the machine where this was diagnosed (2026-08-21, a PCIe LPT card) the desktop had to be powered off at the switch. It is intermittent, because `lp` holds the port only some of the time, so a run that works proves nothing.

`dmesg` says whether you are exposed, at boot:

```
[    2.503789] lp: driver loaded but no devices found      ← lp loads before
 ↪  the card probes
```

```
[    2.824137] parport0: PC-style at 0x3100, irq 16 [PCSPP,TRISTATE]
[    2.929160] lp0: using parport0 (interrupt-driven).     ← and then
 ↪  attaches to it
```

That last line is the one to grep for. To keep it away across reboots:

```
echo 'blacklist lp' | sudo tee /etc/modprobe.d/blacklist-lp.conf
```

Nothing is lost: `lp` is the parallel *printer* driver, and unloading it also leaves the port's IRQ unarmed, which ppdev writes never use. See `tests/test_parallel_port` for the diagnosis in full.

Verify the same way as the rest of this page — from the data, not from memory. A run that obtained the vblank clock says so in its `-info.txt`:

```
# sys vblank_backend: Linux DRM vblank (card /dev/dri/card2, crtc 0, ...)
```

and a run that could not falls back silently to a software-derived onset, which is a difference of several parts per million in a long block.

17.0.6 How to Verify it Worked

Once you've logged back in, you can verify that your Go program (or any process) will have these rights.

1. Check Group Membership: Run the command `groups`. You should see `goxpyriment` in the list.

2. Check the Memory Lock Limit: Run this command to see your current "Max locked memory" limit:

```
ulimit -l
```

If it says **`unlimited`**, you are good to go!

3. Check Real-Time Priority: Run:

```
ulimit -r
```

It should return **`50`**. If it still returns `0`, the grant is not in effect — either the file was never written, or you have not fully logged out and back in.

4. Check from inside your experiment. Any goxpyriment program's system report includes a `Sched:` line, so a recorded run carries its own evidence:

```
Sched:      policy: SCHED_FIFO  priority: 50  REAL-TIME
Sched:     policy: SCHED_OTHER  nice: 0  (real-time available up to 50, not
 ↪  used)
Sched:      policy: SCHED_OTHER  nice: 0  (real-time NOT available to this
 ↪  user)
```

Those three lines are three different situations with three different fixes, and they are worth being able to tell apart after the fact.

The **last** is this setup not being in place — go back to Step 2. The **middle** is the setup working but the program not having asked: for a goxpyriment program that means `-no-realtime` was passed or `RealTimePriority` was set to 0, since otherwise it asks on its own; for anything else it means no `chrt` prefix. Without the line in the data you would be left comparing timing distributions and guessing which had happened.

17.1 Running from a virtual console (VT)

Running with no display server at all — GDM/SDDM stopped, a bare virtual console, SDL on the `kmsdrm` driver — is the configuration this page's groups exist for, and it is the single largest improvement available on Linux. On a Radeon Pro W5700 with everything else held identical it took onset latency from ~55 ms to ~22 ms and the scatter from 0.296 ms to 0.057 ms (measured 2026-08-17 with a photodiode; see The display stack is worth two frames of latency and five times the jitter).

It also changes two things that a desktop session hides, and both have already cost a real capture.

17.1.1 Getting there

```
sudo systemctl stop gdm3          # or sddm, lightdm — whatever your machine
 ↪  runs
# Ctrl-Alt-F3, log in, then:
cd ~/my-experiment && go run .
```

You need the `input` and `video` groups from Step 6 — in a desktop session `systemd-logind` grants those devices by ACL, and the ACL does not follow you to a bare VT. If SDL still fails to go fullscreen, see SDL3 fullscreen in a Linux virtual console; the usual fix is `SDL_VIDEODRIVER=kmsdrm`.

17.1.2 The console's mode is the mode you get

goxpyriment **never changes the display mode.** On KMS/DRM it opens a fullscreen-desktop window and the fullscreen mode it passes is always the display's *current* mode, so resolution and refresh rate come from whatever the kernel already set on that console — not from the monitor's native mode, and not from what the desktop session was using before you stopped it.

That is easy to miss, because the number that changes is not the one you would watch. A Dell U2720Q (native 3840x2160) driven from a VT at **2560x1440** ran at **59.951 Hz** instead of the 59.997 Hz of its native mode — 750 ppm apart, and the monitor was upscaling every frame to its own panel for the whole run. Nothing in the experiment complained; the pacing schedule, the vblank clock and the `-hz` figure passed to the tone all simply worked from the mode that was actually set.

Read back what you got, from the run's own `-info.txt` rather than from memory:

```
# sys physical_resolution: 2560x1440 px
# d name: DP-1
# d refresh_rate_hz: 59.9500
# sys vblank_backend: Linux DRM vblank (card /dev/dri/card2, crtc 1 driving
 ↪  DP-1 2560x1440@59.9514 Hz, ...)
```

Before a run, the same questions from the console:

```
# Which heads are connected, and what modes each offers (first = preferred)
for c in /sys/class/drm/card*-*; do
  [ "$(cat $c/status)" = connected ] && { echo "== $c"; head -3 $c/modes; }
done

# What is actually set right now, with exact pixel clocks
sudo apt install libdrm-tests   # provides modetest; drm_info also works
modetest -c | grep -A2 'connector.*connected'
```

17.1.3 Pinning the mode

The console's mode is set by the kernel at boot, so the place to change it is the kernel command line — one entry per connector, named exactly as it appears in `/sys/class/drm/cardN-<NAME>`:

```
video=DP-1:3840x2160@60
```

Add it to `GRUB_CMDLINE_LINUX_DEFAULT` in `/etc/default/grub`, run `sudo update-grub`, reboot, and check `/proc/cmdline` and the `-info.txt` afterwards. Several connectors take several `video=` entries.

A monitor on USB-C is still `DP-N`. The DRM connector type names the *protocol*, not the plug: USB-C carries DisplayPort over Alt Mode, so a monitor on a USB-C–USB-C cable appears as `DP-1`, `DP-2`, … exactly like one on a full-size DisplayPort socket. There is no `USB-C-1`. On a Precision 5490 all four external outputs are USB-C/Thunderbolt and all four are exposed as `card2-DP-1` … `card2-DP-4`, one per port; each carries its own ACPI firmware node, so a given socket keeps its number across reboots. Plugging the same monitor into a *different* socket will change it — which is the reason to read the name off `/sys/class/drm` rather than remember it:

```
ls -d /sys/class/drm/card*-* | while read c; do
  echo "$(basename $c)  $(cat $c/status)"
done
```

One exception: a monitor reached through an MST hub or a docking station is a branch device, and its connector is created on hotplug rather than fixed in firmware. It has a `path` attribute (`cat /sys/class/drm/card*-DP-*/path`) where a directly-attached one has none, and a boot-time `video=` entry cannot be relied on to find it. For a timing rig, plug the stimulus monitor straight into the machine.

`modetest -s` can also set a mode, but it holds DRM master for as long as it runs and drops the mode when it exits, so it is a diagnostic rather than a way to prepare a run — SDL cannot open the device while it is held.

Whether the native mode is worth insisting on is a measurement, not a rule. What is certain is that a non-native mode makes the monitor scale, and that its refresh rate is a different number from the one on the box. The TTL→photon lag in the 1440p capture above ran 38–55 ms end to end; how much of that the scaler accounted for was not measured, because the native mode was never captured for comparison. If input lag matters to your paradigm, capture both.

17.1.4 One display, or name the one you mean

A VT lights every connected head, and that is where the second trap is. The vblank ioctl names a CRTC by index, not by monitor, and a laptop's internal panel and an external monitor run **different clocks** — on a Precision 5490 they were 1449 ppm apart. Reading the wrong one produced onsets that looked perfectly regular, reported themselves as `hardware-verified`, and walked a whole frame away from the photons and back 44 times in eight minutes.

goxpyriment now resolves the CRTC from the display it is presenting to and says which head it picked in `sys vblank_backend`, and refuses to use a vblank clock it cannot match rather than timing the wrong monitor. Two things are still worth doing:

- Pass `-d N` deliberately and confirm `d name:` in the `-info.txt` is the monitor the participant is looking at.
- Read the end of `sys vblank_resolution:` on any run with `GOXPY_VBLANK=on`. It compares the vblanks actually read against the display drawn on:

  ```
  sys vblank_resolution: frames=30000 … measured=59.9514 Hz
   ↪  nominal=59.9506 Hz (frame period -13 ppm, matches the display)
  ```

  If it says `WRONG DISPLAY`, the onsets in that file are on another monitor's grid.

The simplest way to have neither problem is to blank or unplug the head you are not using, so there is only one mode and one clock on the machine.

17.1.5 ! A Friendly "Warning"

Once this grant is in place, goxpyriment programs run at real-time priority **by default** — you no longer have to remember a `chrt` prefix, and equally you no longer get a reminder that you are asking for it. A busy loop (`for { }` with no sleep, or a spin-wait) is then running above the window manager and the input handling, and the OS will not interrupt it to let you click "Stop".

Two things bound the damage, and it is worth knowing which one you are relying on:

- **In-program elevation raises only one thread** — the experiment's own. The Go runtime's other threads, including the garbage collector, stay at normal priority, so a spinning goroutine occupies one core rather than all of them.
- **`chrt -f 50 prog` raises the whole process**, because the policy is set before `exec` and every thread inherits it. That is the more dangerous of the two, and the one this warning is mostly about.

Either way: keep a terminal with `top` or `htop` open while developing, and use `-no-realtime` when stepping through code in a debugger — a breakpoint hit on a real-time thread can leave the desktop unresponsive until the process is killed.

If a program is unkillable and real-time priority is *not* involved, suspect a different mechanism with the same symptom: a thread blocked in the kernel, in uninterruptible sleep, which no signal can reach. The parallel port has a known way of doing this — see `lp` in Step 6.

The kernel's real-time throttle (`/proc/sys/kernel/sched_rt_runtime_us`, 950 ms per second by default) is a backstop, not a licence: it stops a runaway task locking the machine completely, but a machine at 95 % real-time occupancy is not usable.

17.1.6 ! The throttle also wrecks the timing you asked for

That backstop is not only a protection against you — it is a hazard to the very thing real-time priority was requested for. Once a `SCHED_FIFO` thread reaches 100 % duty, the kernel stops it for the remaining 50 ms of the second, and that suspension lands wherever it lands: in the middle of a pulse, a frame, or a response window.

Measured on a 22-core Linux 7.0 host, with a pinned `SCHED_FIFO 50` thread spinning continuously:

host state	stalls	each	when they landed
idle	**0 in 20 s**	—	—
under `stress-ng --cpu 20`	**24 in 25 s**	51.0 ms	0.999, 2.000, 3.001, 4.002 s …

One per second, to the millisecond — the throttle period exactly. **Load is a necessary condition**, which is the worst way for a fault to behave: on an idle runqueue the kernel borrows unused real-time bandwidth from the other CPUs and the limit is never reached. So it does not happen on the quiet machine you develop on, and does happen on the loaded one you run participants on. A pulse train that spun through its inter-trial gaps at `chrt -f 50` lost up to **49.63 ms** on 23 of 1000 trials this way — about ten times the 4.75 ms worst-case spread that real-time priority was bought to remove in the first place.

The rule that avoids it: sleep, and spin only the last millisecond or two. A wait that sleeps most of its duration never approaches the limit, and the short spin at the end recovers the precision `time.Sleep` cannot give on its own. goxpyriment's frame pacing works this way (`apparatus.paceToFrame`), which takes a 60 Hz present loop from ~100 % duty to ~10 % with no loss of landing accuracy. If you write your own wait, write it the same way — a bare `for time.Now().Before(deadline) {}` over a whole trial is the shape that gets throttled.

If you genuinely need a thread at 100 % duty — a spin-wait on a panel fast enough that there is nothing left worth sleeping — the limit can be lifted:

```
sudo sysctl kernel.sched_rt_runtime_us=-1        # until reboot
```

That removes the backstop along with the throttle, so a runaway real-time loop will then hold a CPU with nothing left to take it back. Reasonable on a dedicated stimulus machine; not on a laptop you also read mail on.

17.1.7 ! The speed the CPU runs at is not fixed either

Real-time priority decides **when** your thread runs. It says nothing about how fast the core runs once it does, nor how long the core takes to wake up, and both default to saving power rather than responding quickly:

- **The frequency governor.** `powersave` and `schedutil` raise the clock only after they observe load, so the first work after an idle gap runs at a lower clock than the work after it.
- **Idle states.** A deeply idle core takes tens to hundreds of microseconds to come back. An experiment that sleeps between frames is idle by design — which is precisely the pattern that pays this cost, and the pattern the previous section tells you to adopt.

To pin the clock for a session:

```
sudo cpupower frequency-set -g performance     # package: linux-tools-common
 ↪  / cpupower
cpupower frequency-info | grep -i "current policy"
```

It does not survive a reboot. Make it persistent through your distribution's usual mechanism (a systemd unit, or `GOVERNOR=performance` in `/etc/default/cpufrequtils` on Debian and Ubuntu) rather than by remembering to type it, since the failure mode is silent.

Measured, on a Radeon Pro W5700 workstation driving a 4K panel over DisplayPort, 1010 cycles per run, onsets anchored on kernel DRM vblanks and verified against a photodiode. Two runs differing only in the governor:

governor	flip -> photons drift	shape
default (`schedutil`)	+11.8 ppm	flat at +0.3 ppm for 316 s, then +10.6 ppm
`performance`	**+0.41 ppm**	no change of regime; 0.059 ms scatter

The default-governor run is the instructive one. Its cadence did not degrade gradually: it held to 0.3 ppm for five minutes and then switched, cleanly, to 10.6 ppm and stayed there — with the *panel* holding its own period to 0.27 ppm across the same instant, so the display was not what moved. Nothing in the recorded configuration changed. A run like that reports excellent frame statistics throughout and would pass any host-side check.

That is the failure worth guarding against: not noise, which shows up in a standard deviation, but two halves of a session that are internally tidy and not comparable to each other. Pinning the governor removed it. The recorded `host cpu_mhz` went from 3600 of a 4800 maximum to 4500.

One pair of runs, one machine — enough to act on, not enough to call it the only mechanism.

Check it from the data rather than from memory. Every run records the clock it saw at start-up in its `-info.txt`:

```
# host cpu_mhz: 3600 (max 4800)
```

A run that starts well below its maximum was not on a pinned clock. As with `sys sched_policy`, the value is in the file so that two runs can be compared afterwards without anyone having to recall what the machine was doing.

Idle states are the untested half. Holding `/dev/cpu_dma_latency` open at 0 to keep cores out of deep idle is the usual next step, and we have not measured it; if you do, measure the pair before and after and add the numbers here. That is what the rest of this page is made of.

> **Note on the grant itself:** goxpyriment only uses `rtprio`. The `nice -20` and `memlock unlimited` lines in Step 2 are there because they are commonly wanted alongside it and cost nothing, not because anything here requires them. Granting `rtprio` alone is enough if you prefer the smaller privilege.

Chapter 18

Setting priority under Windows

If you are here because of EEG or MEG trigger timing, read Minimising trigger-to-stimulus jitter as well: priority is necessary but nowhere near sufficient, and the dominant term is not the one most people expect.

18.0.1 ! None of this has been measured on Windows

The Linux procedure in Setting priority under Linux was verified against `chrt` and `nice` on real hardware, and every figure in it comes from a run. **This page has not been.** goxpyriment's own Windows scheduling code (`sysinfo/scheduling_windows.go`) is likewise written against the documented Win32 API and never exercised — it says so at runtime, in the `Sched:` line of every report.

What follows is the mechanism and the procedure, not a result. Measure it before quoting a number.

18.1 The short version

From **`cmd.exe`** (not PowerShell — see below):

```
start /high my_experiment.exe
```

That is the recommendation. `/realtime` also exists and is discussed below, but it is **not** the default suggestion, for reasons that are worth reading before you use it.

18.2 Why a launcher prefix at all

On Linux, a goxpyriment program raises its own priority at startup — it calls `sysinfo.RaiseToRealTime` and needs no `chrt` prefix, which is what makes the Linux setup a one-time system configuration rather than a thing to remember.

On Windows it does not. `sysinfo/realtime_other.go` deliberately implements `raiseToRealTime` as an error on every non-Linux platform, and `Experiment.Initialize` does not even make the request there — it would fail identically on every machine, so a message about it carried no information. (Earlier versions printed "real-time scheduling not obtained, continuing at normal priority" on every Windows run; a console window opening beside the fullscreen experiment just to show that line was the only visible effect.)

The elevation is Linux-only because the alternative — a portable "raise priority" call that quietly does something different on each OS — would report success while giving you a different guarantee on every machine. Windows has no `SCHED_FIFO` and no POSIX priority to raise; it has *priority classes*, which are a different mechanism with different consequences.

So on Windows the priority decision is made **outside** the program, by however you launch it. `-no-realtime` and `-realtime-priority` have no effect there. The run's `-info.txt` still records the scheduling class the process actually got, so a launcher prefix that did not take effect is visible afterwards.

18.3 Step 1: launch through `start`

`start` is a **`cmd.exe` built-in command**, and its priority switches are the simplest way to set a process's priority class at launch:

switch	priority class
`/low`	`IDLE_PRIORITY_CLASS`
`/belownormal`	`BELOW_NORMAL_PRIORITY_CLASS`
`/normal`	`NORMAL_PRIORITY_CLASS` (the default)
`/abovenormal`	`ABOVE_NORMAL_PRIORITY_CLASS`
`/high`	`HIGH_PRIORITY_CLASS`
`/realtime`	`REALTIME_PRIORITY_CLASS`

A shortcut or a `.bat` file next to the experiment is the practical way to make this reproducible, so that a run started by someone else gets the same treatment as one started by you:

```
@echo off
rem run_experiment.bat — always launch at HIGH, never at NORMAL by accident
start /high /wait my_experiment.exe %*
```

`/wait` keeps the console window attached so you can see the program's own log output, including the `Sched:` line.

18.3.1 ! In PowerShell, `start` is a different command

This is the trap most likely to catch you, because it fails *quietly*:

```
start /high my_experiment.exe        # NOT what you want in PowerShell
```

In PowerShell, `start` is an alias for `Start-Process`, which has no `/high` switch and no priority parameter at all. Depending on the version you will get an error about an unexpected argument, or `/high` will be passed through as an *argument to your program* — which goxpyriment's flag parser will reject, but a different program might silently ignore. Either way the priority class is `NORMAL`.

Two ways out. Call `cmd` explicitly:

```
cmd /c start /high my_experiment.exe
```

or start the process and set the class afterwards:

```
$p = Start-Process .\my_experiment.exe -PassThru
$p.PriorityClass = 'High'
```

The second raises the priority a moment *after* the process starts, so the very beginning of the run is at `NORMAL`. That is harmless for an experiment with an instructions screen and a keypress before the first trial, and not harmless for one that starts measuring immediately.

18.4 Step 2: decide about `/realtime` — the honest answer is "probably not"

`REALTIME_PRIORITY_CLASS` is not the Windows equivalent of Linux's `SCHED_FIFO 50`. It is considerably more aggressive, and Microsoft's own documentation warns against it: threads in this class run above most operating-system threads, including ones handling disk flushing and mouse and keyboard input. A busy loop there can make the machine unresponsive in a way that `/high` cannot.

goxpyriment's frame pacing sleeps most of each wait and spins only the last couple of milliseconds (`apparatus.paceToFrame`), specifically so that it does not sit at 100 % duty — so the worst case is less likely than it would be with a naive spin-wait. But "less likely" is the whole claim, and it is not one this project has tested on Windows.

Two further reasons to prefer `/high` as the default:

- **`/realtime` needs administrator privileges** (`SeIncreaseBasePriorityPrivilege`). Without them the process is created at `HIGH_PRIORITY_CLASS` instead — and **no error is reported**. You get the priority you would have got from `/high`, while believing you got something stronger. This is the same failure shape the Linux page warns about, where a missing `rtprio` grant produces a run that looks identical to one that has it.
- **`/high` is very likely where the return curve flattens.** The gap between `NORMAL` and `HIGH` is the one that stops ordinary background work preempting the experiment. The gap between `HIGH` and `REALTIME` mostly buys precedence over the OS itself, which is not what a dropped frame usually comes from.

If you do use `/realtime`, launch the shell as Administrator, and **check the `Sched:` line afterwards** rather than assuming it took.

18.5 Step 3: the thing that probably matters more — timer resolution

For millisecond stimulus timing on Windows, the system timer resolution is plausibly a larger term than the priority class, and it is the reason `sysinfo/scheduling_windows.go` collects it alongside the priority:

> The timer resolution matters more than the priority class for millisecond stimulus timing [...] the default tick has historically been ~15.6 ms, and a process that has not raised it cannot place an event to the millisecond however high its priority.

A process that has not requested a finer tick can have its sleeps rounded to the nearest ~15.6 ms, which is roughly one 60 Hz frame — no priority class rescues that. The Go runtime does request high-resolution timers on modern Windows, so in practice you are likely to see a much smaller number, but that is a runtime implementation detail rather than a guarantee, which is why goxpyriment **measures** it (`NtQueryTimerResolution`) instead of assuming it.

Read it off the report:

```
Sched:      policy: HIGH_PRIORITY_CLASS  nice: 0  [UNTESTED on Windows; no
 ↪  POSIX
           policy, priority class shown instead; timer resolution 0.500 ms]
```

If that figure reads ~15.6 ms rather than ~1 ms or below, **stop and fix that first** — no amount of priority will compensate, and any timing you record until you do is quantised to a frame.

18.6 Step 4: verify

Every goxpyriment program's system report carries the state it actually ran with, so a recorded run is self-labelling. Print it without running an experiment:

```
Timing-Tests -sysinfo
```

and look at the `Sched:` line:

what you see	what it means
`policy: NORMAL_PRIORITY_CLASS`	the prefix did not take — most likely the PowerShell trap above
`policy: HIGH_PRIORITY_CLASS`	`/high` worked

what you see	what it means
`policy: REALTIME_PRIORITY_CLASS … RE AL-TIME`	`/realtime` worked *and* you had the privilege
`policy: unknown`	`GetPriorityClass` failed; report it as a bug

Note the asymmetry that makes checking worth the two seconds: asking for `/realtime` **without** administrator rights shows `HIGH_PRIORITY_CLASS` here, which is exactly what a successful `/high` shows. The line tells you what you got, not what you asked for.

The `[UNTESTED on Windows …]` note is printed deliberately and will stay until someone runs the verification below on real hardware.

18.7 How to find out whether it helped, on your machine

This is the part that turns the page from advice into a result, and it takes about ten minutes. Nothing above substitutes for it, because the answer depends on your GPU driver, your compositor settings and what else the machine is doing.

Run the frame-interval test twice, changing only the launcher:

```
Timing-Tests -test display -duration-s 300
start /high Timing-Tests -test display -duration-s 300
```

Then compare the two `-info.txt`/`.csv` pairs. What to read, in order:

1. **Dropped frames and the maximum interval** — the tail, not the average. A priority change moves the worst frames; it barely moves the median, so a comparison of means will show almost nothing even when the change helped.
2. **The SD of the frame intervals.**
3. **The `Sched:` line in each `-info.txt`**, which records which run was which so the two files cannot be mixed up afterwards.

Do it on a machine in the state you will actually run participants on. An idle machine hides scheduling problems — that is the single most common way this kind of comparison produces a false negative.

If you run it, the numbers would be genuinely valuable to this project; there are currently none for Windows.

18.8 Other Windows-specific things worth knowing

Listed as pointers rather than recommendations — none has been verified here:

- **Fullscreen optimizations.** Windows 10/11 may run an apparently exclusive fullscreen window through the desktop compositor instead. That is the same class of problem as the compositor issues documented for Linux in TimingTests.md, and it can be disabled per-executable in the file's Properties → Compatibility tab.
- **Variable refresh rate / G-Sync / FreeSync.** Turn it off on a stimulus machine. A panel that varies its own frame period defeats the assumption every frame-counted duration rests on.
- **Power plan.** "High performance" rather than "Balanced"; core parking and aggressive frequency scaling both add wake-up latency.
- **Windows Game Mode and background apps.** Both change scheduling in ways that are documented only loosely. Whatever you choose, keep it fixed across a study and record it — a setting that is invisible afterwards makes two runs incomparable.

18.9 See also

- Setting priority under Linux — the measured version of this page, and the one to read for the *reasoning*, most of which transfers even though the mechanism does not.
- Minimising trigger-to-stimulus jitter
- Timing tests — how to run the measurements above.

Chapter 19

Setting priority under macOS

If you are here because of EEG or MEG trigger timing, read Minimising trigger-to-stimulus jitter as well: priority is necessary but nowhere near sufficient, and the dominant term is not the one most people expect.

19.0.1 ! None of this has been measured on macOS

The Linux procedure in Setting priority under Linux was verified against `chrt` and `nice` on real hardware, and every figure in it comes from a run. **This page has not been.** goxpyriment's own Darwin scheduling code (`sysinfo/scheduling_darwin.go`) is likewise written against the documented BSD/Darwin APIs and never exercised — it says so at runtime, in the `Sched:` line of every report.

What follows is the mechanism and the procedure, not a result. Measure it before quoting a number.

19.1 The short version

```
sudo nice -n -20 ./my_experiment
```

This is worth doing on a dedicated stimulus machine, **with two caveats that are not cosmetic** — one about what it actually buys you, and one about what `sudo` does to your data files. Both are below. Read them before adopting this as a lab habit.

19.2 Why a launcher prefix at all

On Linux, a goxpyriment program raises its own priority at startup — it calls `sysinfo.RaiseToRealTime` and needs no `chrt` prefix.

On macOS it does not. `sysinfo/realtime_other.go` deliberately implements `raiseToRealTime` as an error on every non-Linux platform, and `Experiment.Initialize` does not make the request there at all — it would fail identically on every machine, so the "real-time scheduling not obtained" message that earlier versions printed on every macOS run carried no information and is gone. The reason it is an error rather than a silent success is worth stating, because it is the same reason `nice` is a weaker lever here than it looks:

> macOS has `thread_policy_set`, but [...] the setting that matters most for stimulus timing is something else entirely [...] a per-thread time-constraint policy on macOS. Pretending a portable "raise priority" call covers them would produce a program that reports success while doing something different on each platform.

So on macOS the priority decision is made **outside** the program. `-no-realtime` and `-realtime-priority` have no effect there beyond suppressing the message.

19.3 Caveat 1: `nice` is not the lever that governs real-time behaviour on Darwin

This is the part where the recommendation should be qualified rather than simply repeated.

`nice -n -20` lowers the process's nice value, which raises its priority **within the timeshare scheduling band**. That is a real effect: it makes the experiment less likely to be preempted by other user-space work, which is exactly what you want on a machine that is doing anything else at all.

But on Darwin, "will this thread wake on time" is not primarily decided by the timeshare priority. It is decided by which *scheduling band* the thread is in, and the band that gives hard wake-up guarantees is entered by setting a **per-thread time-constraint policy** (`thread_policy_set` with `THREAD_TIME_CONSTRAINT_POLICY`) — the mechanism Core Audio uses for its render threads. A negative nice value does not move a thread into that band.

goxpyriment does not set such a policy. `sysinfo/scheduling_darwin.go` is explicit that it cannot even *see* one:

```
Sched:      policy: SCHED_OTHER (assumed)  nice: -20  [UNTESTED on macOS;
↪  cannot
           see per-thread THREAD_TIME_CONSTRAINT_POLICY, so RealTime is not
           authoritative]
```

The practical reading: **expect `nice` to reduce preemption by other processes, not to give the experiment thread a deadline guarantee.** It is a real improvement of the "keep other work out of the way" kind, in the same family as quitting Spotlight-heavy applications before a session — and it is a smaller, softer effect than `chrt -f 50` is on Linux. Do not carry a Linux expectation across.

19.4 Caveat 2: `sudo` leaves your data files owned by root

This one will bite you, and it has nothing to do with timing.

`sudo nice -n -20 ./my_experiment` runs the **whole experiment** as root, not just the priority change. Every file it writes is then owned by `root`:

```
-rw-r--r--  1 root  wheel   14523  Aug 15 10:24 Timing-Tests_sub-000_….csv
```

Two consequences. First, the results in `~/goxpy_data/` are no longer yours to move, edit or delete without `sudo`. Second, and worse, if the directory itself ends up root-owned, a **later run without `sudo` may fail to write its data file** — after the participant has done the task.

Mitigations, in order of preference:

1. **Check the ownership after your first `sudo` run**, and fix it once:

   ```
   ls -l ~/goxpy_data | head
   sudo chown -R "$USER" ~/goxpy_data
   ```

2. **Point the run somewhere explicit** with `-outdir`, so a mistake is confined to one directory rather than to your whole data tree.
3. **Decide per-machine, not per-run.** If a dedicated stimulus Mac always runs with `sudo`, that is consistent and fine. Mixing the two on the same data directory is what causes trouble.

There is also an identity question worth knowing about: macOS privacy permissions (screen recording, input monitoring, accessibility) are granted per user and per application, so a program run as root is not necessarily the same subject as the same program run as you. If you are prompted for a permission you thought you had already granted, this is why.

19.5 Step 1: run it

```
sudo nice -n -20 ./my_experiment
```

`nice` needs root only for **negative** values; `nice -n 5` (lower priority) works unprivileged. `-20` is the strongest available; the scale runs −20 (most favourable) to +19.

`renice` can change a process after it has started, which avoids running the whole experiment as root:

```
./my_experiment &
sudo renice -n -20 -p $!
```

The trade is that the first fraction of a second runs at normal priority. Harmless for an experiment with an instructions screen before the first trial; not harmless for one that starts measuring immediately.

To make it reproducible across people in a lab, put it in a one-line script beside the experiment rather than relying on everyone remembering the prefix.

19.6 Step 2: verify

Every goxpyriment program's system report carries the state it actually ran with, so a recorded run is self-labelling:

```
Timing-Tests -sysinfo
```

Read the `nice:` field of the `Sched:` line — `-20` if the prefix took, `0` if it did not.

What the line **cannot** tell you is whether anything is in the time-constraint band, and it says so rather than guessing. `RealTime` stays `false` on macOS even if a thread does hold such a policy; a confident "not real-time" would be an answer this code is not in a position to give.

19.7 How to find out whether it helped, on your machine

This is the part that turns the page from advice into a result, and it takes about ten minutes.

Run the frame-interval test twice, changing only the prefix:

```
Timing-Tests -test display -duration-s 300
sudo nice -n -20 Timing-Tests -test display -duration-s 300
```

Then compare the two `-info.txt`/`.csv` pairs. What to read, in order:

1. **Dropped frames and the maximum interval** — the tail, not the average. A priority change moves the worst frames; it barely moves the median, so a comparison of means will show almost nothing even when the change helped.
2. **The SD of the frame intervals.**
3. **The `Sched:` line in each `-info.txt`**, which records which run was which so the two files cannot be mixed up afterwards.

Do it on a machine in the state you will actually run participants on. An idle Mac hides scheduling problems — that is the single most common way this kind of comparison produces a false negative. Given caveat 1, a **small or absent** difference on an idle machine is the outcome to expect; the interesting comparison is under load.

If you run it, the numbers would be genuinely valuable to this project; there are currently none for macOS.

19.8 Other macOS-specific things worth knowing

Listed as pointers rather than recommendations — none has been verified here, and on current hardware several of these are plausibly larger terms than `nice`:

- **App Nap and Low Power Mode.** Both throttle background and power-constrained work. Low Power Mode on a laptop is the more likely of the two to affect a foreground fullscreen application, and it is a single toggle in System Settings → Battery. **Run on mains power.**
- **ProMotion / variable refresh rate.** MacBook Pro and Studio Display panels can vary their refresh rate. Pin the display to a fixed rate on a stimulus machine: a panel that changes its own frame period defeats the assumption every frame-counted duration rests on. This is the macOS instance of the problem `tests/Timing-Tests -test vrr` exists to probe.
- **Apple silicon efficiency cores.** A thread scheduled onto an E-core behaves differently from one on a P-core, and QoS class is what influences that placement — another respect in which the meaningful lever on macOS is QoS rather than nice.
- **Display mirroring and Sidecar.** Both add a compositing stage between your frame and the panel. Turn them off.

Whatever you choose, keep it fixed across a study and record it — a setting that is invisible afterwards makes two runs incomparable.

19.9 See also

- Setting priority under Linux — the measured version of this page, and the one to read for the *reasoning*, most of which transfers even though the mechanism does not.
- Minimising trigger-to-stimulus jitter
- Timing tests — how to run the measurements above.

Chapter 20

Fullscreen + HIGH_PIXEL_DENSITY issue

20.1 Symptom

On **macOS** (Retina), running an experiment in fullscreen mode shows a gray screen after the initial setup UI. Windowed mode (`-w`) works correctly. The issue was reported for the `retinotopy` example but likely affects any experiment that uses hardcoded pixel dimensions.

20.2 Root cause analysis

20.2.1 The asymmetry in `apparatus/NewScreen`

In `apparatus/screen.go`, the fullscreen and windowed paths differ in one critical flag:

Fullscreen path:

```
window, err := sdl.CreateWindow(title, 0, 0, sdl.WINDOW_HIGH_PIXEL_DENSITY)
// ...
w, h, _ := window.SizeInPixels()
return &Screen{..., Width: int(w), Height: int(h)}
```

Windowed path:

```
window, renderer, err := sdl.CreateWindowAndRenderer(title, width, height,
 ↪  sdl.WINDOW_HIDDEN)
// no WINDOW_HIGH_PIXEL_DENSITY
return &Screen{..., Width: width, Height: height}
```

20.2.2 What `SDL_WINDOW_HIGH_PIXEL_DENSITY` does

Without it, SDL3 follows the OS logical coordinate space: - On a macOS Retina display at 2560×1600 logical points, SDL gives a 2560×1600 renderer — the OS compositor

silently doubles to 5120×3200 physical pixels. - On standard Linux (pixel density = 1), there is no difference.

With it, SDL3 bypasses logical scaling and gives raw physical pixels: - Same Retina display: you get a 5120×3200 renderer directly. - All sizing and coordinate code must work in physical pixels.

20.2.3 Consequence

Mode	Flag present	`Screen.Width/Height`	`CenterToSDL`
Windowed	No	logical pixels (e.g. 1024×768)	correct
Fullscreen	Yes	physical pixels (e.g. 5120×3200 on Retina)	correct

`CenterToSDL` uses `renderer.RenderOutputSize()`, so it is always internally consistent. The problem is **any code that uses hardcoded pixel counts**.

In `retinotopy/main.go`:

```
const WindowWidth  = 768
const WindowHeight = 768

// Texture created at hardcoded logical size:
tex, err := r.Exp.Screen.Renderer.CreateTexture(
    apparatus.PIXELFORMAT_RGBA32,
    apparatus.TEXTUREACCESS_STREAMING,
    WindowWidth, WindowHeight,   // ← wrong in fullscreen HiDPI
)
r.PixelBuffer = make([]byte, WindowWidth*WindowHeight*4)
// ...
r.CombinedTexture.Update(nil, r.PixelBuffer, WindowWidth*4)
```

On a Retina Mac in fullscreen the renderer expects physical pixels (e.g. 5120×3200) but the texture and pixel buffer are 768×768. The stimulus either renders at a tiny fraction of the screen or lands outside the viewport entirely.

20.3 Why the Pi issue was different

The Pi gray screen was caused by a different bug: `binsdl.Load()` tried to extract and load an x86-64 `libSDL3.so` on an ARM64 system, which fails immediately with:

```
binsdl: couldn't sdl.LoadLibrary: /tmp/.../libSDL3.so.0: cannot open shared
 ↪  object file
```

This was originally worked around with build-tag-selected loader files, one of which loaded the *system* SDL3 on linux/arm64. **That workaround is gone**: since go-sdl3 v0.1.1 bundles SDL3 for linux/arm64 too, `control/sdlload_embedded.go` handles every platform with `binsdl/binttf/binimg.Load()` and carries no build tag (removed in commit `53da703`). Nothing needs a system-installed SDL3.

After that fix the Pi works correctly in both windowed and fullscreen — confirming the Pi had no HiDPI issue (pixel density = 1 on a standard monitor).

20.4 The Mac issue — implemented fix

The Mac fullscreen gray screen has been addressed with a framework-level fix in `apparatus/screen.go`.

20.4.1 Approach — `SetLogicalPresentation` (implemented)

`WINDOW_HIGH_PIXEL_DENSITY` is **kept** (it is needed to get correct centering on Linux). After the renderer is created, the framework immediately queries the logical (OS) pixel dimensions via `window.Size()` and calls:

```
renderer.SetLogicalPresentation(logW, logH, sdl.LOGICAL_PRESENTATION_
↪  STRETCH)
```

SDL3 then maps all drawing commands from logical coordinates (e.g. 2560×1600 on a Retina display) to the full physical resolution (e.g. 5120×3200) transparently. The `Screen` struct stores the logical `Width`/`Height`, which is what experiment code should use for coordinate math. `CenterToSDL` and `MousePosition` already use the `LogicalSize` field, so no changes are needed in experiment code.

Platform	`window.Size()`	`window.SizeInPixels()`	Renderer logical space
Linux standard (Pi, desktop)	1920×1080	1920×1080	1920×1080 — no change
macOS Retina 2560×1600	2560×1600	5120×3200	2560×1600 — SDL upscales

Because `Screen.Width/Height` are now always in logical pixels and `SetLogicalPresentation` is active, experiments using hardcoded pixel values (like the retinotopy texture at 768×768) work correctly on all platforms — SDL scales the output to fill the screen without the experiment needing to know the physical pixel count.

Chapter 21

SDL3 Fullscreen in a Linux Virtual Console (TTY)

> This page is the narrow troubleshooting note for one symptom: SDL not going fullscreen on a bare console. For running experiments there properly — the groups it needs, why the console's display mode (not the monitor's native one) is the mode you get, how to pin it, and what a second lit head does to the vblank clock — see Running from a virtual console.

21.1 Problem

When running with no display server (GDM stopped, bare TTY), SDL3 may fail to switch to fullscreen — the console stays visible instead of the experiment window.

21.2 Root Cause

SDL3 auto-detects the video driver. Without X11 or Wayland running, it should fall back to the **KMS/DRM** (`kmsdrm`) driver, but auto-detection may pick a non-functional driver silently.

21.3 Diagnostics

Check which driver SDL3 selects and whether KMS is available:

```
# Explicit KMSDRM driver — should work in a bare TTY
SDL_VIDEODRIVER=kmsdrm go run main.go -test display

# Check if SDL3 was built with KMSDRM support (no display needed)
SDL_VIDEODRIVER=offscreen go run main.go -test display 2>&1 | head -20
```

If `SDL_VIDEODRIVER=kmsdrm` gives "no available video device", SDL3 was not built with KMSDRM support — you need to rebuild SDL3 with KMSDRM enabled, or install a

package that includes it.

21.4 Fix

21.4.1 Option 1 — environment variable (quickest)

```
SDL_VIDEODRIVER=kmsdrm go run main.go -test display
```

Or via the existing wrapper:

```
# e.g. a run_vt.sh wrapper
SDL_VIDEODRIVER=kmsdrm go run "$@"
```

21.4.2 Option 2 — programmatic hint in screen.go

In `apparatus/screen.go`, set the hint before `sdl.Init` when no display server is detected:

```
import "os"

// Before sdl.Init(...)
if os.Getenv("DISPLAY") == "" && os.Getenv("WAYLAND_DISPLAY") == "" {
    sdl.SetHint("SDL_VIDEODRIVER", "kmsdrm")
}
```

21.5 Notes

- `window.SetFullscreen(true)` after creation may be unreliable on KMSDRM; creating the window with fullscreen from the start (as `apparatus/screen.go` already does) is the right approach.
- On a machine that already has a compositor running, `SDL_VIDEODRIVER=wayland` is the right choice; `kmsdrm` is for a bare TTY with no compositor at all.
- Ensure the user running the program is in the `video` (and possibly `input`) group:

  ```
  sudo usermod -aG video,input $USER
  ```

  KMS/DRM access is permission-gated; without group membership SDL will fail to open the device.

Chapter 22

Media — Multi-Movie Playback with Hardware-Verified Display Onsets

This document describes the `media/` package added to goxpyriment for multi-movie playback synchronised to a shared master clock, with hardware-verified display-onset events suitable for wiring to external EEG / MEG / TMS triggers.

22.1 Table of Contents

1. Scope and design
2. Quick start
3. Architecture in one diagram
4. API reference (package `media`)
5. API reference (package `media/present`)
6. Multi-movie synchronisation: what is guaranteed
7. External-trigger synchronisation with frames on screen
8. Platform support (Stage 5)
9. Mapping to PsyScope movie commands
10. References

22.2 1. Scope and design

22.2.1 What `media/` is

A new top-level package alongside `stimuli/` that adds **multi-movie** playback with shared timing. It is the home for everything related to synchronising several video streams against one another and to external hardware (EEG / MEG / TMS triggers, photodiodes).

It imports the semantics of PsyScope Tahoe's movie commands — `MovieDo[ PLAY/PAUSE/STOP/SET/GET ]` actions and `Movie[Done/At/AtDisplay]` + `Display[Onset/Offset]` conditions — adapted to a Go API.

22.2.2 Hard scope choices

- **`.gv` movies only.** Decoding is delegated to the existing pure-Go `funatsufumiya/go-gv-video` library (LZ4-compressed RGBA frames, random access). No ffmpeg, no MPEG-1, no H.264.
- **Silent.** Movies have no audio. The package does not load, decode, mix, or schedule audio of any kind.
- **Pure Go, no cgo.** SDL3 is loaded via `purego` (already used by `go-sdl3`); the macOS Stage 5 backend uses purego for CoreVideo / libSystem; the Linux Stage 5 backend uses the standard `syscall` package.
- **Coexists with `stimuli`.** `stimuli.GvVideo` and `stimuli.PlayGv` remain the single-movie convenience path. `media.Movie` is added alongside for the multi-movie / sync use cases.

22.2.3 What problems it solves

Problem	Solution
Multiple AVPlayer-style movies drift apart over hundreds of pause/resume cycles	One `MasterClock` shared by every `Movie`; positions computed from a single clock reading per draw cycle. Zero cumulative drift by construction.
Three sequential `Pause` calls reading the clock at slightly different times	`BeginBurst()` / `EndBurst()` freezes `MasterClock.Now` so all commands inside the burst observe the same value.
`Movie[At "f:N"]` needs to fire ~one vsync *before* the target frame is displayed so an overlay can land on the same vsync	`OnAt(Target, fn)` evaluates from inside the per-frame decode loop, *before* `FlipTS`.
External EEG trigger must be aligned to actual first-pixel-visible time, not to when our `Present` call returned	Stage 5 `media/present` package; on macOS uses `CVDisplayLink`, on Linux uses `DRM_IOCTL_WAIT_VBLANK`, both report OS-measured vsync.
Need to know post-hoc which exact vsync each trigger landed on	`Onset.TimestampNS` is in the same nanosecond reference as `sdl.TicksNS()` and SDL keyboard event timestamps; subtract directly to compute reaction times.

22.3 2. Quick start

A complete two-movie experiment in ~25 lines, including hardware trigger wiring (silently no-ops if no DLP-IO8 is plugged in):

```
package main

import (
    "log"
    "time"

    "github.com/funatsufumiya/go-gv-video/gvvideo"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/media"
    "github.com/chrplr/goxpyriment/triggers"
)

func main() {
    exp := control.NewExperimentFromFlags("Demo", control.Black,
↪   control.White, 32)
    defer exp.End()

    gvL, _ := gvvideo.LoadGVVideo("clipL.gv")
    gvR, _ := gvvideo.LoadGVVideo("clipR.gv")

    mgr := media.NewMovieManager(exp.Screen)
    defer mgr.Close()

    left, _  := media.NewMovie(mgr, gvL, media.WithTag("L"),
↪   media.WithRepeat(-1))
    right, _ := media.NewMovie(mgr, gvR, media.WithTag("R"),
↪   media.WithRepeat(-1))
    defer left.Close()
    defer right.Close()

    ttl, _, _ := triggers.AutoDetectDLPIO8()
    defer ttl.Close()

    left.OnAtDisplay(media.Frame(60), func(o media.Onset) {
        log.Printf("frame 60 displayed @ %d (%s)", o.TimestampNS, o.Source)
        _ = ttl.Pulse(0, 5*time.Millisecond)
    })

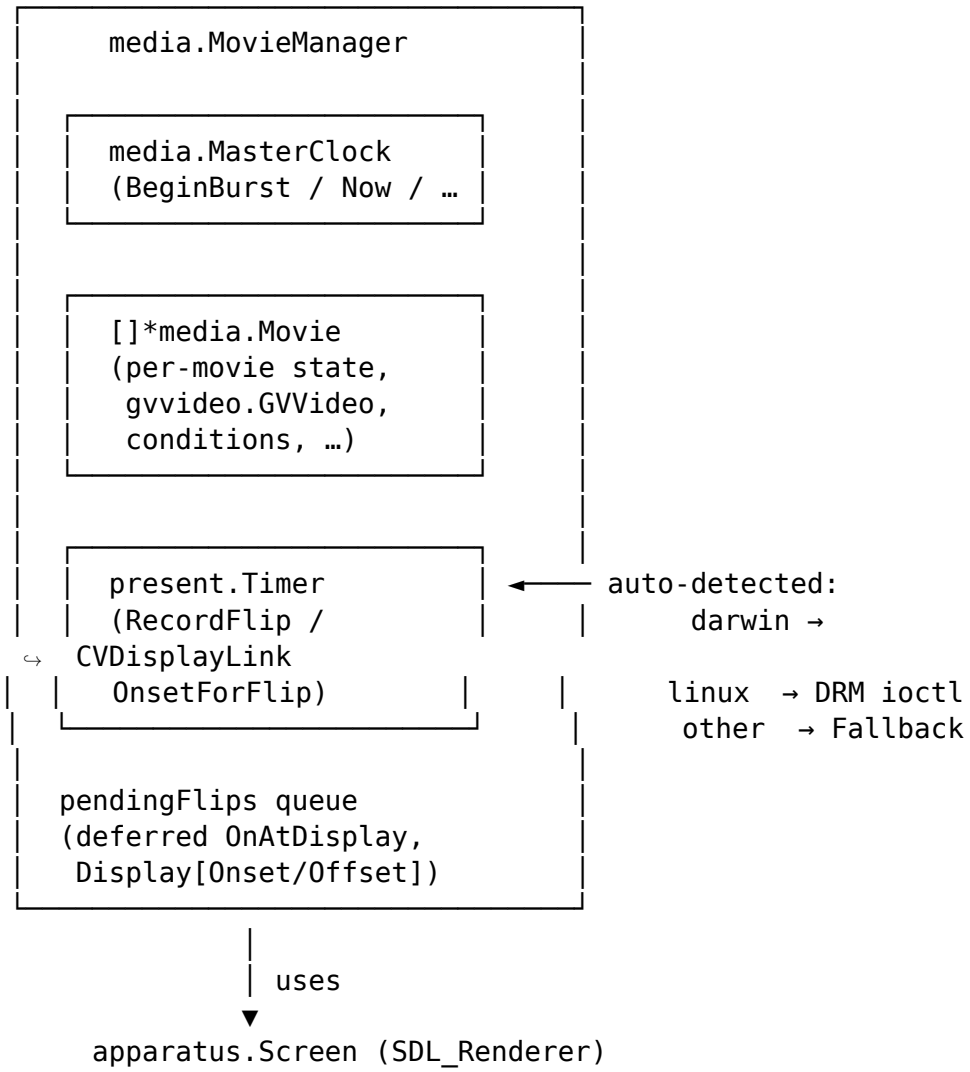
    mgr.BeginBurst(); left.Play(); right.Play(); mgr.EndBurst()

    exp.Run(func() error {
        _, err := mgr.Draw()
        return err
```

```
    })
}
```

A more comprehensive walkthrough with composited overlays, burst pause, seek-back, and `Display[Onset/Offset]` events lives in examples/demo_sync_two_gv_movies/.

22.4 3. Architecture in one diagram



Per-frame call site (mixed compositing form):

```
exp.Screen.Clear()              // background
mgr.DrawWithoutFlip()           // decode + render all movies, fire OnAt
 ↪  look-ahead
otherStim.Draw(exp.Screen)      // overlay text / fixation cross / …
```

```
ts, _ := exp.Screen.FlipTS()    // present to display
mgr.NotifyFlipped(ts)           // fire OnAtDisplay + Display[Onset/Offset]
```

For movie-only frames the all-in-one `mgr.Draw()` collapses these into one call.

22.5 4. API reference (package `media`)

22.5.1 4.1 `MovieManager`

The per-experiment owner of the master clock and the registered movies.

```
func NewMovieManager(screen *apparatus.Screen, opts ...ManagerOption)
 ↪  *MovieManager
```

Constructor. Auto-detects the best `present.Timer` for the platform unless overridden via `WithPresentTimer`. Logs the chosen backend at construction:

```
media: present backend: macOS CVDisplayLink (CoreVideo, hardware-verified)
 ↪  (precision=hardware-verified)
```

```
type ManagerOption func(*MovieManager)

func WithPresentTimer(t present.Timer) ManagerOption
```

Override the auto-detected timer. Pass `present.NewFallback()` to force vsync-estimated mode for testing or to silence the macOS `CVDisplayLink` background thread.

22.5.1.1 Lifecycle

Method	Purpose
`(*MovieManager).Close() error`	Stops the macOS CVDisplayLink thread / closes the Linux DRM fd. Idempotent. **Defer this before `exp.End()`.**
`(*MovieManager).Clock() *MasterClock`	Returns the shared master clock (call `Reset()` at trial boundaries if needed).
`(*MovieManager).LogicalSize() (float32, float32)`	Returns the renderer's coordinate-space size (HiDPI-correct). Use for layout instead of `Screen.Size()`.
`(*MovieManager).LastFlipTS() uint64`	Most recent `FlipTS` passed to `NotifyFlipped`.

22.5.1.2 Movie set

Method	Purpose
`Add(m *Movie)`	Implicitly called by `NewMovie`.
`Remove(m *Movie)`	Deregister a movie.
`Movies() []*Movie`	Snapshot of registered movies in registration order.

22.5.1.3 Burst (atomic command groups)

Method	Purpose
`BeginBurst()`	Freeze `MasterClock.Now` at its current value (nestable).
`EndBurst()`	Decrement freeze counter; clock thaws when it reaches zero.

`MovieManager.Draw` / `DrawWithoutFlip` already wrap their per-frame body in an internal burst, so all movies on a frame share one `MasterClock.Now`. Use the public `BeginBurst` / `EndBurst` to make a sequence of script-style command calls atomic across multiple movies.

22.5.1.4 Per-frame

Method	Purpose
`Draw() (uint64, error)`	All-in-one: clear → decode → render → flip → notify. Returns the post-flip SDL ticks. Movie-only frames.
`DrawWithoutFlip() error`	Decode + render, no clear, no Present. Caller composites other stimuli, then calls `Screen.FlipTS()` and `NotifyFlipped(ts)`.
`NotifyFlipped(ts uint64)`	Publish post-vsync events: advance each movie's `currentFrameIndex`, fire `OnAtDisplay`, fire `Display[Onset/Offset]`. Defers callbacks until the matching hardware-verified vsync timestamp arrives (see §7.4).

22.5.1.5 Display[Onset/Offset]

Method	Purpose
`RegisterStimulusName(name string, visible func() bool) func()`	Declare a non-movie stimulus by name. Visibility predicate is sampled in `DrawWithoutFlip`. Returns an unregister function. (Movies are auto-registered with their tag.)
`OnDisplayOnset(name string, fn func(Onset)) func()`	Fires when `name` transitions from not-visible to visible across two flips. Returns unsubscribe.
`OnDisplayOffset(name string, fn func(Onset)) func()`	Fires when `name` transitions from visible to not-visible. Returns unsubscribe.

22.5.2 4.2 `Movie`

Per-movie state, mirrors `PSMovieDecoder` in PsyScope Tahoe.

```
func NewMovie(mgr *MovieManager, gv *gvvideo.GVVideo, opts ...Option)
 ↪  (*Movie, error)
```

Wraps an already-loaded `gvvideo.GVVideo` and registers it with the manager. The caller retains ownership of `gv` and is responsible for closing the underlying file (matches `stimuli.GvVideo.Unload`).

22.5.2.1 Construction options

```
type Option func(*Movie)

WithTag(tag string)              // identifier; required for
 ↪  Display[Onset/Offset]
WithRate(r float64)             // playback rate; 1.0 = normal, must be > 0
WithRepeat(n int)               // -1 = infinite, 1 = play once (default)
WithFromTime(d time.Duration)   // initial playhead within movie media-time
WithScale(s ScaleMode)          // ScaleNone | ScaleFit | ScaleFill
WithSize(w, h float32)          // explicit destination size; overrides
 ↪  WithScale
WithPosition(p sdl.FPoint)      // center-relative draw position
WithFadeIn(d time.Duration)     // linear opacity 0→1 ramp over d effective
 ↪  time
WithPersistent(b bool)          // last frame stays on screen after Done
```

22.5.2.2 Playback control

Method	Effect
`Play()`	Start (first call) or resume (after `Pause`). Fresh start anchors playhead at `WithFromTime`. Idempotent.
`Pause()`	Freeze at current frame.
`PauseWithLoop(window time.Duration)`	Pause and bounce within ±
`Stop()`	Halt, release GPU texture, reset state. After `Stop` the movie can be `Play`-ed again from `WithFromTime`.
`Close()`	Release GPU resources. Does not close the underlying `gvvideo.GVVideo`.

22.5.2.3 Property access

Method	Returns
`Time() time.Duration`	Effective media time (rate-scaled, cumulative across loops).
`Frame() int`	Cumulative 1-based displayed-frame index (advanced by `NotifyFlipped`). 0 before first frame.
`FPS() float64`	Frames per second from the `.gv` header.
`FrameCount() int`	Frames per loop iteration.
`LoopCounter() int`	0-based loop iteration currently playing.
`Repeat() int`	Configured loop count (-1 = infinite).
`Width(), Height() float32`	Native frame dimensions.
`IsActive(), IsPaused(), IsDone() bool`	State queries.

22.5.2.4 Property mutation

Method	Effect
`SetRate(r float64) error`	Change playback rate; preserves current effective time. r ≤ 0 returns an error.
`SeekTime(d time.Duration) error`	Jump to elapsed media time `d`. Resets condition fired-flags so re-pass over the same target re-fires.
`SeekFrame(n int) error`	Jump to cumulative 1-based frame `n`.

22.5.2.5 Conditions / callbacks

```
// Sealed Target type — only these three constructors:
type Frame int                  // 1-based cumulative target
type AtTime time.Duration       // elapsed media time target
type Done struct{}              // end-of-file target
```

```
// Look-ahead (fires from DrawWithoutFlip, BEFORE the target frame is
 ↪  presented):
OnAt(target Target, fn func(Onset)) func()    // returns unsubscribe
OnDone(fn func(Onset)) func()                 // sugar for OnAt(Done{}, ...)
DoneCh() <-chan Onset                         // channel form for OnDone

// Hardware-verified (fires from NotifyFlipped, AFTER the target frame
 ↪  appears on screen):
OnAtDisplay(target Target, fn func(Onset)) func()
```

The look-ahead vs hardware-verified distinction is critical for trigger alignment — see §7.

22.5.3 4.3 `MasterClock`

Monotonic media clock with freeze-during-burst support.

```
func NewMasterClock() *MasterClock

(*MasterClock).Now() time.Duration   // monotonic since last Reset /
 ↪  construction
(*MasterClock).Reset()               // back to zero
(*MasterClock).BeginBurst()          // freeze Now at current value
 ↪  (nestable)
(*MasterClock).EndBurst()            // unfreeze when freeze count hits zero
(*MasterClock).Frozen() bool         // currently inside a burst?
```

22.5.4 4.4 Onset events

```
type Onset struct {
    TimestampNS uint64       // SDL ticks; same reference as sdl.TicksNS /
 ↪  FlipTS
    Frame       int          // cumulative 1-based; 0 if not applicable
    Source      OnsetSource  // precision class
    Movie       *Movie       // nil for generic Display events of non-movie
 ↪  stimuli
    Name        string       // movie tag for movies; user name for others
}

type OnsetSource int

const (
    VsyncEstimated   OnsetSource = iota  // post-Present FlipTS,
 ↪  vsync-period precision
    HardwareVerified                     // OS-measured first-pixel-visible
 ↪  (sub-ms)
```

```
    LookAhead                           // pre-vsync, fired from decode loop
)
```

22.5.5 4.5 Time-spec helpers

For users translating PsyScope scripts to Go:

```
ParseTimeSpec("s:3,ms:100")  // → 3.1 * time.Second, nil
ParseFrameSpec("f:186")      // → 186, nil
ParseFrameSpec("frame:1")    // → 1, nil
```

Native API uses `time.Duration` and `int` directly — these helpers are a convenience for porting existing scripts.

22.6 5. API reference (package `media/present`)

The presentation-timer subpackage. Each backend reports OS-measured vsync timestamps that pair with `Screen.FlipTS()` values to give hardware-verified display-onset times.

```
type Timer interface {
    RecordFlip(flipTS uint64)
    OnsetForFlip(flipTS uint64) (timestamp uint64, source OnsetSource, ok
    ↪ bool)
    Precision() OnsetSource
    Close() error
    Description() string
}

func AutoDetect(screen *apparatus.Screen) Timer  // platform-best, never nil
func NewFallback() Timer                          // always-available no-op
```

`MovieManager` calls `RecordFlip(ts)` immediately after each `FlipTS`, then `OnsetForFlip(ts)` to read back the matching hardware-verified vsync timestamp. If `ok=false`, the manager defers and retries on the next `NotifyFlipped` (the OS may publish vsync timestamps asynchronously).

Most users never construct a `Timer` directly — `NewMovieManager` handles auto-detection. Use `WithPresentTimer(present.NewFallback())` in tests if you need deterministic vsync-estimated behaviour.

22.7 6. Multi-movie synchronisation: what is guaranteed

22.7.1 6.1 Mechanism

Every per-frame body opens an internal burst, reads `MasterClock.Now` exactly once, and computes each movie's effective media time from that single value:

```
// Inside MovieManager.DrawWithoutFlip:
mgr.clock.BeginBurst()
defer mgr.clock.EndBurst()
now := mgr.clock.Now()          // ← single read for this frame

for _, m := range movies {
    eff := m.effectiveLocked(now)   // each movie computes its own offset
    target := int(eff.Seconds() * m.fps) + 1
    // … decode + render …
}
```

Because every movie computes from the same `now`, the **inter-movie spread within one frame is zero by construction**. There is no rounding error to accumulate. Pause/resume cycles update each movie's `mediaTimeOffset` independently but always by the same delta when issued inside the same burst.

22.7.2 6.2 What that means in numbers

Measurement	Result	Notes
Inter-movie spread at `Play`	**0 ms**	All movies anchored to one `MasterClock.Now` reading inside `BeginBurst/EndBurst`.
Inter-movie spread at first frame on screen	**0 ms**	All movies render in one `Renderer.Present` call → one vsync → one display refresh.
Cumulative drift after 500 pause/resume cycles	**0 ms**	Mathematical: pause/resume only changes per-movie `mediaTimeOffset`; drift would require a per-movie clock, which the design forbids.
Same-vsync rendering	All visible movies share the same vsync	Single `Renderer.Present` call → atomic page flip.

This matches PsyScope Tahoe's measured numbers (see `MultiMovieSyncStrategy.md` §8 in that repo).

22.7.3 6.3 The atomic-burst pattern

Use `mgr.BeginBurst() / mgr.EndBurst()` around any sequence of script- style commands that should observe the same clock:

```
mgr.BeginBurst()
left.Pause()      // captures MasterClock.Now at burst-start time T
right.Pause()     // captures the same T
center.Pause()    // captures the same T
mgr.EndBurst()
```

Without the burst, three sequential `Pause` calls would each read `MasterClock.Now` a few microseconds apart. The burst ensures all three movies pause at the exact same media time, regardless of scheduling jitter between the calls.

22.7.4 6.4 What is *not* synchronised

- **Rendering completion across multiple Renderers.** This package uses one renderer (the `apparatus.Screen.Renderer`), so this is moot. If you build a multi-window experiment, each window has its own `MovieManager` and they synchronise only as well as the OS schedules their flips.
- **Decode latency variation.** Different movies may take different times to LZ4-decompress a frame. The decode happens before the shared `Renderer.Present` call, so a slow decode would push the entire frame past its target vsync. Prefer movies of similar resolution and bitrate; avoid per-frame allocations (the decode buffer is reused).
- **Audio.** Audio is out of scope (see §1).

22.8 7. External-trigger synchronisation with frames on screen

This section is the most important one for EEG / MEG / TMS experiments and the question most users will have. Read it carefully.

22.8.1 7.1 The two timestamps that matter

For a callback registered via `mov.OnAtDisplay(media.Frame(N), fn)`, two distinct timestamps are involved:

Quantity	What it is	Source	Precision
`Onset.TimestampNS`	When frame N's first pixel begins scanning out of the display controller	OS vsync API (CVDisplayLink / DRM_IOCTL_ WAIT_VBLANK)	**Sub-ms** vs scanout start (Stage 5 platforms)

Quantity	What it is	Source	Precision
Wire-arrival of TTL pulse	When the EEG box sees the trigger line go HIGH	DLP-IO8 USB / parallel port / MEGTTLBox + your callback latency	`TimestampNS + 0.5 –5 ms` (constant + small jitter)

`Onset.TimestampNS` is what you want for **post-hoc analysis**: it is in the same nanosecond reference frame as `sdl.TicksNS()` and SDL3 keyboard event timestamps, so reaction times computed as `keyEvent.Timestamp - onset.TimestampNS` are sub-ms accurate.

The wire-arrival is what arrives at your EEG amplifier's TTL input. It lags `TimestampNS` by a small chain of latencies.

22.8.2 7.2 The wire-arrival delay

Breakdown of the ~0.5–5 ms gap between `TimestampNS` and the TTL line going HIGH:

Stage	Typical	Notes
OS publishes vsync timestamp	0 µs (Linux DRM, sync ioctl) to ~1 ms (macOS, async CV callback may defer)	See §7.4
Manager dispatches the callback	~1–100 µs	Mutex + closure invocation
`ttl.Pulse` writes to the device	0.5–2 ms (DLP-IO8 USB-CDC); <1 µs (parallel port); ~1 ms (MEGTTLBox 115200 baud)	Bus + firmware
Total	**~0.5 ms (parallel) to ~5 ms (USB)**	Stable / calibratable

The delay is **constant and stable**. Calibrate once with a photodiode + TTL recording (see §7.5) and subtract the measured offset in your analysis pipeline.

22.8.3 7.3 LookAhead vs HardwareVerified vs VsyncEstimated

The three `OnsetSource` values describe the precision class of `Onset.TimestampNS`:

Source	Where it fires	TimestampNS meaning	Use case
`LookAhead`	Inside `DrawWithoutFlip`, before `Present`	`sdl.TicksNS()` at fire time (NOT a vsync)	Set state for the same-vsync compositor (e.g., "show overlay this frame")
`VsyncEstimated`	After `Present`, in `NotifyFlipped`	Post-`Present` `FlipTS` value	Vsync-period precision (~16 ms at 60 Hz). Default fallback when no Stage 5 backend is available.
`HardwareVerified`	After `Present`, in `NotifyFlipped`, possibly deferred to next flip	OS-measured first-pixel-visible time	Sub-ms precision vs scanout start. Available on macOS + Linux.

Callback	Source on Stage 5 platforms	Source on fallback platforms
`mov.OnAt(t, fn)`	`LookAhead`	`LookAhead`
`mov.OnDone(fn)`	`LookAhead`	`LookAhead`
`mov.OnAtDisplay(t, fn)`	`HardwareVerified`	`VsyncEstimated`
`mgr.OnDisplayOnset(name, fn)`	`HardwareVerified`	`VsyncEstimated`
`mgr.OnDisplayOffset(name, fn)`	`HardwareVerified`	`VsyncEstimated`

When the timer is `VsyncEstimated`, the manager logs a one-time warning on the first display callback registration. When the timer is `HardwareVerified`, the warning is suppressed.

22.8.4 7.4 The deferral pattern (macOS specifically)

On macOS, the `CVDisplayLink` callback fires on a background thread asynchronously to `Present`. In the rare race where the callback hasn't yet published a vsync timestamp by the time `NotifyFlipped(ts)` runs, the manager:

1. Stashes this flip's callbacks in `pendingFlips`.
2. Continues normally (no error).
3. On the *next* `NotifyFlipped` (one vsync later, ~16 ms wall-clock), tries `OnsetForFlip(stashed_ts)` again. The CVDisplayLink callback has surely fired by now, so the lookup succeeds and the callbacks fire — with the **correct** hardware-verified timestamp for the stashed flip.

The trigger pulse therefore arrives ~16 ms after the visual onset in this race case (the wire is one vsync late), but the `Onset.TimestampNS` you receive is still sub-ms accurate for *frame N's actual visual onset*. Post-hoc analysis is unaffected; only the real-time wire timing is delayed by one vsync.

If a stash sits in the queue for more than `~3 × frame period` (~50 ms at 60 Hz), the manager gives up waiting and fires the callbacks with `(FlipTS, VsyncEstimated)` as a fallback, logging a single warning. This handles compositor stalls and display-link issues without silently inaccurate timestamps.

On Linux this race does not happen: `RecordFlip` queries DRM synchronously right after `FlipTS`, the kernel returns the matching vblank's count and timestamp before `OnsetForFlip` is called, and the callback fires immediately on the same `NotifyFlipped`.

22.8.5 7.5 Calibration

For sub-ms wire-side alignment to visual onset, you must measure the constant delay of your specific hardware:

1. Display a flashing white square in the corner of the screen.
2. Place a photodiode on the square; route both the photodiode signal and the TTL line to a recording amplifier with sub-ms sampling (most EEG amplifiers can do this directly).
3. Measure `delta = wire_HIGH_time − photodiode_first_light` over many trials. Take the median.
4. In your data analysis, subtract `delta` from every TTL marker timestamp.

After calibration, wire-side alignment to visual onset is bounded by hardware jitter (typically tens of µs) plus your measurement uncertainty. The `tests/Timing-Tests/` framework in this repository can drive this calibration measurement automatically.

22.8.6 7.6 What is *not* measured

The Stage 5 backends report when the **display controller** starts scanning out, not when LCD photons actually emit. Two layers of photon-side latency are invisible to any software-only path on these platforms:

- **Panel response** — 1–10 ms typical for IPS / VA / OLED panels at 60 Hz, longer for cheap monitors with image processing turned on ("game mode" usually disables this). Fixed offset per monitor.
- **Scanout phase** — at 60 Hz, the top of the screen receives its new pixels at vsync; the bottom receives them ~16 ms later (full scanout = one frame period). The reported timestamp is **vsync** = start-of-scanout, so a stimulus at the bottom of the screen has up to a frame period of additional latency before its photons emit. Place timing-critical stimuli near the top, or use a photodiode at their actual location.

These limits also apply to Apple's `addPresentedHandler` and Vulkan's `VK_EXT_present_timing` — they are intrinsic to the display pipeline, not specific to this implementation. Photon-precise onset still requires a photodiode.

22.8.7 7.7 Recommended pattern

```
// At experiment start:
mov.OnAtDisplay(media.Frame(N), func(o media.Onset) {
    // Always log the hardware-verified timestamp, even if you also pulse a
     ↪  wire:
    exp.Data.Add(trialID, "stim_onset_ns", o.TimestampNS, o.Source.String())

    // Pulse the trigger immediately. The pulse arrives ~0.5–5 ms after
     ↪  o.TimestampNS;
    // calibrate the constant and subtract in analysis.
    _ = ttl.Pulse(0, 5*time.Millisecond)
})
```

Two records per event: the timestamp (hardware-verified, sub-ms) goes into the data file; the wire pulse goes into the EEG amplifier. Align post-hoc by subtracting the calibrated wire-arrival delay.

22.9 8. Platform support (Stage 5)

Platform	Backend	Precision	Status	Prerequisites
macOS (Apple Silicon + Intel)	`CVDisplayLink` via `purego` (CoreVideo + libSystem)	`HardwareVerified`, sub-ms	**Shipped**	None — frameworks are part of the OS
Linux (X11 + Wayland)	`DRM_IOCTL_WAIT_VBLANK` via `syscall` on `/dev/dri/cardN`	`HardwareVerified`, sub-ms	**Shipped**	User must be in `video` group (default on most distros). At least one DRM device.
Windows	(none yet)	`VsyncEstimated` only	**Not yet shipped**	—
FreeBSD / OpenBSD / others	(none)	`VsyncEstimated` only	Falls back to `Fallback`	—

The startup log line confirms the chosen backend:

```
media: present backend: macOS CVDisplayLink (CoreVideo, hardware-verified)
 ↪  (precision=hardware-verified)
media: present backend: Linux DRM vblank (DRM_IOCTL_WAIT_VBLANK,
 ↪  hardware-verified) (precision=hardware-verified)
media: present backend: vsync-estimated (post-Present FlipTS, no OS
 ↪  integration) (precision=vsync-estimated)
```

If auto-detection fails (e.g., Linux without `video` group, missing `/dev/dri/cardN`), the manager logs the failure once and falls back gracefully:

```
present: Linux DRM vblank unavailable (open /dev/dri/cardN: permission
 ↪  denied); falling back to vsync-estimated
media: present backend: vsync-estimated (post-Present FlipTS, no OS
 ↪  integration) (precision=vsync-estimated)
```

22.9.1 8.1 What Windows users get today

Windows builds compile cleanly (the `vblank/autodetect_other.go` file matches `!darwin && !linux`) and use the `VsyncEstimated` fallback. This means:

- All multi-movie synchronisation works (§6 guarantees are independent of Stage 5).
- All `Movie[At]` look-ahead callbacks work (LookAhead is software-only, always available).
- All `OnAtDisplay` / `Display[Onset/Offset]` callbacks fire — but with `Source: VsyncEstimated`, accurate to vsync-period (~16 ms at 60 Hz) rather than sub-ms.
- A one-time warning is logged at the first display-callback registration:

  ```
  media: display-onset precision is vsync-period (~16.666ms at this
   ↪  refresh rate). Install a sub-ms presentation backend (Stage 5:
   ↪  Vulkan VK_EXT_present_timing or Metal addPresentedHandler) for
   ↪  hardware-verified timing.
  ```

Workaround for Windows users today: photodiode calibration (§7.5) using the `VsyncEstimated` timestamp as a reference. The constant offset includes one extra source of jitter (the ~vsync-period uncertainty), but for low-trial-rate studies this is often acceptable.

22.9.2 8.2 What needs to be done for Windows

To bring Windows up to parity with macOS / Linux, a Stage 5 backend needs to be added at `vblank/dxgi_windows.go` (next to the existing `vblank/cvdisplaylink_darwin.go` and `vblank/drm_linux.go`; `media/present` is a thin adapter over that package). The most promising APIs:

API	Pros	Cons
`IDXGISwapChain::GetFrameStatistics` (DXGI 1.0+)	Available on every Windows since Vista. Returns `PresentRefreshCount` + `PresentQPCTime`.	DXGI swapchain owned by SDL_Renderer. We can't construct our own without taking over rendering.
`DwmGetCompositionTimingInfo`	Returns `qpcCompose` and `qpcVBlank` for the desktop window manager. Doesn't require swapchain ownership.	DWM-specific (composition mode); behaviour with fullscreen exclusive may differ.

API	Pros	Cons
D3DKMTWaitForVerticalBlankEvent + D3DKMTGetScanLine (gdi32)	Direct kernel-mode access; very precise.	Undocumented headers, requires libloaderapi workaround for the kernel32 entrypoint.

Recommended path: **DwmGetCompositionTimingInfo** via purego on dwmapi.dll. It's the cleanest API that doesn't require swapchain ownership and works alongside SDL_Renderer.

Skeleton:

```
//go:build windows

package present

// Use purego.Dlopen("dwmapi.dll") to get a handle, then bind:
//   DwmGetCompositionTimingInfo(HWND, *DWM_TIMING_INFO) HRESULT
// where DWM_TIMING_INFO contains qpcVBlank (QueryPerformanceCounter
// units) for the most recent vblank. Convert QPC ticks to SDL ticks
// via QueryPerformanceFrequency at startup.

func newWindowsBackend(screen *apparatus.Screen) (Timer, error) { ... }
```

Estimated effort: ~250 lines of Go (similar shape to the existing macOS backend), plus ~100 lines of tests (purego symbol-loading, QPC → SDL-ticks conversion, struct layout pinning).

Until that lands, Windows experiments needing sub-ms wire alignment should rely on photodiode calibration (§7.5).

22.10 9. Mapping to PsyScope movie commands

Users porting PsyScript-based experiments will recognise this table. Volume-related rows are explicitly unsupported (audio is out of scope — see §1).

PsyScope construct	goxpyriment Go API
EventType Movie, Stimulus "clip.gv", MovieTag M1	gv, _ := gvvideo.LoadGVVideo("clip.gv"); media.NewMovie(mgr, gv, media.WithTag("M1"))
MovieRate 1.0	media.WithRate(1.0) / mov.SetRate(...)
Repeat 3 / Repeat -1	media.WithRepeat(3) / WithRepeat(-1)
FromTime "s:5"	media.WithFromTime(5*time.Second)
Flags Scale_X Scale_Y	media.WithScale(media.ScaleFit)
FadeIn 1000	media.WithFadeIn(1*time.Second)

PsyScope construct	goxpyriment Go API
`ClearType NO_CLEAR`	`media.WithPersistent(true)`
`MovieDo[ PLAY ]`	`mov.Play()`
`MovieDo[ PAUSE ]`	`mov.Pause()`
`MovieDo[ PAUSE ... LoopRegion=ms:N ]`	`mov.PauseWithLoop(±N*time.Millisecond)`
`MovieDo[ STOP ]`	`mov.Stop()`
`MovieDo[ SET Rate=R ]`	`mov.SetRate(R)`
`MovieDo[ SET Time=... ]`	`mov.SeekTime(...)`
`MovieDo[ SET Frame=N ]`	`mov.SeekFrame(N)`
`MovieDo[ GET Time/TimeMs/Frame/FrameCount/FPS/Counter/Repeat ]`	`mov.Time()`, `Frame()`, `FPS()`, `FrameCount()`, `LoopCounter()`, `Repeat()`
`MovieDo[ SET Volume=... ]`, `GET Volume`	**Not supported** (audio out of scope)
`Movie[ Done ]`	`mov.OnDone(fn)` or `<-mov.DoneCh()`
`Movie[ At THISMOVIE "f:N" ]`	`mov.OnAt(media.Frame(N), fn)`
`Movie[ At THISMOVIE "s:T,ms:U" ]`	`mov.OnAt(media.AtTime(d), fn)`
`Movie[ AtDisplay THISMOVIE "f:N" ]`	`mov.OnAtDisplay(media.Frame(N), fn)`
`Display[ Onset ]` / `Display[ Offset ]`	`mgr.OnDisplayOnset(name, fn)` / `OnDisplayOffset(name, fn)`
`SerialOut[ "0xFF" ]` action	call inside the OnDisplay/OnAtDisplay callback into `triggers.OutputTTLDevice`
`THISMOVIE` keyword	irrelevant — the closure already captures `mov`
`Instances: "-1"`	irrelevant — Go calls execute once

Burst-pause atomicity (the goxpyriment equivalent of three sequential `MovieDo[ PAUSE ... ]` script lines hitting the same draw cycle):

```
mgr.BeginBurst()
movieA.Pause(); movieB.Pause(); movieC.Pause()
mgr.EndBurst()
```

22.11 10. References

In this repository:

- examples/demo_sync_two_gv_movies/ — comprehensive walkthrough example
- tests/Timing-Tests/ — empirical timing-measurement framework

External / upstream:

- PsyScope Tahoe `MultiMovieSyncStrategy.md` — describes the original Apple-side architecture this package mirrors (CMTimebase, AVAssetReader, Metal addPresentedHandler).
- PsyScope Tahoe `CrossPlatformAnalysis.md` — feasibility study that led to the present `media/` package.

- `VK_EXT_present_timing` Vulkan spec — the future direct-Vulkan path (not yet implemented; would replace `SDL_Renderer` entirely).
- Apple `CVDisplayLink` reference — the API the macOS backend uses.
- Linux DRM uAPI `drm_wait_vblank` — the API the Linux backend uses.

Chapter 23

Running goxpyriment experiments in a web browser (WASM)

Status (2026-07-13): a real experiment runs end-to-end in the browser. `examples/parity_decision` — instructions screen, fixation cross, 10 RT trials with keyboard responses, feedback logic, and CSV + info-file download — was run to completion in headless Chrome, driven by DevTools-protocol key events. The data files that Chrome downloaded are well-formed goxpyriment output with correct browser metadata (`video_driver: emscripten`). See What remains for the open items.

> An earlier version of this document described a manual Emscripten workflow (hand-built `SDL3.js`, an `EXPORTED_FUNCTIONS` list, a custom `index.html`). That workflow is **obsolete**, replaced by the `wasmsdl` bundler described below. A condensed record of the old recipe is kept in the appendix as a reference for rebuilding the SDL wasm blob.

23.1 Architecture

Two WASM runtimes cooperate in the same page:

1. **`sdl.wasm`** — SDL3 + SDL3_ttf + SDL3_image + SDL3_mixer (with a Web Audio backend) compiled by Emscripten into a single module, loaded by `sdl.js`.
2. **`main.wasm`** — the Go experiment binary compiled with `GOOS=js GOARCH=wasm`, loaded by Go's `wasm_exec.js`.

The Go side talks to the Emscripten side through `go-sdl3`'s js bindings (`syscall/js` calls into `Module._SDL_*`), sharing one Emscripten heap.

23.1.1 Where the pieces live

Piece	Location
go-sdl3 fork with the js/wasm target	github.com/chrplr/go-sdl3-wasm, branch **wasm-render-fixes** (local clone: ~/00_git/go-sdl3-wasm). The module path is unchanged (github.com/Zyko0/go-sdl3), and goxpyriment's go.mod has a replace pointing at a pinned pseudo-version of the fork; vendor/ is kept in sync with GOWORK=off go mod vendor
Prebuilt sdl.js / sdl.wasm + bundler	cmd/wasmsdl in the fork — embeds the blobs and an index.html, and builds/serves a complete browser bundle. Rebuild recipe: .docker/emscripten-build/Dockerfile in the fork
goxpyriment js platform code	control/platform_js.go (URL-parameter flags, no dialog, audio device open), apparatus/screen_newscreen_js.go (canvas window), apparatus/screen_present_js.go (RAF-synced flips), results/output_file_wasm.go + results/data_wasm.go (session → one .zip download, or to the JATOS server), results/jatos_wasm.go (jatos.js bridge) — all build tag js
Export-list generator	cmd/gen-wasm-exports — scans go-sdl3's js bindings + goxpyriment's own calls **as compiled for GOOS=js** (files excluded by build constraints, and triggers/, are not counted), emits wasm/exported_functions.json for emcc -sEXPORTED_FUNCTIONS=@…, and **lists the go-sdl3 calls whose js bindings are still panic-stubs** (the remaining-work list)

23.1.2 The go-sdl3 replace — what dependents need to know

goxpyriment's go.mod pins the fork with a line of the form

```
replace github.com/Zyko0/go-sdl3 => github.com/chrplr/go-sdl3-wasm v0.1.2
```

(the authoritative version is whatever go.mod at the repo root says — it is bumped whenever the fork changes. It is a released tag today; while the fork is being developed between tags it is a pseudo-version of the form v0.1.2-0.<timestamp>-<hash> instead.)

Go applies replace directives **only in the main module**, so a project that imports goxpyriment as a dependency does *not* inherit this line. Such a project resolves upstream

`Zyko0/go-sdl3` — fine on desktop (behaviour is unchanged there), but the browser build will hit `panic("not implemented on js")` stubs. **To build your own experiment for the browser, copy the current `replace` line from goxpyriment's `go.mod` into your project's `go.mod`.** (It works because the fork keeps the upstream module path.)

When developing the fork itself, point the replace at the local clone instead (`go mod edit -replace github.com/Zyko0/go-sdl3=/home/cp983411/00_git/go-sdl3-wasm`, then `GOWORK=off go mod vendor`), and restore the pinned GitHub pseudo-version before committing (`git log -1 --format=%cd-%h --date=format:%Y%m%d%H%M%S` in the fork gives the timestamp/hash for a fresh pseudo-version; the base tag is `v0.1.1`, so the form is `v0.1.2-0.<timestamp>-<12-char-hash>`).

23.2 Building and serving an example

From the repo root:

```
# Build a self-contained bundle (index.html + sdl.js + sdl.wasm + main.wasm
# + wasm_exec.js) into _build/wasm/parity_decision/
make wasm-parity_decision

# Or build + serve on http://localhost:8080 in one step, then open
# http://localhost:8080/?s=1
make wasm-parity_decision-serve
```

The targets run the `wasmsdl` bundler out of the pinned go-sdl3 fork (`go run github.com/Zyko0/go-sdl3/cmd/wasmsdl …`), so they need no local fork clone and no Emscripten install. `wasmsdl serve` also sends the COOP/COEP headers that unlock the high-resolution browser clock (see "Timing in the browser").

WASM cannot be loaded from `file://` URLs; for the `build` output any static server works (`python3 -m http.server`), but plain servers don't send the COOP/COEP headers — timestamps then tick at ~100 µs instead of ~5 µs.

23.2.1 Per-example launcher pages

By default the bundle gets wasmsdl's page: a bare canvas that starts the experiment the moment the wasm loads. An example can ship its own landing page instead by adding `examples/NAME/web/index.html`; the `wasm-%` targets detect it and pass it to the bundler with `-html`. The bundler writes `sdl.js`, `sdl.wasm`, `wasm_exec.js` and `main.wasm` alongside it, which is all the page has to load.

`examples/Memory_span/web/` is the worked example (see its README). The same `web/index.html` path is what `docs/StandaloneExampleRepo.md` expects when an example is split out into its own repo with a GitHub Pages build, so a launcher written here travels with the example. A launcher page is worth writing when the experiment needs any of:

- **A participant-ID field.** `GetParticipantInfo` cannot run on js, and typing a URL parameter is not something to ask a participant to do. The page writes the field

back as `?s=<id>` with `history.replaceState` before starting Go, which is where `platformPrepareFlags` reads it from.
- **A user gesture before the first sound.** Browsers create the AudioContext suspended. A Start button doubles as the gesture that resumes it.
- **Instructions in HTML**, so the participant reads them before the canvas takes over the window.

Two things such a page must get right:

- **Start Go from a fresh task** (`setTimeout(…, 0)`), not from inside the click handler or a `requestAnimationFrame` callback. The experiment blocks its main goroutine for the whole session, and the wasm scheduler can only park it — returning to the JS event loop so DOM events keep reaching SDL — when it is not nested inside another callback's stack.
- **Do not resize the canvas with CSS after SDL has sized its window**, unless the aspect ratio is preserved. Emscripten maps pointer events through `getBoundingClientRect`, so a uniform `transform: scale()` is safe (and is how Memory_span's page fits the canvas into a short viewport), but `object-fit: contain` is not: it letterboxes the bitmap inside an element box that Emscripten still assumes the bitmap fills, and every click lands off target.

23.2.2 Session settings via URL parameters

There is no command line in a browser. On `GOOS=js`, `control.NewExperimentFromFlags` synthesizes `os.Args` from the page URL's query string, so the standard flags — and any experiment-specific flags the program registered — work unchanged:

`http://localhost:8080/?s=3&w` equivalent to: `-s 3 -w`

Keys that don't match a registered flag are ignored with a console note. The participant-info dialog never opens in the browser (it needs its own SDL window); when `s` is absent the subject ID defaults to 0, exactly like `-headless`.

Programs that call `control.GetParticipantInfo` directly are answered from the same query string, by `platformParticipantInfo` in `control/platform_js.go`. Each field is looked up by its own `Name`, falling back to the cached value and then to its `Default`:

`?subject_id=3&level=2&fullscreen=false`

`subject_id` also accepts the short `s` that works everywhere else. A `FieldSelect` takes either the exact option text or a **1-based index** into its options — the escape hatch for options like `"Level 2 — Name"`, which are miserable to type into a URL. A `FieldCheckbox` takes `true`/`false`/`1`/`0`/ `yes`/`no`, and a bare key means true. Every resolved value is logged to the console, since there is no dialog to read the settings back off afterwards.

23.2.3 Headless verification (no display needed)

```
cd /tmp/hello_bundle && python3 -m http.server 8123 &
google-chrome --headless=new --use-gl=swiftshader --no-sandbox \
  --window-size=1100,850 --virtual-time-budget=6000 \
  --screenshot=/tmp/hello.png --enable-logging=stderr \
```

```
  "http://localhost:8123/?s=1" 2>&1 | grep "INFO:CONSOLE"
```

Go panics (with full stack traces) appear as `INFO:CONSOLE` lines; the screenshot shows what rendered. Each remaining stub panics with its binding's file:line, which tells you exactly what to un-stub next.

23.3 Publishing to downloads.pallier.org

Every tagged release publishes 86 of the 98 examples (as of 2026-09-17) as browser builds, served from the Cloudflare R2 bucket. A visitor follows a **Run** link on https://downloads.pallier.org/builds/latest/ and the experiment starts — no download, no install, no Gatekeeper.

```
builds/{sha}/wasm/_runtime/{sdl.js,sdl.wasm,wasm_exec.js}   shared, 5.3 MB
builds/{sha}/wasm/{app}/index.html                          launcher page
builds/{sha}/wasm/{app}/main.wasm                           the experiment
```

`examples/installers/build-wasm-apps.sh` drives it, and `cmd/gen-wasm-launcher` writes the pages. Three things about that pipeline are worth knowing:

- **The runtime is shared.** `wasmsdl` writes five files per bundle, but three of them are byte-identical everywhere. They are published once and every page loads them from `../_runtime/`, which requires setting Emscripten's `Module.locateFile` — the glue fetches `sdl.wasm` by name, not through a tag, so without that line a page loads and then cannot find its runtime. Sharing saves ~415 MB per build.
- **An example may keep its own launcher.** When `examples/NAME/web/index.html` exists it is used as the base and only the runtime paths are rewritten, so `Memory_span` and `Reading-1` keep their bespoke instructions. Everything else gets a page generated from the example's `meta.yaml`.
- **12 examples are excluded**, listed with reasons in `examples/installers/wasm-skip.txt`. They read stimuli from disk, which compiles for js and fails at run time — so the list is maintained by hand rather than inferred from a build. Converting one to `//go:embed` and deleting its line publishes it.

The bucket sends the COOP/COEP headers through a Cloudflare response-header rule, so published experiments get the ~5 µs clock; see `docs/copy_apps_to_cloudflare_R2.md` for the rule and the storage budget.

23.4 Running on JATOS

A browser build can run as a study on a JATOS server, which stores the results instead of having the participant download them: `make jatos-NAME` builds the study archive. **How to deploy and use it is in Deploying on a JATOS server**; what follows is for maintainers.

- **Detection.** `results.JatosAvailable()` checks for the global `jatos` object, which only exists on a page that loaded `jatos.js` — served by the JATOS server itself,

not shipped in the study. Without it, every path is the plain browser behaviour above.

- **One promise chain.** Every jatos.js call returns a promise. They are queued on one chain (`results/jatos_wasm.go`), so appends arrive in order and the trial loop never waits on the network; `Finalize` waits for the whole chain. The chain is built from plain JS functions only (bound methods), never `js.FuncOf`: a Go callback cannot run once the Go program has exited, and a crashed session exits with appends still queued.
- **Parameters.** JATOS redirects the study link to the component's own URL and keeps the link's query string only in `jatos.urlQueryParameters`; `control.queryPairs` merges it with `location.search`.
- **The page.** `gen-wasm-launcher -jatos` renders the launcher with `jatos.js`, no participant-ID box, Start enabled only after `jatos.onLoad`, and a handler on `go.run`'s promise that closes the study run from the outcome the Go side left in `window.goxpyrimentOutcome`. Hand-written `web/index.html` pages are not adapted; the generated page is used for every example.
- **The archive.** `cmd/gen-jatos-jzip` writes the `.jas` study description (schema version 3, taken from JATOS's own test archive) and zips the bundle. UUIDs are derived from the example name, so importing a rebuilt archive updates the existing study; the assets folder is `goxpy_<name>` to avoid collisions on a shared server. The runtime files are bundled per study (+5.3 MB): JATOS studies cannot share an assets folder.
- **Server behaviour, measured on cortex.jatos.org (JATOS 3.11, 2026-10-04).** The component page has no COOP/COEP headers (`crossOriginIsolated` is false, so timestamps tick at ~100 µs rather than ~5 µs, see below) and no CSP. `.wasm` files are served as `application/octet-stream` and uncompressed, which `instantiateStreaming` rejects: the JATOS page compiles `main.wasm` from an ArrayBuffer instead (`sdl.js` falls back by itself). Result-file names containing whitespace or punctuation such as `:`, `/`, `,` or `!` are refused with `400 Bad filename` (the full set is `IOUtils.REGEX_ILLEGAL_IN_FILENAME` in the JATOS source); `results.JatosFilename` replaces them with `_`.

23.5 What works today (verified 2026-07-13)

- `GOOS=js GOARCH=wasm go build` for the library (all packages except `triggers/`, which needs a serial port and stays desktop-only) and examples.
- `wasmsdl build` bundles a goxpyriment example; the page boots both WASM modules.
- Full `control.Experiment` initialization: SDL + TTF init, canvas window + renderer, embedded font via the in-memory IOStream path (`IOFromDynamicMem/Write/Seek`), keyboard/mouse wiring, system- and display-info gathering, data-file creation.
- Text rendering (`stimuli.TextBox/TextLine` with wrap alignment and string metrics), fixation cross, `SetLogicalSize` letterboxing.
- The full trial machinery inside `exp.Run`: `ShowInstructions`, `Blank/ShowTimed/Wait`, `ShowAndGetRT` with hardware-timestamp RTs, `exp.Data.Add`, ESC/quit handling via `pumpFrame` (`HasEvent/GetKeyboardState`).
- Keyboard events reach SDL from the canvas; RTs come back as plausible mil-

lisecond values (timestamp *granularity* not yet measured — see below).
- `results` output: at experiment end the browser downloads **one** archive holding the `.csv` and the `-info.txt` (verified: the files land intact and unzip to what a native run writes). It must stay one download — see "Why the results arrive as a .zip" below.
- Audio playback: the buzzer/feedback sounds, `PlaySync`/`PlayAsync`, and `Tone` all work (see "Audio in the browser" below); confirmed by ear in an interactive session, not just headlessly.
- Mouse-driven responses: `examples/Memory_span` runs in the browser with its on-screen button grid, including `Renderer.RenderCoordinatesFromWindow` mapping window pixels into a `SetLogicalSize` letterboxed space (verified 2026-08-27 by driving headless Chrome over the DevTools protocol: sequence presentation, click responses, correct/incorrect feedback, and the staircase all behave). The end-of-session download was verified for `parity_decision`, not re-verified for a full 30-trial Memory_span run.

23.5.1 Why the results arrive as a .zip

`results.DataFile.Finalize` writes **one** file in the browser: a `.zip` containing the `.csv` and the `-info.txt` under the names the desktop build gives them. Unzipping a browser session yields byte-for-byte what a native run writes, so nothing downstream has to know where the data came from.

This is not packaging convenience — it is a correctness fix. Until 2026-08-29 the browser build fired the two downloads back to back, and the second one was routinely lost: browser sessions arrived with their metadata and no results.

The mechanism, measured against Chrome's own `downloads` table (`~/.config/google-chrome/Default/History`): when the browser is set to *ask where to save each file*, two downloads requested milliseconds apart are serialized into two modal Save-As dialogs, and only the first is shown. The second stays queued, and is cancelled the moment the page goes away — which is precisely what a participant does when the session ends. Chrome records the casualty as `state=CANCELLED`, `interrupt_reason=USER_CANCELED`, `0` bytes received and no chosen filename. Because `Finalize` wrote the info file first, the metadata always survived and the results never did.

Two things that look like the cause are not, and were each disproven by experiment before this fix landed:

- **The wasm instance exiting.** A probe that kept the module alive for two minutes after `Experiment.End()` behaved no differently from one that exited immediately: in the immediate case the Save-As dialog still appeared and the file still completed 34 s *after* the Go program had returned from `main`. What cancels a pending download is the **document** being destroyed (navigating away, closing the tab), not the Go program finishing.
- **Revoking the object URL right after `click()`.** It looks like the classic antipattern. A controlled test (one download pair per page load, no user gesture) delivered 5/5 with the immediate revoke and 3/5 with a deferred one. The revoke stays where it is.

So: **never add a second download to the browser path.** If a future experiment needs to hand back another artefact, add it to the archive. (Under JATOS there is no download: the files are uploaded — see "Running on JATOS" above.)

23.5.2 The key design points

- **Blocking experiment code works in the browser — but only on the main goroutine.** Go's wasm scheduler parks a blocked main goroutine and returns to the JS event loop, so DOM events fire and SDL sees them. Code that blocks inside a `js.FuncOf` callback instead wedges the tab (the scheduler spins in `findRunnable`). Consequently `sdl.RunLoop` on js paces frames with `requestAnimationFrame` but runs the experiment logic on the goroutine that called `Run`; the RAF callback only signals a channel. (`SDL_Delay` is a no-op on js — it would busy-wait; use `time.Sleep`.)
- **Assets must be embedded.** There is no filesystem; `//go:embed` + `FontFromMemory` / `sdl.IOFromBytes` is the portable path (and was already the recommended pattern on desktop). Path-based loaders fail on js.

23.6 What remains for a complete port

All five phases of the roadmap are **done**: branch consolidation and vendor/fork unification; IOStream + info bindings, URL-parameter setup, demo_hello_world rendering; `parity_decision` running end-to-end (keyboard trials, RTs, CSV download) under the redesigned main-goroutine `RunLoop`; timestamp granularity measured and fixed; flips aligned to `requestAnimationFrame` with measured 60.00 Hz pacing and zero dropped frames; audio playback; and packaging (`make wasm-NAME` / `wasm-NAME-serve` targets, obsolete artifacts removed, `GOOS=js` builds in CI via `.github/workflows/go-build.yml`).

Open decision: PR the fork's js/wasm work upstream to `Zyko0/go-sdl3`, or keep maintaining the fork.

Fixed since: `Experiment.Initialize` recorded the audio device name via `AudioDeviceID.Name()`, whose js binding is still a panic-stub — so every browser experiment aborted during initialization, before its first frame, with `panic: not implemented on js`. The call now goes through a build-tagged `platformAudioDeviceName` (`control/platform_js.go`) that reports `"Web Audio (browser)"` instead; there is no physical device to name when SDL routes everything through one Web Audio context. (Fixed 2026-08-27. The stub itself is still worth un-stubbing in the fork, but nothing depends on it now.)

Known gaps:

- Fullscreen and multi-monitor are meaningless in a canvas; the js `NewScreen` always opens 1024×768 (or the requested size). A "resize canvas to viewport" option would be nicer for participants.
- Less-common APIs are still stubbed (gamepad/joystick enumeration, audio recording, `WaitEvent`/`WaitEventTimeout`, video playback). They panic with a clear console message when hit. (`SetRenderLogicalPresentation` used to be on

this list; it is implemented in the vendored fork and present in `wasm/exported_functions.json`.)
- `GetParticipantInfo` never opens its dialog on js — it cannot, needing its own SDL window and an `sdl.Quit()` afterwards — and is answered from URL parameters instead (see "Session settings via URL parameters"). Until 2026-08-31 there was no such check, and all eleven examples that call it directly died in the browser inside the dialog at `Renderer.CurrentOutputSize`, a panic-stub, showing the participant a black page after they pressed Start.

23.7 Audio in the browser

Audio playback works (verified 2026-07-13): `platformInitAudio` on js opens the default playback device exactly like on desktop, and the whole goxpyriment audio API — `exp.Audio.PlayBuzzer/PlayCorrect/PlaySync/PlayAsync`, `Sound.PreloadDevice/Play/Wait`, `Tone` — runs on SDL's Emscripten Web Audio backend. Verified in `parity_decision`: the device opens as 48 kHz stereo F32LE with a 2048-frame buffer, the AudioContext reaches `running`, and the buzzer feedback plays on incorrect trials.

Browser specifics:

- **First sound needs a user gesture.** Browsers create the AudioContext suspended; SDL's Emscripten backend resumes it automatically on the first click or keypress. Any experiment that starts with a "press SPACE" screen satisfies this without extra code. Sounds triggered *before* any interaction stay silent until the first one.
- **Latency is higher than desktop.** The Emscripten device reports a 2048-frame buffer at 48 kHz ≈ 43 ms; `SetAudioSampleFrames` (an SDL hint) is not honoured the same way by the Web Audio backend. Treat audio onset timing as approximate in the browser; `PlaySyncedWithFlip`'s desktop guarantees do not transfer.
- If the device cannot be opened, the experiment continues with a silent no-op `AudioManager` instead of crashing (browser audio support varies).

23.8 Timing in the browser

23.8.1 Clock / timestamp resolution (measured 2026-07-13, headless Chrome)

SDL timestamps in the browser (`sdl.TicksNS` and input-event timestamps, i.e. what `ShowAndGetRT` / `GetKeyEventTS` use) tick at:

Configuration	Resolution
plain page	~100 µs
cross-origin isolated page (COOP/COEP headers)	~5 µs

`wasmsdl serve` sends the COOP/COEP headers, so served experiments get the ~5 µs clock; any other hosting needs `Cross-Origin-Opener-Policy: same-origin` and `Cross-Origin-Embedder-Policy: require-corp` to match.

These numbers required a fix in the fork (commit `5eb7455`): out of the box, **every SDL timestamp was quantized to 1 ms**, because SDL3's `SDL_GetPerformanceCounter` probes `CLOCK_MONOTONIC_RAW`, Emscripten's WASI shim rejects that clock id, and SDL silently falls back to `gettimeofday = Date.now()`. The fork's `sdl.js` asset patches `emscripten_date_now` to `performance.timeOrigin + performance.now()`, and the Dockerfile re-applies the patch on rebuild.

Note: Go's `time.Now()` on js still has 1 ms resolution (the wall clock is `Date.now()`). This is one more reason for the existing rule: RTs come from the SDL event clock, never wall-clock deltas.

23.8.2 Frame pacing (measured 2026-07-13, headless Chrome)

On js, `Screen.Update()` (and therefore `Flip`/`FlipTS`/`Show`) presents to the canvas and then parks until the browser's next `requestAnimationFrame` tick — the browser's VSYNC equivalent (implemented in `apparatus/screen_present_js.go` on top of `sdl.WaitAnimationFrame` in the fork). This is required for correctness, not just pacing: canvas updates are only composited when the page yields, and the desktop busy-wait would freeze the tab. A 250 ms fallback tick prevents deadlock in background tabs (where browsers throttle RAF) — but run experiments in a **focused, foreground tab**.

Measured flip-loop intervals (299 frames, alternating full-screen colors):

Loop	mean	SD	min–max	dropped frames
`Update` loop	16.666 ms (60.00 Hz)	0.12 ms	16.1–17.3 ms	0
`Flip` loop	16.666 ms (60.00 Hz)	0.11 ms	16.3–17.1 ms	0

Caveat: headless Chrome ticks a virtual 60 Hz compositor; on real hardware the rate follows the display (e.g. 120 Hz laptop panels) and jitter depends on system load. The hardware-locked VSYNC timing available on desktop (DRM ioctl, CoreVideo) is not available in a browser, and there is no photodiode validation of actual pixel onset yet. Experiments that require sub-millisecond stimulus onset precision (rapid RSVP, subliminal priming) should be run natively. Cognitive tasks (choice RT, memory, attention) work fine.

23.9 Notes for developers

- **Un-stubbing a go-sdl3 js binding** is usually just deleting the `panic("not implemented on js")` first line, *but audit the marshalling*: the fork's `CLAUDE.md` documents the rules (opaque handles vs input structs vs out-params, `float32` passed directly, `size_t` as i32 not BigInt, u64 returns via `internal.GetInt64`, struct returns via `internal.NewObject`, `HEAPU8` fetched fresh).
- **`Module._SDL_Foo is not a function`** at runtime means the C symbol is missing from the `sdl.wasm` export list — regenerate the list with `go run ./cmd/gen-wasm-exports/` and relink the blob (`.docker/emscripten-build/Dockerfile` in the fork).

- **After changing the fork**, re-sync goxpyriment's vendor tree: `GOWORK=off go mod vendor` at the repo root.

23.10 Appendix: historical manual Emscripten recipe

The early-2026 attempt built the SDL blob by hand. Kept here (condensed) as a reference; the current blob is built by `.docker/emscripten-build/Dockerfile` in the fork and additionally bundles SDL3_image and SDL3_mixer.

```
# Emscripten SDK
git clone https://github.com/emscripten-core/emsdk.git && cd emsdk
./emsdk install latest && ./emsdk activate latest && source ./emsdk_env.sh

# SDL3 (static)
git clone https://github.com/libsdl-org/SDL.git -b release-3.2.x SDL3-src
emcmake cmake -S SDL3-src -B SDL3-wasm -DCMAKE_BUILD_TYPE=Release \
  -DSDL_SHARED=OFF -DSDL_STATIC=ON -DSDL_TESTS=OFF -DSDL_EXAMPLES=OFF
emmake make -C SDL3-wasm -j$(nproc)
cmake --install SDL3-wasm --prefix "$PREFIX"

# FreeType, then SDL3_ttf against the same prefix
emcmake cmake -S freetype-src -B freetype-wasm -DCMAKE_BUILD_TYPE=Release
emmake make -C freetype-wasm -j$(nproc) && cmake --install freetype-wasm
 ↪  --prefix "$PREFIX"
emcmake cmake -S SDL3_ttf-src -B SDL3_ttf-wasm -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_PREFIX_PATH="$PREFIX" -DSDL3TTF_HARFBUZZ=OFF -DSDL3TTF_
   ↪  SAMPLES=OFF
emmake make -C SDL3_ttf-wasm -j$(nproc) && cmake --install SDL3_ttf-wasm
 ↪  --prefix "$PREFIX"

# Link one module; export list generated by cmd/gen-wasm-exports
emcc -o sdl.js "$PREFIX"/lib/libSDL3.a "$PREFIX"/lib/libSDL3_ttf.a
 ↪  "$PREFIX"/lib/libfreetype.a \
  -s MODULARIZE=0 -s ENVIRONMENT=web -s ALLOW_TABLE_GROWTH=1 \
  -s EXPORTED_RUNTIME_
   ↪  METHODS='["HEAPU8","stackAlloc","stackSave","stackRestore","getValue
   ↪  ","setValue","stringToUTF8OnStack","UTF8ToString","addFunction"]' \
  -s EXPORTED_FUNCTIONS=@wasm/exported_functions.json
```

Pitfalls recorded at the time: use `-DCMAKE_PREFIX_PATH` (where `SDL3Config.cmake` was installed), not `-DSDL3_DIR` at the raw build dir; SDL3_ttf pins a minimum SDL3 version — if they mismatch, rebuild SDL3 from the tag SDL3_ttf requires; all libraries must be linked into **one** module because the Go bindings share a single Emscripten heap; force-link static archives (`-Wl,--whole-archive`) or `EXPORT_ALL` drops unreferenced symbols.

Chapter 24

Deploying an experiment on a JATOS server

JATOS (Just Another Tool for Online Studies) is a server that runs experiments in participants' browsers and stores their results on the server. A goxpyriment experiment can be packaged as a JATOS study and run there without changing its code: instead of each participant downloading a `.zip` of results and sending it to you, the data arrives on the server, block by block, as the session runs.

This page explains how to build the study, put it on a server, recruit participants and get the data back. For how browser builds work in general, see WASM build.

In short

1. Make sure the experiment runs in a browser: stimuli embedded, `exp.Data.Save()` after each block, tested with `make wasm-NAME-serve`.
2. Build the study archive: `make jatos-NAME` → `_build/jatos/NAME.jzip`.
3. Import it in JATOS with **Import study**.
4. Run it yourself with the **Run** button and check the result arrives.
5. Create a study link and send it to participants.
6. Download the results with **Export Results**.

24.1 What you need

- A JATOS server and an account on it. Your institution may run one; otherwise MindProbe offers free hosting for researchers, and JATOS can be installed on any machine (see the JATOS documentation). Studies built here have been tested on JATOS 3.11.
- A clone of this repository and Go (the same setup as for building the examples; see Installation). No other tools: the build produces the study archive itself.
- An experiment that runs in a browser (next section).

24.1.1 JATOS vocabulary

The JATOS interface uses a few words of its own:

- **Study** — one experiment as it exists on the server. Each `.jzip` you import is one study.
- **Component** — one page of a study. A goxpyriment study has a single component, the experiment; you can add others (a consent form, a questionnaire) in JATOS.
- **Worker** — JATOS's word for a participant. You are a worker too when you test your own study.
- **Study link** — the URL you give participants. Links are grouped into **batches**; the imported study comes with one batch, `Default`, which is all you need.
- **Result** — one run of the study by one worker. It holds the *result data* (text) and the *result files* (uploaded files); see Getting the results.

24.2 1. Make sure the experiment runs in a browser

JATOS runs the browser (WebAssembly) build of the experiment, so anything that works with `make wasm-NAME-serve` works on JATOS. Before deploying:

1. **Embed every stimulus file** (images, sounds, fonts, lists) with `//go:embed`. A browser has no file system: an experiment that reads files from disk at run time compiles, then fails when it starts. The examples listed in `examples/installers/wasm-skip.txt` are in that situation.
2. **Call `exp.Data.Save()` after every block.** Online, it is what sends the block's rows to the server. If a participant closes the tab halfway, you keep every block saved so far; without `Save()` calls, nothing reaches the server until the very end.
3. **Try it locally:**

   ```
   make wasm-Simon_task-serve      # then open http://localhost:8080/?s=1
   ```

   Run through it once and check the `.zip` it downloads at the end.

The experiment must live in its own directory under `examples/`, with a `main.go` and a `meta.yaml`. The directory name is the `NAME` in `make wasm-NAME-serve` and `make jatos-NAME`, and becomes the study's title in JATOS.

24.2.1 If your experiment is in its own folder

If you built your experiment in a folder of your own, as in Creating Your Own Experiment, move a copy into the repository first. The browser build needs the repository: it relies on the browser version of SDL that the repository includes and on its `make` targets.

1. Clone the repository, if you have not already, and go into it:

   ```
   git clone https://github.com/chrplr/goxpyriment
   cd goxpyriment
   ```

2. Copy your folder into `examples/`, **leaving out `go.mod` and `go.sum`**: the `examples` directory already has its own, which cover every goxpyriment package. Keep the name short, without spaces (`my_experiment`, not `My Experiment`). Below, `../my_experiment` is your folder; adjust the path to wherever it is.

```
mkdir examples/my_experiment
cp -r ../my_experiment/* examples/my_experiment/
rm -f examples/my_experiment/go.mod examples/my_experiment/go.sum
```

3. Create `examples/my_experiment/meta.yaml` with three lines:

   ```
   category: experiment
   description: Short description shown to participants and in JATOS
   reference: (a citation, or leave empty)
   ```

   The `description:` line is shown on the page participants open and as the study description in JATOS.

4. Check it in the browser with `make wasm-my_experiment-serve`, then continue with step 2 using `make jatos-my_experiment`.

Edit the copy under `examples/` from then on, or copy your changes over before each rebuild.

24.3 2. Build the study archive

```
make jatos-Simon_task
```

This writes `_build/jatos/Simon_task.jzip` (about 3 MB). A `.jzip` is JATOS's own study-archive format: the files of the study plus a description of it. The archive holds:

File	Role
`index.html`	The page participants see: a short description and a **Start** button
`main.wasm`	Your experiment, compiled for the browser
`sdl.js`, `sdl.wasm`	The SDL3 graphics/audio runtime
`wasm_exec.js`	Go's browser support script

On Windows, run the same command from Git Bash, or run the script it calls directly: `bash examples/installers/build-jatos-study.sh Simon_task`.

24.4 3. Import it into JATOS

1. Log in to your JATOS server.
2. Use **Import study** and select the `.jzip`.
3. The study appears in your study list under the example's name, with one component (the experiment). Its files are stored on the server in a folder named `goxpy_<name>`.

Updating the study. Fix the experiment, rebuild with `make jatos-NAME`, and import the new `.jzip`. Because every build of the same example carries the same study identifiers, JATOS recognises it and asks whether to overwrite the existing study and its files: answer yes to both. The study's links stay valid.

24.5 4. Try it yourself first

Before sending a link to anyone, run the study as a participant would:

1. Open the study in JATOS and click its **Run** button. This runs the study as your own *JATOS worker*, the worker JATOS keeps for researchers testing their studies. No study link is needed.
2. Click **Start** and go through the experiment to the end.
3. Go back to the study and open **Results**: your run should be there, `FINISHED`, with its result data and its two result files (see Getting the results). Download the files and check them as you would after a lab session.

Delete this test result afterwards (select it under **Results** and delete it) so it is not mixed with real data.

24.6 5. Create study links

Open the study and go to **Study links**. JATOS offers several kinds of link; the usual choices are:

Link type	Use it for
Personal Single	One link per participant, usable once. Best for lab-recruited participants.
General Single	One link for everybody, each browser can run it once.
General Multiple	One link, any number of runs. Use it for piloting and testing.

A *Single* link really is single: once it has been used, opening it again shows "Study can be done only once" — even after a failed or abandoned run. Test with a *Multiple* link.

24.6.1 Passing parameters

Anything appended to the link after `?` reaches the experiment exactly as on the command line:

```
https://jatos.example.org/publix/AbCdEfGh?s=12            subject 12
https://jatos.example.org/publix/AbCdEfGh?s=12&w          …in a 1024×768
 ↪ window
https://jatos.example.org/publix/AbCdEfGh?level=2         any flag the
 ↪ program defines
```

- **Subject ID.** Without `s`, the participant is numbered with the JATOS *study result ID*, which is unique for each run on the server. File names and the `subject_id` column then never collide, and the number matches the row in JATOS's results table.
- **Recruitment platforms** (Prolific, MTurk, SONA…) append their own parameters, for example `?PROLIFIC_PID=…`. The experiment ignores parameters it has no flag for, and JATOS records them with each result. For Prolific, JATOS can also redirect participants back with a completion code: set the study's *End redirect URL* in the study properties.
- **Participant information.** Fields of `exp.GetParticipantInfo` are filled from parameters of the same name (`?age=31&handedness=2`), as in any browser build. No dialog is shown online.

24.7 6. What participants see

1. The study link opens a page with the experiment's name, its description and a **Start** button. There is no participant-ID box.
2. Clicking **Start** runs the experiment in the browser window, as on a desktop. The click is needed: browsers allow sound and keyboard focus only after a user action.
3. At the end, JATOS's end page is shown (or the participant is redirected, if the study has an end redirect URL). Nothing is downloaded.

Ask participants to use a computer with a keyboard, Chrome, Edge or Firefox, and to keep the tab in the foreground: browsers slow down background tabs.

24.8 7. Getting the results

Open the study and go to **Results**. Each run is a row, with its state (`FINISHED`, `FAIL`, `DATA_RETRIEVED`…), the worker, the duration and the parameters it was started with. Two kinds of data are stored for each run:

In JATOS	Contents	When it is written
Result data	The CSV text, one block appended after another	At every `exp.Data.Save()`
Result files	`<name>.csv` and `<name>-info.txt`, the same two files a desktop run writes	At the end of the session

- Use the **result files** for analysis: they are byte-for-byte what a desktop session produces, so existing analysis scripts work unchanged. One difference: JATOS refuses spaces and some punctuation in file names, so those characters are replaced by `_` — `Simon Task_sub-012_….csv` arrives as `Simon_Task_sub-012_….csv`.
- The **result data** is the safety net. It is there even for a participant who closed the tab halfway, and it is what you see when you open a result in the JATOS interface. Its content is the CSV: header line, then the rows of each saved block.

To download, select the runs and use **Export Results**. You can export the result data (text), the result files (a zip of folders, one per run), or everything at once. JATOS also has a REST API to fetch results from a script; see the JATOS API documentation.

24.8.1 When something goes wrong

What happened	Participant sees	In JATOS
Session completed	JATOS end page	`FINISHED`, result data and both result files
Participant pressed **ESC**	JATOS end page	`FINISHED`, with the blocks saved before ESC
Tab closed or browser crashed	—	Run left unfinished; result data holds the saved blocks
Experiment error	An error message, which stays on screen	`FAIL` with the error message; result data holds the saved blocks
Results could not be sent at the end	A message asking them to send you the `.zip` their browser just downloaded	`FAIL`, with the reason

The last case is a fallback: jatos.js retries each failed request several times first, and only if the server still cannot be reached does the browser download the usual `.zip`, so that the data is not lost.

24.9 Limits and caveats

- **Timing.** In a browser, onsets are tied to the screen refresh as on desktop, but timestamps have ~0.1 ms resolution rather than the ~5 µs of a local or cross-origin-isolated page: JATOS does not send the headers that would enable the finer clock. This is far below behavioural response variability, but do not use an online study for sub-millisecond measurements.
- **Data size.** By default a JATOS server accepts up to 5 MB of result data per run (`jatos.resultData.maxSize`) and 30 MB per result file (`jatos.resultUploads.maxFileSize`). Ordinary behavioural CSVs are far smaller; an experiment logging every frame or mouse sample might not be.
- **Reloading the page** is refused: JATOS ends the run as failed rather than restarting the experiment and mixing two attempts in one result.
- **Custom launcher pages.** Examples that ship their own `web/index.html` (`Memory_span`, `Reading-1`) are deployed with the standard page; their extra instructions are not shown on JATOS.
- **Download size.** Each study carries its own copy of the SDL runtime, and JATOS serves the files uncompressed, so a participant downloads about 11 MB before Start is enabled (the `.jzip` itself is ~3 MB).

- **Several components.** The experiment can be one component among others (a consent form before it, a questionnaire after): when it finishes, JATOS moves on to the next component. Add the other components in the JATOS interface.

24.10 Troubleshooting

- **The console shows "wasm streaming compile failed … Incorrect response MIME type", then "falling back to ArrayBuffer instantiation".** Harmless. JATOS serves `.wasm` files with a generic content type; the SDL runtime retries in a way that does not care, and the experiment's own module is loaded that way from the start.
- **The page stays on "Loading…".** Open the browser's developer console (F12). A message about a file that cannot be found usually means the study's files were not imported completely: import the `.jzip` again.
- **"Study can be done only once".** The link is a *Single* link that has already been used, possibly by you. Create a new one, or test with a *General Multiple* link.
- **The experiment starts but a stimulus is missing, or it stops at once.** The experiment probably reads a file from disk; see step 1. Check with `make wasm-NAME-serve`, which behaves the same way.
- **No result files, only result data.** The session did not reach `exp.End()`: the participant closed the tab, or the experiment stopped with an error (the run is then marked `FAIL` with the message).

Chapter 25

Packaging Your GoXpyriment Experiment

This guide explains how to package and distribute your `goxpyriment` experiments as standalone binary executables and installers (e.g., `.exe`, `.deb`, `.rpm`, `.zip`) for different platforms and architectures.

> To split **one example from this repo's `examples/`** into its own published GitHub repository — with release CI and an optional in-browser (WebAssembly) build on GitHub Pages — see Publishing an example as a standalone repository.

25.1 Prerequisites

1. **GoReleaser**: The primary tool for automating the build and release process. Install it from goreleaser.com.
2. **No SDL3 installation needed**: `goxpyriment` uses `github.com/Zyko0/go-sdl3/bin/binsdl` which embeds pre-built SDL3 and SDL3_ttf libraries for Linux, macOS, and Windows (both amd64 and arm64) directly inside the Go binary. There is nothing extra to bundle or install.

25.2 Project Structure Recommendation

For a distributable experiment, keep your assets organized:

```
my_experiment/
├── assets/             # Large files (videos, high-res images)
├── assets_embed/       # Small files (fonts, icons, sound effects)
├── main.go             # Your experiment logic
├── go.mod
└── .goreleaser.yaml    # GoReleaser configuration
```

25.3 Step 1: Handling Assets

25.3.1 A. Embedding (Small Assets)

For small files (fonts, small sounds), use Go's `//go:embed` directive. This includes the assets directly inside the binary.

```
//go:embed assets_embed/font.ttf
var MyFont []byte
// ...
exp.LoadFontFromMemory(MyFont, 32)
```

25.3.2 B. Bundling (Very Large Assets)

For assets that are too large to embed in the binary (e.g., video files), bundle them alongside the executable and look for them relative to the executable path at runtime. For images, CSVs, audio clips, and similar small-to-medium files, prefer embedding (`//go:embed`) over bundling.

25.4 Step 2: GoReleaser Configuration

Create a `.goreleaser.yaml` in your project root. Here is a template based on the `retinotopy` example:

```
version: 2

before:
  hooks:
    - go mod tidy

builds:
  - id: experiment
    env:
      - CGO_ENABLED=0  # Required for easy cross-compilation with purego
    goos:
      - linux
      - windows
      - darwin
    goarch:
      - amd64
      - arm64
    main: ./main.go
    binary: my_experiment

archives:
  - id: default
    name_template: "{{ .ProjectName }}_{{ .Os }}_{{ .Arch }}"
    formats: [zip]
```

```
    files:
      - assets/**/*
      - README.md

nfpms:
  - id: linux-packages
    package_name: my-experiment
    maintainer: Your Name <you@example.com>
    description: My Awesome Behavioral Experiment
    formats:
      - deb
      - rpm
    contents:
      - src: assets/**/*
        dst: /usr/share/my-experiment/assets
```

25.5 Step 3: Building

25.5.1 Local Snapshot Build

To test the packaging locally without creating a GitHub release:

```
goreleaser build --snapshot --clean
```

The resulting binaries and packages will be in the `dist/` folder.

25.5.2 Official Release

1. Commit your changes.
2. Tag your repository: `git tag -a v1.0.0 -m "First release"`
3. Push the tag: `git push origin v1.0.0`
4. Run GoReleaser (or use a GitHub Action like `retinotopy_build.yml`):

```
goreleaser release --clean
```

25.6 Platform-Specific Notes

25.6.1 Windows

SDL3 is embedded in the binary by `binsdl`, so users can simply unzip and run the experiment without installing anything extra.

25.6.2 Linux

Generating `.deb` or `.rpm` files via `nfpms` (included in GoReleaser) allows users to install your experiment using `sudo apt install ./my_experiment.deb`.

25.6.3 macOS

For macOS, GoReleaser creates a binary. To create a proper `.app` bundle or `.dmg`, you may need additional tools like `gon` for notarization or custom scripts to create the folder structure: `MyExperiment.app/Contents/MacOS/my_experiment`.

25.7 Using GitHub Actions

You can automate this entire process by using the `retinotopy_build.yml` workflow. Whenever you push a new version tag (e.g., `v1.2.3`), GitHub will automatically build all versions and attach them to a new Release page on your repository.

Chapter 26

Publishing an example as a standalone repository

This guide turns one directory from `examples/` into its own self-contained GitHub repository, so a colleague can:

- download a **prebuilt binary** for Linux/macOS/Windows and run it, and
- optionally **play it in the browser** (a WebAssembly build on GitHub Pages).

It is the process used for the two reference repos — copy either as a template:

Repo	Kind	Browser build?
`chrplr/Language-Localizer-French-audio`	audio experiment, ships `.wav` assets	no
`chrplr/Rush-Hour`	mouse experiment, everything embedded	yes (Pages)

> **Just need to hand off a zip, not publish a repo?** Use the Makefile wrapper from the repo root:
>
> ```
> make share-<ExampleName>                    # → _
>  ↪  build/share/<ExampleName>/
> make share-<ExampleName> VERSION=v0.12.5     # pin a specific
>  ↪  goxpyriment release
> ```
>
> That exports a standalone *module* (buildable on its own with `go run .`, no workspace) to zip and email — see `examples/README.md` in the repository. This guide goes further: it turns that output into a *published repository* with release CI and an optional in-browser build. Start from `share.sh` (Step 1), then add the pieces below.

26.1 When to use it

Publish an example this way when you want an external audience to run it without the goxpyriment workspace. Only publish a **browser** build for paradigms that tolerate browser timing (choice RT, memory, attention, puzzles) — not ones that need sub-millisecond onset control (rapid RSVP, subliminal priming). See WASM.md.

26.2 Prerequisites

- The `gh` CLI, authenticated (`gh auth status`).
- For a **browser** build: the example must embed *all* its assets with `//go:embed` (no filesystem exists in the browser), and it should avoid APIs still stubbed on js — see WASM.md.
- Use a goxpyriment release that contains the browser fixes ($\geq$ **v0.12.5**): the js `Save()` no-op (one download at the end, not per block) and the mouse path fix (`RenderCoordinatesFromWindow`). Earlier releases crash or spam downloads in the browser.
- **Pin a release that delivers the session as a single `.zip`.** Until 2026-08-29 the browser build fired two downloads (the `.csv` and the `-info.txt`) milliseconds apart, and browsers set to *ask where to save each file* silently dropped the second one — sessions arrived with their metadata and no results. See "Why the results arrive as a .zip". Check which release you are pinning before you publish anything that will collect real data, and make sure your `web/index.html` describes the download the pinned version actually produces.

26.3 Step 1 — Generate the standalone module

From the goxpyriment repo root, export the example with the latest release pinned (or pass a specific version):

```
bash examples/share.sh <ExampleName> <version> <output-parent-dir>
# e.g.
bash examples/share.sh Rush-Hour v0.12.5 ~/00_git
# → ~/00_git/Rush-Hour/  with a generated go.mod/go.sum requiring
 ↪  goxpyriment v0.12.5
```

`share.sh` copies the directory, drops the workspace `go.mod`/`go.work` coupling, writes a fresh `go.mod` requiring the *published* goxpyriment, and runs `go mod tidy`. The result builds on its own with `go build .`.

> **`share.sh` starts with `rm -rf` on its destination.** Point the third argument at a parent directory where `<ExampleName>/` does **not** yet exist. Re-running it over a repository you have already started would delete the `.git` directory and everything you added in the steps below.

`make share-<ExampleName>` is the convenient wrapper, but it always writes to `_build/share/<ExampleName>/`; call `share.sh` directly with the third argument when you want the module created straight where the repo will live (e.g. `~/00_git`).

26.4 Step 2 — Prune goxpyriment-internal files

The copy still contains files that only make sense inside the monorepo. Remove:

```
cd ~/00_git/<ExampleName>
rm -rf .claude meta.yaml description.md   # gallery/agent metadata, not
 ↪  needed standalone
```

Keep the `.go` sources, embedded assets, tests, and `README.md`.

If the example ships its own launcher page (`web/index.html`, as `examples/Reading-1` and `examples/Memory_span` do), `share.sh` has copied it along with everything else — you do **not** need Rush-Hour's bare page. But it was written for the monorepo, so grep it and `web/README.md` for references that no longer resolve:

- `make wasm-<Name>-serve` — that target exists only in the goxpyriment repo.
- `docs/WASM.md` and other relative doc links — repoint them at https://chrplr.github.io/goxpyriment/WASM/.
- comparisons to sibling examples ("unlike `Memory_span/web`, …").

While you are in there, add a link back to the published repository: a demo page with no way to reach the source or the desktop build is a dead end for anyone who wants to actually run the experiment.

26.5 Step 3 — Add the repo scaffolding

Add these files (copy and adapt from the reference repos):

- **`LICENSE`** — pick a license. goxpyriment itself is Apache-2.0, but **you are its author**, so you may license your example however you like (Rush-Hour is MIT). If you change the license, update the per-file header comments to match.
- **`build.sh`** / **`build.bat`** — a one-command local build (`CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o <Name> .`). These let a colleague build a *locally-signed* binary that avoids OS gatekeeper warnings.
- **`.gitignore`** — ignore the built binary. Note `go build .` produces a binary named after the **module** (lowercase, e.g. `rush-hour`) *and* `build.sh` produces the `-o` name (e.g. `Rush-Hour`); ignore **both**, plus `dist/` and `_build/`.
- **`README.md`** — adapt the example's README: change `go run ./examples/<Name>` to `go run .`, and add "Binaries" (Releases) and, if applicable, "Play in your browser" sections. The reference READMEs include the standard notes about unsigned macOS/Windows binaries.
- For a **browser** build only: **`web/index.html`** — if the example already had one, `share.sh` copied it and Step 2 tells you what to rewrite in it; otherwise copy Rush-Hour's, which wires the canvas, `sdl.js`, `wasm_exec.js` and `main.wasm` and nothing else. A page that asks for a participant ID before starting (`examples/Reading-1/web/index.html`) is the fuller template: it writes the ID back as `?s=<id>` with `history.replaceState`, which is how the Go side receives it.

26.6 Step 4 — Add the CI workflows

26.6.1 `.github/workflows/release.yml` (native binaries)

Cross-compiles Linux/macOS/Windows on every push and, on a `v*` tag, publishes a GitHub Release with archives + `SHA256SUMS`. `CGO_ENABLED=0`; go-sdl3 is pure Go and embeds SDL for every platform, so all three cross-compile from Linux. This path uses **upstream** go-sdl3 — no `replace`. Copy Rush-Hour's verbatim and change the binary name.

26.6.2 `.github/workflows/pages.yml` (browser build — optional)

Builds the WebAssembly bundle and deploys it to GitHub Pages. Two things make it work (copy Rush-Hour's file and change nothing but the repo name in comments):

1. It checks out the **`chrplr/go-sdl3-wasm`** fork (branch **`wasm-render-fixes`**), which supplies the js/wasm SDL bindings and the `wasmsdl` bundler (ships `sdl.js` + `sdl.wasm`).
2. It runs `go mod edit -replace github.com/Zyko0/go-sdl3=../go-sdl3-wasm` **in CI only** (never committed), so the browser build uses the fork while native/release builds keep using upstream go-sdl3.

   Gotcha — branch ref, not pinned commit. `pages.yml` tracks the fork's *branch HEAD*, whereas goxpyriment's own `go.mod` pins a specific fork pseudo-version. A fix that lives only in goxpyriment's vendored copy (or an uncommitted fork edit) will **not** reach your repo's browser build until it is committed and pushed to `wasm-render-fixes`. This is the trap that broke Rush-Hour's first deploy.

26.7 Building the browser bundle locally

CI is not the only way to get a bundle, and you should get one before you publish — a Pages deploy is a slow way to discover that the page is broken. The standalone repo needs the same two ingredients the workflow uses:

```
cd ~/00_git/<ExampleName>
git clone -b wasm-render-fixes https://github.com/chrplr/go-sdl3-wasm
 ↪  ../go-sdl3-wasm
go mod edit -replace github.com/Zyko0/go-sdl3=../go-sdl3-wasm   # NEVER
 ↪  commit this
(cd ../go-sdl3-wasm && go run ./cmd/wasmsdl serve \
    -html "$PWD/../<ExampleName>/web/index.html" "$PWD/../<ExampleName>")
# → http://localhost:8080/?s=1, with the COOP/COEP headers
go mod edit -dropreplace github.com/Zyko0/go-sdl3                # before
 ↪  committing
```

Use `wasmsdl build -out _build/wasm …` instead of `serve` to inspect the bundle. It is about 10 MB (roughly half `sdl.wasm`, half `main.wasm`), which is why neither reference repo commits it: the workflow rebuilds it on every push, and committing 10 MB per

rebuild would bloat the history fast. Make sure `.gitignore` covers wherever you put it.

Verify the `replace` is gone before you commit — `git diff go.mod` — or the published module will point at a path that exists only on your machine.

26.8 Step 5 — Create the GitHub repo and enable Pages

```
cd ~/00_git/<ExampleName>
git init -b main && git add -A && git commit -m "Initial commit"

gh repo create chrplr/<ExampleName> --public --source=. --remote=origin
 ↪  --push

# Browser build only: enable Pages with GitHub Actions as the source
gh api -X POST repos/chrplr/<ExampleName>/pages -f build_type=workflow
```

The first push runs both workflows. The Pages URL is `https://chrplr.github.io/<ExampleName>/`.

26.9 Step 6 — Cut a release

Binaries are published only on a version tag:

```
git tag v0.1.0 && git push origin v0.1.0   # → release.yml builds +
 ↪  publishes the Release
```

Native binaries are independent of the browser build — tag whenever the source is ready.

26.10 Updating the pinned goxpyriment later

When a newer goxpyriment fixes something you need, re-pin and redeploy:

```
go get github.com/chrplr/goxpyriment@vX.Y.Z && go mod tidy
git commit -am "Bump goxpyriment to vX.Y.Z" && git push   # redeploys Pages
```

> **Gotcha — fresh tags and the checksum DB.** Right after you push a goxpyriment tag, `sum.golang.org` may not have indexed it yet and `go get` fails with a `500`. Fetch straight from the tag once with `GOPROXY=direct GOSUMDB=off go get github.com/chrplr/goxpyriment@vX.Y.Z`; the correct hashes land in your `go.sum`, so CI (which reads `go.sum`) is fine.

26.11 Verifying a browser build

Serve the bundle and open it in a real, focused, foreground tab:

```
# from the goxpyriment repo, before you split it out:
make wasm-<ExampleName>-serve      # http://localhost:8080/?s=1

# or the built standalone repo's Pages URL after deploy
```

Sanity checks: the experiment renders and responds; a normal end triggers exactly **one** download, a `.zip` holding the `.csv` and the `-info.txt` (not one download per block, and not two files); on a crash you see the error overlay and **no** download (see WASM.md). Check the browser console for panics — each names the file:line of any still-stubbed binding.

Two things that will mislead you while checking by hand:

- **An unfocused window throttles `requestAnimationFrame` to ~1 Hz.** goxpyriment notices and logs `WARNING: measured refresh 1.00 Hz differs from nominal 60.00 Hz`. A session driven from a background window is not a timing test, and a canvas that looks frozen is usually just being throttled.
- **A download that has not appeared is not necessarily lost.** If the browser asks where to save each file, the file lands only when the dialog is answered; Chrome's own record is in the `downloads` table of `~/.config/google-chrome/Default/History` (copy it first — it is locked while Chrome runs), where `state=2` with `interrupt_reason=40` and an empty `target_path` means the dialog was never answered.

26.12 See also

- `examples/README.md` (in the repository) — `make share-NAME` (the zip-and-email path)
- WASM.md — how the browser build works, timing, and known gaps
- How_to_package_your_experiment.md — the GoReleaser alternative for native binaries

Chapter 27

How `control/experiment.go` works

`control/experiment.go` defines the **`Experiment` facade** — the single object an experiment program interacts with. It owns every subsystem (screen, input, audio, data, design) and exposes thin convenience methods over them. Here's how it works, following the lifecycle:

27.1 1. The `Experiment` struct (the facade)

`Experiment` (lines 70–108) bundles pointers to the focused subsystem packages:

- `Screen` (`apparatus.Screen`) — window + renderer
- `Keyboard` / `Mouse` (`apparatus`) — input
- `Audio` / `AudioDevice` — sound
- `Data` (`results.DataFile`) — CSV logging
- `Design` (`design.Experiment`) — trial/block structure

Plus config (`BackgroundColor`, `Fullscreen`, `ScreenNumber`, fonts) and internal state: `event EventState`, `quitFlag` (atomic, set by the signal handler), and `endOnce` (ensures the data footer is written exactly once).

The doc comment (lines 47–54) states the design intent: **most methods are thin delegations** to subsystem packages, keeping the user API simple.

27.2 2. Construction & initialization

- **`NewExperiment(...)`** (148) — just fills the struct; no SDL yet.
- **`NewExperimentFromFlags(...)`** (177) — the standard entry point. Parses `-w` / `-d N` / `-s N`, builds the struct, then calls `Initialize()` and `log.Fatal`s on failure.
- **`Initialize()`** (476) does the real setup, in order:
  1. Loads the embedded SDL/TTF/IMG binaries (reusing loaders cached by `GetParticipantInfo` to avoid a double-load crash on macOS).
  2. `sdl.Init` + `ttf.Init`, audio device, then `apparatus.NewScreen`.
  3. Wires `Keyboard`/`Mouse` with **poll closures** (lines 522–549) — these call back into `e.PollEvents`, so all input flows through the experiment's single event

drain.
4. Loads the default Inconsolata font, creates the `DataFile`, writes system/display/participant metadata into its header.
5. Spawns a **signal handler goroutine** (595) for Ctrl-C/SIGTERM: it sets `quitFlag`, calls `finalizeData()`, and force-exits after 3s. It deliberately never calls SDL teardown (concurrent CGo from two threads → SIGSEGV).

27.3 3. The run loop

`Run(logic func() error)` (936) is the heart:

- `runtime.LockOSThread()` pins the goroutine — SDL video/render/events must all stay on one thread (the **single-thread contract**).
- It calls `sdl.RunLoop`, and each frame: recovers from `exitPanic`, calls `pumpFrame()`, then your `logic()` callback.
- Return `control.EndLoop` from `logic` to exit cleanly.

`pumpFrame()` (428) checks `quitFlag`, pumps OS events, and checks for QUIT/ESC — **without draining the queue**, so keyboard/mouse events keep their original hardware timestamps for `GetKeyEventTS`.

27.4 4. Event handling

`PollEvents(handler)` (630) is the one place SDL events are drained. It resets transient state, loops over events, records the first key/mouse/keyup of the cycle plus their nanosecond timestamps, and sets the **sticky** `QuitRequested` on ESC or window-close. The `Keyboard`/`Mouse` closures and `WaitAnyEventTS` all sit on top of this.

27.5 5. Convenience methods (the ergonomic layer)

These compose the common patterns so experiment code stays short:

- `Show` / `ShowTS` (208, 222) — clear+draw+flip; `ShowTS` returns the VSYNC flip timestamp for precise RT.
- `ShowTimed`, `ShowAndGetRT`, `ShowInstructions`, `ShowEndMessage`, `Blank`, `Wait` — each replaces a 2–3 line idiom (documented inline).
- `WaitAnyEventTS` (252) — blocks for any device event with a hardware timestamp.

The `exitPanic` sentinel (136) lets these methods abort to `Run`'s recover on ESC/quit without threading an error through every line of user code.

27.6 6. Teardown

`End()` (877) — deferred by the caller. Calls `finalizeData()` (writes the end-time footer + saves, guarded by `endOnce`), closes font/screen/audio/microphone, then `ttf.Quit`/`sdl.Quit`, and unloads the embedded library blobs in reverse order. `Fatal()` (910) is the post-init replacement for `log.Fatalf` that cleans up first.

The rest of the file is **delegation wrappers** — `AddBlock`, `ShuffleBlocks`, `AddBWSFactor`, etc. forward straight to `e.Design`; `SetVSync`, `Flip`, `LoadFont` forward to `e.Screen`. That's the facade pattern the doc comment describes: a simple surface, with the real logic living in each subsystem package.

Chapter 28

Library usage: reducing direct SDL use

The examples in `examples/` sometimes call the underlying **go-sdl3** (`sdl`) library directly. The goxpyriment library provides wrappers and re-exports so that typical experiment code can avoid importing `github.com/Zyko0/go-sdl3/sdl` for common operations.

28.1 Run-loop exit and error handling

- **Return** from your `exp.Run(...)` callback with `control.EndLoop` for normal exit (e.g. ESC or window close), instead of `sdl.EndLoop`.
- **Check** the error after `exp.Run(...)` with `control.IsEndLoop(err)` instead of comparing to `sdl.EndLoop`:

```
err := exp.Run(func() error {
    // ...
    return control.EndLoop
})
if err != nil && !control.IsEndLoop(err) {
    log.Fatal(err)
}
```

28.2 Key and button constants

Use **control** key and button constants so you don't need the `sdl` import for key names:

- Keys: `control.K_SPACE`, `control.K_ESCAPE`, `control.K_S`, `control.K_F`, `control.K_J`, `control.K_Q`, `control.K_R`, `control.K_G`, `control.K_B`, `control.K_Y`, `control.K_N`, `control.K_1`–`control.K_4`, `control.K_KP_1`–`control.K_KP_4`.
- Buttons: `control.BUTTON_LEFT`, `control.BUTTON_RIGHT`.
- Type: `control.Keycode` (alias for `sdl.Keycode`).

Example: `exp.Keyboard.WaitKeys([]sdl.Keycode{control.K_SPACE}, -1)` can stay as-is (Keycode is the same type), or use `[]control.Keycode{control.K_SPACE}` if you only import control.

28.3 Points and colors

- **Points:** use `control.Point(x, y)` or `control.Origin()` instead of constructing `sdl.FPoint{X: x, Y: y}` or `sdl.FPoint{X: 0, Y: 0}`.
- **Colors:** use `control.RGB(r, g, b)` or `control.RGBA(r, g, b, a)` for ad-hoc colors, and predefined colors like `control.DarkGray`, `control.LightGray`, `control.Black`, `control.White`, etc.

28.4 Screen

- Use **`exp.Screen.ClearAndUpdate()`** instead of calling `exp.Screen.Clear()` then `exp.Screen.Update()` when you clear the backbuffer and present in one step.

28.5 Timing

- Use **`clock.Wait(ms)`** for delays (milliseconds) instead of `sdl.Delay(ms)`. This keeps timing in one place and avoids pulling in SDL for the main loop throttle (e.g. `clock.Wait(10)` in event loops).

28.6 When you still need SDL

You will still need the `sdl` import for:

- Constructing **sdl.FRect** or other SDL types not yet wrapped (e.g. in `demo_play_two_videos` for destination rects).
- Any SDL API not yet exposed by goxpyriment (e.g. raw `sdl.PollEvent` in special cases like loading screens).
- Passing SDL types into stimuli that accept them (e.g. `sdl.Color` is still used by stimuli; control re-exports colors and RGB/RGBA helpers).

Over time, more of these can be moved behind goxpyriment APIs so that most experiments only need `control`, `io`, `misc`, and `stimuli`.

Chapter 29

Viewing the documentation locally

There are two complementary sets of documentation, and you can serve either of them on your own machine:

- **The API / source-code reference** — generated from the Go source by pkgsite.
- **This documentation site** — the hand-written guides you are reading now, built with zensical.

29.1 API reference (pkgsite)

`pkgsite` serves the Go source documentation (every package, type, and function) as a local website.

Install it once:

```
go install golang.org/x/pkgsite/cmd/pkgsite@latest
```

Then run it from the repository root and open http://127.0.0.1:8080 in your browser:

```
cd goxpyriment
pkgsite
```

To jump straight to a package, append its import path, for example the `stimuli` package:

```
http://localhost:8080/github.com/chrplr/goxpyriment@v0.0.0/stimuli
```

29.2 This documentation site (zensical)

The guides on this site are built with zensical. Install it once (it is listed in `docs/requirements.txt`):

```
pip install -r docs/requirements.txt
```

(or follow the zensical get-started guide).

Then, **from the repository root**, start a live-reload preview at http://127.0.0.1:8000:

```
zensical serve        # or: make serve
```

To produce a static HTML build in `site/` instead:

```
zensical build --clean   # or: make docs
```

!!! note Run these commands from the repository root. zensical auto-discovers its configuration file (`zensical.toml`), which lives at the root next to the `Makefile`.